



**Development of a Browser Extension for Phishing
Website and Email Detection Using Supervised Learning**

By

Hemanthrajah A/L N Krishnarajah

TP070844

APD3F2601CS(CYB)

A report submitted in partial fulfilment of the requirements for the degree of

B.Sc. (Hons) Computer Science Specialism in Cyber Security

at Asia Pacific University of Technology and Innovation.

Supervised by: Dr. Kamalakannan Machap

2nd Marker: Ts. Dr. Manimegalai Rajenderan

2026

DECLARATION OF THESIS CONFIDENTIALITY

Author's full name: **HEMANTHRAJAH A/L N KRISHNARAJAH**

IC No./Passport No.: **040303-14-1799**

Thesis/Project title: **DEVELOPMENT OF A BROWSER EXTENSION FOR PHISHING
WEBSITE AND EMAIL DETECTION USING SUPERVISED LEARNING**

I declare that this thesis is classified as:

- ☐ CONFIDENTIAL
☐ RESTRICTED
☒ OPEN ACCESS

I acknowledged that Asia Pacific University of Technology & Innovation (APU) reserves the right as follows:

1. The thesis is the property of Asia Pacific University of Technology & Innovation (APU).
2. The Library of Asia Pacific University of Technology & Innovation (APU) has the right to make copies for the purpose of research only.
3. The Library has the right to make copies of the thesis for academic exchange.

Author's Signature: 

Date: 27 August 2026

Supervisor's Name: **DR. KAMALAKANNAN MACHAP**

Date: 27 August 2026

Signature: 

Please fill in **all** the following details for library cataloguing purposes.

First Name: Hemanthrajah
Middle Name (only if applicable):
Last Name: N Krishnarajah
Title of the Final Year Project / Dissertation / Thesis: Development of a Browser Extension for Phishing Website and Email Detection Using Supervised Learning
Abstract: Phishing remains a persistent cybersecurity threat because deceptive websites and emails exploit user trust to obtain credentials, personal information, or financial value. This final year project designs and implements PhishGuard, a Chrome Manifest V3 browser extension that combines supervised learning with contextual risk analysis and user-facing protection. The completed system uses a Random Forest website classifier and a calibrated LinearSVC email classifier, a Flask API deployed on Render, and Neon PostgreSQL for authenticated account, security-event, and encrypted vault data. PhishGuard provides website protection, Gmail message scanning, explainable risk evidence, personal trust controls, multi-factor authentication, security dashboards, PDF reporting, and a client-side encrypted password vault. The website model achieves 83.15% accuracy with a 2.59% false-positive rate, while the email model achieves 97.18% accuracy, 98.94% precision, 93.27% recall, and 96.02% F1-score on its leakage-safe held-out evaluation. Contextual regression gates address important real-world cases without altering the frozen model scores. The project supports SDG 9 through safer digital innovation and SDG 16 through stronger protection against cyber-enabled fraud.
A few keywords associated with the work: Keywords — <i>phishing detection, browser extension, supervised learning, phishing websites, phishing emails, cybersecurity</i>
General Subject: Computer Science, Cybersecurity
Date of Submission: 2 nd September 2026

ACKNOWLEDGEMENT

To begin with, the developer would like to thank the project supervisor, Dr. Kamalakannan Machap, with all his heart, as he has been of great help, giving guidance and constructive feedback during the preparation of this investigation report. His remarks served to narrow down the focus of the research, enhance the quality of the analysis, and stay research focused. Time, guidance and support provided at the meetings and review sessions played a vital role in transforming this project into a more structured and academically based work.

The developer would also wish to express gratitude to Ts. Umapathy Eaganathan to brief, guide material and general coordination of the Final Year Project process. The format and expectations relayed by the project management team allowed the developer to realize the format that the investigation report should have and get each part of the report completed in a more organized way. These resources were particularly useful during the planning of the scope of the project, the ethics requirements, and the process of data collection.

The respondents who also participated in the survey that was conducted during this research are also given special thanks. Their time, opinions and feedback offered viable understanding of phishing awareness, user behavior, and expectations in regard to browser-based protection. The answers offered by the survey directly influenced the requirement analysis of the suggested system and assisted in making sure that the project was based on real user concerns and not technical assumptions only.

Lastly, the developer would like to acknowledge the family and friends that have been patient, encouraging, and supportive of the completion of this report. Their drive and knowledge were a valuable source of strength in revision, analysis and formatting. The investigation phase of the project would have been very hard without their assistance.

ABSTRACT

Phishing remains a persistent cybersecurity threat because deceptive websites and emails exploit user trust to obtain credentials, personal information, or financial value. This final year project designs and implements PhishGuard, a Chrome Manifest V3 browser extension that combines supervised learning with contextual risk analysis and user-facing protection. The completed system uses a Random Forest website classifier and a calibrated LinearSVC email classifier, a Flask API deployed on Render, and Neon PostgreSQL for authenticated account, security-event, and encrypted vault data. PhishGuard provides website protection, Gmail message scanning, explainable risk evidence, personal trust controls, multi-factor authentication, security dashboards, PDF reporting, and a client-side encrypted password vault. The website model achieves 83.15% accuracy with a 2.59% false-positive rate, while the email model achieves 97.18% accuracy, 98.94% precision, 93.27% recall, and 96.02% F1-score on its leakage-safe held-out evaluation. Contextual regression gates address important real-world cases without altering the frozen model scores. The project supports SDG 9 through safer digital innovation and SDG 16 through stronger protection against cyber-enabled fraud.

***Keywords* — phishing detection, browser extension, supervised learning, phishing websites, phishing emails, cybersecurity**

Table of Contents

CHAPTER 1: INTRODUCTION	12
1.1 Introduction.....	12
1.2 Problem Background	14
1.3 Project Aim	16
1.4 Objectives	16
1.5 Scope.....	16
1.6 Potential Benefits and Target Users	18
1.7 Overview of the Final Report	19
1.8 Project Plan	20
1.8.1 Part 2 Development and Completion Plan.....	23
CHAPTER 2: LITERATURE REVIEW	24
2.1 Domain Research	24
2.1.1 Present-day Phishing Scenario and Threat Development.	25
2.1.2 Phishing Websites	26
2.1.3 Phishing Emails	28
2.1.4 Structural, Textual, and Behavioral Indicators.....	30
2.1.5 Dataset Quality, Drift, and Generalization.....	31
2.1.6 Supervised Learning and Algorithms Choice.	33
2.1.7 Browser Extension as a Delivery Model.	34
2.1.8 Extension-Based Detection: Security, Privacy, and User Trust.	35
2.1.9 Summary of Domain Research	36
2.2 Similar Systems / Works	37
2.2.1 Typical features of Comparable Systems / Works.	37
2.2.2 Strengths and Weaknesses of Similar Systems	38
2.2.3 Conclusion	41
2.3 Technical Research	42
2.3.1 Functional and Non-Functional Requirements	42
2.3.2 Software Environment	43
2.3.3 Hardware Environment.....	45
2.3.4 Technical Architecture Considerations	46
2.3.5 Design Browser Extension.....	46
2.3.6 Evaluation Meters and Technical feasibility.	47
2.3.7 Technical Research Summary.	48
2.3.8 Operational considerations and Security Controls.....	48
2.3.9 Implementation Priority and Final Outcome	49
CHAPTER 3: METHODOLOGY	50
3.1 System Development Methodology.....	50
3.1.1 Importance of System Development Methodology	51
3.1.2 Methodology Options Considered	52

3.1.3 Comparative Analysis of Agile, RAD, and Kanban	53
3.1.4 Justification for Selecting Agile	55
3.1.5 Agile Process and Activities in the Selected Approach	56
3.1.6 Agile Iteration Roadmap Applied in Part 2	57
3.2 Data Gathering Design	58
3.2.1 Target Population and Sampling Method	59
3.2.2 Survey Instrument Design	60
3.2.3 Data Collection Procedure	60
3.2.4 Reason behind the selection of Survey Method	61
3.2.5 Supporting Document Analysis	62
3.2.6 Ethical Considerations	62
3.3 Analysis	63
3.3.1 Cross-Question Interpretation and New Themes	91
3.3.2 Final list of user requirements	92
3.3.3 Mapping Survey Results to Design Decisions	94
3.3.4 Implications for Model Design and Extension Workflow	95
3.3.5 Implemented System Architecture	96
3.3.6 Limitations of the Data Gathering and Requirement Analysis	97
CHAPTER 4: DESIGN AND IMPLEMENTATION	99
4.1 Introduction	99
4.2 Design	99
4.2.1 Overall System Architecture	99
4.2.2 Use Case Design	102
4.2.3 Activity Diagram Design	103
4.2.4 Sequence Diagram Design	108
4.2.5 Security, Trust, and Model Release Design	111
4.3 Database Design	116
4.3.1 Entity Relationship Design	116
4.3.2 Database Table Structure	117
4.3.3 Database Portability and Migration Design	117
4.4 Interface Design	117
4.5 Implementation	127
4.5.1 Distribution and Extension Installation	127
4.5.2 Account Registration and Multi-Factor Authentication	130
4.5.3 Website Protection and Manual Scanning	136
4.5.4 Dashboard, Reporting and Personal Trust	142
4.5.5 Encrypted Password Vault	145
4.6 Sample Code Implementation	147
4.6.1 Backend Website Prediction API	148
4.6.2 Website Model and Contextual Risk Analysis	151

4.6.3 Email Model and Contextual Risk Analysis	153
4.6.4 Authentication, MFA and Security Events	156
4.6.5 Chrome Extension Website Protection	161
4.6.6 Gmail Email Protection.....	164
4.6.7 Security Dashboard and PDF Reporting.....	165
4.6.8 Client-Side Encrypted Password Vault	166
4.7 Summary	170
CHAPTER 5: RESULT AND DISCUSSION	171
5.1 Introduction.....	171
5.2 Testing Design / Plan	171
5.2.1 Automated Unit and Regression Testing.....	171
5.2.2 User Acceptance Testing Design.....	185
5.2.3 Testing Environment and Acceptance Criteria.....	187
5.3 System Testing and Discussion.....	187
5.3.1 Automated Python Test Execution Record	187
5.3.2 Node.js Extension Test Execution	194
5.3.3 Website Model Evaluation	195
5.3.4 Email Model Evaluation	198
5.3.5 Security and Regression Discussion	201
5.3.6 User Acceptance Testing Execution and Results	201
5.3.7 Completed User Acceptance Testing Forms	203
5.3.8 Testing Discussion	215
5.4 Summary	216
CHAPTER 6: CONCLUSION	217
6.1 Critical Evaluation	217
6.2 Limitation.....	218
6.3 Recommendation	219
REFERENCES	220
Appendix A: PPF – Title Registration Proposal	224
Appendix B: Ethics Form	225
Appendix C: Log Sheets	229
Appendix D: A3 Project Poster.....	236
Appendix E: Gantt Charts	237
Appendix F: Selected Source Code Excerpts	239
Appendix G: Respondent Demographic Profile	242

LIST OF FIGURES

Figure 1: Visual Representation of Phishing Threats.....	25
Figure 2: Graphical Representation of Structural, Textual, and Behavioral Indicators in Phishing Detection.....	30
Figure 3: Graphical Representation of Supervised Learning and Classification Algorithms ..	33
Figure 4: Python Logo	43
Figure 5: Jupyter Logo.....	44
Figure 6: HTML, CSS & JavaScript Logo	44
Figure 7: Hardware Specifications Required for Browser Extension and Supervised Learning Development.....	45
Figure 8: Key Reasons for the Importance of System Development Methodology	51
Figure 9: Graphical Representation of the Agile Process and Activities in the Selected Agile Approach.....	56
Figure 10: Participant Consent.....	67
Figure 11: Age Group	68
Figure 12: Gender	69
Figure 13: Occupation.....	70
Figure 14: Email Usage Frequency.....	71
Figure 15: Frequency of Web Site Browsing.....	72
Figure 16: Phishing Website Awareness	73
Figure 17: Phishing Emails Awareness	74
Figure 18: Generalized knowledge of Phishing Attacks.....	75
Figure 19: Suspicious emails	76
Figure 20: Vexed Suspicion of Web.....	77
Figure 21: Trust in Self-Detection	78
Figure 22: Signs That Users Can be Suspicious of Phishing.....	79
Figure 23: Largest perceived risk of phishing	80
Figure 24: Mistakenly Clicking Suspicious Links.....	81
Figure 25: Soberness of Phishing in the present Digital Age	82
Figure 26: Usefulness of the Proposed Browser Extension.....	83
Figure 27: Most Helpful Extension Feature.....	84
Figure 28: Type of Preferred Alerts	85
Figure 29: Significance of Detection Speed	86
Figure 30: Ease of Use Importance.....	87
Figure 31: Readiness to Use the Extension.....	88
Figure 32: Other Features Wanted	89
Figure 33: Recommendations	90
Figure 34: PhishGuard System Architecture.....	100
Figure 35: PhishGuard Component Diagram	101
Figure 36: PhishGuard Use Case Diagram	102

Figure 37: Website Scan Activity Diagram.....	104
Figure 38: Email Scan Activity Diagram.....	105
Figure 39: Authentication and MFA Activity Diagram.....	106
Figure 40: Password Vault Activity Diagram	107
Figure 41: Website Scan Sequence Diagram	108
Figure 42: Email Scan Sequence Diagram	109
Figure 43: PDF Security Report Sequence Diagram	110
Figure 44: Password Vault Sequence Diagram	111
Figure 45: PhishGuard Risk Decision Flow	113
Figure 46: Machine Learning Development and Release Pipeline.....	114
Figure 47: PhishGuard Entity Relationship Diagram	116
Figure 48: Authentication and MFA Wireframe	118
Figure 49: Main Extension Popup Wireframe	119
Figure 50: Website Scan and Warning Wireframe	120
Figure 51: Manual Scan Wireframe.....	121
Figure 52: Gmail Email Protection Wireframe.....	122
Figure 53: Security Dashboard Wireframe	123
Figure 54: Personal Trust, Settings and Account Security Wireframe.....	124
Figure 55: Password Vault Wireframe	125
Figure 56: Security Report Export Wireframe.....	126
Figure 57: PhishGuard Public Website	127
Figure 58: PhishGuard Extension Download and Installation Instructions.....	128
Figure 59: Loading the Unpacked PhishGuard Extension in Chrome.....	129
Figure 60: PhishGuard Sign-in Interface	130
Figure 61: PhishGuard Account Creation Interface.....	131
Figure 62: Mandatory MFA Enrollment after Account Creation.....	132
Figure 63: One-Time Recovery Codes after Account Creation.....	133
Figure 64: Progressive MFA Lockout after Repeated Incorrect Codes	134
Figure 65: Suspicious Sign-In Activity Security Alert	135
Figure 66: Authenticated PhishGuard Popup.....	136
Figure 67: Current Website Scan Result	137
Figure 68: High-Risk Website Warning Page	138
Figure 69: Trust Website Confirmation from the Warning Page	139
Figure 70: Manual Website Scan Result	140
Figure 71: Low-Risk Email Scan Result	141
Figure 72: Security Dashboard Overview.....	142
Figure 73: Security Events and Personal Trust Management	143
Figure 74: Generated Security Scan Report.....	144
Figure 75: Password Vault Unlock	145
Figure 76: Password Vault Dashboard.....	146

Figure 77: Add Password to the Encrypted Vault	147
Figure 78: Website Phishing Prediction Pipeline.....	148
Figure 79: Machine-Learning Model Registry	149
Figure 80: Website Feature-Order Consistency Check.....	150
Figure 81: Contextual Website Risk Analysis.....	151
Figure 82: Website Model Evaluation and Regression Gate	152
Figure 83: Email Detection Orchestration	153
Figure 84: Hybrid Email Risk Adjustment	154
Figure 85: Email Regression Gate and Threshold Selection	155
Figure 86: Password Validation and Argon2 Hashing	156
Figure 87: TOTP Secret Protection and Authenticator Enrollment	157
Figure 88: Progressive MFA Lockout and Security Events.....	158
Figure 89: MFA Challenge and Security-Event Models.....	159
Figure 90: Registration, Login and MFA API Routes.....	160
Figure 91: Background Website Scan Request	161
Figure 92: Warning Page Continue and Trust Actions.....	162
Figure 93: Manifest V3 Permissions and Content Scripts	163
Figure 94: Automatic Gmail Scan Request and Race Protection	164
Figure 95: Authenticated PDF Download Handling.....	165
Figure 96: Vault Profile Creation and Data-Encryption-Key Wrapping.....	166
Figure 97: AES-GCM Vault Entry Encryption and Decryption	167
Figure 98: Phishing-Aware Manual Credential Fill.....	168
Figure 99: Ciphertext-Only Backend Vault Validation	169
Figure 100: Website Model Held-out Performance	196
Figure 101: Website Model Confusion Matrix	197
Figure 102: Email Model Held-out Performance	199
Figure 103: Email Model Confusion Matrix	200
Figure 104: PPF	224
Figure 105: Ethics Form Page 1.....	225
Figure 106: Ethics Form Page 2.....	226
Figure 107: Ethics Form Page 3.....	227
Figure 108: Ethics Form Page 4.....	228
Figure 109: Meeting Log 1	229
Figure 110: Meeting Log 2	230
Figure 111: Meeting Log 3	231
Figure 112: Meeting Log 4	232
Figure 113: Meeting Log 5	233
Figure 114: Meeting Log 6	234
Figure 115: Meeting Log 7	235
Figure 116: PhishGuard A3 Project Poster	236

Figure 117: FYP Part 1 (Investigation Report) Gantt Chart	237
Figure 118: FYP Part 2 Gantt Chart.....	238
Figure 119: Website Prediction API Source Code	239
Figure 120: Email Detection Orchestration Source Code.....	240
Figure 121: Authentication and MFA API Routes Source Code.....	241

LIST OF TABLES

Table 1: Detailed Project Plan.....	22
Table 2: Comparison of Similar Systems and Implemented PhishGuard System	41
Table 3: Comparison of Agile, RAD, and Kanban	55
Table 4: Summary of Survey Demographic Results.....	64
Table 5: Phishing Awareness and Exposure Summary	64
Table 6: Preferred Indicators of Phishing	65
Table 7: Desired Browser Extension Features	66
Table 8: Consolidated User Requirements.....	94
Table 9: PhishGuard Use Case Specification Summary	103
Table 10: Major PhishGuard Security Trust Boundaries	115
Table 11: Backend Database Tables.....	117
Table 12: Python Automated Test Plan	184
Table 13: Node.js Extension Test Plan.....	185
Table 14: UAT Scenario Design.....	187
Table 15: Testing Acceptance Criteria	187
Table 16: Python Automated Test Execution Summary	194
Table 17: Node.js Extension Test Execution	195
Table 18: Website Model Held-out Metrics	195
Table 19: Email Model Leakage-safe Held-out Metrics.....	198
Table 20: UAT Execution Summary	202
Table 21: Respondent Demographic Profile	243

CHAPTER 1: INTRODUCTION

1.1 Introduction

The growing reliance on online platforms has turned the manner in which individuals interact, labor, study and conduct daily transactions. Banking portals, learning management systems, cloud storage, e-commerce websites, social media accounts, and organizational resources have come to be mediated by web browsers and email services. This reliance has brought about a digital world where speed and convenience have become the order of the day. Simultaneously, though, it has also opened up more avenues to cybercriminals to take advantage of users by interfaces and messages that look normal, familiar, and credible.

Phishing is one of the most problematic and harmful types of cyber threats that affect ordinary users. Phishing attack tries to fool the user into taking an unsafe measure like clicking an infected link, downloading an infected file or typing credentials into a fake form. The deceit is normally based on impersonation. The attackers replicate authoritative brands, organizations, college systems, delivery services, or financial systems so that they can make the victim think that the communication is real. Technical success of phishing can therefore be less about hacking into a system and more about convincing the user to open the door willingly.

Phishing has become more serious due to the improvement of the quality of attacks. The previous phishing websites were usually easier to identify as they had bad grammar, lack of brand consistency, or even suspicious links. Conversely, current phishing attacks are often conducted with high-quality imitated webpages, persuasive headings, temporal rogue web domains, credible-sounding sender names, and well-timed social-engineering wording. Recent publications by the Anti-Phishing Working Group and the 2024 Internet Crime Report by the FBI indicate that phishing and spoofing are still among the most commonly reported types of cyber-enabled frauds, which implies that the issue is both very relevant to individuals and organizations (APWG, 2025; FBI IC3, 2025).

The old-fashioned anti-phishing tools like blacklists, reputation services, static rules, and general spam filters are also valuable, but not enough by themselves. Blacklists are responsive and rely on the detection of a bad site or spammer and only after detection can blocking commence.

Patterns that are known can be detected by rule-based systems, yet they cannot cope with attackers who modify the phrasing of messages, new domains, or modify technical characteristics enough to be detected. Practically, this implies that users can still be exposed to zero-day phishing attacks before a defensive system has sufficient evidence to identify them via older, more static approaches.

This is a more adaptive problem with supervised learning. A supervised-learning model can be trained on labelled data and deployed to label new examples by examining the underlying patterns that distinguish phishing and legitimate content, as opposed to relying solely on manually written rules. These patterns can consist of structural information provided by the URLs, textual information provided by the email messages and behavioral or contextual information accumulated by the webpage content. Recent work still demonstrates that supervised-learning schemes are competitive and workable in phishing detection, especially when the aim is to construct precise classification pipelines, not with the computational expense of vast end-to-end deep-learning systems (Asiri et al., 2023; Fares et al., 2024).

Technical accuracy cannot suffice in itself. A detection system should also be able to reach the user at the appropriate time and in a form that is easily understood. That is why this project concentrates not only on the detection model, but also on the delivery mechanism. The solution here is a browser extension which uses supervised learning on phishing websites and emails. This is a viable method since users will see suspicious web pages, content on webmail and login pages in real time on the browser. The project will deliver timely protection and meaningful user support to users through a combination of a usable extension interface and a detection model based on supervised learning.

1.2 Problem Background

Phishing remains effective as the attack is focused on human trust, habitual behavior, and time constraints instead of relying solely on technical shortcomings within software. People are used to clicking on an email message, clicking on a link on a message that looks like they know, and they log into a service without a thorough examination of the page they have entered. Hackers take advantage of these habits to create interactions that appear natural to avoid suspicion. A phishing email that pretends to be a campus alert, a failed delivery notice or a bank security message can lure a user to rush into quick action before further verification occurs.

The second aspect of the issue is the rising complexity of phishing. Phishing websites can now be constructed in a visual form almost identical to that of legitimate websites, and phishing emails can be sent with no apparent errors that formerly revealed them. Attackers take advantage of branding resources, temporary domains, redirection paths and well-constructed urgency messages to enhance believability. Other campaigns combine several tactics as one e.g. an email might be used to generate urgency and a spoofed website to steal the credentials of the victim. Recent sources indicate that such mixed strategies have rendered the distinction between suspicious and legitimate content a more challenging task to assess by human judgment, alone (Naqvi et al., 2023; Kavya et al., 2024).

Conventional defense strategies have a number of drawbacks within this setting. The blacklist-based methods are still effective against well-known bad destinations but fail to stop newly established phishing infrastructure. Rule-based systems have the con of the converse, but similar set of flaws: rule-based systems may be comprehensible and can be executed quickly, but they become fragile when the lexical, structural, or content-level characteristics of attackers change. Even the general email filters cannot always recognize spam and real phishing intent accurately. This leads to the user being exposed to bad pages or messages even with the common defensive layers in place.

Technical detection research and practical delivery to users, too, have a gap between them. A large body of research indicates that classification does well in controlled settings, although the

applicability of a detection model is determined by the manner in which the output is delivered to the user in real browsing. An alarm that comes too late disrupts the workflow too often or does not define risk in a sensible manner may not have the desired protective result. That is, phishing detection is not only a classification problem, but also a human-computer interaction problem. The browser extension format is appealing because the model can work in a similar environment where the user makes a trust decision.

That is why an adaptive, quick and practical solution is obviously needed, which can be used every day in the online environment. This requirement can be satisfied with a supervised-learning-based browser extension which provides the combination of data-driven classification and direct functionality protection to the user. The project aims to explore that hybrid: how phishing sites and email-related content can be identified with high precision with the current supervised-learning algorithms, and how the detection output can be presented in a way that fits well into the user's web experience, like a browser extension.

1.3 Project Aim

Design and develop a supervised learning-based system capable of accurately and efficiently detecting phishing websites and phishing emails by analyzing their structural, textual, and behavioral features, in order to enhance cybersecurity and protect users from online threats.

1.4 Objectives

- To analyze the common characteristics and techniques used in phishing websites and phishing emails.
- To collect and preprocess relevant datasets for phishing websites and email detection.
- To develop supervised machine learning models for classifying websites and emails as either legitimate or phishing.
- To evaluate the performance of the developed models using standard classification metrics.

1.5 Scope

The final project covers the design, implementation, deployment, and evaluation of a browser-based phishing protection system named PhishGuard. The scope retains the original research focus on supervised-learning detection of phishing websites and phishing emails, while the completed prototype also includes supporting security and usability mechanisms required for realistic operation.

1. Website Phishing Detection. PhishGuard extracts a controlled set of lexical and hostname-oriented URL features and applies a frozen Random Forest classifier. A bounded contextual analysis layer handles combinations of structural risk evidence, including prize or redirect-style patterns, punycode, IP-based credential destinations, suspicious query structures, and brand-impersonation characteristics. Raw model output is preserved separately from contextual and personalized risk.

2. Email Phishing Detection. The system uses a calibrated LinearSVC classifier with TF-IDF word unigrams and bigrams. Gmail is supported for automatic extraction of the currently opened visible message, while manual pasted email text is available for other providers. Sender identity, visible links, personal trusted-sender decisions, and high-severity contextual indicators are evaluated separately from the base text-model probability.

3. Browser Extension and Hosted Backend. The user-facing component is a Chrome Manifest V3 extension implemented using HTML, CSS, and JavaScript. The production configuration communicates through HTTPS with a Flask API hosted on Render. The deployed API loads the saved scikit-learn model artifacts and uses Neon PostgreSQL through SQLAlchemy and psycopg for persistent account-related data. Localhost and SQLite remain development alternatives rather than the primary deployed demonstration path.

4. Authentication and Account Security. Every normal user session requires password verification followed by Time-based One-Time Password multi-factor authentication or a one-time recovery code. Passwords and recovery codes use Argon2id hashing, MFA secrets are encrypted at rest, sessions are opaque and revocable, failed-password traffic is source-throttled, and repeated MFA failures use progressive lockouts with user-owned security events.

5. Security Dashboard, Trust, and Reporting. PhishGuard records a privacy-safe user-scoped scan history in the extension, provides filters and recent security activity, supports exact sender, email-domain, exact website, and registrable-domain personal trust decisions, and

generates authenticated individual or date-range PDF security reports from signed scan receipts. Personal trust changes the user-specific contextual presentation without rewriting the original ML result.

6. Client-Side Encrypted Password Vault. The extension includes a separate vault master password, PBKDF2-HMAC-SHA-256 key derivation, a randomly generated data-encryption key, AES-256-GCM authenticated encryption, secure password generation, encrypted save/update flows, and phishing-aware manual fill. The Flask backend and Neon database store only opaque ciphertext and wrapped-key material; they do not receive the vault master password or plaintext vault entries.

7. Constraints and Exclusions. PhishGuard remains an undergraduate prototype and does not claim to replace enterprise secure email gateways, endpoint protection, commercial password managers, or live reputation services. Automatic email extraction is limited to Gmail, the website model remains URL-feature-oriented, live WHOIS/domain-age lookups are intentionally excluded from the runtime path, and the system still requires realistic Chrome and user acceptance testing before a final production claim can be made.

1.6 Potential Benefits and Target Users

The suggested project has its tangible and intangible advantages. Tangible benefits are those that can be easily noticed or quantified in the process of developing and evaluating a system. Here, the key concrete deliverables include the prototype of a browser extension, a trained supervised-learning model or models, quantifiable evaluation outcomes based on classification measures, and a specified user-facing warning workflow. These results are testable, compare and write. When effectively used the system can also mitigate the chances that a user clicks on a malicious site or acts recklessly on a site that is potentially suspicious by giving a warning when the risk occurs.

Intangible benefits are as well crucial. By demonstrating what kinds of signals users should be suspicious of, the project can raise their awareness of phishing tricks. It may also help to instill user confidence by providing support during the decision-making process, as opposed to compelling the user to wholly depend on his or her intuitions. Academically, the research will add value to the applied body of knowledge in cybersecurity by demonstrating how to combine supervised learning, user feedback and delivery via a browser into a single, practical anti-phishing

concept. The report also enhances the developer with the knowledge of secure design, phishing and user centered detection systems.

The general internet users, university students, working adults, and staff members who use browsers and email to fulfill their daily activities are the target users of the proposed system. University students are particularly of interest since they often deal with login pages, institutional notices, cloud applications, and webmail applications, which makes them an appealing group of victims of phishing. Adults at work and general users also matter since phishing is not restricted to the formal organizational environment; it is often used to target personal finance, delivery services, social media, and productivity tools. Hence, the purpose of the proposed browser extension is not to be as technical as it can be understood and useful to both academic and non-academic users.

1.7 Overview of the Final Report

Chapter 1 introduces the phishing problem and the rationale for PhishGuard, then defines the final aim, objectives, completed scope, target users, benefits, and the full project plan from the investigation stage through the 2 September 2026 submission.

Chapter 2 presents the literature review and technical research that informed the final system. It examines phishing websites and emails, structural and textual indicators, dataset quality and generalization, supervised-learning choices, browser-extension delivery, similar systems, security and privacy considerations, and the implemented software, deployment, and architecture decisions.

Chapter 3 explains the methodology used to move from investigation to implementation. It justifies Agile development, documents the survey and requirement-analysis process, interprets 44 valid responses, maps those findings to the implemented warning, privacy, trust, performance, and usability decisions, and records the Agile iterations applied during Part 2.

Chapter 4 documents the completed design and implementation of PhishGuard. It covers the Manifest V3 extension, Render-hosted Flask API, Neon PostgreSQL persistence, database structure, wireframes, final interfaces, website and email model development, contextual risk analysis, authentication and MFA, Gmail protection, dashboard and reporting, encrypted password vault, deployment, source-code evidence, and final system screenshots.

Chapter 5 presents testing, results, and discussion. It records the website and email model evaluation, automated and regression testing evidence, security and release-gate checks, and the user acceptance testing plan and analysis section for the intended target audience. UAT outcomes are reported only after completed tester evidence is available.

Chapter 6 concludes the project through critical evaluation of the completed system, its main contributions and strengths, acknowledged limitations, and recommendations for future work including broader webmail support, independent modern datasets, stronger distributed rate limiting, automated end-to-end browser testing, and optional external reputation signals.

1.8 Project Plan

The project plan represents the complete Final Year Project rather than only the Investigation Report period. Part 1 ran from 26 January 2026 to 22 April 2026 and covered problem definition, Chapters 1 and 2, survey and ethics preparation, the confirmed survey collection window from 15 April to 17 April 2026, Chapter 3 analysis, supervisor review, and submission of the investigation work. Part 2 began on 11 June 2026 and continues through architecture refinement, implementation, deployment, testing, Chapters 4 to 6, appendices, final quality assurance, and the final submission on 2 September 2026.

The confirmed academic milestones are retained in the plan: title confirmation on 11 February 2026; Part 1 supervisor meetings on 31 March, 14 April, 20 April, and 22 April; survey collection from 15 April to 17 April; FYP Part 2 commencement on 11 June; the second briefing on 18 June; the third briefing class on 9 July; the final FYP briefing class on 13 August 2026 from 8:30am to 10:30am; and final submission on 2 September 2026. The development ranges between these milestones are reasonable estimates reconstructed from the completed PhishGuard feature set and are intentionally overlapped to reflect the Agile workflow used throughout Part 2.

Table 1 presents the complete project plan from the investigation stage through Chapters 4, 5, and 6 and final submission. The detailed Part 1 and Part 2 Gantt charts are retained in Appendix D. Section 1.8.1 provides a more detailed Part 2 development and completion plan aligned to the implemented system.

WBS	Detailed Task / Activity	Start	End	Deliverable / Output
P1-T1	Review FYP/IR requirements, refine the phishing-protection domain, and establish the investigation backlog.	26 Jan 2026	04 Feb 2026	Investigation scope and work plan
P1-T2	Prepare the title proposal and obtain project-title confirmation.	05 Feb 2026	11 Feb 2026	Confirmed project title
P1-T3	Draft Chapter 1: problem background, aim, objectives, scope, benefits, report overview, and initial project plan.	12 Feb 2026	03 Mar 2026	Chapter 1 draft
P1-T4	Complete Chapter 2 literature review, similar-system comparison, and technical research.	24 Feb 2026	31 Mar 2026	Chapter 2 draft
P1-T5	Design the questionnaire and prepare ethics-related supporting material.	18 Mar 2026	07 Apr 2026	Survey and ethics evidence
P1-T6	Supervisor Meeting 1 and first integrated review of the investigation work.	31 Mar 2026	31 Mar 2026	Supervisor feedback
P1-T7	Revise the integrated Part 1 report and finalise data-collection preparation.	01 Apr 2026	14 Apr 2026	Pre-collection IR draft
P1-T8	Supervisor Meeting 2 and pre-data-collection review.	14 Apr 2026	14 Apr 2026	Revision actions
P1-T9	Distribute the questionnaire and collect valid target-user responses.	15 Apr 2026	17 Apr 2026	Survey dataset
P1-T10	Analyse survey findings and complete Chapter 3 methodology, requirements, and design implications.	18 Apr 2026	20 Apr 2026	Chapter 3 draft and requirements
P1-T11	Supervisor Meeting 3 and final Part 1 correction pass.	20 Apr 2026	20 Apr 2026	Pre-submission review
P1-T12	Supervisor Meeting 4 and Part 1 submission completion.	22 Apr 2026	22 Apr 2026	Completed Investigation Report
P2-T1	FYP Part 2 commencement, backlog review, architecture confirmation, and design refinement.	11 Jun 2026	02 Jul 2026	Part 2 plan and updated design
P2-T2	Develop and integrate website model, contextual URL analysis, and website warning workflow.	19 Jun 2026	16 Jul 2026	Website phishing protection
P2-T3	Develop email model refinements, contextual analysis, and Gmail protection workflow.	03 Jul 2026	30 Jul 2026	Email/Gmail phishing protection
P2-T4	Implement authentication, mandatory TOTP MFA, recovery codes, sessions, lockouts, and security events.	10 Jul 2026	30 Jul 2026	Account-security workflow

P2-T5	Implement the Security Dashboard, authenticated PDF reporting, personal trust, and encrypted password vault.	17 Jul 2026	13 Aug 2026	Supporting security features
P2-T6	Configure hosted deployment, database persistence, integration checks, and regression protection.	31 Jul 2026	20 Aug 2026	Integrated deployed prototype
P2-T7	Complete Chapter 4 design and implementation evidence, code screenshots, system screenshots, and explanations.	14 Aug 2026	26 Aug 2026	Chapter 4 completed
P2-T8	Execute automated/system testing and UAT activities and analyse results for Chapter 5.	18 Aug 2026	30 Aug 2026	Chapter 5 testing and discussion
P2-T9	Complete Chapter 6 critical evaluation, limitations, and recommendations.	24 Aug 2026	31 Aug 2026	Chapter 6 completed
P2-T10	Finalize APA references, appendices, poster, Turnitin evidence, formatting, and submission QA.	24 Aug 2026	01 Sep 2026	Submission package
P2-T11	Submit the final FYP.	02 Sep 2026	02 Sep 2026	Final submission

Table 1: Detailed Project Plan

1.8.1 Part 2 Development and Completion Plan

Part 2 is scheduled from 11 June 2026 to 2 September 2026. The early weeks are allocated to architecture refinement, dataset/model confirmation, and browser-protection integration; July concentrates on website and email workflows together with authentication and MFA; late July to mid-August covers dashboard, reporting, vault, deployment, and integration; and the final weeks are reserved for regression testing, Chapter 4 documentation, Chapter 5 testing and UAT, Chapter 6 evaluation, appendices, poster preparation, Turnitin, and final submission. The ranges below are realistic project estimates aligned with the completed system and the confirmed briefing dates rather than an attempt to treat every code commit as the start of development.

WBS	Detailed Task / Activity	Start	End	Deliverable / Output
P2-01	FYP Part 2 commencement and backlog review.	11 Jun 2026	11 Jun 2026	Part 2 kickoff
P2-02	Architecture, database, interface, and implementation-plan refinement.	12 Jun 2026	02 Jul 2026	Updated technical design
P2-03	Second FYP briefing.	18 Jun 2026	18 Jun 2026	Briefing milestone
P2-04	Website dataset/model confirmation, contextual URL analysis, and Chrome website protection.	19 Jun 2026	16 Jul 2026	Website protection workflow
P2-05	Third briefing class.	09 Jul 2026	09 Jul 2026	Briefing milestone
P2-06	Email model refinement, contextual email analysis, summaries, and Gmail integration.	03 Jul 2026	30 Jul 2026	Email/Gmail protection workflow
P2-07	Authentication, TOTP MFA, recovery codes, session security, progressive lockouts, and security events.	10 Jul 2026	30 Jul 2026	Secure account workflow
P2-08	Security Dashboard, scan history, trust controls, and PDF reporting.	17 Jul 2026	06 Aug 2026	Dashboard/reporting workflow
P2-09	Client-side encrypted password vault, generator, save/fill controls, and backend ciphertext storage.	24 Jul 2026	13 Aug 2026	Encrypted vault
P2-10	Hosted Render/Neon deployment, integration, and regression testing.	01 Aug 2026	20 Aug 2026	Deployed integrated prototype
P2-11	Final FYP briefing class, 8:30am-10:30am.	13 Aug 2026	13 Aug 2026	Final briefing milestone
P2-12	Chapter 4 design/implementation writing, screenshots, diagrams, and source-code evidence.	14 Aug 2026	26 Aug 2026	Chapter 4 completed
P2-13	Chapter 5 automated/system testing, browser validation, UAT execution, results analysis, and discussion.	18 Aug 2026	30 Aug 2026	Chapter 5 completed
P2-14	Chapter 6 critical evaluation, limitations, recommendations, final	24 Aug 2026	01 Sep 2026	Chapter 6 and final report package

WBS	Detailed Task / Activity	Start	End	Deliverable / Output
	references, appendices, poster, and Turnitin.			
P2-15	Final FYP submission.	02 Sep 2026	02 Sep 2026	Submitted FYP

CHAPTER 2: LITERATURE REVIEW

2.1 Domain Research

This chapter reviews the phishing-detection domain from the general threat context to the technical decisions that informed the completed PhishGuard system. It covers phishing websites, phishing emails, supervised-learning design choices, browser-extension delivery, similar systems, dataset quality, privacy, performance, generalization, and deployment considerations. The purpose is not simply to list previous work, but to establish the research and technical foundation against which the implemented system can be understood and evaluated.

The priority of the sources used in this chapter is only recent and credible materials that are released after 2022 to ensure that the investigation is up to date with the current research on phishing. This is crucial since the world of phishing threats is rapidly changing. Attackers modify the lexical format of URLs, switch channels of delivery, enhance the quality of email wording and integrate technical tricks with social engineering more and more. Thus, literature on the topic created in the recent past is more helpful than older literature in comprehending the current limitations of browser-based phishing protection.

2.1.1 Present-day Phishing Scenario and Threat Development.



Figure 1: Visual Representation of Phishing Threats

Recent findings have always indicated that phishing is no longer an email specific issue. It has become a large ecosystem of fake interactions that encompasses cloned sites, brand impersonation, counterfeit document-sharing alerts, account-checking, and fraudulent delivery notices, and credential-gathering campaigns spread via various channels. According to Naqvi et al. (2023), phishing is a dynamic and adaptive type of attack that has remained uneliminated due to its use of people, interfaces, and trust relationships as opposed to just focusing on technical vulnerabilities. They also conclude in their review that effective mitigation needs a mix of prevention, detection, user education and responsive reporting, which also suggests that a single fixed defensive layer might not suffice.

Although awareness of phishing has grown, the Anti-Phishing Working Group reported almost one million phishing attacks in the fourth quarter of 2024 alone, which shows that the phishing activity, on a high scale, does not decrease (APWG, 2025). Similarly, the 2024 Internet

Crime Report by FBI reported phishing and spoofing among the most commonly reported internet crimes, which confirms that the threat is still impacting significant user numbers in practice (FBI IC3, 2025). Such official reports are helpful since they supplement academic results with working evidence: phishing is not merely a lab issue, but a constantly occurring user-facing threat.

Attackers are becoming more adept at using technical evasion in conjunction with social engineering finesse as phishing campaigns continue to become increasingly sophisticated. Contemporary-day campaigns deploy temporary infrastructure, redirection chains, Unicode or lexical manipulation, realistic forms, and credible-looking sender identities. Sometimes the email itself is not the goal, it is a bait that directs the user to a page that harvests credentials or performs some other action that is malicious. This cross-channel behavior is the key in the current project since a browser extension that checks websites and not email-linked behavior would only cover a portion of the decision chain, which results in victimization.

Recent reviews also suggest that the development of phishing is driving researchers towards more adaptive, explainable and deployable solutions well near the user. Kavya et al. (2024) suggest that future-oriented phishing detection should be concerned not only with the classification accuracy, but also with the robustness, interpretability, and the response to the novel variants of attack. This is important in the context of an undergraduate project as it implies that a viable solution must be able to compromise between model complexity and maintainability as well as human usability. An example of a lightweight supervised-learning pipeline that can be delivered as a browser extension is thus quite consistent with the pragmatic path indicated by the existing literature.

2.1.2 Phishing Websites

Phishing sites are a type of fraudulent webpage created to mimic the appearance of a valid online site to gather information, spread malware, or make dangerous user choices. The fundamental deceit is normally pictorial and circumstantial. A web page can replicate the color scheme, layout, and logos of a bank, cloud service provider, university portal, or online shopping site, and make the user think they have arrived at a secure location. The misrepresentation in most instances is

supported by lexical similarity in the URL, counterfeit security language or a well-timed email spoil that causes the page to seem anticipated.

Safi and Singh (2023) note that phishing-website research has made extensive use of machine-learning methods because they can model lexical, host-based, and content-oriented features more flexibly than fixed rules. They also show that dataset size, age, source, and feature design directly affect generalisation. A classifier that performs well on one benchmark may not behave identically on a different threat distribution. This consideration is directly relevant to PhishGuard because the implemented browser extension is intended for ordinary web interactions rather than a single curated benchmark environment.

Contemporary phishing sites manipulate URLs and visible trust cues to conceal risk. Common patterns include long URLs, suspicious subdomains, deceptive tokens such as login, verify, or secure, IP-address destinations, shortened links, and visually similar domains. Website risk can also be reflected by page structure, form behavior, embedded links, favicon sources, scripts, and external-resource dependencies. However, not every possible feature is equally lightweight, reliable, or appropriate to collect from a browser extension. PhishGuard therefore uses a deliberately controlled website feature set and supplements the frozen model with a separate contextual-risk layer rather than attempting to collect every theoretically available signal.

Ongoing studies continue to add to feature engineering of phishing sites. Omari (2023) investigated gradient boosting to detect phishing and found that structured sets of features can lead to a great classification strength being relatively interpretable. Rashid et al. (2024) further stressed on the significance of generalization in datasets and the collection environment. Their work particularly comes in handy with this project since they warn against overconfidence on models that work well in just one set of data. In order to be useful in practice, a browser extension should not be too specific to a particular training distribution.

A second important finding from recent literature is that user-facing detection must consider how risk is communicated, not only whether a site is classified as malicious. A browser extension cannot assume that users will inspect raw URL features or interpret technical model output. PhishGuard therefore converts classification and contextual evidence into concise,

actionable feedback through risk labels, warning pages, explanations, and explicit user choices. This principle is reflected in the implemented interface documented in Chapter 4.

2.1.3 Phishing Emails

Phishing emails are also one of the most frequent and most effective delivery mechanisms of phishing attacks as email is a daily tool that is used in academic, personal, and business life. The phishing email can demand a verification of the password, purport that an account will be closed, impersonate a document-sharing service, or generate a sense of urgency about paying, tax, delivery, or security. The message is meant to break habitual behavior, and it is meant to compel the user to take immediate action before thorough scrutiny is carried out.

Email phishing is not the same as generic spam since the goal is typically more targeted and more dangerous. The message can include a link to a bogus login form or prompt the user to open a weaponized attachment or gather data by use of a reply-based scam. Fares et al. (2024) demonstrate that machine-learning algorithms like the Random Forest, the Support Vector machine and the XGBoost can still be used to classify email-phishing when trained on meaningful features and assisted by an appropriate choice of features. Their work is particularly applicable to this project since it shows that only classical supervised-learning models can be used to detect email-based phishing and not large and resource-intensive ones.

The feature-design questions are also presented by the email domain of phishing detection. Other characteristics are lexical and textual e.g., suspicious words, urgency language or strange grammar. Others include the structural ones, e.g. inconsistency in senders, mismatch in domain, linkage patterns or indication of attachment. Meléndez, Ptaszynski, and Masui (2024) compared models built with traditional machine-learning with transformers-based models in detecting phishing emails and concluded that more recent language-centric models have the potential to offer high-quality results, albeit with the caveat of needing to carefully design datasets, trade-offs in computation, and resilience to the changing content of emails. In the present project, this justifies a practical way: classical supervised-learning practices may be used in case the objective is to develop a viable and fairly lightweight extension-oriented solution.

A deployment challenge also exists because a browser extension does not automatically receive unrestricted access to native email clients, and even webmail content is constrained by browser permissions, privacy expectations, and page structure. The completed PhishGuard email workflow therefore focuses on browser-accessible signals: the currently opened Gmail message can be extracted automatically, while manual pasted text supports other providers. Sender identity, visible links, message text, and contextual evidence are then analysed without requiring mailbox-wide access.

Phishing email is particularly important to the user-centred design because it often begins a multi-stage attack. A suspicious message may not directly steal credentials, but it can establish trust and direct the user to a fraudulent website. PhishGuard therefore treats email and website phishing as related protection paths within the same extension. The Gmail warning flow, website warning page, trust controls, and shared dashboard allow the user to review those related risks through one system.

2.1.4 Structural, Textual, and Behavioral Indicators.

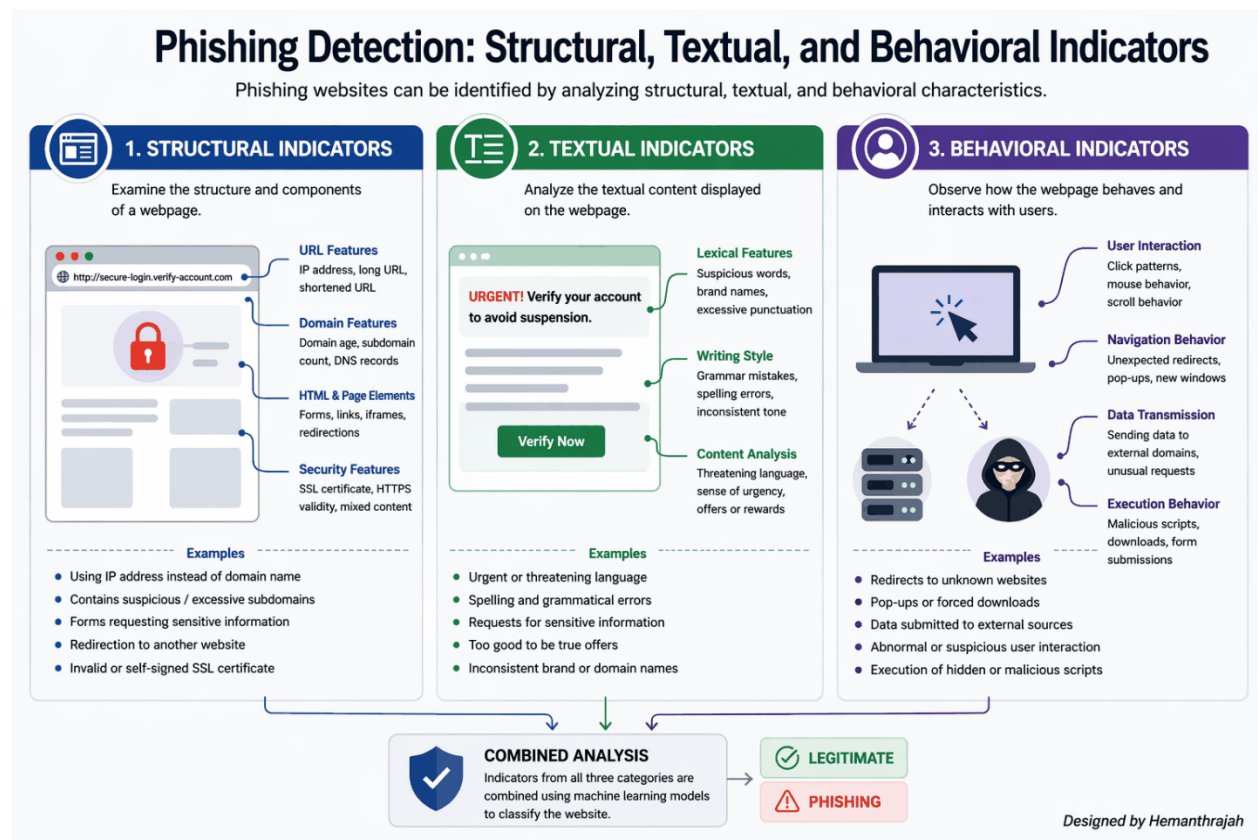


Figure 2: Graphical Representation of Structural, Textual, and Behavioral Indicators in Phishing Detection

PhishGuard does not rely on a single type of phishing indicator. The completed design combines structural, textual, and selected behavioural or contextual cues according to the protection path. Structural indicators include measurable properties of URLs, domains, paths, tokens, and related destination characteristics; email analysis additionally uses message text, sender information, and visible links. This combination is suitable for browser-integrated protection because the selected signals can be collected quickly without requiring heavy visual processing of every page.

Textual indicators are linguistic indicators in an email or webpage. This can encompass urgency words, words that request action, or words that attempt to evoke fear or compliance, lexical patterns related to credential gathering, and wording that does not match the purported identity of the sender or service. Text messages come in handy since phishing is usually based on

persuasion rather than technical deception. Meanwhile, textual analysis should be approached with precaution, since valid messages may also include an element of urgency or security jargon. This is the reason why textual features should be applied in conjunction with other cues and not a decision rule by itself.

Behavioral indicators are more contextual. Behavioral in the context of this project does not imply gathering invasive long-term browsing telemetry. Rather, it means observable context surrounding a suspicious interaction: such as whether a page immediately requests credentials, whether the destinations of links and links match that destination visible text, whether a sender is new, or whether the page tries to prompt a quick decision to trust or not via interface design. These indicators are used to fill the gap between raw features and risk to the user. They can be especially helpful in a browser extension since they are more intuitively aligned to the type of warning language a user can comprehend.

The recent research on explainable phishing detection makes the argument of cautious feature selection more robust. Shafin (2025) suggested an explainable feature-selection framework of identifying web phishing based on machine learning and established that not every available webpage feature is equally useful in a model. This is significant as large feature sets may also lead to complexity, which does not enhance feasible performance. In a project that is based on extensions, features and explainability are particularly desirable: the model must be correct enough to be useful, but simple enough to warrant and implement without unwarranted overhead.

Collectively, the literature suggests that a practical phishing-detection system should combine useful signals while remaining selective about what is actually processed. PhishGuard follows this principle. The final website and email pipelines use defensible feature and text representations that balance predictive value, interpretability, runtime cost, privacy, and maintainability, while contextual evidence is kept separate from the frozen model outputs.

2.1.5 Dataset Quality, Drift, and Generalization

Among the most significant problems of phishing detection research, the fact that datasets may become obsolete in the short term should be mentioned. Domains are dropped, campaigns are

changed and over time the linguistic nature of phishing messages changes. A model that has been trained on an older benchmark can thus have a good historical performance but a poor generalization in the real world. This problem has been brought to focus of recent studies. Rashid et al. (2024) directly consider the aspect of generalization among phishing URLs datasets and demonstrate that cross-dataset transferability is not an easy thing. The result is of significant importance to the present project since a browser extension is useful only when it is able to support the data that is not in the identical format as the training set.

The composition of the dataset also matters since phishing detection tends to be based on the class imbalance and heterogeneity of the sources. Phishing datasets can be collected based on reporting feeds, open repositories, synthetic collections, or hand-curated benchmarks. Popular-domain lists, or ordinary email corpora, or web archives may serve as legitimate samples. Every decision influence the task in a different way. As an example, just training on blatantly malicious URLs and very popular legitimate domains can provide a false sense of an easy line between the two. However, in a real-world browser context the model can come across ambiguous content that is between those extremes.

The other problematic area is the availability of features in datasets. Other datasets include webpage content, host features, or email bodies whereas some offer URL-level attributes only. This has a direct repercussion on reproducibility and deployment. An extension that is based on capabilities not provided by most of the public sources could be hard to test reliably. In comparison, a system that is built based on a smaller yet more reproducible set of features could be simpler to train, compare, and justify. Due to this reason, the current project focuses on feasibility and consistency rather than on the maximum number of features.

It is also mentioned in the literature that interpretability is important in the case of drift and generalization. When a model starts to give up on new data, a set of interpretable features makes it easier to diagnose why. Recent explainable-ML research in phishing detection supports the perception that transparent feature contribution is not only a scholarly matter, but also practically applicable to maintain and refine via iteration (Shafin, 2025). This underpins the choice of supervised-learning models in the project as more easily comparable, tuned, and explained within the constraints of an undergraduate cybersecurity project.

2.1.6 Supervised Learning and Algorithms Choice.

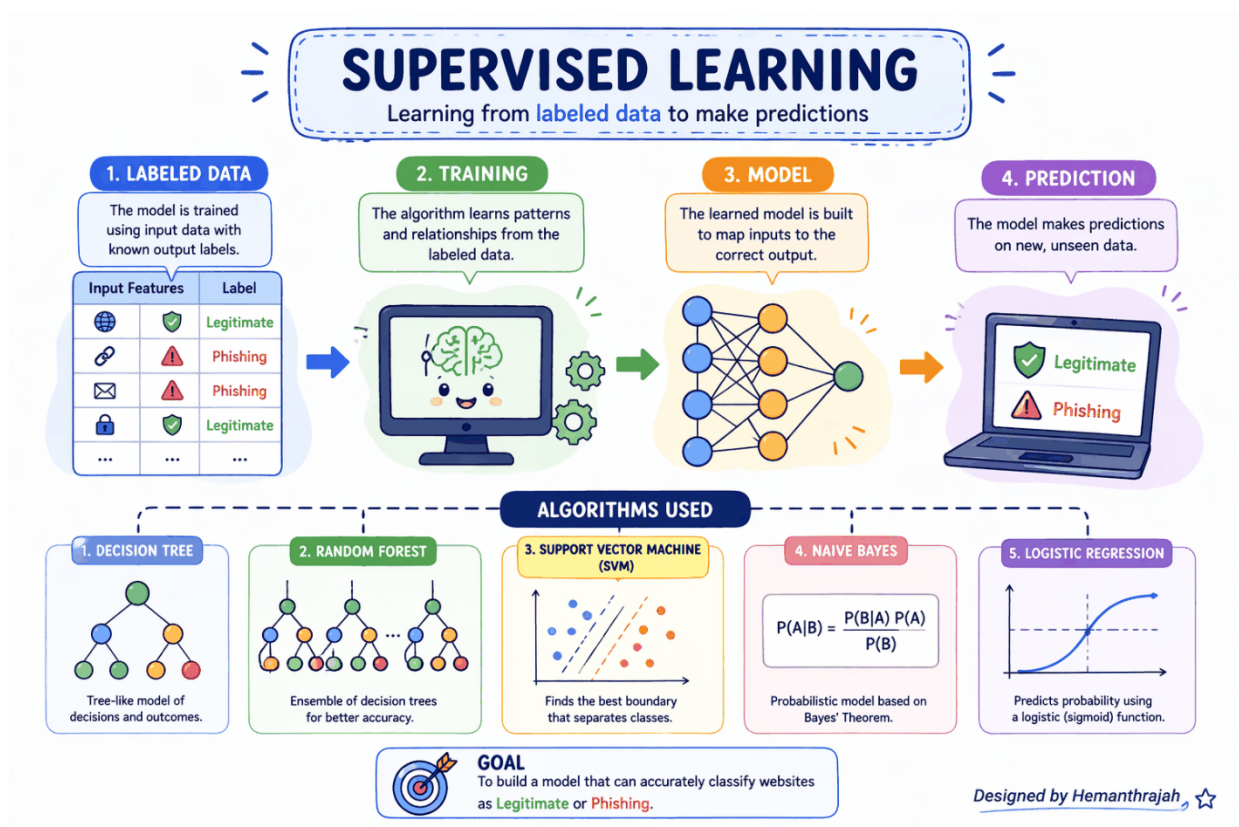


Figure 3: Graphical Representation of Supervised Learning and Classification Algorithms

Supervised learning is still appealing to phishing detection since it occupies a viable space between flexibility, interpretability, and resource utilization. The model in a supervised environment acquires the knowledge of labelled phishing and legitimate samples and uses the association between features and labels to categorize new examples. This simple paradigm is highly applicable to phishing since several existing public datasets already include binary labels and the overall system goal in this project is also binary: is a webpage or email-linked artefact to be considered as legitimate or suspicious.

Logistic Regression is an example of a classical model that can be used as a baseline since it is easy, quick, and somewhat interpretable. It tends to work well in situations where the feature space is highly linearly related to the class label, and it can be used to give a clear point of reference to more complex models. Random Forest is useful due to its ability to model non-linear interactions between features and its resistance to moderately noisy features. Support Vector Machine is

appealing in the sense that it can work well in classification, with well-crafted feature spaces, and can be efficient when the problem is well-constrained. These features render the three appropriate to the current project.

Current studies still confirm these models in phishing situations. In a study by Fares et al. (2024) on the use of both random forest, SVM, and XGBoost to detect email phishing, all three algorithms were useful and SVM had the greatest accuracy in their context. Omari (2023) proved that the use of gradient boosting techniques can also be quite efficient when it comes to phishing websites classification. Even though the results of transformer and deep-learning models can be very strong in certain circumstances, Meléndez et al. (2024) demonstrate that the decision between the traditional and transformer-based approaches should be made based on the context, data characteristics, and constraints of the deployment and not just on the trend.

In this project, a fully deep-learning-heavy design is undesirable, and instead, supervised learning is more appropriate due to the following reasons. To begin with, the extension should be lightweight and responsive. Second, the goal of the investigation phase is not to come up with a maximum scale benchmark outcome, but to create an acceptable and executable anti-phishing concept. Third, classical supervised-learning models are more readily compared systematically with precision, recall, F1-score, and confusion analysis. This makes them more applicable to a final-year project setting where transparency, viability and critical analysis are of value.

2.1.7 Browser Extension as a Delivery Model.

An example of a viable delivery mechanism of anti-phishing is a browser extension since the browser is usually where a trust decision is made by the user. The user clicks a link and arrives on a page, or reads a message in webmail, or a login prompt. When protection is provided retrospectively most of the value is lost. An extension in the browser removes that distance by enabling the detection logic or risk-evaluation workflow to execute within the identical space of interaction where the suspicious content is shown.

New studies confirm this trend. Wu, Kuo, and Yang (2023) have introduced a machine-learning-based browser extension (to detect phishing websites) and demonstrated that even real-

time browser-integrated warning can be of practical benefit in the context of anti-phishing. Linh et al. (2024) applied the existing concept of browsers to a deep-learning framework implemented as extensions, showing that the desire to deploy at the client-side remains. These systems can be applied to the current project since they demonstrate that phishing detection is no longer restricted to either offline classifiers or server-side filters, it can also be provided as a user-centered tool that can be used during browsing.

But browser extensions also bring about limitations in design. They should be light, secure, permission-conscious and understandable to the user. Any extension that introduces perceptible latency, demands excessive access, or generates noisy alerts can be technical but practically ineffective. Thus, the extension model does not eliminate the complexity in design, it merely shifts the focus. The project should not only concentrate on the performance in classification but should also concentrate on the interface simplicity, warning timing, risk explanation, and trust in the extension itself.

2.1.8 Extension-Based Detection: Security, Privacy, and User Trust.

A browser extension that analyses web pages or email-related content must be designed carefully around security and privacy because extensions can access browsing context, page content, or interaction events. PhishGuard therefore treats minimum necessary access as a design requirement. The final Manifest V3 extension limits its permissions and host access to the functions required for scanning, storage, downloads, tab coordination, Gmail protection, and communication with the local or production backend.

Privacy wise, the extension must not gather too much personal information and must keep browsing data that has no connection to phishing detection. In case of possibilities, feature extraction can be restricted to the amount required to do classification and generate warnings. There should be no retention of sensitive data unless it is necessary to conduct tests and the system must be transparent on what it is analyzing. This is especially critical in any functionality involving email, since users might consider inbox-linked data more sensitive than regular webpage information.

Explanation is also important to user trust. When the extension merely states that a page is dangerous with no explanation, a portion of the users might not pay attention to the warning, particularly in cases where the page itself seems valid. The responses to the surveys used in this project show that the vast majority of users like to receive direct warnings in the form of pop-ups, and a considerable number of users also like to understand why the content is suspicious. This implies that the extension must record layered communication: a brief initial warning on speed, and optionally, on confidence and learning.

Another practical consideration is security throughout the extension lifecycle. The system must be maintainable so that browser code, contextual checks, or model artifacts can be updated without making the runtime path unnecessarily complex. PhishGuard follows this principle through a modular Chrome extension, hosted Flask API, separately versioned model artifacts, regression tests, and bounded contextual logic. A smaller and auditable protection path is preferable to an unnecessarily feature-heavy design that is difficult to justify or maintain.

2.1.9 Summary of Domain Research

The sources examined in this section demonstrate that phishing is a large-scale and constantly developing cyber threat, particularly via websites and email-operated exchanges. It also demonstrates that supervised learning remains a viable and plausible route to the classification of phishing, as long as the model is backed by the right features and the latest data. Lastly, the literature shows that browser-integrated protection is a good direction due to its proximity of detection to the actual decision point of the user. The combination of these findings proves the main assumption of the given project: a browser extension built on supervised learning has a chance to be both academically and practically justified.

2.2 Similar Systems / Works

This section reviews recent systems and research that are closely related to the implemented PhishGuard browser extension. The comparison emphasizes credible and recent work rather than relying only on older landmark systems. The objective is to identify the strengths and weaknesses of current phishing-detection approaches and to position PhishGuard in relation to browser-integrated warning systems, real-time website detection, email-phishing classification, and hybrid protection frameworks.

Because PhishGuard integrates website and email phishing protection within a browser extension, the comparison is broader than a single class of previous work. It considers browser-extension systems, real-time web-detection systems, email classifiers, and hybrid frameworks. This comparison shows that the completed project is not only a classifier and not only an interface; it combines supervised-learning inference, contextual analysis, user-facing intervention, account security, reporting, and credential protection in one prototype.

2.2.1 Typical features of Comparable Systems / Works.

One of the best recent examples of phishing detection offered in the form of a browser extension is provided by Wu et al. (2023). Their work is highly applicable as it shows that machine-learning-based scoring of risks can be directly incorporated into the operation of a browser and be applied to give a user real-time feedback. The main characteristic of this system is its practical client-facing nature. Rather than making phishing detection a phishing offline classification activity, it puts the model as a part of the browsing process.

The study by Linh et al. (2024) is also dedicated to the extension-based phishing detection, though it is more deep-learning-oriented. A different route of design is emphasized in their work: they do not use lightweight classical models as the primary source of power, instead deep-learning-based detection is investigated in the context of extensions. This makes it handy as a benchmark of comparison in terms of the trade-off between sophistication and runtime overhead.

Fares et al. (2024) concentrate on phishing email detection and compare the models of supervised learning to classify email content. Even though their work is not packaged as a browser extension, it holds significance as it focuses on the email aspect of the phishing issue with approaches that are not too complicated to be implemented by undergraduates. The property that renders the work relevant is that it focuses on feature selection and classical supervised learning and not on opaque large-scale architectures.

Melendez et al. (2024) contrast the use of traditional machine-learning models with transformer models to detect phishing emails. Their contribution is valuable as it demonstrates that the classical approaches are not necessarily nullified by the newer language models. Rather, a model is effective based on context, data, and deployment objectives. This analogy is very applicable to the current project in the sense that it supports the notion that feasibility, interpretability and user-facing deployment constraints should be used in determining the model choice.

Asiri et al. (2024) suggest a PhishingRTDS, a deep-learning based real-time phishing detection system. This system is applicable as it is a more sophisticated and holistic detection framework with the objective of providing protection in real time. It assists in putting the ongoing project in comparison to a more complicated state-of-the-art orientation. Equally, Rashid et al. (2024) and Shafin (2025) are valuable, as they discuss two concerns that are of great importance in real-world implementation: generalization across datasets and explainable feature selection. These are not the same extension products as Wu et al. (2023) but are what a sound browser-side solution must and must not rely on.

2.2.2 Strengths and Weaknesses of Similar Systems

Immediacy is the best strength of new browser-extension systems. When detection is implemented in the browser, the user can be cautioned during the presence of risk, as opposed to once a suspicious interaction has taken place. Wu et al. (2023) demonstrate this strength by demonstrating the possibility of a machine-learning-based extension to present direct phishing warnings as a person browses. The shortcoming, however, is that extension-based systems have to

be lightweight and permissioned. The richness of a model does not necessarily make it more effective when it creates delay, complexity, or privacy issues.

The systems developed based on deep learning, like those mentioned by Linh et al. (2024) and Asiri et al. (2024), tend to be more powerful in modelling and detecting in real-time. Their great advantage lies in the fact that they can learn more intricate patterns than simple rules which have to be engineered manually. The weakness of their practicality in the context of this project is that heavier models may be more difficult to justify, explain, and integrate into a lightweight extension prototype. They might also need additional calculation and additional fine-tuning than would be optimal in a final-year project based on feasibility and usability.

Another strength of email-centred classifiers such as Fares et al. (2024) and Meléndez et al. (2024) is that they demonstrate effective supervised-learning detection from textual content. These studies support the email-analysis component of PhishGuard, but a classifier alone does not solve the delivery problem. The completed system addresses that gap by placing the result inside the user's browser through Gmail banners, popup results, manual email scanning, and supporting contextual explanations.

Studies that focus on generalisation and explainability contribute a different strength. Rashid et al. (2024) emphasise robustness across datasets, while Shafin (2025) highlights transparent and effective feature selection. These contributions expose weaknesses that high benchmark scores can hide, including poor transferability and unnecessary feature complexity. PhishGuard applies these lessons by using supervised-learning components that can be evaluated independently, explained through explicit features or text representations, and protected with regression tests as contextual behavior evolves.

System / Work	Recent paper / citation / link	System type	Main strengths	Main limitations	Relevance to proposed system
---------------	--------------------------------	-------------	----------------	------------------	------------------------------

Browser-extension phishing detection	Wu, Kuo, & Yang (2023) Link: https://doi.org/10.1109/AsiaJCIS60284.2023.00023	Machine-learning browser extension	Real-time browser-side warning, practical user delivery	Focused mainly on phishing websites, extension constraints remain	Very high
Deep-learning extension-based real-time detection	Linh et al. (2024) Link: https://doi.org/10.11591/ijece.v14i3.p3021-3035	Browser extension with deep-learning pipeline	Immediate client-side protection with stronger modelling capacity	Heavier architecture and potentially higher runtime cost	High
PhishingR TDS	Asiri et al. (2024) Link: https://doi.org/10.1016/j.cose.2024.103843	Real-time phishing detection system	Strong modern architecture and emphasis on real-time detection	More complex than a lightweight undergraduate prototype	High
Email phishing detection with classical ML	Fares et al. (2024) Link: https://doi.org/10.1016/j.procs.2024.11179	Supervised-learning email classifier	Good performance with SVM, RF, and XGBoost; feasible baseline approach	Does not directly address browser delivery or interface design	High
Traditional vs transformer email detection	Meléndez et al. (2024) Link: https://doi.org/10.3390/electronics13244877	Comparative phishing-email study	Clarifies trade-offs between classical ML and transformer models	Not itself a deployable user-facing system	Medium-High

Cross-dataset URL generalization	Rashid et al. (2024) Link: https://doi.org/10.1016/j.comnet.2024.110398	URL-based phishing detection generalization study	Highlights robustness and transferability issues	Not packaged as a client-facing product	Medium
Explainable feature selection for web phishing	Shafin (2025) Link: https://doi.org/10.1016/j.dsm.2024.08.004	Explainable phishing-feature engineering framework	Improves interpretability and feature efficiency	Focus is feature selection rather than full delivery workflow	Medium
PhishGuard (implemented system)	Current project / Link: N/A	Hosted supervised-learning Chrome extension for website and Gmail phishing protection	Combines frozen models, contextual risk analysis, warnings, MFA, dashboard/reporting, personal trust, and encrypted vault	Prototype scope; Gmail-specific automatic email extraction; website model remains URL-oriented; final UAT evidence is still required	Current system

Table 2: Comparison of Similar Systems and Implemented PhishGuard System

2.2.3 Conclusion

The comparison shows that PhishGuard sits between two common strands of recent work. Browser-integrated systems emphasize immediate warning and client-side protection, while classification studies concentrate on algorithms and feature sets for phishing websites or emails. The implemented system combines these strands: classical supervised-learning models provide measurable detection components, while the Chrome extension, contextual risk layer, MFA-protected account functions, trust controls, dashboard, reports, and encrypted password vault convert those results into a practical security workflow.

This is a suitable position for a final-year project. It is not too ambitious to be meaningful, as it deals with websites and email-linked phishing indicators, but is limited enough to be implementable within resource and time constraints. The recent literature thus augurs in favor of the project direction: the lightweight, supervised-learning, browser-based anti-phishing solution is no trade-off in relevance, but a logical and justifiable design choice.

2.3 Technical Research

This section connects the literature review to the technical environment that was ultimately used to implement PhishGuard. The completed system combines an offline supervised-learning workflow with a Chrome Manifest V3 client, a Flask API deployed on Render, and Neon PostgreSQL for persistent account-security and encrypted-vault records. The technical research therefore considers not only model development, but also browser permissions, hosted inference, authenticated services, privacy-sensitive storage, deployment constraints, and user-facing response time.

The final architecture separates responsibilities deliberately. Python and scikit-learn are used for dataset preparation, model training, evaluation, and backend inference; HTML, CSS, and JavaScript implement the browser extension; Flask exposes authenticated scan, account, reporting, and vault APIs; Render hosts the production service; Neon PostgreSQL stores persistent account-related records; and Chrome storage retains user-scoped scan history, trust decisions, and settings that do not need centralized persistence. This separation keeps the machine-learning workflow maintainable while allowing the extension to remain responsive and permission-conscious.

2.3.1 Functional and Non-Functional Requirements

The implemented functional requirements extend beyond a standalone classifier. PhishGuard must analyse website URLs and visible Gmail messages, return a risk score and explanation, warn when the configured intervention threshold is reached, support manual website and email scans, maintain user-scoped recent history, allow carefully bounded personal trust

decisions, protect authenticated features with mandatory MFA, generate individual and date-range PDF reports, and provide an encrypted password vault whose plaintext remains in the browser. These functions correspond to the final scope documented in Chapter 4.

The main non-functional requirements are responsiveness, clarity, privacy, least-privilege access, secure authentication, maintainability, and predictable false-positive behavior. The extension should deliver a decision quickly enough for normal browsing, explanations should remain understandable to non-specialists, and personal browsing history should not be centralized unnecessarily. Security-sensitive backend data must be authenticated and ownership-checked, model changes must be versioned and regression-tested, and personal trust must never silently rewrite the frozen model output. These requirements directly shaped the final separation between browser-side history/trust, hosted model services, and encrypted persistent account data.

2.3.2 Software Environment



Figure 4: Python Logo

Python is the main language used for the machine-learning and backend workflow. Pandas and NumPy support dataset cleaning and feature preparation, while scikit-learn provides the Random Forest website classifier, calibrated LinearSVC email classifier, preprocessing utilities, probability calibration, and evaluation metrics. Flask exposes the model and authenticated application services as a production API, allowing the browser extension to call the same versioned inference logic used during testing.



Figure 5: Jupyter Logo

Jupyter Notebook was useful during exploratory analysis and model experimentation, while Visual Studio Code became the primary development environment for Python, Flask, HTML, CSS, JavaScript, tests, and documentation. The production backend is deployed on Render and connects to Neon PostgreSQL through SQLAlchemy and psycopg. Localhost and SQLite remain development fallbacks, but the report and screenshots describe the hosted demonstration path.

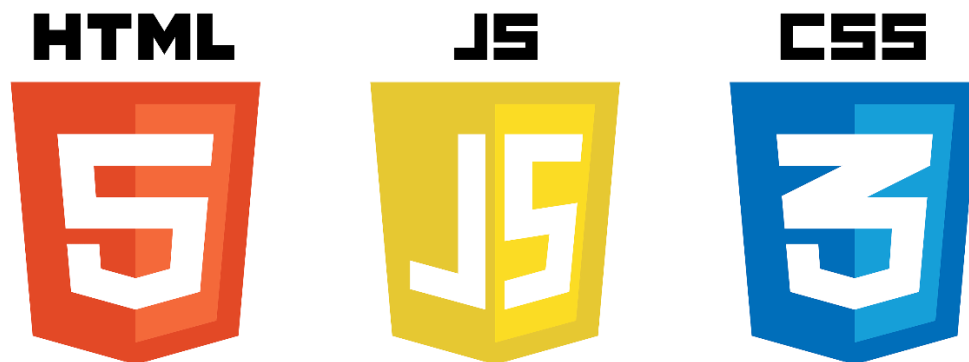


Figure 6: HTML, CSS & JavaScript Logo

The browser component is implemented with standard web technologies under Chrome Manifest V3. HTML structures the popup, dashboard, settings, and vault interfaces; CSS provides the visual system; and JavaScript handles extension events, active-tab scanning, Gmail content-script communication, authentication state, trust management, report downloads, and client-side vault cryptography. The final manifest requests only the capabilities required by these workflows and limits host access to the local development endpoint and the production Render API.

2.3.3 Hardware Environment

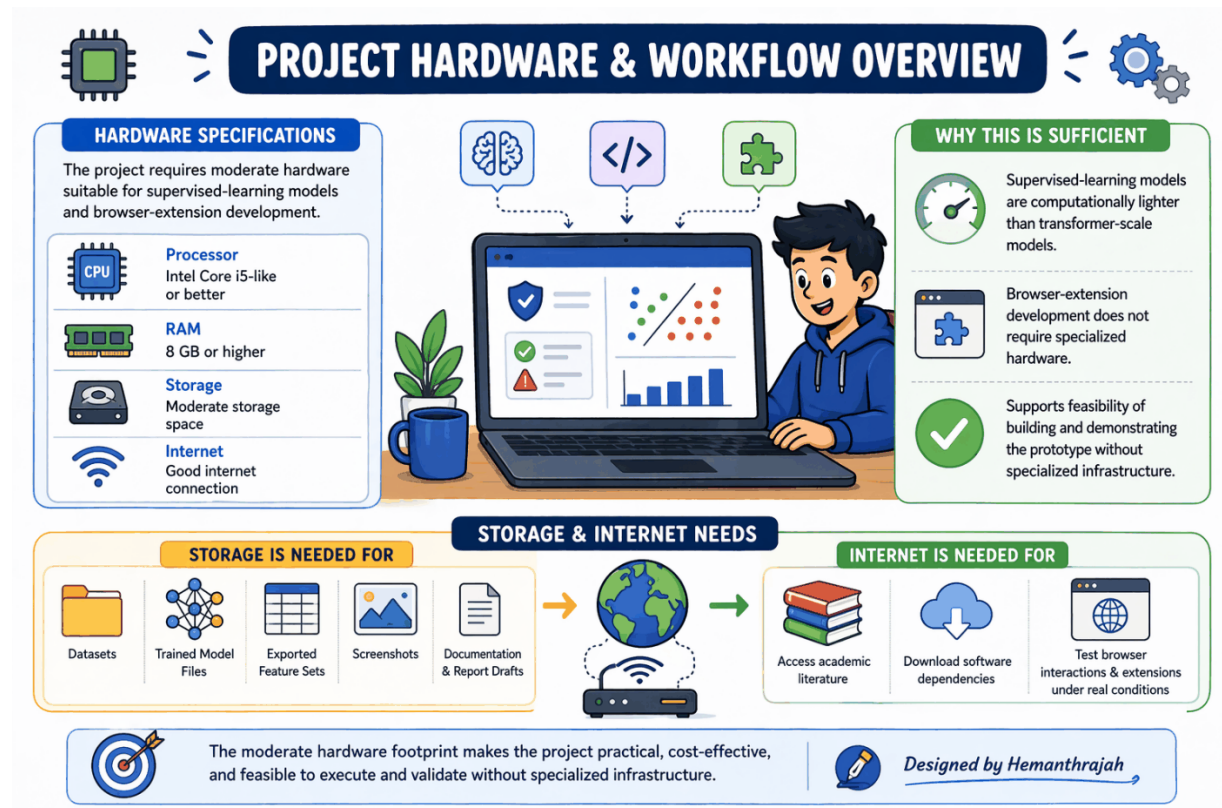


Figure 7: Hardware Specifications Required for Browser Extension and Supervised Learning Development

The completed project does not require specialist hardware. Development and evaluation can be performed on a typical modern laptop or desktop with an Intel Core i5-class processor or equivalent, at least 8 GB of RAM, adequate local storage, and a stable internet connection. The selected classical supervised-learning models do not require a dedicated GPU, and the deployed demonstration offloads authenticated inference and persistence to the hosted backend rather than requiring the end user to run a local Python service.

Local storage is required for datasets, trained model artifacts, test fixtures, screenshots, code evidence, and report material. Internet access is required for dependency installation, deployment, communication with the Render API and Neon database, and realistic testing of browser and Gmail interactions. This moderate hardware footprint supports the feasibility of the prototype and makes the system demonstrable without specialized laboratory infrastructure.

2.3.4 Technical Architecture Considerations

The implemented system uses a hosted hybrid architecture. The Chrome extension is the client-facing protection layer and performs browser-specific tasks such as active-tab access, Gmail content extraction, local history, personal trust, settings, and vault cryptography. Scan and authenticated application requests are sent over HTTPS to the Flask API hosted on Render, where the frozen website and email model artifacts are loaded for inference. This design keeps model management centralized without moving every user-owned state item into the backend.

Data placement follows the minimum persistence required by each feature. Neon PostgreSQL stores user accounts, encrypted MFA material, recovery-code hashes, session/challenge metadata, security events, vault profiles, and encrypted vault entries. By contrast, normal scan history, trusted websites, trusted senders/domains, and user settings remain user-scoped Chrome storage. PDF reports are generated from authenticated, ownership-checked signed scan receipts rather than from a centralized browsing-history table.

The architecture also separates base machine-learning output from contextual and personalized interpretation. The website Random Forest and calibrated email LinearSVC remain frozen model components whose raw results can be evaluated independently. Contextual evidence can raise or constrain final risk, while personal trust can reduce user-specific risk only within bounded rules and cannot retrain the model. This separation improves auditability, regression testing, and maintainability as phishing examples and interface requirements evolve.

2.3.5 Design Browser Extension.

The final extension follows a layered interface rather than presenting a large technical dashboard as the first interaction. The popup provides protection status, scan controls, recent results, and entry points to the Security Dashboard and Password Vault. High-risk website navigation uses a dedicated warning page, while Gmail uses an in-page banner. Detailed evidence remains available through Scan Details and reporting instead of overwhelming the initial warning.

Risk communication is intentionally probabilistic. PhishGuard distinguishes the raw model probability from the final contextual risk and uses labels such as Low Risk, Suspicious, and High Risk rather than claiming certainty. This is important because supervised-learning decisions can still produce false positives and false negatives. The interface therefore combines the score with reasons, trust status, and explicit user actions such as go back, continue once, or review the evidence.

Permissions and data access are kept proportional to the implemented features. Manifest permissions support active-tab scanning, storage, downloads, tab coordination, and required extension actions, while Gmail content scripts are restricted to the Gmail workflow. The design also separates sensitive operations: normal browsing history is local, account and security endpoints require authenticated sessions, and vault plaintext encryption/decryption occurs in the browser before ciphertext is synchronized.

2.3.6 Evaluation Meters and Technical feasibility.

Evaluation was designed around security-relevant measures rather than accuracy alone. Website and email classifiers are assessed using precision, recall, F1-score, confusion matrices, balanced accuracy, false-positive and false-negative rates, and, for the calibrated email model, probability-quality measures. The final release evidence also distinguishes model metrics from contextual-regression tests so that improvements to practical risk handling do not get misreported as changes to the frozen classifier.

Technical feasibility also includes runtime and integration behavior. Feature extraction, API response time, Gmail scan race protection, authentication flows, dashboard operations, report generation, and vault save/fill paths must remain responsive enough for normal use. For this reason, Chapter 5 combines model results with automated unit/regression evidence and realistic browser-oriented validation rather than treating offline classifier performance as the only success criterion.

2.3.7 Technical Research Summary.

The technical research established a practical stack that was subsequently implemented in PhishGuard. Python and scikit-learn provide the model workflow, Flask supplies the authenticated API, Chrome Manifest V3 provides browser integration, Render provides hosted execution, and Neon PostgreSQL provides persistent storage for account-sensitive data. The final architecture remains feasible on ordinary hardware, separates privacy-sensitive state appropriately, and supports transparent model evaluation and iterative refinement. Chapter 4 documents the resulting implementation in detail.

2.3.8 Operational considerations and Security Controls.

PhishGuard is itself security-sensitive because it interacts with web pages, Gmail, authentication state, reports, and credentials. The final extension therefore follows least-privilege design and restricts browser/host access to what the implemented workflows require. The production API also applies origin restrictions and authenticated ownership checks rather than exposing account-sensitive operations as anonymous endpoints.

Privacy is handled by separating local user-owned data from persistent backend data. Routine scan history, personal trust, and settings stay in Chrome storage, while the backend stores only the persistent records required for accounts, MFA, sessions, security events, and encrypted vault synchronization. The vault master password and plaintext credentials are never sent to the backend; encryption and decryption occur client-side.

Authentication controls include Argon2id password and recovery-code hashing, encrypted MFA secrets, mandatory TOTP enrollment, opaque revocable sessions, source-level throttling of failed password attempts, progressive MFA lockouts, recovery codes, and user-visible security events. These controls are documented and tested separately from the phishing models because compromising the protection tool would undermine the purpose of the system itself.

Operational quality also depends on controlling false positives and regressions. PhishGuard therefore keeps raw model results separate from contextual/personalized risk, caps

trust reductions when severe evidence is present, performs suffix-safe domain matching, and maintains dedicated website/email regression tests. This makes changes to warning behavior reviewable and reduces the chance that a convenience feature silently weakens high-risk protection.

2.3.9 Implementation Priority and Final Outcome

The first implementation priority was the core website protection loop: inspect the active destination, obtain a model/contextual result, and intervene when the configured high-risk threshold is reached. This became the current-site scan, automatic website protection, warning-page workflow, scan details, and bounded website-trust handling documented in Chapter 4.

The second priority was email protection because the survey showed frequent exposure to suspicious messages. This became a calibrated text classifier combined with contextual email evidence, Gmail content-script extraction, visible risk banners, local summaries, and exact-sender or confirmed-domain trust rules. Severe evidence prevents personal trust from suppressing a dangerous message.

The third priority was secure account and evidence support. Mandatory MFA, recovery codes, session validation, progressive lockouts, security events, user-scoped scan history, the Security Dashboard, and authenticated PDF reporting were added so that the system could operate as a realistic security application rather than as a demonstration classifier only.

The final implementation priority was credential protection and release hardening. The client-side encrypted password vault, password generator, safe save/fill separation, Render/Neon deployment, automated regression tests, code evidence, screenshots, and documentation were completed before the final testing and submission phase. This sequence reflects the Agile principle of establishing the protective core first and adding secondary capabilities only after the main warning workflows were stable.

CHAPTER 3: METHODOLOGY

3.1 System Development Methodology

This chapter explains the systematic approach used across both the investigation and implementation stages of PhishGuard. The project combines research, survey-based requirement gathering, dataset preparation, supervised-learning evaluation, browser-extension development, hosted backend integration, security controls, and iterative refinement. Agile was selected because these activities required repeated review as model behavior, browser constraints, false positives, and usability issues became clearer during development.

The methodology serves two purposes in the completed project. First, it explains how literature and survey findings were converted into requirements and then into implemented PhishGuard features. Second, it provides a traceable reason for the order in which website protection, email protection, authentication, dashboard/reporting, vault functionality, testing, and documentation were introduced. This traceability is important because the final system evolved beyond the narrower Investigation Report concept while still preserving the original phishing-detection aim.

The section therefore retains the original comparison of Agile, RAD, and Kanban, but evaluates them against what the project actually required: iterative model tuning, staged security-sensitive implementation, user-interface refinement, deployment changes, regression testing, and supervisor/user feedback. The selected Agile process is then described using the iterations that were applied during Part 2 rather than only as a future recommendation.

3.1.1 Importance of System Development Methodology

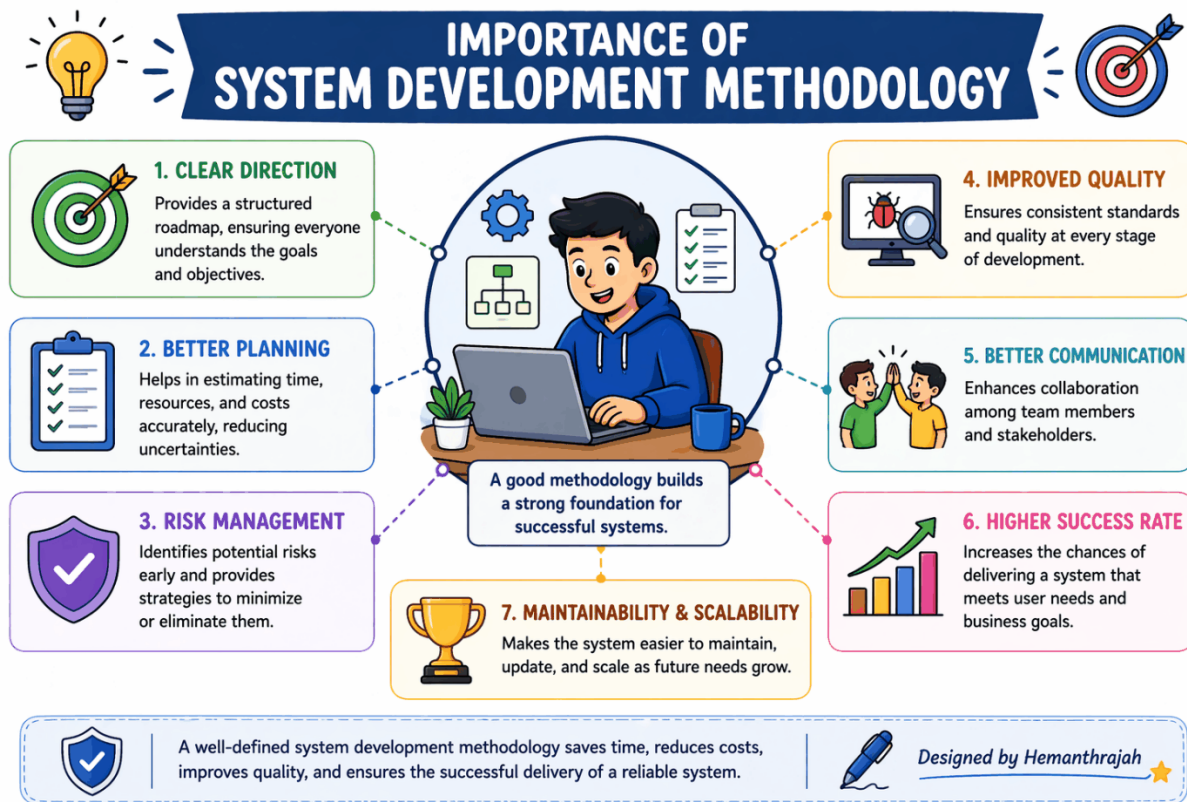


Figure 8: Key Reasons for the Importance of System Development Methodology

A defined methodology gave structure to a project that contained several parallel workstreams. Literature review, survey analysis, dataset cleaning, model development, extension design, authentication, backend deployment, testing, and report writing could not be completed effectively as unrelated tasks. Agile provided staged deliverables and review points so that a working protective loop could be established first and then expanded without losing traceability to the original requirements.

Risk management is also supported by methodology. There are several types of risk in this project: dataset risk (when publicly available phishing datasets might be out of date or skewed); model risk (when a seemingly high-quality classifier does not generalize); usability risk (when warnings are obtrusive or too weak); and schedule risk (when time might be wasted unless experimental usage is constrained). The development approach must be appropriate in order to

expose these risks at an early stage as opposed to letting them occur until the implementation process is in its latter stages.

Another reason why methodology is important is quality assurance. Quality in a project such as this is multi-dimensional. It comprises technical quality, e.g. the relevance of features and the classification functionality; interface quality, e.g. comprehensibility of warnings and responsiveness; and documentation quality, e.g. traceable decisions and requirements that are well-defined. A methodology which promotes frequent inspection, small steps and correction through iteration is more likely to encourage these various types of quality than a methodology which does not provide feedback until the end.

Lastly, methodology aids in communication with stakeholders. Despite being an academic project, significant stakeholders, such as the supervisor, the project manager, and the users, who will be represented by survey responses, will be involved. The approach taken must enable the feedback with the said stakeholders to be easily integrated without leading to scope and direction chaos. Due to this fact, the methodology adopted by the project should be flexible and at the same time disciplined.

3.1.2 Methodology Options Considered

Agile, Rapid Application Development (RAD) and Kanban are the three methodologies that were taken into consideration in this project. They were chosen on the basis of their being iterative to a certain extent and as such, each of them commonly relates to projects where refinement and user input can be improved. Waterfall was thought of conceptually as a conventional baseline, but since the current project involves changing feature selection, experimentation of the model, and interface polishing, the primary comparison is between approaches that are more adaptive in real practice.

Agile is commonly linked to the short iterations, the frequent re-evaluation, and the constant stakeholder engagement. It is especially applicable to the projects that have requirements that can only be clarified by development, not by being fully stable. Agile would be appealing in

the context of this project since the performance of phishing detection, warning presentation, and extension behavior will tend to be enhanced by repeating the process of testing and revising.

RAD focuses on the speed of prototyping and speedy user-driven change cycles. Visual prototyping and immediate feedback are key to the project, and this makes it often useful. In the case of a browser extension, RAD seems appealing, as the interface concepts can be prototyped in a short time. Nevertheless, there is an element in the current project, machine-learning, which introduces an element of technical experimentation which does not necessarily lend itself well to speed. An approach which excessively emphasizes the importance of fast prototyping may generate pressure to make commitments to interface decisions before the classification pipeline has been fully justified.

Kanban on the other hand is particularly robust in visibility of tasks and management of workflow. It assists teams to visualize work, manage work in progress, and prevent bottlenecks. This may come in handy handling the small tasks in a project that is in the final year. But Kanban alone lacks the boundary of iteration and formal review moments as Agile has. In a project that requires a frequent check of the model outputs and warning messages to users, more powerful iteration cycles are useful.

3.1.3 Comparative Analysis of Agile, RAD, and Kanban

The primary advantage of Agile is that it does not consider change to be disruptive but expected. This is precisely the type of project which is helped by such an attitude. The developer can start with a set of planned URLS features and then find out in the course of testing that some of them are weak, redundant, or impractical in the extension workflow. Likewise, a style of alert that appears reasonable in theory might turn out to be obnoxious when a prototype is tried. The structure of iteration in Agile enables the absorption of such findings as productive rather than failures. The project is able to proceed through the planning cycles, building cycles, checking and refining cycles without losing the overall direction.

RAD does have its share of that flexibility, but the focus is altered. Its major advantage is speed. In the event that the primary issue of the project was to create and perfect an interface in

the shortest period of time, RAD could be the most appealing alternative. However, interface iteration is not the only thing needed in the current project. It involves evidence-based comparison of supervised-learning models, meticulous feature refinement, classification performance evaluation, and a rational tradeoff between performance and extension efficiency. These are iterative activities, but do not necessarily happen fast as required by RAD. A methodology that gives too much preference to visible prototypes at an early stage may downplay the quality of data and evaluation.

Kanban can be very useful in work visibility and management of the flow of tasks. It could be handy to use it to track literature-review activities, dataset-cleaning activities, documentation activities, and implementation milestones. However, a phishing-detection extension can be not only the problem of task management. It is an issue of staged learning and confirmation. The developer must have clear review points where the model family selected, the model interaction flow, and the warnings shown to the user are re-evaluated. Kanban can aid with that, but it lacks the iterative rhythm that Agile offers.

Recent comparison studies concerning software-development methods support this argument. Jain, Jotwani, and Agrawal (2024) highlight that Agile models are particularly effective when dealing with dynamic environments that need continual feedback and flexibility. Tetteh (2024) also emphasizes that various agile models are different in style yet have in common the suitability to projects where the requirements become clear as time goes by. Sitorus et al. (2025) compare Agile to more conservative methods and highlight the flexibility and speed of Agile. These are the observations that are highly consistent with the requirements of the current project.

According to the comparison, Agile was selected and applied as the main methodology. RAD remained useful as a reference for rapid interface prototyping, and Kanban-style task visibility was useful for tracking concurrent work, but Agile provided the clearest structure for repeated model, security, integration, and usability review. This made it the most suitable primary methodology for the completed phishing-detection browser extension.

Aspect	Agile	RAD	Kanban
Primary emphasis	Iterative development with structured review cycles	Rapid prototyping and quick user feedback	Workflow visibility and limiting work in progress
Strength for this project	Supports model tuning, interface refinement, and requirement evolution	Useful for fast interface mock-ups	Useful for tracking many concurrent tasks
Main weakness for this project	Requires discipline in iteration planning	May encourage interface speed over careful ML evaluation	Provides weaker iteration structure for validation-heavy work
Fit with phishing detection model development	High	Medium	Medium
Fit with browser extension refinement	High	High	Medium
Overall suitability	Selected methodology	Useful comparison method	Useful supporting management perspective

Table 3: Comparison of Agile, RAD, and Kanban

3.1.4 Justification for Selecting Agile

Agile was appropriate because the system changed materially as implementation progressed. Dataset inspection affected the final feature sets, false-positive behavior required contextual analysis and regression protection, browser constraints influenced where state could be stored, and account security requirements expanded into mandatory MFA, security events, and encrypted vault support. Iterative review allowed these changes to be incorporated without abandoning the original project objective.

The project outputs also suited incremental delivery. The protective core could be developed as separate but connected increments: models and backend inference, website warning, Gmail protection, authentication, dashboard/reporting, vault functionality, deployment, and then release hardening. Each increment produced something testable before the next layer was added, which reduced the risk of discovering integration problems only at the end.

Agile also aligned with the survey evidence because speed, clarity, ease of use, and visible warnings required repeated interface and behavior checks rather than a single design decision. The final warning page, Gmail banner, risk explanations, trust controls, and dashboard views were therefore refined alongside the detection logic instead of being treated as a separate cosmetic stage.

3.1.5 Agile Process and Activities in the Selected Approach

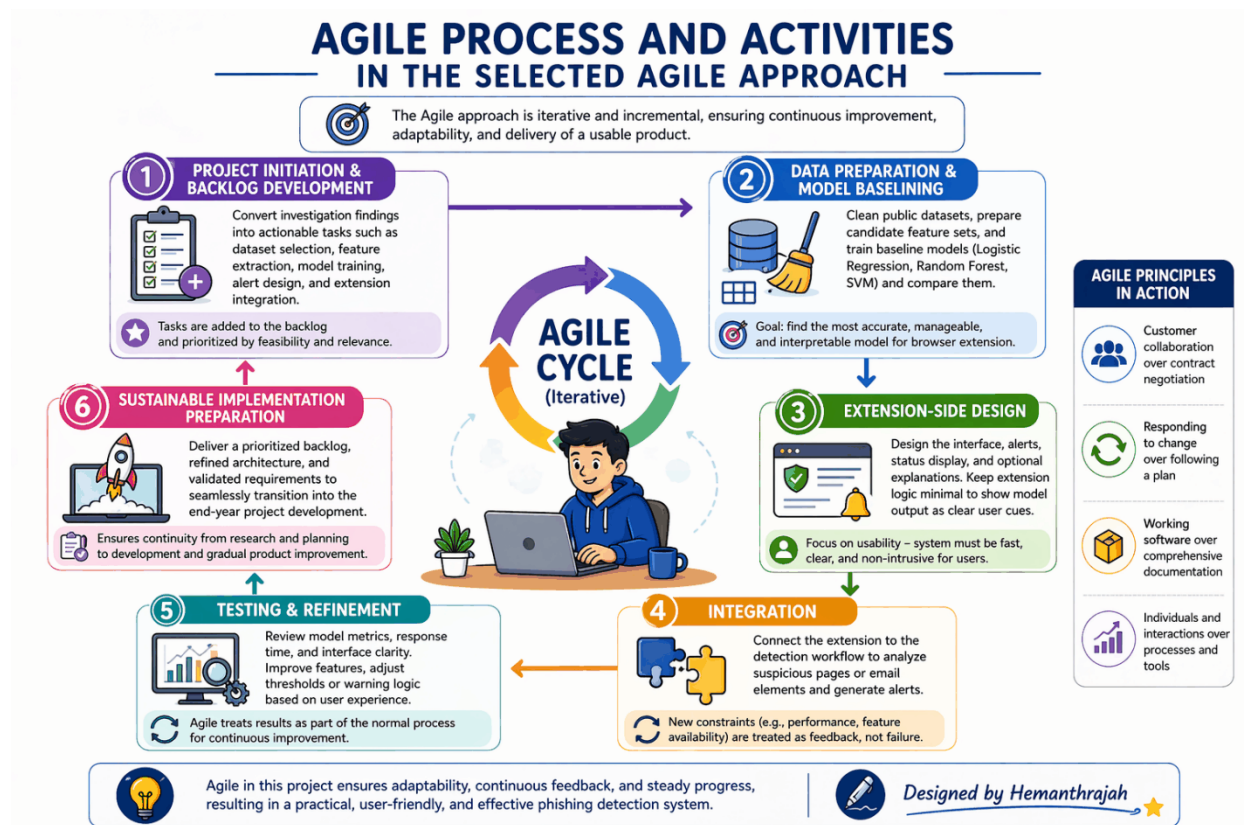


Figure 9: Graphical Representation of the Agile Process and Activities in the Selected Agile Approach

The Agile process used in this project was organized around a prioritized backlog. Part 1 produced the initial requirements, architecture direction, survey findings, and research evidence. At the start of Part 2, those items were converted into implementation tasks covering website detection, email detection, browser warning flows, authentication, reporting, trust, vault protection, deployment, testing, and documentation.

The first implementation cycles concentrated on confirming datasets, feature preparation, model artifacts, and backend inference. The website Random Forest and calibrated email LinearSVC were kept as independently measurable model components, while the backend API established a stable point for the extension to obtain predictions and explanations.

The next cycles focused on browser-side protection. Website scanning and warning behavior were integrated first, followed by Gmail extraction and email risk banners. Low-fidelity interface designs were then refined into the final popup, warning page, dashboard, settings, account-security, and vault interfaces shown in Chapter 4.

Integration cycles added the supporting security services required by the final system: password authentication, mandatory TOTP MFA, recovery codes, sessions, progressive lockouts, security events, personal trust, signed report generation, and the encrypted password vault. Hosted Render and Neon configuration were introduced as the production demonstration path while localhost remained available for development.

Testing and refinement were continuous rather than postponed until the end. Website and email regressions, authentication behavior, reporting, dashboard functions, vault save/fill rules, and deployment configuration were repeatedly checked as features changed. False-positive cases were used to refine contextual analysis while the frozen model outputs were preserved for honest evaluation.

The final Agile cycle concentrates on evidence and completion: Chapter 4 implementation documentation, Chapter 5 testing and UAT, Chapter 6 critical evaluation, APA references, appendices, poster preparation, Turnitin, and final submission. This keeps the final report connected to the implemented artifact instead of describing an earlier proposed architecture.

3.1.6 Agile Iteration Roadmap Applied in Part 2

Iteration 1 - Part 2 initiation and design refinement (11 June to 2 July 2026). The investigation findings were converted into a current backlog, the hosted extension/backend architecture was confirmed, and database, interface, and security responsibilities were refined. The second briefing on 18 June provided an early academic checkpoint during this stage.

Iteration 2 - Detection core and website protection (19 June to 16 July 2026). Website data and model assumptions were confirmed, the Random Forest inference path was integrated with the Flask API, and Chrome website scanning, contextual risk analysis, warning behavior, and regression examples were developed. The third briefing class on 9 July occurred while this work was active.

Iteration 3 - Email protection and account security (3 July to 30 July 2026). The calibrated email classifier, contextual email analysis, Gmail content-script flow, summaries, and trusted-sender behavior were developed in parallel with password authentication, mandatory TOTP MFA, recovery codes, session validation, progressive lockouts, and security-event handling.

Iteration 4 - Supporting security features and deployment (17 July to 20 August 2026). The Security Dashboard, scan details, personal trust management, authenticated PDF reports, encrypted password vault, Render deployment, Neon persistence, and integration/regression checks were completed. The final FYP briefing class on 13 August 2026 served as the final formal briefing milestone during implementation.

Iteration 5 - Documentation, testing, and completion (14 August to 2 September 2026). Chapter 4 was aligned to the final source code and screenshots, Chapter 5 was prepared around automated/system testing and UAT, Chapter 6 was completed with critical evaluation and recommendations, and the remaining period was allocated to appendices, poster, Turnitin, formatting, and final submission on 2 September 2026.

3.2 Data Gathering Design

Information collection was required because PhishGuard is intended for real users rather than as a purely theoretical classifier. The investigation therefore needed evidence about how frequently the target audience uses browsers and email, which phishing indicators they already recognise, how confident they are when assessing suspicious content, and what behavior they expect from a browser extension. The purpose of the data-gathering process was to identify user requirements that could later be traced to the completed interface and protection workflow.

The investigation-report guideline permits several data-gathering procedures, including surveys, interviews, document analysis, and observation. For this project, the primary technique was an anonymous online survey, supported by document analysis of recent academic literature, official reports, and technical sources. The survey provided direct user opinions, while the document analysis strengthened the academic and technical basis used to design and implement PhishGuard.

3.2.1 Target Population and Sampling Method.

This study has a target audience of individuals who regularly use browsers and email services. This covers university students, working adults and ordinary users who are likely to come across phishing attempts within their normal academic, personal or work settings. Students are particularly applicable since they come into contact with the login pages, university portals, cloud solutions, and email-based messages regularly. Adults who work are also applicable since phishing is often targeted at business email, financial dealings, and daily web services.

The survey employed a convenient sampling method; a practical convenience-based sampling and expansive sharing on the internet. The link of the survey was given out via direct contacts and free online. Even though it is not a probability sample, it can be used in an undergraduate investigation report in which the aim is to collect requirements as opposed to making inferences about a large population. The 44 responses gathered are more than the minimum number of participants required according to the guideline and offers sufficient range in terms of age, occupation, and daily use of technology to facilitate requirement analysis.

The survey was not aimed at gathering sensitive identifying information. No names or student IDs, no personal email addresses, no passwords were requested. This design was done intentionally since the subject matter itself is related to phishing and trust. It would have been unethical to request unneeded personal information, and this would be contrary to the spirit of good ethics and the targeted privacy-focused nature of the project.

3.2.2 Survey Instrument Design

The questionnaire was structured into parts to enable the results to be viewed progressively. The initial part entails demographic data, including age bracket, gender, and occupation. The second one looks at the frequency of technology use, i.e. email and site use, as they influence phishing exposure. The third examines the awareness and experience, such as whether participants have heard of phishing websites and phishing emails, whether they have come across suspicious content and how confident they are about it. The fourth one is devoted to the expectations of the offered browser extension, such as the preferred types of warnings, desired capabilities, the significance of response speed, and readiness to use the system. The last part is an invitation to feedback and improvement suggestions which are open-ended.

This design was selected as it reflects the reasoning behind requirement identification. The first step of determining how a browser extension should perform is to determine who the probable users are, how vulnerable they are to the issue, what indicators they already depend on, and what form of intervention they would be willing to receive. The combination of closed-ended and open-ended questions is in favor of the quantitative and qualitative interpretation. Through multiple-choice and Likert-style questions, clear tendencies can be measured, whereas the open-ended feedback questions can pick up ideas which are not reflected by the fixed alternatives.

The questionnaire used simple, non-technical wording so that participants without specialist cybersecurity knowledge could respond meaningfully. This was important because PhishGuard targets ordinary internet users as well as technically experienced users. An overly technical instrument could have biased the sample toward participants with above-average security knowledge and reduced the usefulness of the findings for real-world interface design.

3.2.3 Data Collection Procedure

The survey was shared online and remained open on 15 April 2026-17 April 2026. Participants were free to fill out the form during this time, read the ethics notice, give consent, and take their time to fill out the questionnaire. Since it was an online form, respondents were able to

answer at any point of location thereby eliminating participation barriers and enhancing convenience. This strategy was particularly appropriate when it came to dealing with a project that dealt with the use of the web and email since it aligned with the digital lifestyles of the target market.

The importance of confidentiality was highlighted during the introduction of the form. The respondents were told that the continued participation was at their own will and that no personal identity was going to be reported in the investigation report and that the responses would not be utilized in any personal capacity. The shape had a contact email to provide feedback or enquiries. This design facilitated the development of respondent trust and fit the data-collection approach with the privacy sensitive theme of the project itself.

After the response period was over the dataset was exported to spreadsheet to be analyzed. The cleaning of responses was then performed by verifying incomplete responses, duplicated consent responses and empty-value patterns. The last dataset consisted of 44 valid responses. This data is used in the analysis in Section 3.3.

3.2.4 Reason behind the selection of Survey Method.

There are a number of reasons why the survey method is justified. To begin with, it is efficient. It gives the opportunity to gather data on a greater number of people than would otherwise be possible with interviews within the same time frame. Second, it yields structured data which can be summarized easily to be analyzed during requirement analysis. Third, it will be appropriate for a project where the target users will be familiar with working with online forms and browser-based systems. This minimizes grit in engagement and enhances practical suitability.

Another benefit of the survey is that it facilitates breadth and comparability. By having all the respondents respond to the same set of core questions, the developer is able to compare the degrees of awareness, confidence and preference across the sample in a uniform manner. This helps in identifying those system requirements that are widely shared and those that are more optional or secondary. As an illustration, when the majority of the respondents consider speed and ease of use to be very important, they become obvious non-functional priorities.

Observation is also inappropriate as compared to the survey method in the given project. This would ethically be difficult and possibly unrealistic without putting participants in contrived circumstances in order to observe phishing behavior directly. Similarly, interviews would be richly detailed but not as broad. The survey thus presents the most viable balance of practicality, ethics and usefulness to the current investigation stage.

3.2.5 Supporting Document Analysis

Besides the survey, document analysis was employed to enhance the study. It consisted in the analysis of recent journal articles, conference papers, and official reports about the trends in phishing, phishing website detection, phishing email detection, browser extension design, and supervised learning. The purpose of document analysis is different compared to the survey. Whereas the survey will determine what is demanded and expected by the user, document analysis will determine what is supported by the current state of research both technically and conceptually.

This supporting analysis was especially important because phishing detection is a rapidly changing field. Prioritising recent literature and current official sources reduced dependence on outdated assumptions and kept the technical direction aligned with present concerns such as dataset drift, explainable feature selection, generalisation, browser-side delivery, privacy, and user trust. The resulting evidence informed the architecture and implementation decisions documented in Chapters 2 and 4.

3.2.6 Ethical Considerations

Ethics were significant during the design of the survey and the framing of the project in general. Respondents had been assured that their answers would be utilized in an academic way and that they could withdraw their answers at any time by merely failing to fill in the form. There were no false prompts, fake phishing, or gathering of sensitive security credentials. These limitations were required because the project was not supposed to add more risk to the study of a topic of security sensitivity.

The technical orientation of the project is also ethics-conscious. Because PhishGuard can analyse browser and email-related content, the final design follows the minimum access required for each feature and avoids unnecessary retention of personal browsing information. Routine scan history, trust decisions, and settings remain user-scoped in Chrome storage, while the backend stores only the persistent account, security, and encrypted-vault records required by the implemented workflows. These controls translate the investigation-stage privacy requirements into the completed architecture.

3.3 Analysis

The questionnaire produced 44 valid responses and provides the empirical foundation for the requirement analysis that informed PhishGuard. The discussion is intentionally detailed because the purpose is not merely to report percentages, but to explain how the findings affected the completed system. Each question is therefore interpreted in terms of both the observed response pattern and its relevance to browser-integrated phishing protection.

The responses are interpreted in the same broad sequence as the questionnaire: demographics, phishing awareness and experience, followed by expectations for the browser-extension concept presented to respondents. This order first establishes who participated, then shows their exposure to phishing and confidence in recognising it, and finally identifies the features and interaction patterns they expected. These findings are consolidated into the final user requirements and mapped to implemented design decisions later in this chapter.

Demographic item	Most frequent response	Interpretive note
Age group	18-24 (63.6%)	Majority are digitally active young adults and likely frequent browsers and email users.
Gender	Male 56.8%, Female 43.2%	Relatively balanced samples reduce single-group bias.
Occupation	Student (54.5%)	Strong relevance to academic browsing and email contexts.

Email use	Daily (68.2%)	Supports strong need for email-related phishing awareness.
Website use	Daily (81.8%)	Strong justification for a browser-based delivery model.

Table 4: Summary of Survey Demographic Results

Indicator	Key result	Implication
Heard of phishing websites	86.4% yes	Awareness is high but not universal.
Heard of phishing emails	86.4% yes	Supports combined website and email focus.
Received suspicious email	75.0% yes	Email is a major practical attack channel.
Visited suspicious website	72.7% yes	Browser-side warnings are highly relevant.
Clicked suspicious link by mistake	52.3% yes	Awareness does not fully prevent risky behavior.
Average seriousness rating	4.64 / 5	Respondents see phishing as a serious current threat.

Table 5: Phishing Awareness and Exposure Summary

Indicator	Selections	Interpretation
Suspicious URL or website address	38 (86.4%)	Strongly recognized as a phishing cue.
Requests for passwords or banking details	37 (84.1%)	Strongly recognized as a phishing cue.
Unknown sender email address	34 (77.3%)	Strongly recognized as a phishing cue.
Unexpected attachments or links	33 (75.0%)	Strongly recognized as a phishing cue.
Offers that seem too good to be true	30 (68.2%)	Recognized by many users but not universally.
Urgent or threatening language	29 (65.9%)	Recognized by many users but not universally.
Poor grammar or spelling mistakes	26 (59.1%)	Useful cue, but more dependent on user judgement.

Table 6: Preferred Indicators of Phishing

Feature / Preference	Result	Design implication
Extension usefulness	95.5% Yes	Strong justification for the project concept.
Most useful feature	Warning message when a suspicious website is detected (52.3%)	Direct intervention is preferred.
Preferred alert style	Pop-up warning message (47.7%)	Warnings should be visible and immediate.
Importance of speed	Average 4.64 / 5	Inference must be fast and lightweight.
Importance of ease of use	Average 4.52 / 5	Interface simplicity is essential.
Willingness to use extension	77.3% Yes, 22.7% Maybe	Adoption potential is high if trust and usability are maintained.

Table 7: Desired Browser Extension Features

Question 1 – Participant Consent

I confirm that I have read and understood the information provided above. I understand that my participation is voluntary, and I agree to take part in this study.

44 responses

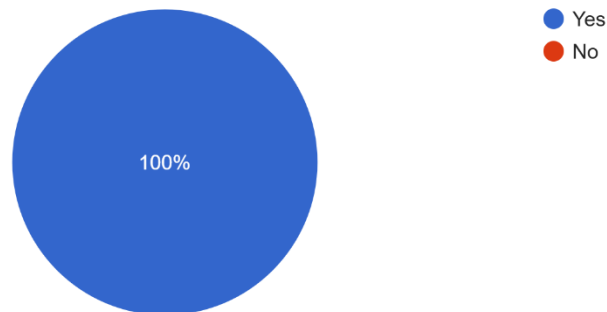


Figure 10: Participant Consent

Valid consent was present in all 44 responses that were given. It is a mere statistical finding, but a methodological point. It implies that all the analyzed entries were the result of a participant who read the introductory information and was clearly willing to participate in the study. There were no missing consent responses that would have to be disqualified, and this reinforces the ethical soundness of the data and eliminates uncertainty as to whether the results can be legitimately applied in the investigation report.

Regarding the research-design approach, the unanimous consent outcome also indicates that the front page of the form was not too complicated and comprehensible. This is important since a confusing consent statement may deter participation or may result in low consistency in the quality of response. The transparency of the consent process is critical in this project since the subject of the survey itself is trust, phishing, and suspicious online behavior. A properly designed consent step thus helps the credibility of the survey process and the project in general.

Question 2 – Age Group

What is your age group?

44 responses

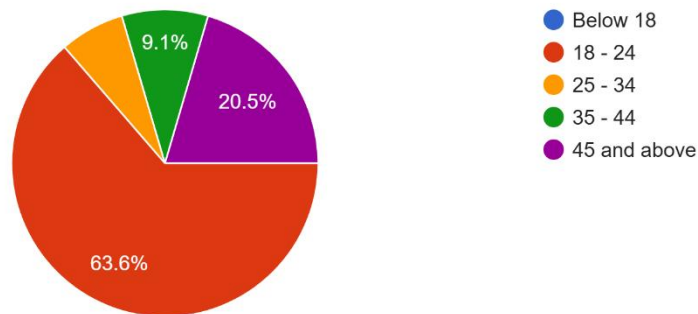


Figure 11: Age Group

The age group distribution reveals that 18 -24 is the most prevalent with 28 representing 63.6% of the sample. Smaller subsets consist of respondents that are between 25 and 34, between 35 and 44 and above. This composition can help the project as it would provide good representation to younger digitally active users but also include older adults whose browsing behavior and level of caution might vary.

The preponderance of younger respondents would indicate that the extension would be effective in high-frequency, high-paced browsing where convenience is important. Meanwhile, the fact that older respondents are there implies that the design cannot presuppose extremely technical instincts. An overly subtle warning interface can be overlooked by users in a hurry, whereas an overly technical warning can be misinterpreted by less security-confident users. The age distribution hence justifies the requirement of a design that is both immediate and comprehensible among mixed user groups.

Question 3 – Gender

What is your gender?

44 responses

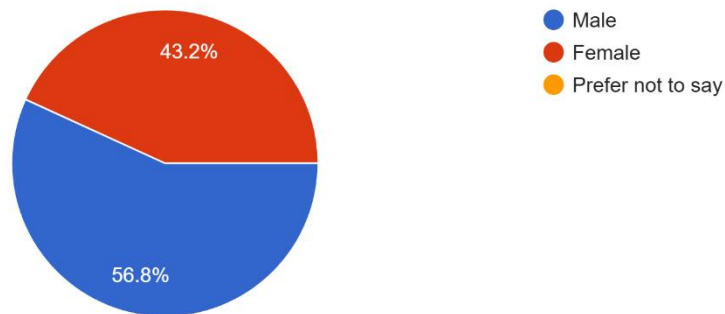


Figure 12: Gender

The gender distribution is reasonably balanced: 25 respondents (56.8%) identified as male and 19 (43.2%) as female. The sample is not perfectly even, but neither group dominates the dataset to the extent that the requirement analysis would represent only one gender perspective. This supports the use of the findings for a general-purpose browser extension such as PhishGuard, while the convenience-sampling limitation is still acknowledged in Section 3.3.6.

This balanced pattern, in terms of requirement-analysis, minimizes the probability of the project inheriting preferences which are highly skewed toward a single respondent segment. Rather, the usefulness, warning style, speed, and ease of use outcomes may be viewed as widely applicable design indicators. The gender finding is not then a design requirement, but it enhances the degree of confidence in the representativeness of the overall preference patterns that will be found later in the survey.

Question 4 – Occupation

How often do you use email services?

44 responses

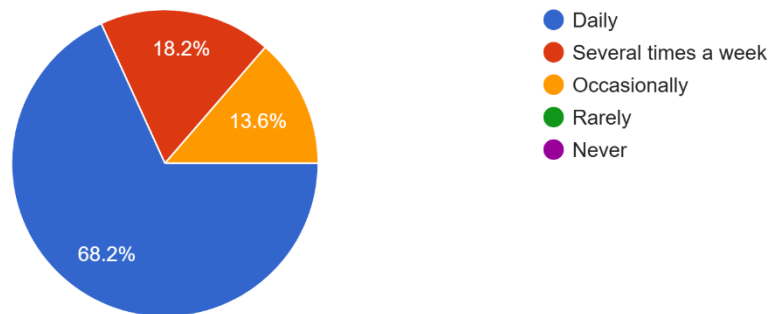


Figure 13: Occupation

Occupation results indicate that students represent the biggest group (24 respondents 54.5%). The number of employed respondents is also a significant population, with self-employed and unemployed respondents having smaller but significant percentages. Such a distribution is feasible since it is appropriate in the desired breadth of the project: it is very relevant to students, but not to them only.

The practical implications of the occupational diversity are practical. Phishing may affect students through business email, finance-related messages, and workflow tools as well as employed users through contact with university portals, cloud services, and links to shared documents. An extension that is too specific to context might not work in the other context. The survey thus recommends a solution that is generic enough to fit in the various real-world environments yet at the same time is particularly applicable to the student users.

Question 5- Email Usage Frequency.

How often do you use email services?

44 responses

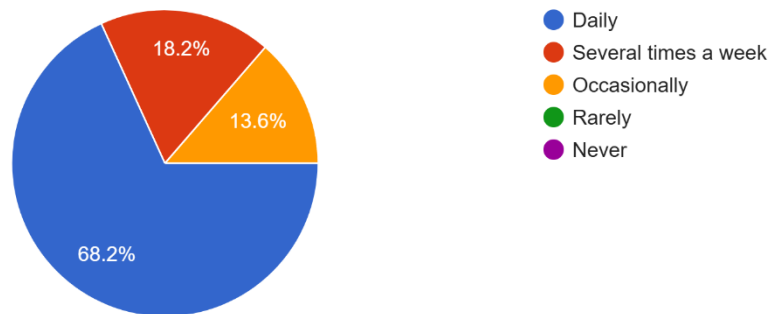


Figure 14: Email Usage Frequency

There is a high frequency of email-use in the sample. The number of respondents who use email daily was 30 (68.2%), and the rest used email several times a week or sometimes. None of the respondents said that he or she never used email. This is a good reason why email-related phishing cues should become a part of the project. In case the target audience uses email regularly, then email can be one of the most plausible locations of the phishing exposure.

The finding also indicates that email protection should not be treated as an incidental feature. Although browser-visible Gmail content and link-related evidence are more practical for the implemented extension than unrestricted inbox analysis, email remains a common starting point for phishing journeys. For this reason, the completed system includes a dedicated email classifier, contextual email analysis, Gmail risk banners, sender/domain trust controls, and manual text scanning in addition to website protection.

Question 6 - Frequency of Web Site Browsing.

How often do you browse websites for personal, academic, or work-related purposes?

44 responses

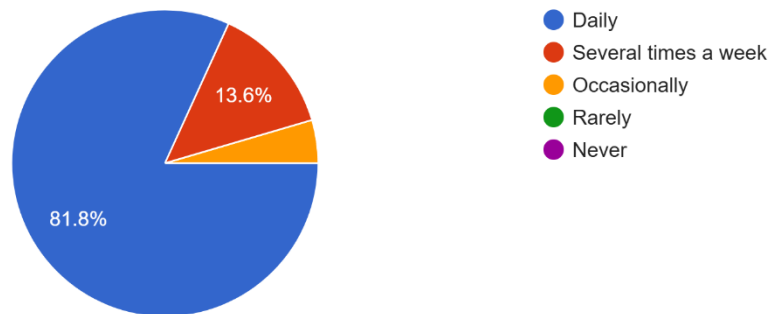


Figure 15: Frequency of Web Site Browsing

Browsing the web is even more common than emailing. Those who have been browsing on a daily basis were 36 respondents (81.8%), and almost all the others browse a few times a week or sometimes. This outcome presents one of the most illustrative reasons as to why the project should be delivered in this model. Since the browser is heavily used by the target population, it is a suitable location to place phishing protection.

The outcome also validates the significance of performance as a prerequisite. Slow, unstable, and disruptive extensions will soon become annoying when users browse on a daily basis and probably on a large number of pages. The fact that the browsing frequency is high thus lends more weight to the fact that the adopted supervised-learning method should be coupled with an efficient inference and a low-friction interface.

Question 7 -Phishing Website Awareness.

Have you heard of phishing websites before?

44 responses

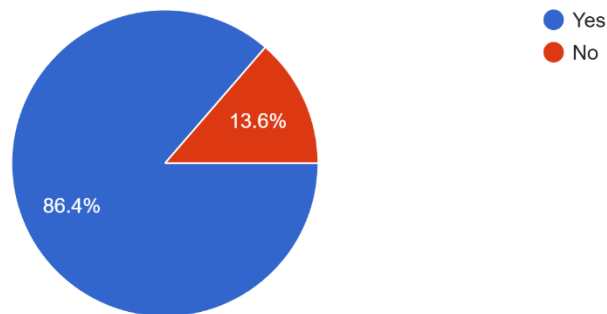


Figure 16: Phishing Website Awareness

Phishing websites awareness is quite good, with 38 respondents (86.4%) saying that they have heard of phishing websites. Nevertheless, there were a fewer number of people who responded with 'No'. This is a significant gap. It demonstrates that general awareness is not universal even in a digitally active sample, although it is very high. It should therefore not be assumed that all users have a common security vocabulary/experience on which the project is based.

Design wise this outcome favors the introduction of a text that is explicit and not too technical as a warning. In case a part of users still does not understand the terminology of phishing, then the extension should inform about danger in easily understandable language. As an illustration, a message that describes that a web page is purporting to be a legitimate site would probably be more efficient than a message that utilizes only the language of specialists or numeric model outputs.

Question 8 - Phishing Emails Awareness.

Have you heard of phishing emails before?

44 responses

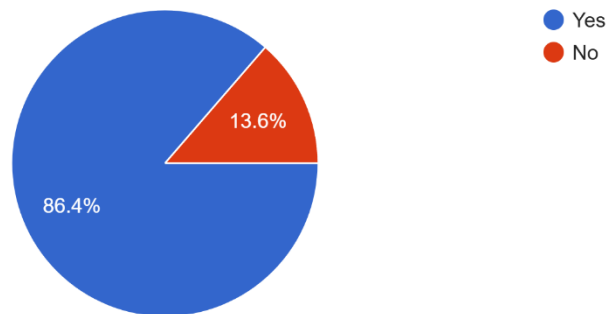


Figure 17: Phishing Emails Awareness

The level of awareness of phishing emails is almost similar to that of websites awareness, with 38 respondents (86.4%) reporting that they have heard about phishing emails. This similar finding indicates that respondents are more likely to acknowledge phishing as a general threat that is present on the internet instead of a channel-specific threat. Awareness, however, does not necessarily mean the ability to identify the less obvious or contemporary forms.

Project implication is that awareness should be supported by the system. Even when the user is aware of the existence of phishing, he or she may not evaluate a particular message or link appropriately in a realistic browsing situation. This is why the extension can be considered useful even to respondents, who already realize the overall idea of phishing.

Question 9 - Generalized knowledge of Phishing Attacks.

How would you rate your overall understanding of phishing attacks?

44 responses

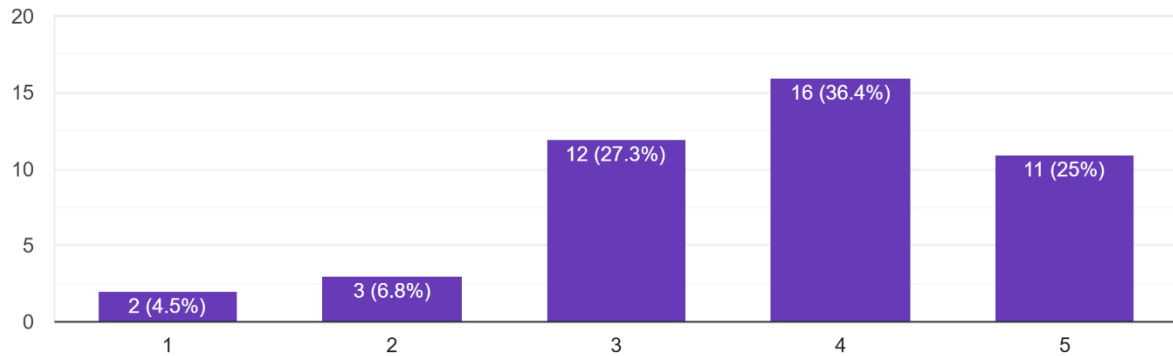


Figure 18: Generalized knowledge of Phishing Attacks

The overall self-rated level of knowledge of phishing attacks is 3.70 out of 5 with the majority of the answers being 3, 4 and 5. This shows that there is moderate to high perceived knowledge but not universal confidence. Proliferation is practical since it represents a realistic base of users: neither completely ignorant nor knowledgeable. This type of user base is just the type of audience that can use supportive detection tools.

Awareness and understanding are also a difference here. Some respondents might know about phishing and yet be confused about what to make of the suspicious links, cloned web sites, or persuasive wording. It is at this point that the explanatory role of the extension plays a significant role. The system need not simply tell the user that something is suspicious but should tell the user why they are suspicious, particularly in cases where the user is not a knowledge expert but is merely an intermediate knowledge user.

Question 10 – Suspicious emails.

Have you ever received a suspicious email asking for personal, login, or financial information?

44 responses

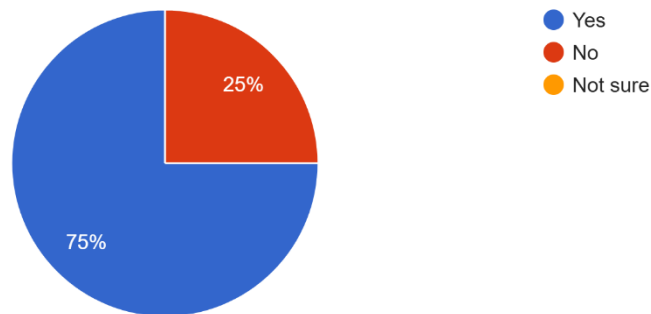


Figure 19: Suspicious emails

Suspicious email experiences are common. 33 (75.0) respondents answered yes to the question that they have been asked to provide personal, login or financial information in a suspicious email. It is among the most formidable practical evidence in the survey since it indicates that phishing exposure is not hypothetical among the sample. It has already become a part of the online experience of the respondents.

This discovery hugely supports the rationale behind the project. The extension that is able to assist with phishing indicators in emails is a responding solution to a widespread, not a niche issue. It also implying that the value of the extension can be judged not only by the performance of the abstract model, but by whether the extension can be able to assist the respondents in case they encounter a suspicious email in practice.

Question 11 - Vexed Suspicion of Web.

Have you ever visited a website that appeared suspicious or untrustworthy?

44 responses

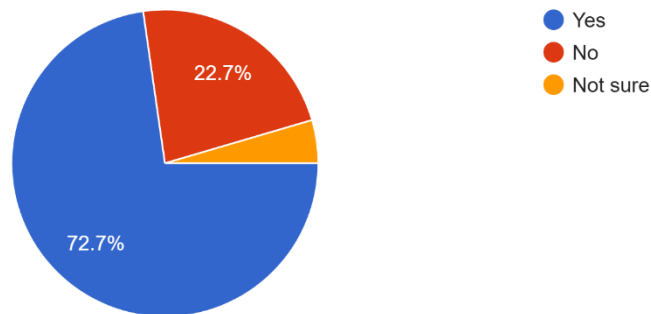


Figure 20: Vexed Suspicion of Web

The exposure to suspicious sites is also prevalent. 32 (72.7%) out of the 44 respondents reported having visited a site that looked suspicious or untrustworthy, the rest reported no or they were not certain. It is particularly telling that there is some ambiguity in certain answers. It indicates that users cannot necessarily know whether their suspicion is warranted, creating an opportunity to make decision-support tools.

This question hence greatly supports the browser-extension model. When users often visit dubious sites and are not necessarily sure how to read them, an in-context warning system can be useful. The extension is not only aimed at substituting judgement but empowering it in the most vulnerable moment the user could be deceived at.

Question 12 - Trust in Self-Detection.

How confident are you in identifying phishing websites or phishing emails on your own?

44 responses

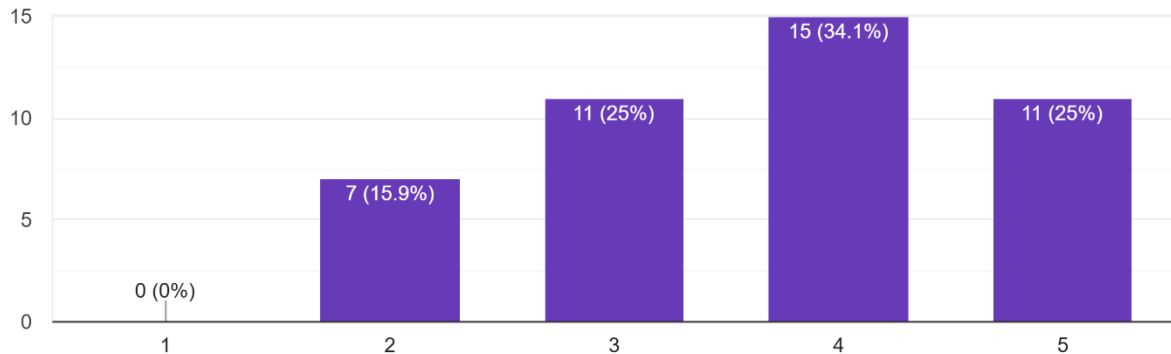


Figure 21: Trust in Self-Detection

Self-detection of phishing has an average score of 3.68 out of 5. The distribution indicates that a substantial percentage of the respondents feel moderately confident, and the scores are not all at the upper end of the scale. Such an outcome is beneficial due to the disclosure of a realistic tension: users tend to believe that they can be aware of phishing, yet this is not always the case. In the context of cybersecurity, biased trust can be harmful when it results in a decreased level of caution.

The question is further amplified when it is taken along with Question 10, 11 and 15. Users have moderate confidence, yet, they also have a high exposure to suspicious content, and a large percentage of inadvertent clicks. Collectively, these findings indicate that perceived competence does not eradicate real risk. The extension is thus warranted as a supplementary layer even with those users who feel that they are fairly competent individually.

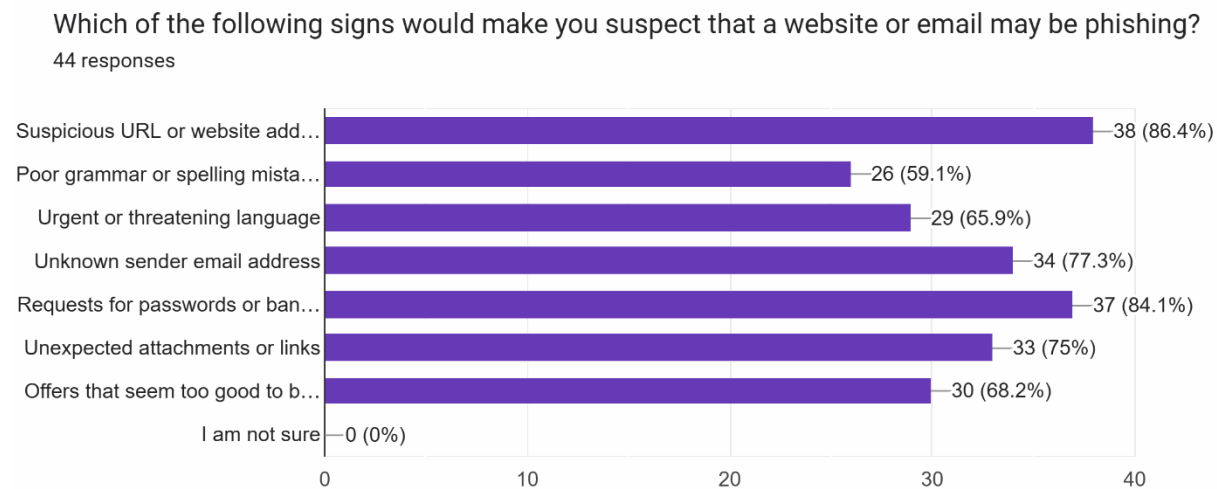
Question 13 - Signs That Users Can be Suspicious of Phishing.

Figure 22: Signs That Users Can be Suspicious of Phishing

The cues that prompt them to suspect phishing were varied, though suspicious URLs, requests of passwords or banking information, unfamiliar sender addresses and unanticipated attachments or links were particularly highly chosen by the respondents. This trend indicates that users depend on a set of indicators other than a single indicator. That is a great design lesson. When it is already a multi-factor user judgement, then the extension should also be multi-factor in that it should also combine various types of signals instead of being based on a simplistic rule.

The strong preference for URL-related and credential-related cues is directly relevant to the implemented PhishGuard workflow. These signals can be processed efficiently and translated into warnings that users can understand. At the same time, the more moderate selection of grammar-related cues shows why obvious writing mistakes cannot be treated as a reliable standalone phishing signal. The completed system therefore combines model evidence with structural and contextual indicators rather than depending on a single familiar stereotype.

Question 14- largest perceived risk of phishing.

In your opinion, what is the biggest risk of phishing attacks?

44 responses

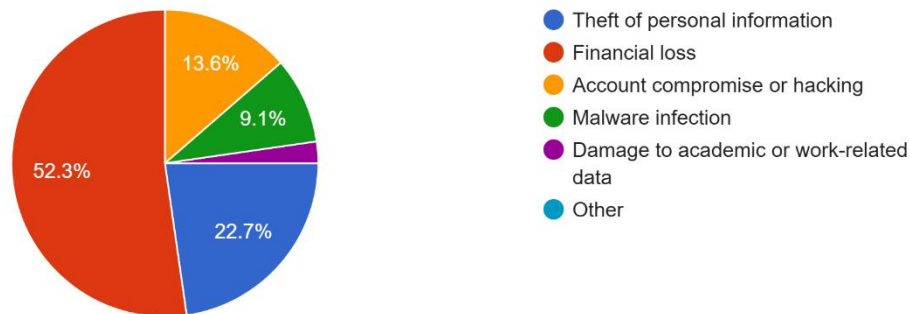


Figure 23: Largest perceived risk of phishing

In a question on the greatest risk of phishing, the greatest number of responses were on financial loss. Other reactions were stealing personal information, compromising the account, malware, and destroying academic or work data. This demonstrates that the users understand phishing as a risk of an immediate threat of harm to their fraud, but they are also aware of the impacts going into privacy, account management, and operational interference.

The significance of this finding is that it affects the motivation of users. When individuals perceive the possibility of a significant personal loss because of the underlying threat, they tend to appreciate a protection tool. With a properly designed extension, adoption barriers can be reduced, as the respondents already associate phishing with serious consequences. Protective value should consequently be communicated in the project, since the users are already aware of the severity of the possible consequences.

Question 15 -Mistakenly Clicking Suspicious Links.

Have you ever clicked on a suspicious link by mistake?

44 responses

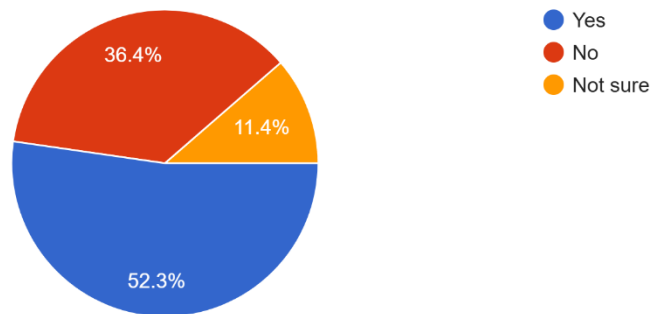


Figure 24: Mistakenly Clicking Suspicious Links

More than half of the respondents (23 respondents, 52.3%) admitted that they had clicked on a suspicious link by mistake. This has been found to be one of the most revealing in the whole survey. It demonstrates that the knowledge of phishing does not always protect against unsafe behavior. The real-world decisions are made fast, and in some cases, they may be distracted or even under extreme pressure but even the users of the theory are still likely to make a mistake in practice.

This result provides very strong support for the project. The extension is not there since the users have no idea that phishing exists; the extension is there since there is no use of awareness only. The system can interrupt risky behavior at the point in the browsing workflow when human judgement is most likely to be pressured, routine or overconfident by making warnings part of the browsing experience.

Question 16 - Soberness of Phishing in the present Digital Age.

How serious do you think phishing attacks are in today's digital environment?

44 responses

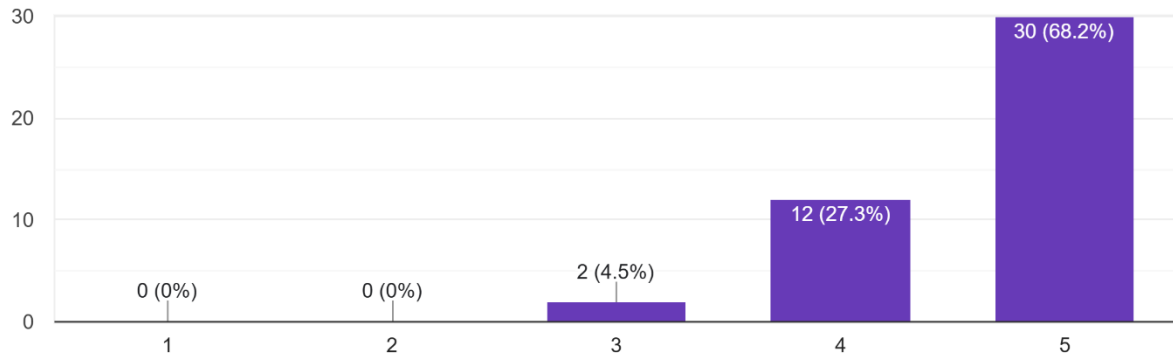


Figure 25: Soberness of Phishing in the present Digital Age

The gravity of phishing was also considered to be very high with the mean score of 4.64 out of 5. The majority of the respondents picked the greatest value on the scale and very few picked low rating. This demonstrates a very close correspondence between the perception of threat by the user and the essence of project motivation. The respondents do not consider phishing as a small or an occasional inconvenience; they consider it to be a big problem in the present digital environment.

The implications of this high seriousness rating are also design implications. Users who already perceive phishing as significant will probably accept more forceful warning and might be more open to a minor level of defensive friction when the extension is efficient. That is, the extension can be assertive but in the warning style without appearing irrelevant as long as the warnings are correct and timely.

Question 17 - Usefulness of the Proposed Browser Extension.

Do you think a browser extension that detects phishing websites and phishing emails would be useful?

44 responses

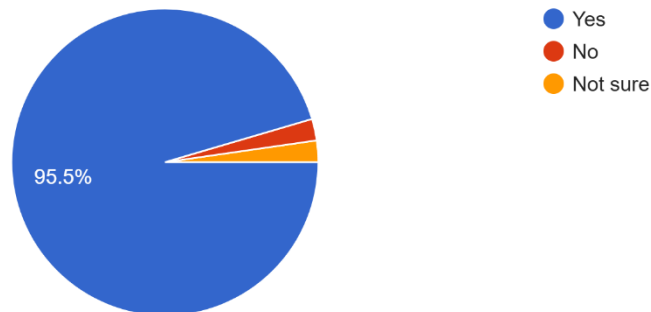


Figure 26: Usefulness of the Proposed Browser Extension

An overwhelming majority of respondents (42, or 95.5%) considered the browser-extension concept proposed in the questionnaire useful. Only isolated responses selected no or not sure. This result provided direct support for continuing with a browser-integrated protection approach. The final PhishGuard implementation retains that delivery model and can now be evaluated against the expectations recorded during the investigation.

The quality of this finding is particularly useful since viability of academic projects is not solely technical. A system must also have perceived usefulness. High usefulness perception implies that subsequent prototype testing can be more concerned with the way of implementing the idea effectively, but not whether the delivery model per se is sensible to users.

Question 18 -Most Helpful Extension Feature.

Which feature would you find most useful in a phishing detection browser extension?

44 responses

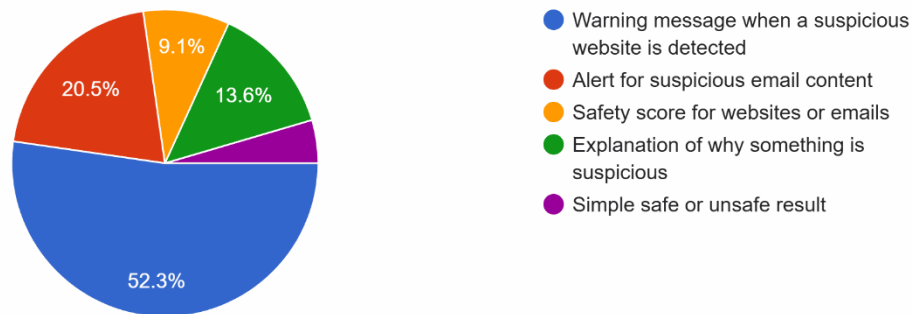


Figure 27: Most Helpful Extension Feature

The most frequently chosen one-useful feature was a warning message about suspicious websites, which was chosen by the respondents the most. The alerts about suspicious email contents, description of why something is suspicious, safety ratings, and easy safe or unsafe were also chosen, however, with lower frequencies. This ranking is significant as it indicates that direct intervention and supplementary explanation are preferred by the users first and second, respectively.

In terms of design, the message is clear: the extension must focus on giving immediate warnings rather than using passive indicators or concealed scores. But explanation must not be expelled altogether, since interpretation is still important to a significant minority. A layered design is thus the most suitable whereby an immediate warning is displayed initially, and more details can be obtained when necessary.

Question 19 - Type of Preferred Alerts.

What type of alert would you prefer from the browser extension?

44 responses

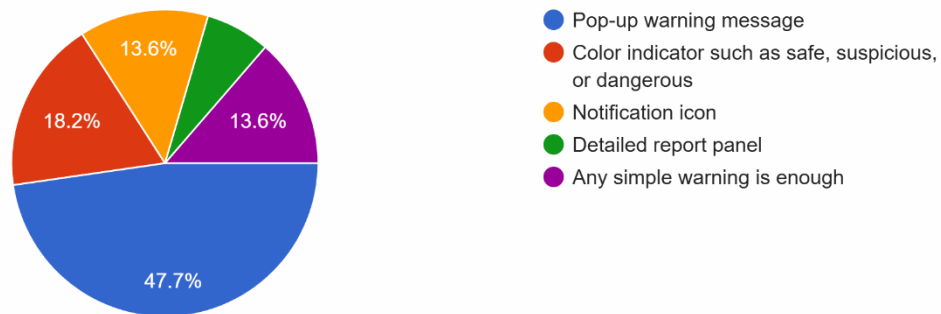


Figure 28: Type of Preferred Alerts

In terms of alert type, a pop-up warning message will be the most preferred type, color indicators, and simple notifications. The number of respondents who chose a detailed report panel as the main alert style is very low. This means that users desire the extension to display risk in a manner that is intuitive and instantaneous and not hidden from an elaborate technical perspective.

What it means is that the extension should not rely on the user to proactively open a dashboard to learn about risk. Rather, the main warning must manifest itself during the suspicion. The detailed views of the report may be additional features of the report that should be available to users that require more detailed description, yet the primary interface should be simplified and action oriented.

Question 20 - Significance of Detection Speed.

How important is it for the browser extension to provide detection results quickly?

44 responses

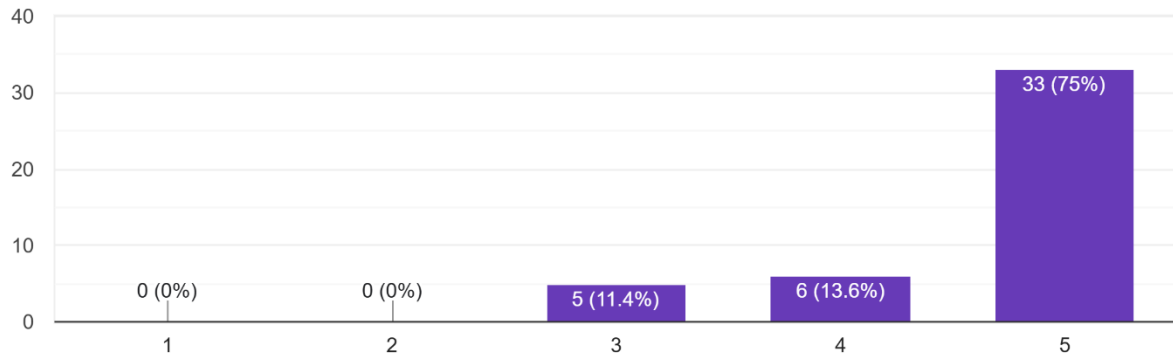


Figure 29: Significance of Detection Speed

Very high detection speed with an average score of importance of 4.64 out of 5. Most of the respondents chose the highest score. This renders speed as non-functional requirement and not an advantage. In that case, when the extension causes the page to interact at a slower pace and/or it feels heavier, the user can turn it off despite the classification model being correct.

This finding is especially significant to technology selection. It favors the selection of computationally manageable supervised-learning models and discourages prototypically heavy architecture. It also warrants the focus on efficient feature extraction and lean extension design rather than feature extravagance.

Question 21 - Ease of Use Importance.

How important is it for the browser extension to be easy to use?

44 responses

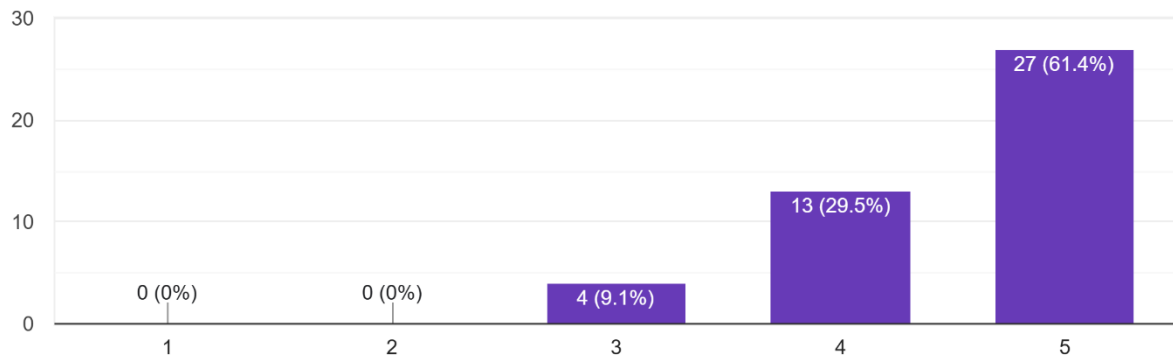


Figure 30: Ease of Use Importance

The ease of use was also very highly rated with the average score being 4.52/5. This proves that the extension should be easily comprehensible by users without the need to have cybersecurity knowledge. An instrument that is precise yet incomprehensible would not be satisfactory to the expectations highlighted in the survey.

The implication is that the extension should minimize interaction burden. Warnings messages must be concise, the textual meaning of statuses must be self-evident, and the user must not have to work out technical values to take safe actions. The ease of use thus becomes a fundamental success factor in oneness with model performance.

Question 22 - Readiness to Use the Extension.

Would you be willing to use a browser extension that helps detect phishing threats while browsing websites or checking emails?

44 responses

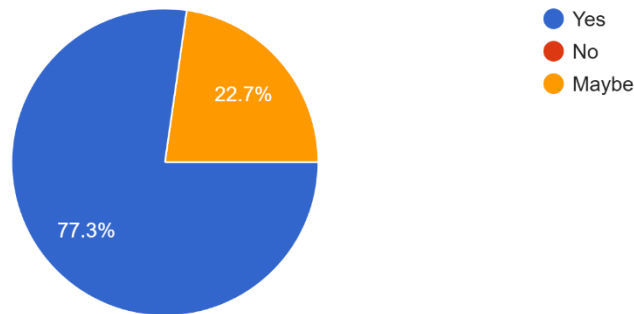


Figure 31: Readiness to Use the Extension

There is a high willingness to use the extension, 34 respondents (77.3) responded yes, and the other respondents answered maybe and not no. It is interesting that there is no explicit rejection. It implies that the idea of phishing-detection extension is tolerable to users provided the information of its implementation including privacy, speed, and quality of alerts are managed properly.

The fact that maybe responses were there is still important though. This means that there are conditionally interested and not entirely committed users. The said users will be convinced of such attributes as pragmatism does the extension feel trustworthy, is it lightweight, is it reasonable in explaining risk, is it considerate of privacy. This supports the significance of the quality of implementation in the following project stage.

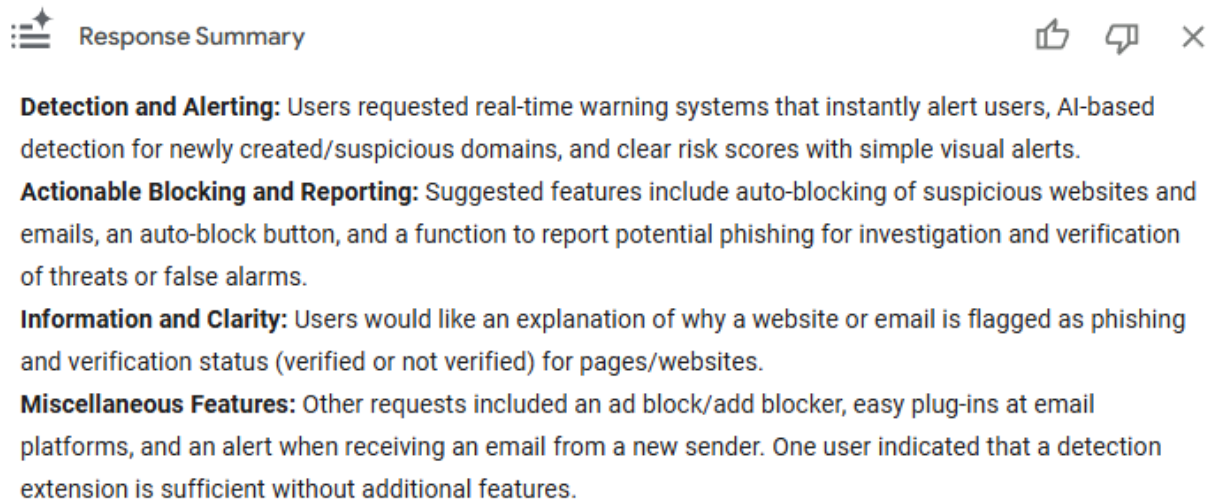
Question 23- Other Features Wanted.

Figure 32: Other Features Wanted

The open-ended answers on other desired features show that there are the same priorities of users. The features requested by the respondents included more robust real-time alerts, optional automatic blocking, suspicious links or pages scan features, sender-awareness features, and how to report suspicious materials. Many of the respondents also showed interest in risk scores, page verification cues, and silent background scanning which does not involve a lot of manual work.

These recommendations show that the users desire the extension to not just label a page as suspicious. They anticipate having a friendly aid that can step in, clarify and potentially lead to safer user behavior in the long run. Concurrently, certain of the more sophisticated requests, like automatic blocking and scanning email more deeply, can add complexity to implementation or sensitivity to privacy. This implies that they must be handled as future-improvement candidates unless they can be incorporated in the prototype in a responsible manner.

Question 24 -Recommendations.**Response Summary**

User Experience and Interface (UI): Suggestions include a cleaner, less intrusive, and more user-friendly UI; providing clear, minimal, and high-confidence alerts; and offering a detailed warning message instead of just a pop-up, explaining what could potentially happen before offering options to "go back" or "proceed."

Functionality and Detection: Recommendations focus on real-time alerts, auto-blocking detection, and the ability to detect fake pages by comparing them to the original website. Integrating AI to detect phishing threats was also suggested.

Integration and Scope: Suggestions include incorporating scanning capabilities for incoming emails (like Gmails) for phishing indicators and allowing the extension to categorize emails by risk level within email portals (like Gmail or Outlook). For enterprise systems, integration across the browser, email, and enterprise systems was noted.

Privacy and Control: One concern raised was that improving such an extension might seriously evade privacy, suggesting the extension should only activate when the user chooses to verify a site.

Detection Techniques: Suggestions include combining URL, content, behavior, and identity signals for detection, and ensuring regular updates to address new phishing techniques.

Figure 33: Recommendations

The last open-ended question on how it can be improved gives great insight into refinements. Some of the replies focused on less intrusive and clean user interfaces, more understandable warning messages, more concise explanations of the reasons behind a page being flagged, and periodic updates to match the current phishing methods. There were privacy concerns by others, particularly with regards to any functionality that reads email content or does more extensive background checks. These issues are very topical as they demonstrate that the users do not consider the extension by detection value only; they also take into consideration the way it works respectfully and transparently.

These recommendations support the final design philosophy of PhishGuard. The extension should be noticeable when risk is high but should avoid becoming noisy, intrusive, or technically overwhelming during normal use. The completed combination of real-time warnings, concise risk explanations, optional detail, user-specific trust, and proportionate browser permissions reflects the expectations expressed by respondents in the survey.

3.3.1 Cross-Question Interpretation and New Themes.

When the survey is viewed as a unit and not as individual questions, there are a number of powerful themes. The first one is that phishing is not a new and unknown issue to the target audience. The level of awareness regarding phishing sites and phishing emails is high, yet the direct exposure level in regard to suspicious messages, suspicious websites, and clicking dangerous links remains high as well. This combination is significant as it demonstrates that awareness does not solve the problem on its own. Users require support when they interact.

The second theme is that the browser is a legitimate place of intervention. The frequency of website use is very high, with most respondents using websites on a daily or weekly basis, and they are also highly supportive of the notion of a browser extension as a helpful security tool. This offers firsthand empirical evidence to the delivery model adopted by the project. The extension is not being forced upon the users in an unnatural manner; it is in line with their real technology usage, and their own perception of what will be beneficial.

The third theme is that usability cannot be discussed without security effectiveness. Speed and ease of use were continually rated as very important by respondents. They also favored conspicuous yet simple notifications like pop-up warnings and color notifications in place of technical visages. That is, it will not be about the classification metrics only but about the ability of the extension to act in a manner acceptable by the user in the course of normal browsing.

Fourth theme is that users appreciate the explanations, but not at the expense of clarity. Although a direct warning message is the most desirable option, open-ended responses, and multiple-choice responses demonstrate a significant need to know the purpose of something suspicious. This implies that the extension would be communicating in layers: immediate risk first,

explanation second. An overly silent system cannot be relied upon, but an overly talkative system cannot be utilized.

Lastly, the open-ended answers show a progressive anticipation of active protection. Users cited automatic blocking, sender-awareness indicators, real time scanning, report features, and on-going updates. These are not all to be or should be implemented in the initial prototype. Nevertheless, they demonstrate that the target audience wants an anti-phishing extension to be a proactive tool of security but not a passive informational extension.

3.3.2 Final list of user requirements.

The survey-derived requirements are divided into functional and non-functional requirements. They originated in the investigation stage, but the final report can now compare them with the completed PhishGuard implementation. Core requirements such as direct website warnings, email-related protection, rapid response, clear explanations, ease of use, and privacy-conscious handling were implemented. Additional security functions such as mandatory MFA, security events, personal trust, PDF reporting, and the encrypted password vault were added during Part 2 because they were necessary for realistic and secure operation of the final prototype.

Requirement type	Requirement	Priority	Rationale from analysis
Functional	The extension shall analyze website-related indicators and classify the current browsing target as legitimate or phishing.	Must Have	High daily browsing frequency and strong concern about suspicious websites.
Functional	The extension shall analyze browser-visible email-linked phishing cues where doing so is technically and ethically practical.	Must Have	Email is used frequently and suspicious emails are common.
Functional	The extension shall display a clear warning message when high-risk phishing indicators are detected.	Must Have	Users strongly prefer direct warning messages and pop-up alerts.
Functional	The extension should provide a short explanation or supporting reason for the warning.	Should Have	Many users want to know why something is suspicious.
Functional	The extension may provide optional safe/suspicious/dangerous status indicators or simple risk scoring.	Could Have	Some users prefer quick visual interpretation.
Functional	The system may include a user-reporting or feedback mechanism for suspicious items in later iterations.	Could Have	Open-ended feedback requested report functions and improvement loops.
Non-functional	The extension should return results quickly and should not noticeably slow normal browsing.	Must Have	Speed was one of the highest-rated importance factors.
Non-functional	The extension should be easy to use and understandable without cybersecurity expertise.	Must Have	Ease of use was rated highly and awareness is not universal.

Non-functional	The warning design should be visible enough to interrupt risky action but not so intrusive that users disable the extension.	Must Have	Users want direct alerts, but several comments requested a cleaner interface.
Non-functional	The extension should request only necessary browser permissions and avoid unnecessary retention of personal data.	Must Have	Open feedback highlighted privacy sensitivity.
Non-functional	The selected supervised-learning models should be transparent enough for comparison and feasible for extension-oriented deployment.	Should Have	Project scope values feasibility, maintainability, and evaluation clarity.

Table 8: Consolidated User Requirements

The requirement list remained a useful design baseline during Part 2. Direct warning capability, website and email coverage, speed, clarity, and conservative privacy behavior correspond closely to the final extension workflows. The implemented system also preserves layered information: the initial warning is concise, while detailed scan evidence, history, security activity, trust controls, and PDF reports are available when users need more context.

The final system still follows the survey preference for focused protection rather than uncontrolled feature growth. Advanced functions were included only where they supported a clear security or usability need, and several enterprise-scale capabilities remain deliberately out of scope. This keeps the prototype defensible: it concentrates on phishing detection, timely intervention, authenticated user support, privacy-conscious evidence, and protected credential handling rather than attempting to replace a complete enterprise security platform.

3.3.3 Mapping Survey Results to Design Decisions

The survey became most valuable when its findings were translated into implementation decisions. Respondents used websites frequently and strongly supported browser-based protection, so the final system retained the Chrome extension as the primary delivery point instead of exposing the models as a disconnected classifier. The popup, current-site scan, warning page, and Gmail

banner all place risk information directly in the environment where the user is making a trust decision.

Speed and ease-of-use ratings influenced the decision to keep the main interaction concise. PhishGuard does not require users to open the Security Dashboard before protection occurs. High-risk website decisions can interrupt navigation immediately, Gmail results are shown in-page, and the popup provides a compact recent result. More detailed evidence is available only when the user asks for Scan Details, history, or a PDF report.

The strong preference for visible warnings influenced the high-risk intervention design, but survey demand for explanation also led to layered evidence. The final system therefore combines a direct risk label with reasons, raw model/contextual separation, trust status, and explicit next actions. This preserves rapid decision support without hiding the basis for the warning.

The survey also supported privacy-conscious design. In the final architecture, routine scan history, trust decisions, and settings remain in user-scoped Chrome storage rather than a centralized browsing-history database. Backend persistence is limited to account/security records and encrypted vault data, and the vault master password and plaintext credentials remain browser-side.

Taken together, the survey did more than justify the project topic. It influenced where protection appears, how much information is shown at first, how optional detail is exposed, how privacy-sensitive state is stored, and why the extension avoids forcing personal trust decisions into the underlying machine-learning models. These mappings can now be evaluated against the completed interface and testing evidence in Chapters 4 and 5.

These design mappings provide a traceable link between the investigation evidence and the implemented PhishGuard behaviours discussed in Chapter 4.

3.3.4 Implications for Model Design and Extension Workflow

The survey indicated that practical robustness and understandable protection were more valuable than maximum model complexity. The final implementation follows that principle by using classical supervised-learning models with explicit feature/text pipelines and by keeping

contextual reasoning separate from the frozen model probability. This makes model behavior easier to evaluate and lets the extension explain high-risk combinations without pretending that contextual rules are part of the original classifier metric.

URL-based signals became the primary website-model input because they are fast and consistently available in the browser workflow. For email, visible message text, sender information, links, and contextual indicators are combined with the calibrated text probability. The final system therefore uses signals that can be acquired without expensive visual processing or invasive mailbox-wide access.

Threshold handling was implemented as a product decision rather than as a single model number. The website model has a raw decision threshold for evaluation, while the extension uses a higher intervention threshold for disruptive warnings. Contextual evidence and trust rules operate within bounded limits, and severe evidence restricts how much personal trust can reduce risk. This directly addresses the survey tension between avoiding missed threats and preventing alert fatigue.

Primary and secondary actions are clearly separated in the completed interface. The immediate goal is to warn or inform the user; secondary capabilities include detailed evidence, history, reporting, trust management, and account security review. This layered workflow lets casual users act quickly while still supporting deeper inspection when needed.

The completed system also supports learning through explanation. Website warnings, Gmail banners, Scan Details, and PDF reports expose the indicators behind a decision, allowing users to see why a destination or message was considered risky. This extends the value of the tool beyond blocking a single event and supports the survey expectation for understandable security assistance.

3.3.5 Implemented System Architecture

The implemented PhishGuard architecture can be understood as five cooperating layers. The first layer is browser capture and interaction: the Manifest V3 extension observes the active tab when requested, runs the Gmail content script on supported Gmail pages, presents the popup

and warning interfaces, stores user-scoped history/trust/settings, and performs client-side vault cryptography.

The second layer is the hosted application API. Flask, deployed on Render, receives website and email scan requests and provides authenticated account, MFA, security-event, reporting, and vault endpoints. CORS and extension-origin handling restrict how the production API is accessed, while localhost remains available for development. This layer also validates sessions and ownership for protected operations.

The third layer is analysis. A frozen Random Forest processes the website feature vector and a calibrated LinearSVC processes email text. Contextual website/email evidence is evaluated separately from the base model output, and personal trust is applied only through bounded user-specific rules. This preserves the ability to report model metrics honestly while still improving practical warning behavior for combinations not captured well by the base classifier.

The fourth layer is persistence and security state. Neon PostgreSQL stores user, recovery-code, session, challenge, security-event, vault-profile, and encrypted-vault-entry records. Normal browsing history and personal trust are intentionally not stored in central history tables. Vault entries remain opaque ciphertext to the backend because the master-password-derived cryptographic operations are performed in the browser.

The fifth layer is user feedback and evidence. Website warnings, Gmail banners, popup results, the Security Dashboard, security activity, trust settings, Scan Details, and PDF reports translate technical results into user decisions. This layered architecture is the practical outcome of the investigation requirement analysis and is documented through the diagrams, code evidence, and screenshots in Chapter 4.

3.3.6 Limitations of the Data Gathering and Requirement Analysis

The survey was appropriate for collecting user requirements, but several limitations affect how the findings should be interpreted. First, convenience sampling was used because the questionnaire was distributed online rather than through controlled random sampling. The results

are therefore useful for identifying the needs of the participating target users, but they should not be treated as population-level estimates of phishing awareness or extension adoption.

Second, the survey relies on self-reported knowledge, confidence, and previous behavior. A participant's confidence in identifying phishing does not necessarily equal real-world detection ability, and memories of suspicious messages or links may be incomplete. This limitation does not invalidate the requirement analysis because the project also needs to understand perceived needs and expectations, but the findings should be interpreted as user-reported evidence rather than objective behavioural measurement.

Third, the investigation survey is not an observational usability experiment. It measures reported awareness, preferences, and past experience rather than directly observing how participants respond to PhishGuard warnings. Part 2 therefore supplements the survey with automated and system testing and a user acceptance testing exercise described in Chapter 5. The report does not treat UAT as completed until the corresponding tester evidence and results are available.

Finally, phishing tactics and user expectations change over time. The survey captures the views of the respondents during the April 2026 investigation period and therefore represents a time-bounded requirements snapshot. This limitation reinforces the value of the Agile methodology: requirement interpretations can be revisited as implementation, testing evidence, and new phishing behaviours reveal the need for refinement.

CHAPTER 4: DESIGN AND IMPLEMENTATION

4.1 Introduction

This chapter documents the completed design and implementation of PhishGuard. In contrast to the Investigation Report, which described a proposed browser-extension architecture, the discussion here is grounded in the latest submitted source code, model metadata, automated tests, deployment configuration, system screenshots, and code screenshots. The final prototype has evolved into a hosted phishing-protection platform consisting of a Manifest V3 Chrome extension, a Flask API deployed on Render, supervised-learning model artifacts, Neon PostgreSQL persistence for account-sensitive data, user-scoped browser history and trust settings, and a browser-side encrypted password vault.

The chapter follows the structure required by the FYP documentation guideline: design, database design, interface design, execution, end-product screenshots, and summary. The implementation is described as a security system rather than only a classifier because the final user experience depends on authentication, warning timing, personal trust decisions, reporting, Gmail behavior, and credential protection as well as the underlying machine-learning models.

4.2 Design

4.2.1 Overall System Architecture

The final PhishGuard architecture uses a Manifest V3 Chrome extension as the client-facing protection layer, a Flask API hosted on Render for model inference and authenticated services, Neon PostgreSQL for persistent account/security/vault records, and user-scoped Chrome storage for privacy-sensitive local history and trust preferences. Chrome's extension APIs are explicitly designed around runtime, tabs, scripting, and storage capabilities, which supports this event-driven separation of browser-side and server-side responsibilities (Google Chrome Developers, n.d.).

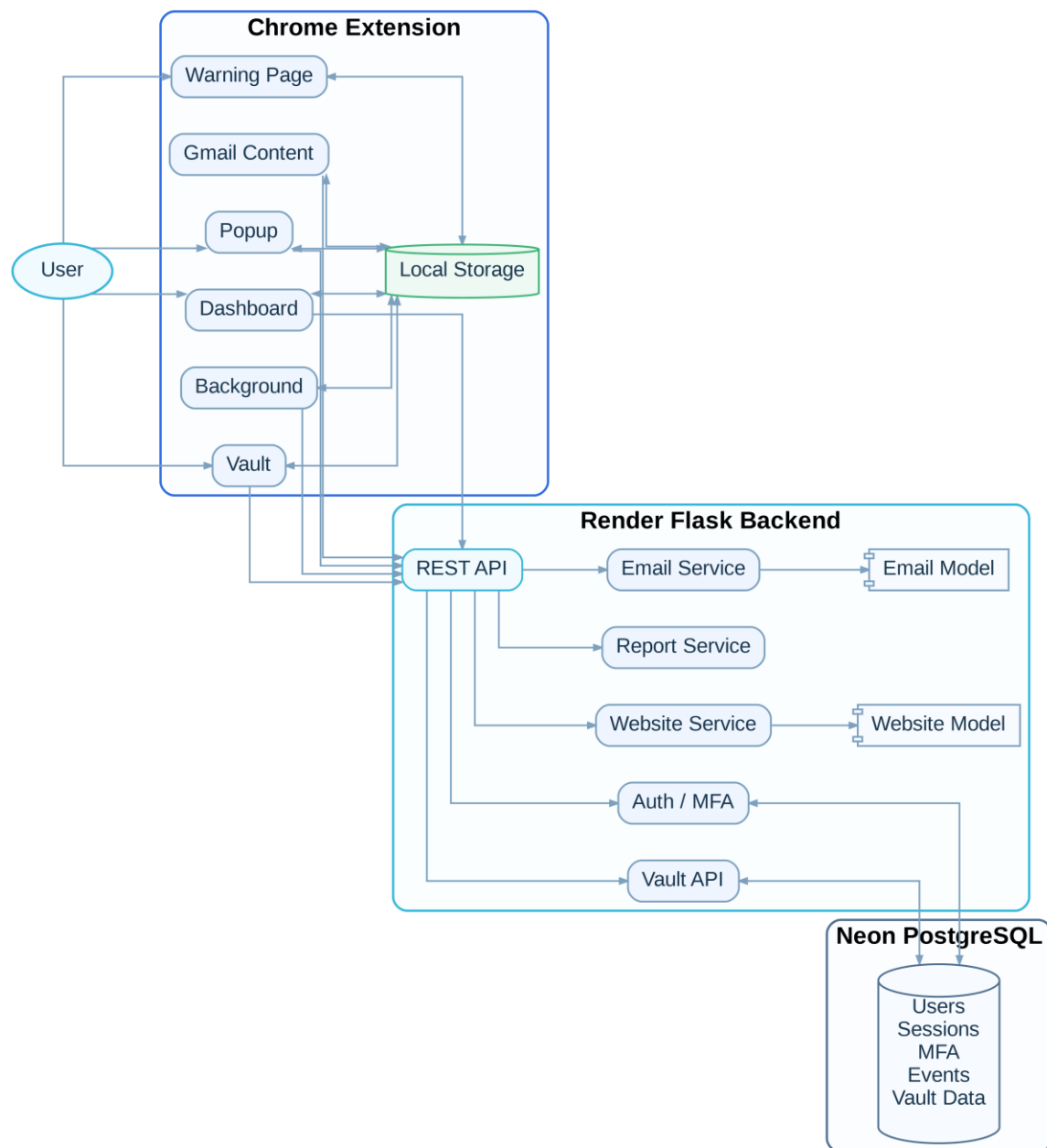


Figure 34: PhishGuard System Architecture

The architecture reflects the current deployed demonstration flow. The production extension communicates with the Render-hosted Flask API over HTTPS; Flask loads the frozen models and connects to Neon PostgreSQL. User-scoped browsing history and personal trust are retained in the browser, whereas account security and opaque encrypted vault data are persistent backend data.

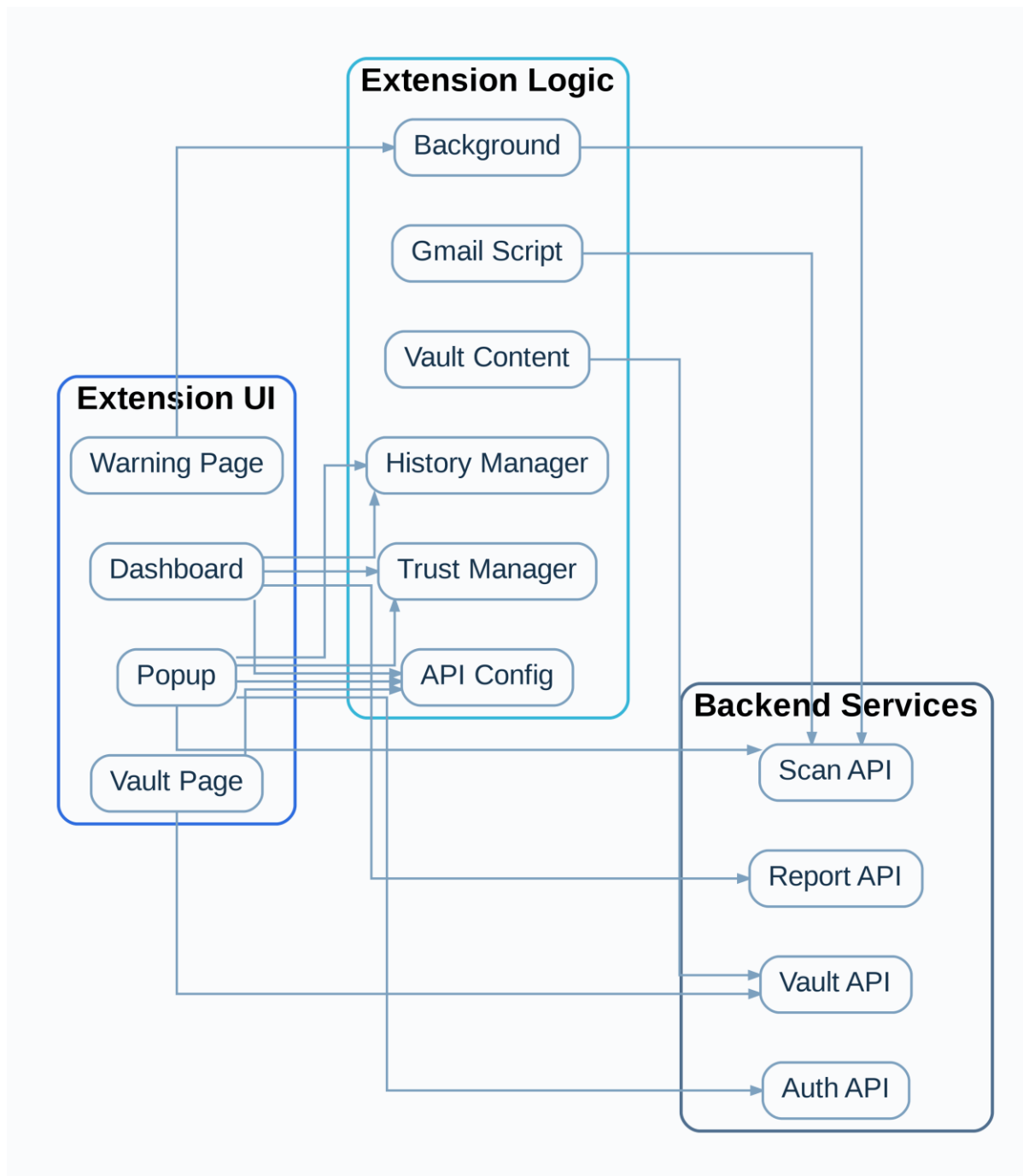


Figure 35: PhishGuard Component Diagram

The component view separates the popup, background service worker, Gmail content script, Security Dashboard, and Password Vault from the backend authentication, scanning, reporting, and vault services. Shared API and authentication helpers reduce duplicated security-sensitive request logic.

4.2.2 Use Case Design

The primary actor is the authenticated user. Registration and mandatory MFA enrollment are prerequisite use cases because the scan APIs and protected dashboard are not exposed to password-only sessions. Once authenticated, the user may perform current-site scans, manual URL or email scans, enable automatic website or Gmail protection, review recent scans, manage trust, export reports, and use the encrypted password vault.

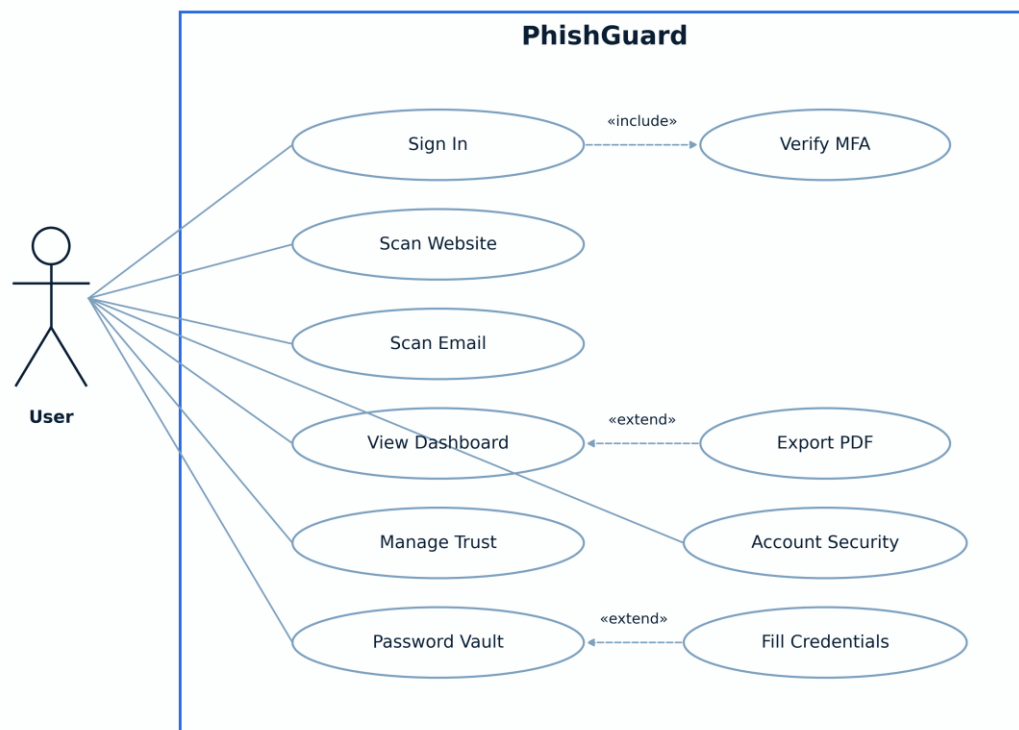


Figure 36: PhishGuard Use Case Diagram

The use case overview summarizes the major user-facing functions implemented in the final prototype. Supporting operations such as session validation, rate limiting, encryption, and signed report validation occur behind these user-facing actions.

ID	Use Case	Actor	Precondition	Expected Outcome
UC-01	Register and enrol MFA	New user	Valid profile, password and TOTP	Account created; recovery codes acknowledged; authenticated session issued
UC-02	Login	Registered user	Correct password + TOTP/recovery	Authenticated session created
UC-03	Automatic website protection	Authenticated user	Website protection enabled	Supported navigation scanned; warning shown when intervention threshold is met
UC-04	Manual website scan	Authenticated user	URL supplied	Raw model, contextual risk and explanation returned
UC-05	Gmail email scan	Authenticated user	Email protection enabled and message opened	Visible message scanned and banner rendered
UC-06	Manage personal trust	Authenticated user	Sender/website/domain selected	User-specific trust stored and personalized risk refreshed
UC-07	Security Dashboard	Authenticated user	History exists	Filters, trends, scan details, security events and report actions displayed
UC-08	PDF report export	Authenticated user	Owned signed receipt or date range	PDF generated in memory and downloaded
UC-09	Password Vault	Authenticated user	Vault master password provided	Encrypted entries can be created, viewed, updated or deleted
UC-10	Fill credentials	Unlocked vault	Matching HTTPS tab and safe fill policy	Credential fields filled only after destination/risk checks

Table 9: PhishGuard Use Case Specification Summary

4.2.3 Activity Diagram Design

Activity diagrams are used to show the decision-oriented execution flow for the most security-sensitive processes. Website scanning contains separate model, contextual, personalization, and intervention decisions. Gmail scanning contains message identity and race-protection logic because Gmail is a single-page application. Authentication separates password verification from MFA verification, and vault actions maintain a separate cryptographic and destination-security boundary.

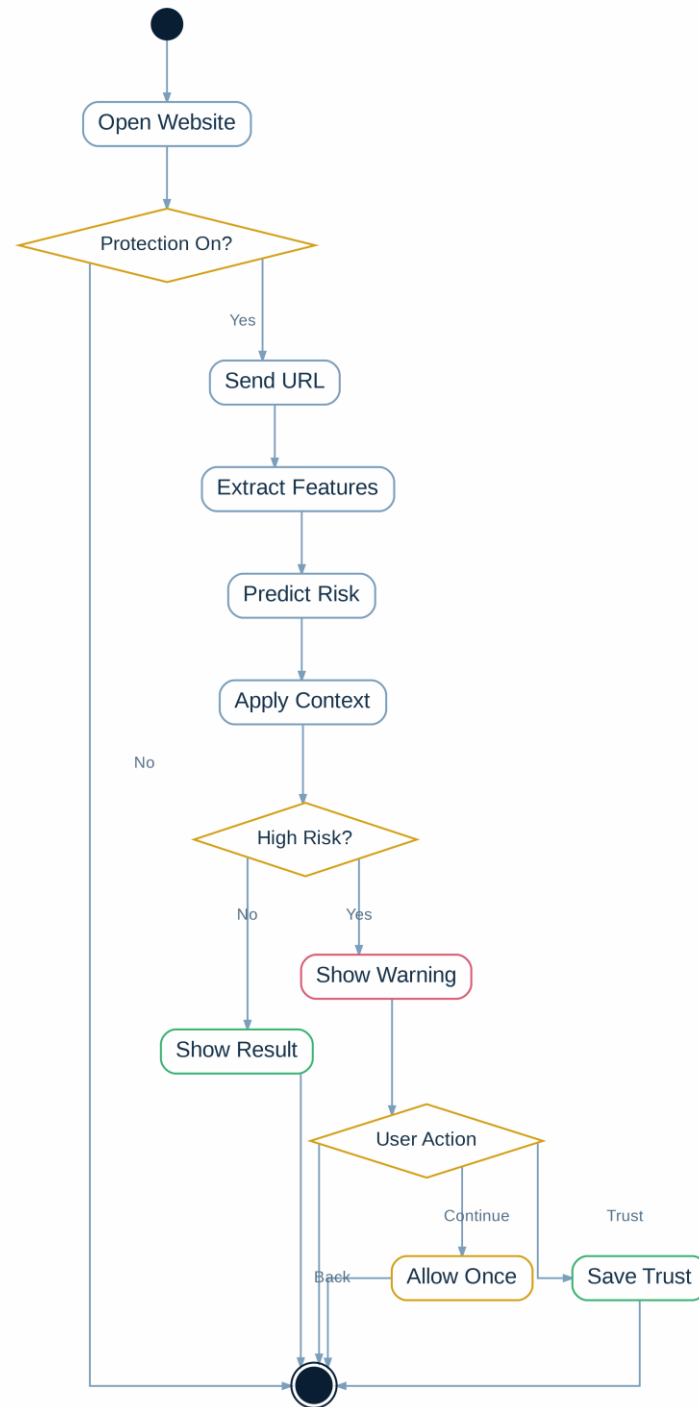


Figure 37: Website Scan Activity Diagram

The website flow demonstrates that final contextual risk and personal trust are separate from the raw Random Forest output and that warning navigation is a final extension policy decision.

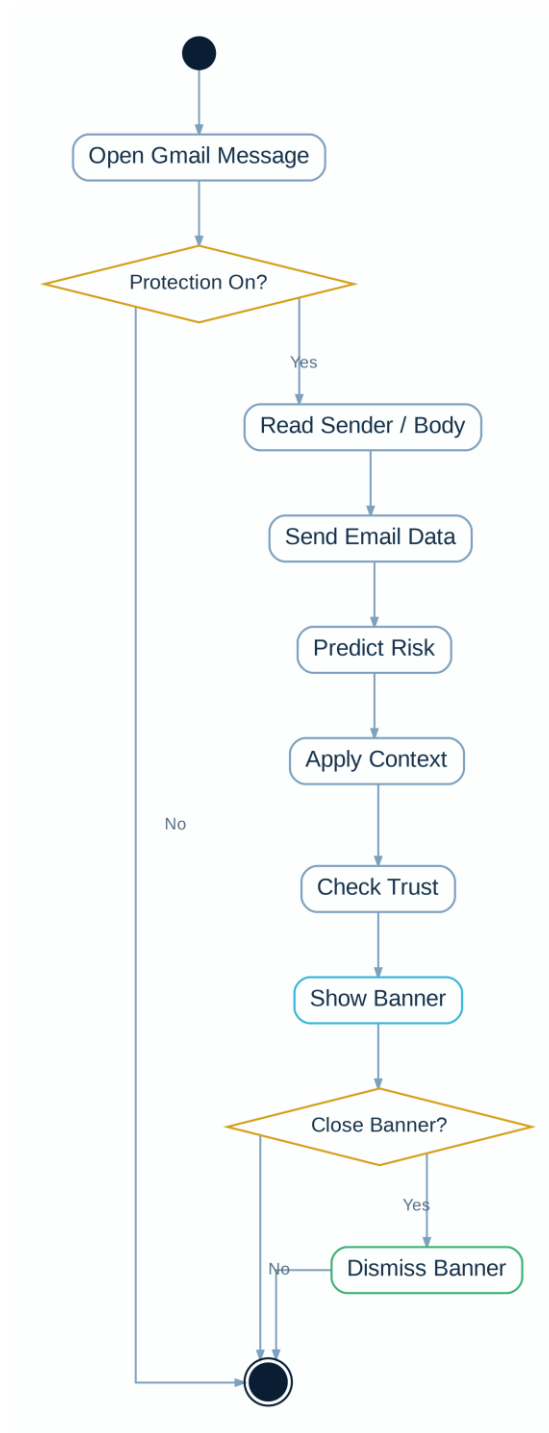


Figure 38: Email Scan Activity Diagram

The Gmail flow includes message identity, debounce, service-worker API access, stale-response prevention, and user-controlled banner dismissal.

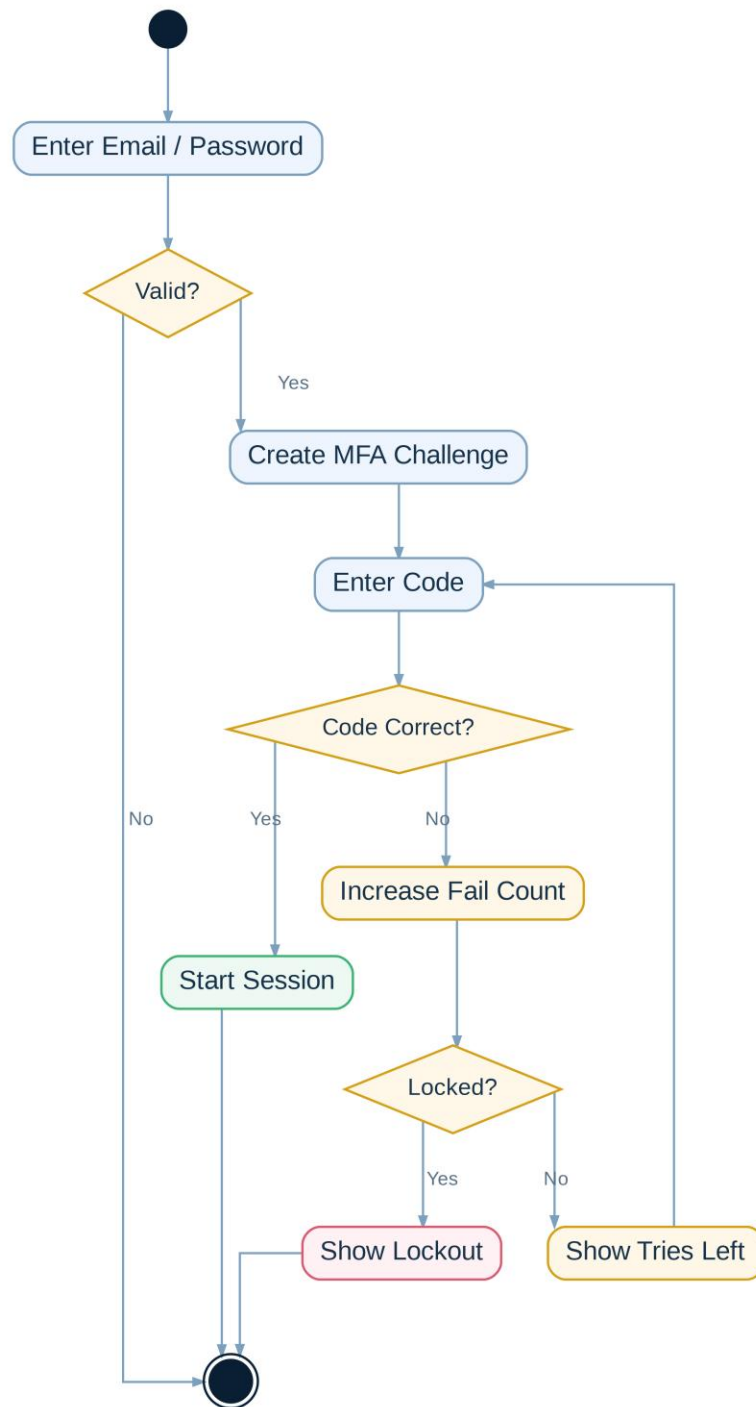


Figure 39: Authentication and MFA Activity Diagram

Password verification must succeed before the MFA stage. This prevents wrong-password activity from directly consuming account-level MFA attempts.

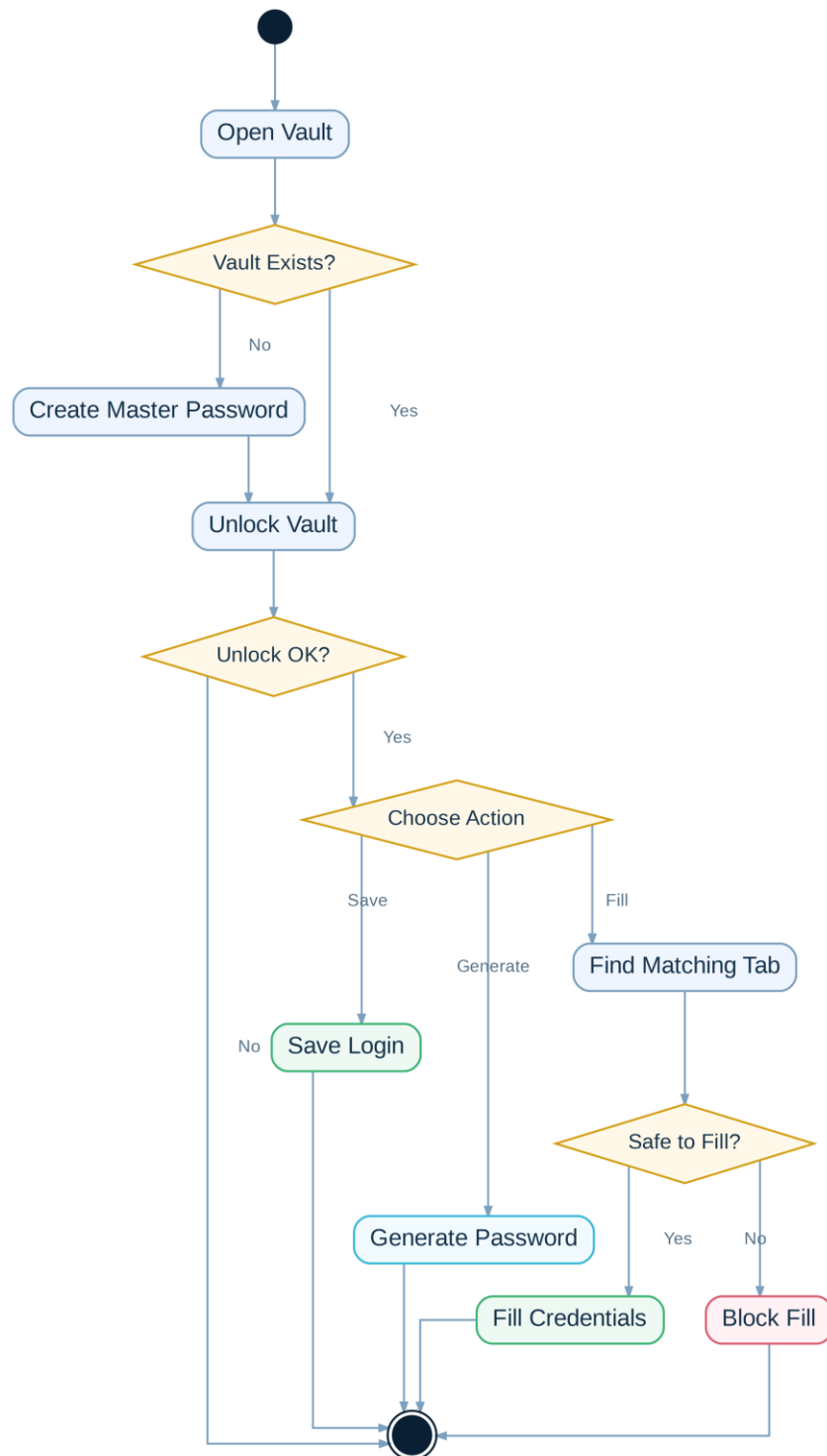


Figure 40: Password Vault Activity Diagram

Vault cryptography remains in the browser. Save and fill are separate policies; fill is subject to destination and risk checks.

4.2.4 Sequence Diagram Design

The sequence diagrams emphasize the distribution of responsibility between browser components and backend services. The extension does not directly expose database credentials or model files to web pages. Gmail uses the background service worker for the privileged cross-origin request, while report generation uses an authenticated user-bound receipt to prevent insecure direct object references.

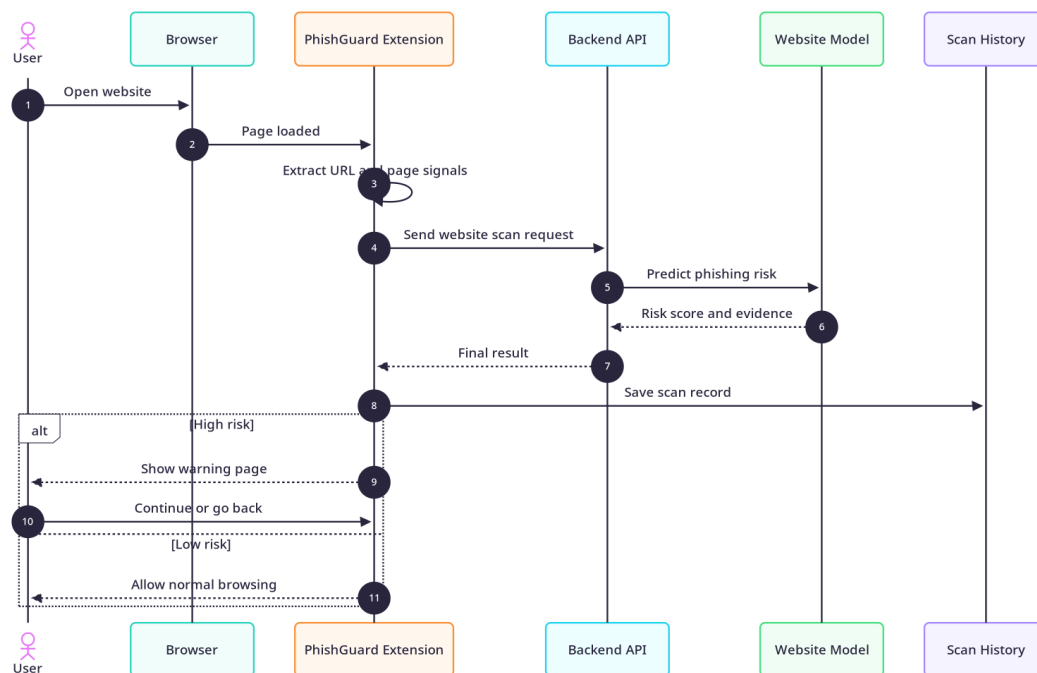


Figure 41: Website Scan Sequence Diagram

PhishGuard sends the current URL to the backend for model and contextual analysis, records the final risk, and shows a warning only when the intervention threshold is reached.

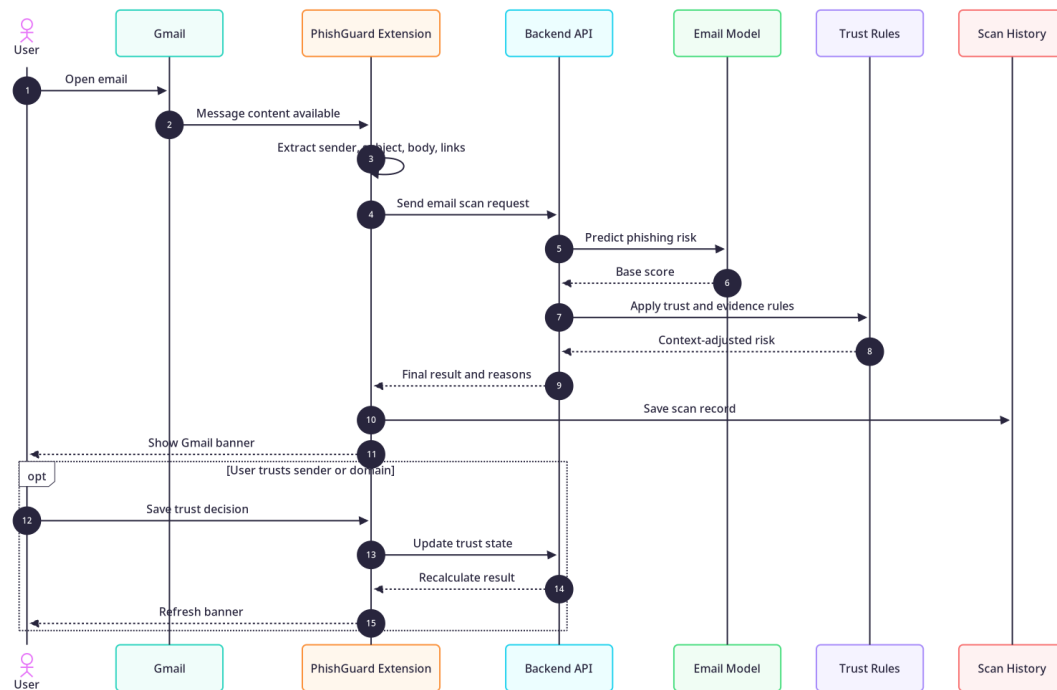


Figure 42: Email Scan Sequence Diagram

When a Gmail message is opened, the content script extracts the visible sender, subject, body, and links and submits them for analysis. The backend obtains the frozen email-model probability, applies contextual evidence and any permitted personal trust signal, then returns the final risk and explanation. The result is recorded in scan history and shown in the Gmail banner; trust changes can personalize the result without altering the underlying model probability.

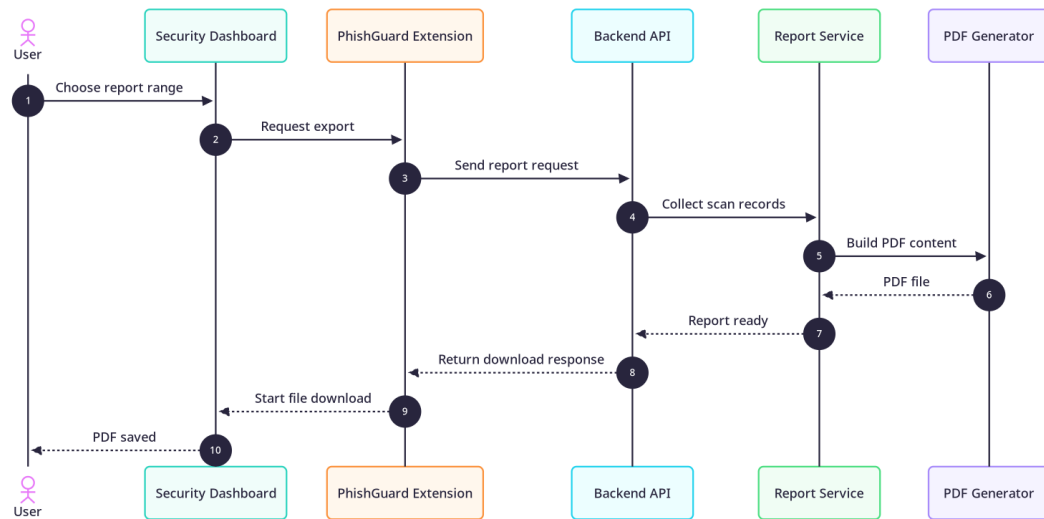


Figure 43: PDF Security Report Sequence Diagram

The report sequence starts when the user chooses an individual scan or date range in the Security Dashboard. The authenticated request reaches the backend, which validates ownership and the requested scan records before generating the PDF. The completed report is returned to the browser as a download, ensuring that only records belonging to the signed-in user are exported.

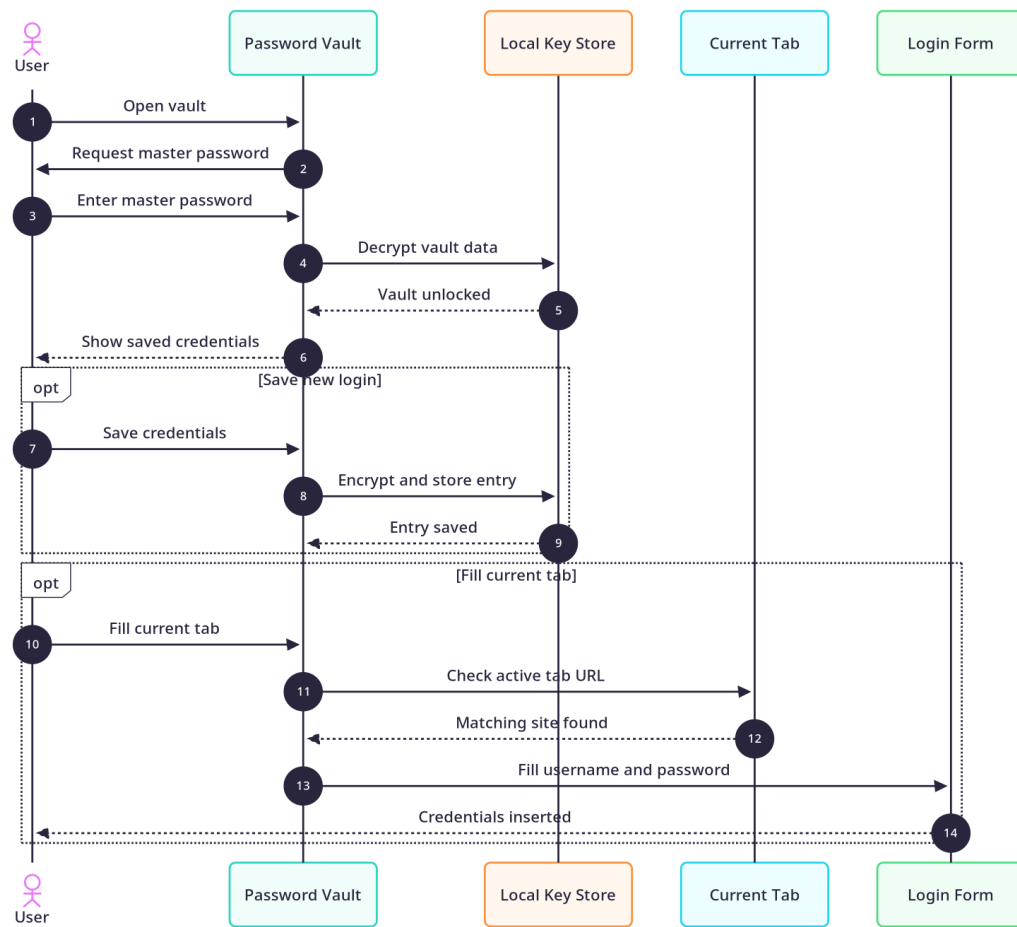


Figure 44: Password Vault Sequence Diagram

The vault sequence keeps sensitive cryptographic work inside the browser. After the user enters the master password, PhishGuard derives the required key material and decrypts the encrypted vault profile and entries locally; the backend stores and returns ciphertext only. New credentials are encrypted before synchronization, and Fill Current Tab is permitted only after destination and risk checks succeed.

4.2.5 Security, Trust, and Model Release Design

Several explicit trust boundaries are implemented. Web pages cannot choose arbitrary backend request origins; the extension uses a central production API configuration. Temporary MFA challenges are not normal authenticated sessions and cannot call protected scan routes. Trusted sender and website decisions remain user-specific personalization signals rather than modifications to the saved machine-learning models. Password-vault encryption keys are not sent to Flask, and the backend accepts ciphertext-shaped vault payloads only. The same design section also separates model training and release from runtime contextual analysis so that evaluation results remain traceable to the frozen supervised-learning artifact.

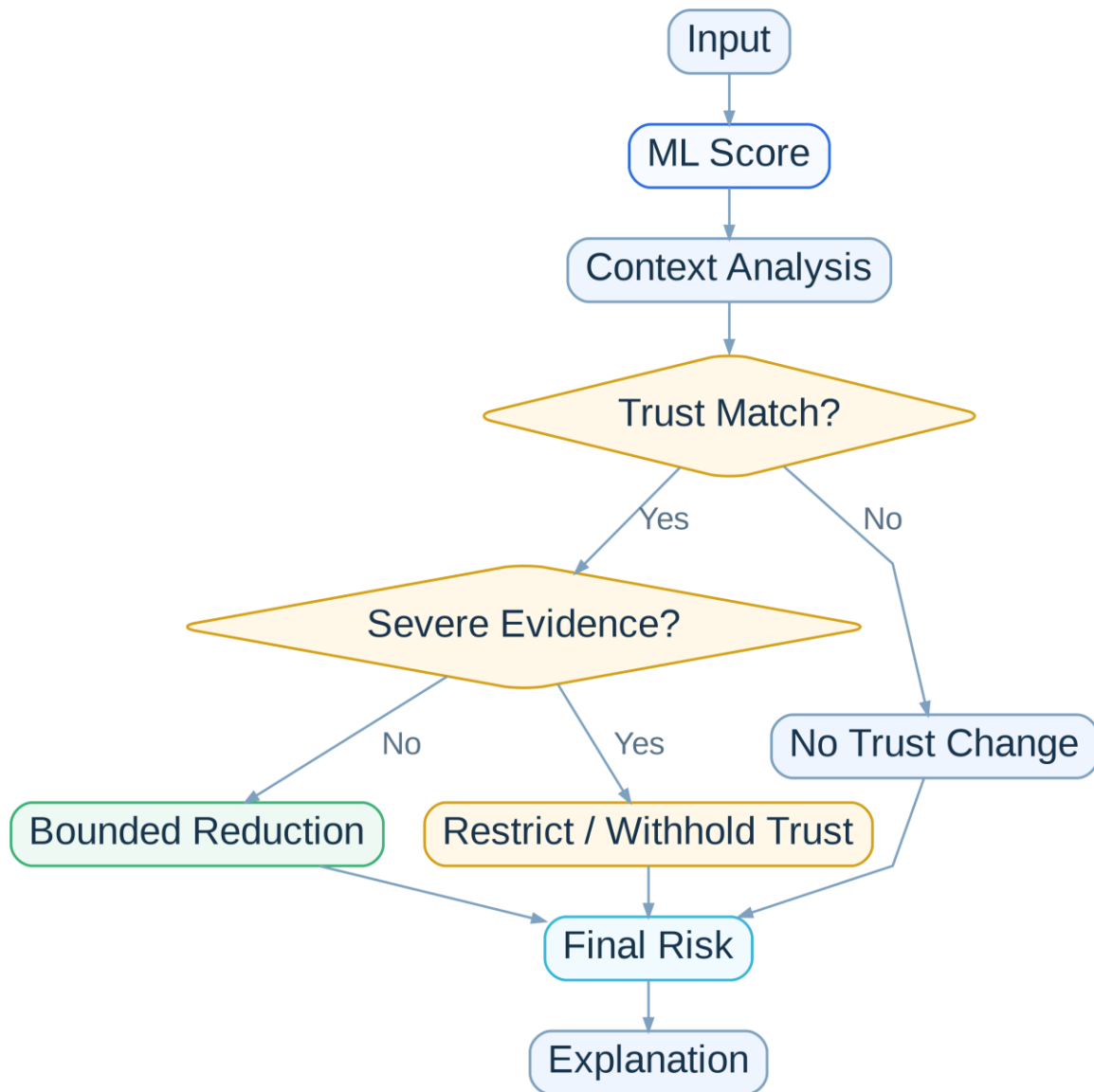


Figure 45: PhishGuard Risk Decision Flow

The risk design preserves the raw supervised-learning probability separately from contextual security analysis and user-specific trust. Personal trust is therefore a bounded personalization layer rather than hidden model retraining.

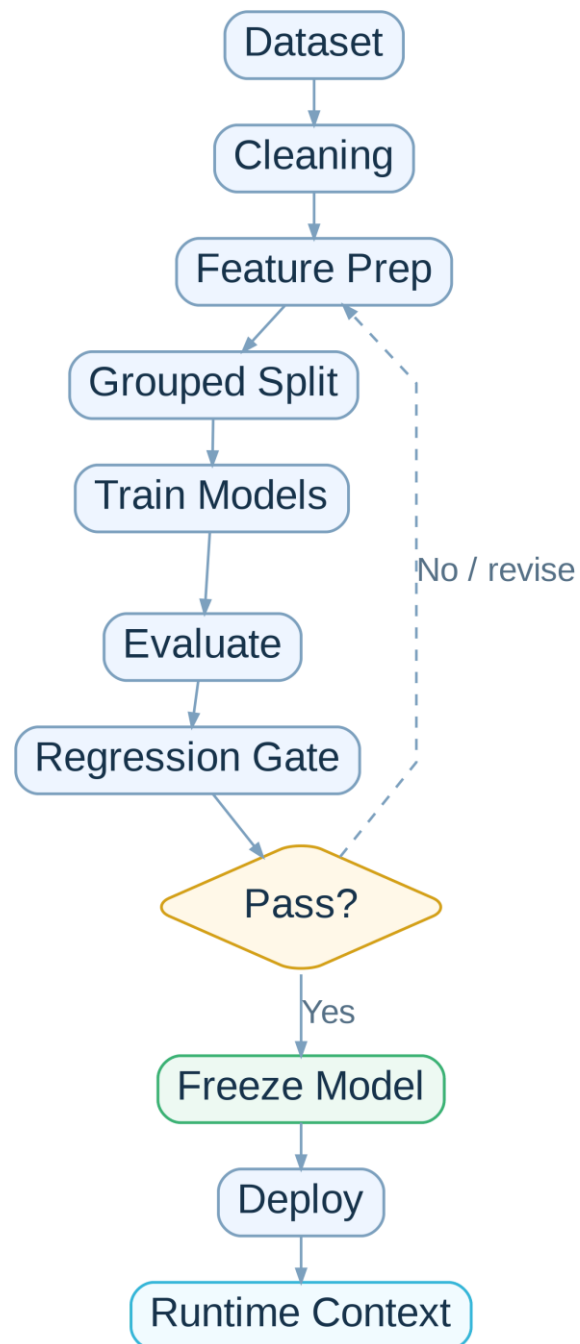


Figure 46: Machine Learning Development and Release Pipeline

The model-development pipeline is kept deliberately concise in the design view. Labelled datasets are cleaned, converted into the required feature representation, separated into training and validation partitions, evaluated against performance and regression criteria, and only then frozen for deployment. Runtime contextual analysis remains a separate layer so deployment logic does not silently alter the trained model artifact.

Boundary	Implementation Control	Security Purpose
Browser page → extension	Only supported content-script/message interfaces are accepted	Prevents arbitrary page code from controlling privileged extension requests
Extension → Render API	HTTPS, explicit API origin, bearer session, CORS allow-list	Protects authenticated communication and narrows origins
Password stage → MFA stage	Temporary challenge token; no scan access	Prevents password-only access
User trust → detector	Trust adjustment is separate from raw model/context score	Preserves auditability and prevents hidden retraining
Vault → backend	Ciphertext, IV, wrapped-key data only	Maintains client-side encryption boundary
Dashboard → report	Signed user-bound receipt + authenticated session	Prevents report ID tampering/IDOR

Table 10: Major PhishGuard Security Trust Boundaries

4.3 Database Design

4.3.1 Entity Relationship Design

The deployed backend uses SQLAlchemy models that are compatible with both development SQLite and production PostgreSQL. The production Render deployment uses Neon PostgreSQL. The database contains account and security state rather than a centralized copy of complete browsing history. The extension history and personal trust lists remain user-scoped browser data, while the password vault stores only encrypted payloads and key-wrapping metadata on the server.

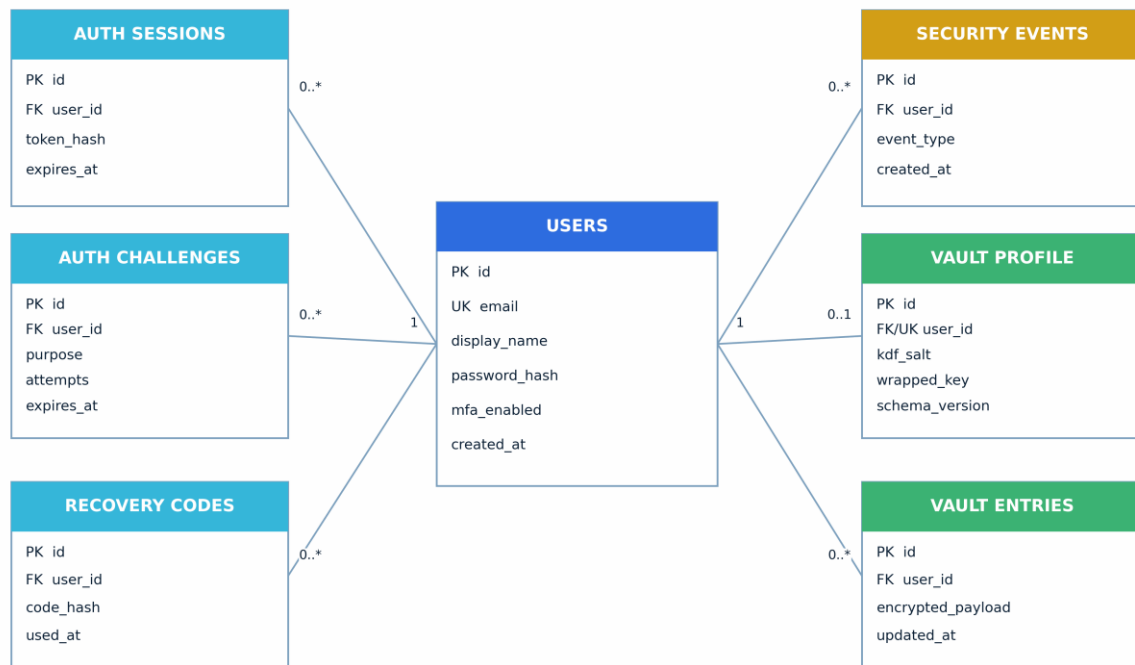


Figure 47: PhishGuard Entity Relationship Diagram

All persistent authentication and vault entities are owned through the users table. Foreign-key cascade deletion is used for user-owned data. VaultProfile is one-to-one with a user, while recovery codes, sessions, challenges, security events, and encrypted entries can have multiple rows per user.

4.3.2 Database Table Structure

Table	Purpose	Principal Fields
users	Account identity, password hash, MFA state, progressive MFA lockout state and timestamps	id, display_name, email, password_hash, mfa_secret_encrypted, mfa_enabled, mfa_failed_attempts, mfa_lockout_level, mfa_locked_until, created_at, last_login_at
recovery_codes	One-use account recovery codes stored only as Argon2id hashes	id, user_id, code_hash, created_at, used_at
auth_sessions	Expiring and revocable normal sessions; only token hashes are persisted	id, user_id, token_hash, created_at, expires_at, last_seen_at, revoked_at
auth_challenges	Short-lived password-verified challenges for MFA/reset workflows	id, user_id, token_hash, purpose, attempts, expires_at, locked_until, consumed_at
security_events	User-owned privacy-safe security events such as MFA lockout	id, user_id, event_type, lockout_level, lockout_seconds, created_at, acknowledged_at
vault_profiles	Opaque client-encryption profile and wrapped vault data key	id, user_id, kdf_algorithm, kdf_salt, kdf_parameters, wrapped_vault_key, wrapped_key_iv, schema_version
vault_entries	Browser-encrypted credential records	id, user_id, encrypted_payload, iv, schema_version, created_at, updated_at

Table 11: Backend Database Tables

4.3.3 Database Portability and Migration Design

Configuration normalizes SQLite and PostgreSQL URLs into the correct SQLAlchemy dialect and enables PostgreSQL through psycopg in production. Alembic migrations are used for schema evolution so deployed data is not recreated when new features are introduced. The vault migration adds encrypted vault tables, while the later security migration adds persistent MFA lockout and security-event state. The current final implementation does not require a database table for every browser preference because user-specific extension history and trust state are deliberately local.

4.4 Interface Design

Before the final PhishGuard interfaces were implemented, low-fidelity wireframes were prepared to define information hierarchy, navigation, control placement, and the separation between quick browser-extension actions and full-page security functions. The wireframes intentionally use monochrome boxes, text lines, image placeholders, and CTA-style controls rather than the final dark visual theme. This keeps Section 4.4 focused on interface planning, while the completed system interfaces are presented later in Section 4.5 as implementation evidence.

Authentication and MFA Wireframe

LOW-FIDELITY WIREFRAME

PHISHGUARD

Sign In

Email

Password

SIGN IN

After password verification:

6-digit MFA code

VERIFY **RECOVERY CODE**

PhishGuard

SECURITY ILLUSTRATION

Figure 48: Authentication and MFA Wireframe

The authentication and MFA wireframe establishes the security sequence that a user must complete before gaining access to protected PhishGuard functions. The first stage is intentionally limited to the account email and password so the primary credential step remains familiar and uncluttered. Only after those credentials are accepted does the interface introduce the second factor, either a TOTP code from an authenticator application or an approved recovery path. This separation makes it clear that password verification alone is not equivalent to a completed login. The wireframe also reserves space for validation messages, recovery guidance, and progression controls so authentication errors can be explained without exposing sensitive information. The final implementation follows this planned structure through password verification, mandatory MFA enrolment, recovery codes, and MFA-protected sessions, demonstrating how the early interface design was translated into the completed account-security workflow.

Main Extension Popup Wireframe

LOW-FIDELITY WIREFRAME

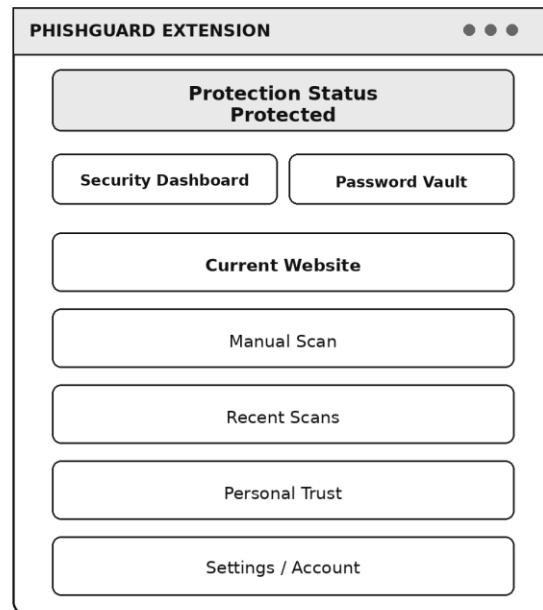


Figure 49: Main Extension Popup Wireframe

The main popup wireframe defines the information hierarchy for the small browser-extension window, where screen space is limited and the most frequently used controls must remain immediately accessible. Protection status and the main navigation shortcuts are positioned near the top so the user can quickly confirm whether PhishGuard is active and open the Security Dashboard or Password Vault when deeper interaction is required. The centre of the layout is reserved for the current website, scan status, risk result, and manual scanning because these are the core day-to-day protection functions. Less frequent functions such as recent history, personal trust, settings, and account controls are placed lower in the interface to avoid overwhelming the first view. This arrangement influenced the final authenticated popup by prioritising immediate scanning and protection feedback while keeping secondary security and management features available through a compact, scrollable structure.

Website Scan and Warning Wireframe

LOW-FIDELITY WIREFRAME

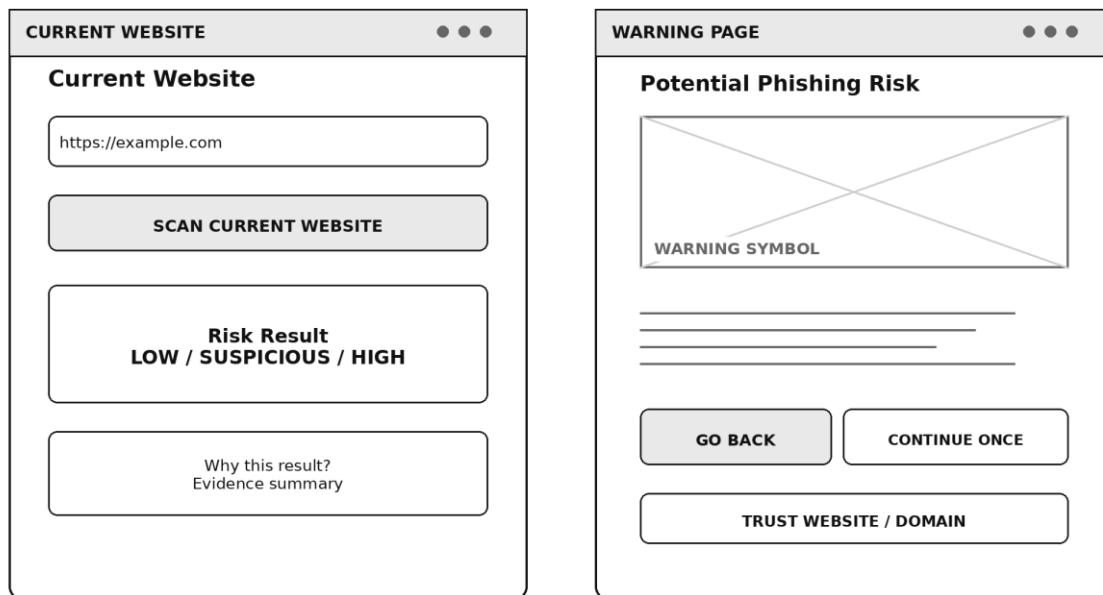
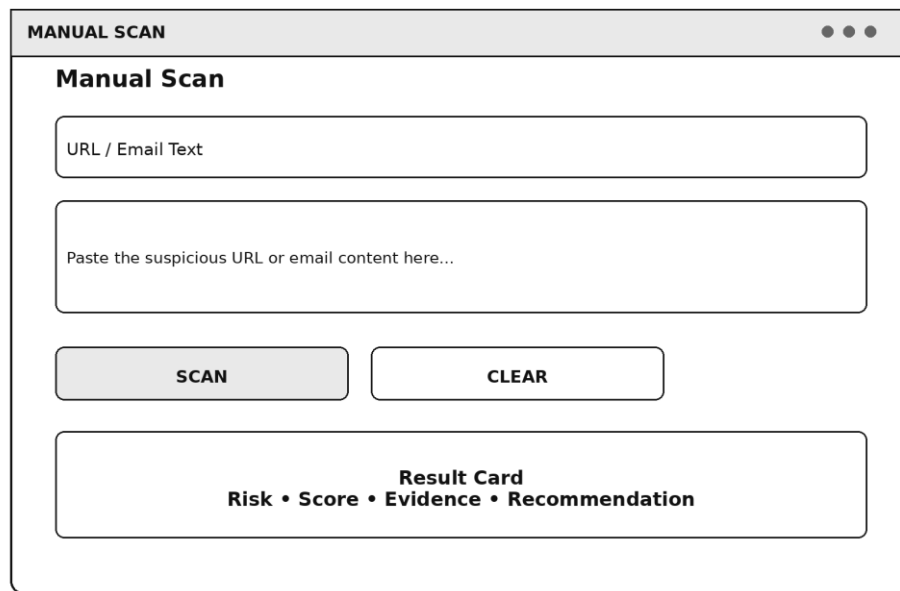


Figure 50: Website Scan and Warning Wireframe

The website scan and warning wireframe models two different interaction states rather than treating every website result in the same way. During normal browsing, the popup shows the active URL, scan action, risk result, and concise supporting evidence so low-risk or non-blocking results can be reviewed without interrupting the user's activity. When the final risk reaches the intervention threshold, the design moves the decision into a dedicated warning page that is more difficult to overlook. The warning state separates the safest action, Go Back, from the temporary Continue Once option and the more persistent Trust Website decision. This distinction was planned to prevent a quick override from becoming an unintended permanent exception. The final implementation preserves this structure and also keeps raw model output, contextual evidence, and personal trust logically separate so the warning can be both actionable and explainable.

Manual Scan Wireframe

LOW-FIDELITY WIREFRAME



The wireframe shows a window titled "MANUAL SCAN" with a standard macOS-style title bar (three dots). Inside the window, the title "Manual Scan" is at the top. Below it is a text input field labeled "URL / Email Text". Underneath that is a larger text area with the placeholder text "Paste the suspicious URL or email content here...". Below the text area are two buttons: "SCAN" and "CLEAR". At the bottom of the window is a large rectangular box labeled "Result Card" which contains the text "Risk • Score • Evidence • Recommendation".

Figure 51: Manual Scan Wireframe

The manual-scan wireframe supports proactive checking when the user wants to assess suspicious content before opening it in the browser. A focused input area allows a URL or supported email text to be pasted directly, while clearly separated Scan and Clear controls keep the interaction simple. The result area is positioned within the same view so the user receives the risk level, explanation, and relevant evidence without navigating to another page. This layout is important because manual scanning should use the same risk language as automatic protection rather than appearing to be a separate or less reliable feature. The wireframe therefore emphasises consistency, minimal interaction steps, and readable feedback. In the completed system, the manual website scan reuses the backend detection and contextual-analysis pipeline, allowing copied links to be assessed before navigation and ensuring that manual and automatic scanning produce comparable explanations and risk terminology.

Gmail Email Protection Wireframe

LOW-FIDELITY WIREFRAME

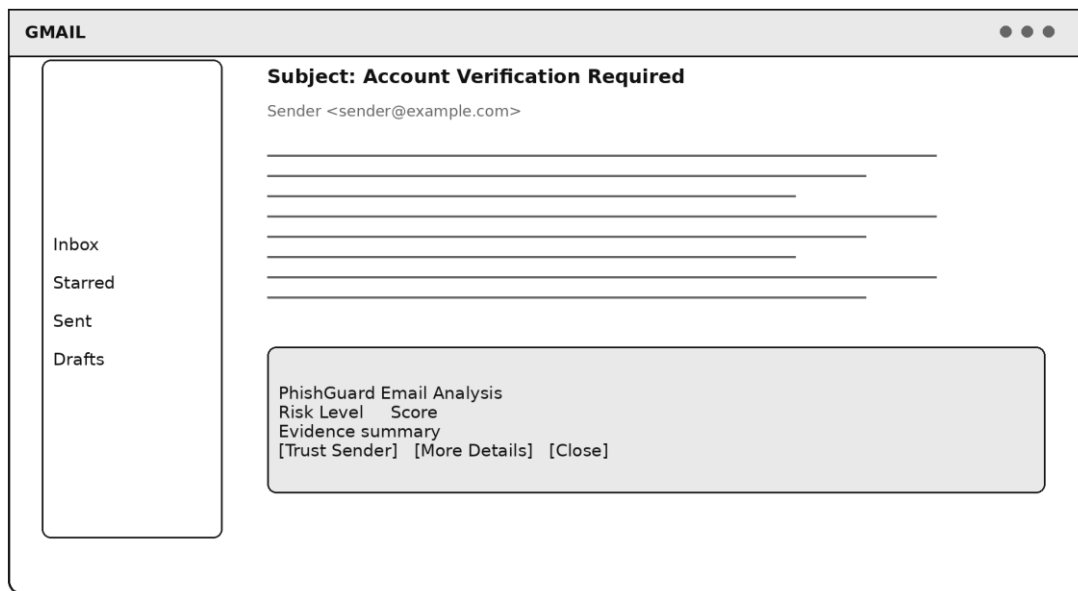


Figure 52: Gmail Email Protection Wireframe

The Gmail email-protection wireframe places PhishGuard feedback directly inside the message-reading context so the user can evaluate risk while the email is still visible. The planned banner contains a concise message summary, the risk assessment, an expandable explanation area, and actions for sender trust, suspicious reporting, or dismissal. Keeping these controls within Gmail reduces the need to switch to the extension popup or Security Dashboard for every message. The design also supports the privacy goal of analysing only the currently opened browser-visible message rather than requiring unrestricted mailbox access. This wireframe later developed into the implemented Gmail banner, which follows message navigation, avoids stale results when users switch emails quickly, and keeps personal trust separate from the frozen email-model probability. The final result therefore preserves the original design objective of providing immediate, understandable, and context-sensitive email protection without interrupting normal webmail use.

Security Dashboard Wireframe

LOW-FIDELITY WIREFRAME

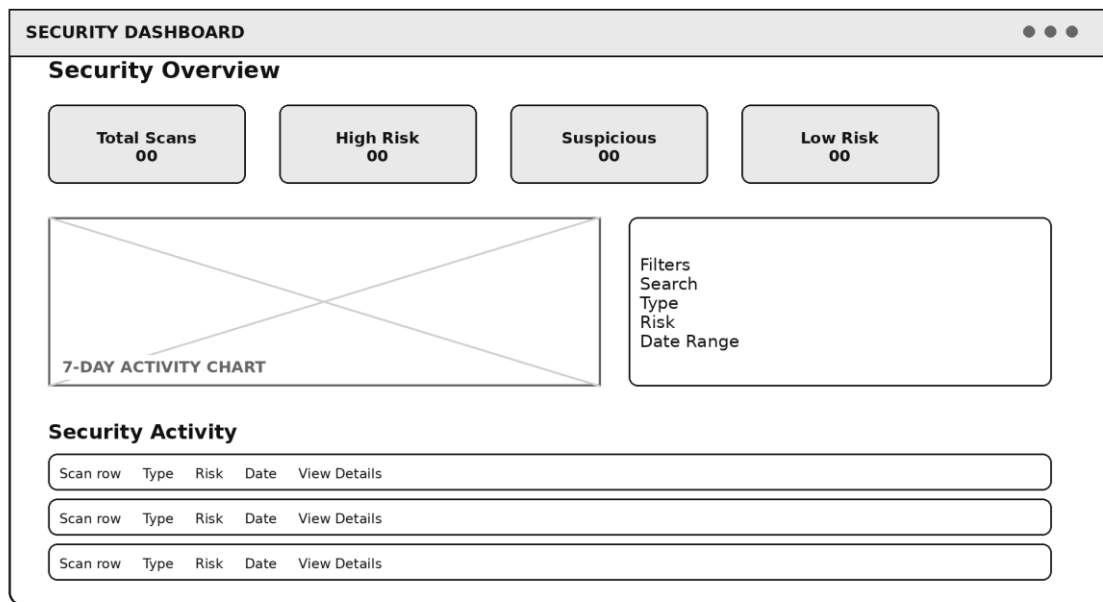


Figure 53: Security Dashboard Wireframe

The Security Dashboard wireframe is designed as the detailed review area for information that cannot be presented effectively inside the compact extension popup. Summary cards and a recent trend view appear first to give the user an immediate overview of scan activity and risk distribution. Search, filtering, and individual scan records are placed below this overview so the interface supports a clear drill-down path from general status to specific evidence. The planned structure also leaves room for scan details, security events, personal trust, and report actions without mixing them into a single dense table. This hierarchy influenced the final dashboard, where routine scan history remains user-scoped while users can still search activity, inspect reasons behind decisions, manage trust, review account-security events, and export reports. The design therefore balances quick situational awareness with deeper investigation while avoiding unnecessary central collection of complete browsing history.

Personal Trust, Settings and Account Security Wireframe

LOW-FIDELITY WIREFRAME

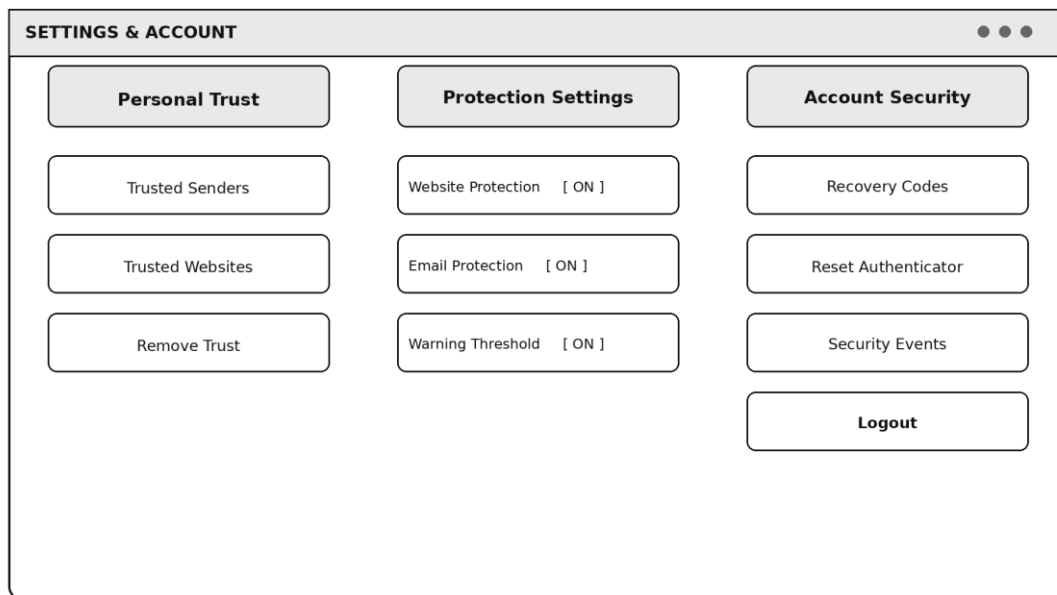
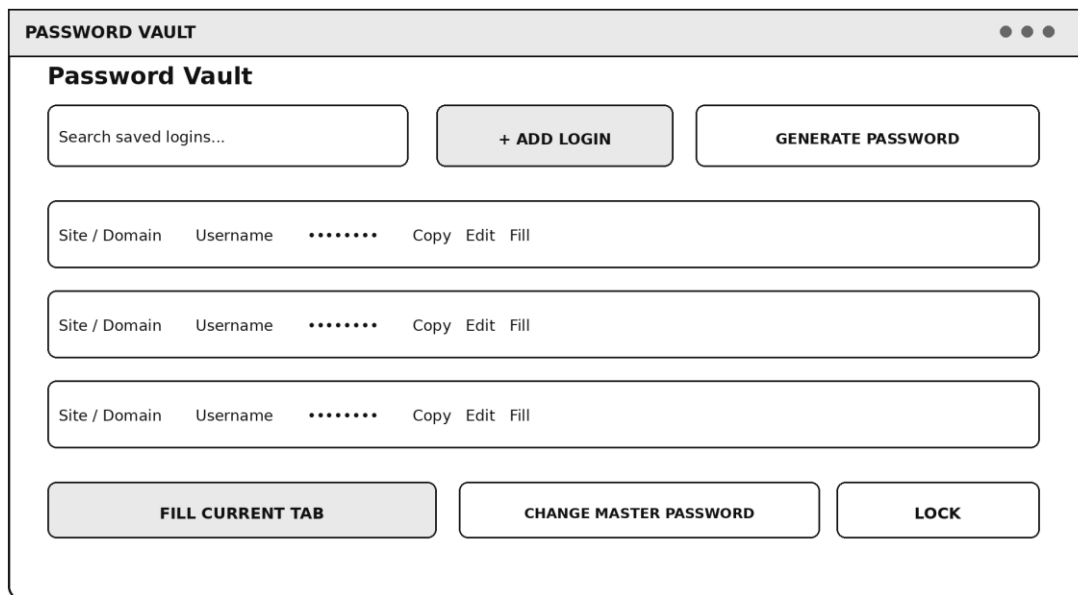


Figure 54: Personal Trust, Settings and Account Security Wireframe

This wireframe separates three types of configuration that have different security meanings: personal trust decisions, protection settings, and account-security actions. Trusted senders, domains, and websites are grouped together so users can review and revoke personalised exceptions without confusing them with global detection controls. Protection settings are placed in a separate area because enabling or disabling website or email protection changes system behavior more broadly than trusting one destination or sender. Account-security functions such as recovery-code management, authenticator reset, security-event review, password changes, and logout are kept within their own section because they affect authentication rather than phishing classification. This separation reduces the risk of accidental configuration changes and improves mental clarity for non-technical users. The final interface follows the same principle by making trust reversible, keeping it separate from the trained models, and treating sensitive account operations as deliberate security actions.

Password Vault Wireframe

LOW-FIDELITY WIREFRAME



The wireframe shows a window titled "PASSWORD VAULT" with a standard macOS-style title bar (three dots in the top right). Inside the window, the title "Password Vault" is at the top left. Below the title, there are three main sections. The first section contains a search bar labeled "Search saved logins..." and two buttons: "+ ADD LOGIN" and "GENERATE PASSWORD". The second section contains three identical credential cards. Each card has a "Site / Domain" label, a "Username" label, a password field represented by seven dots, and three action buttons: "Copy", "Edit", and "Fill". The third section contains three buttons: "FILL CURRENT TAB", "CHANGE MASTER PASSWORD", and "LOCK".

Figure 55: Password Vault Wireframe

The Password Vault wireframe uses a dedicated full-page interface because secure credential management requires more space and more deliberate interaction than the extension popup can provide. The design reserves separate areas for searching saved entries, adding a login, generating a strong password, viewing credential cards, filling the current tab, changing the vault master password, and locking the vault. These actions are deliberately separated so saving, viewing, and filling credentials do not become one automatic operation. The planned layout also supports the security boundary between account authentication and vault access: being signed in to PhishGuard does not automatically expose stored credentials. In the completed implementation, the vault must be unlocked independently, encryption and decryption occur in the browser, and Fill Current Tab remains subject to destination and risk checks. The wireframe therefore anticipates both usability requirements and the client-side cryptographic controls used by the final system.

Security Report Export Wireframe

LOW-FIDELITY WIREFRAME

The wireframe shows a window titled "EXPORT SECURITY REPORT" with three dots in the top right corner. Inside the window, the title "Export Security Report" is at the top. Below it are three buttons: "Individual Scan", "Last 7 Days", and "Custom Range". Under these buttons are two input fields: "Start Date" and "End Date". Below the date fields is a large, prominent button labeled "GENERATE PDF". At the bottom of the window is a rectangular area labeled "PDF PREVIEW" in the bottom-left corner, which is crossed out with a large 'X' to indicate it is a placeholder for the report output.

Figure 56: Security Report Export Wireframe

The security-report export wireframe provides several reporting paths because users may need evidence for one scan or for activity across a period of time. The design separates an individual-scan report from preset recent periods and a custom date-range option, allowing the user to choose the amount of evidence required without navigating through multiple report pages. The Generate PDF action is visually prominent, while the report preview area communicates that the output will contain a structured security record rather than a simple screenshot of the dashboard. Date inputs and report-selection controls are kept close together so invalid or unclear ranges can be corrected before generation. The completed reporting workflow follows this design by validating the authenticated user and report range, generating individual or range-based PDFs, and presenting risk, evidence, and explanation information without including reusable authentication secrets. This makes reporting a controlled evidence-export function rather than unrestricted history download.

4.5 Implementation

Section 4.5 presents a reduced set of representative system screenshots arranged in the order a user would encounter the completed PhishGuard system. The flow begins with the public website, direct download, extraction and Chrome Load unpacked installation, then continues through account creation and multi-factor authentication, website protection, dashboard/reporting functions, and the encrypted password vault. Redundant interface states from earlier drafts have been removed so that each screenshot contributes distinct implementation evidence.

4.5.1 Distribution and Extension Installation

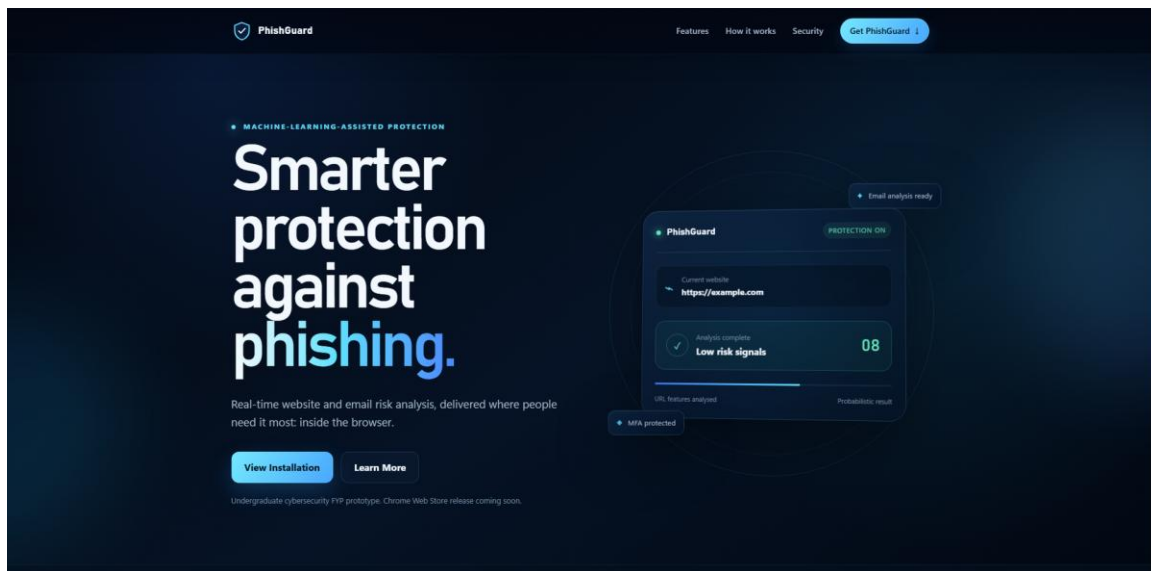


Figure 57: PhishGuard Public Website

The PhishGuard public website acts as the first contact point for users before the browser extension is installed. The landing page introduces the project as machine-learning-assisted protection against phishing websites and phishing emails and presents the main value proposition in non-technical language. Navigation links provide access to the feature, workflow, security, and installation information, while the Get PhishGuard and View Installation controls direct the user toward the current installation process. This page is intentionally separate from the authenticated extension so that installation guidance can be accessed without creating an account first. The interface also identifies the system as an undergraduate cybersecurity FYP prototype and indicates that a Chrome Web Store release is still planned, which accurately reflects the current distribution stage of the project.

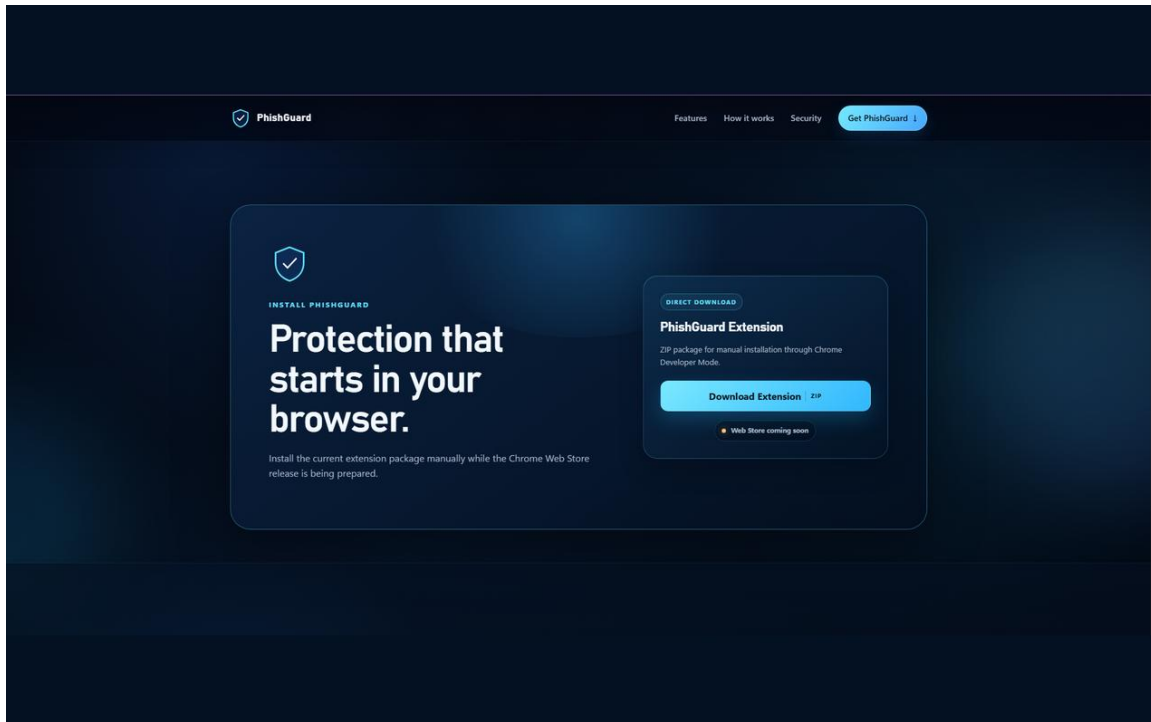


Figure 58: PhishGuard Extension Download and Installation Instructions

The installation section provides the current direct-download distribution path for PhishGuard. Instead of requiring an activation or beta-access code, the user can download the extension ZIP package directly from the website and follow the manual Chrome installation instructions. The page explains that the package must be extracted before it can be loaded through Chrome Developer Mode and clearly labels the Web Store release as a future distribution option. Keeping the installation instructions on the public website reduces confusion because the user completes the download and setup steps before interacting with account registration, MFA, or phishing protection. This also reflects the final simplified deployment design in which normal PhishGuard authentication is responsible for user access rather than a separate pre-registration access layer.

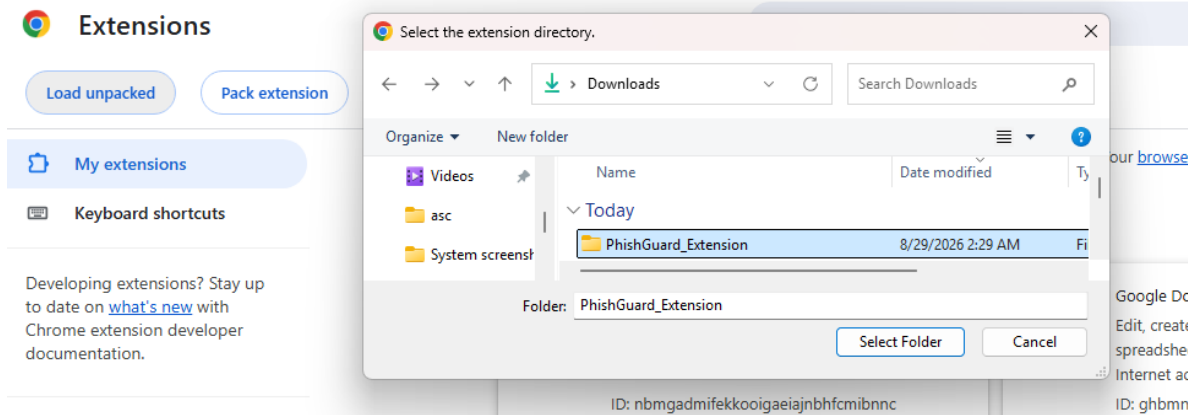


Figure 59: Loading the Unpacked PhishGuard Extension in Chrome

This screenshot records the final manual installation step after the PhishGuard ZIP package has been downloaded and extracted. The user opens `chrome://extensions`, enables Developer mode, selects Load unpacked, and chooses the extracted PhishGuard_Extension directory. Chrome then loads the Manifest V3 extension from that folder and assigns an extension identifier for the local installation. This step is important because the current project is distributed as an unpacked prototype rather than through the Chrome Web Store. Once installation is complete, the user can open the PhishGuard popup and continue to account creation or sign-in; no separate beta activation stage is required in the current system.

4.5.2 Account Registration and Multi-Factor Authentication

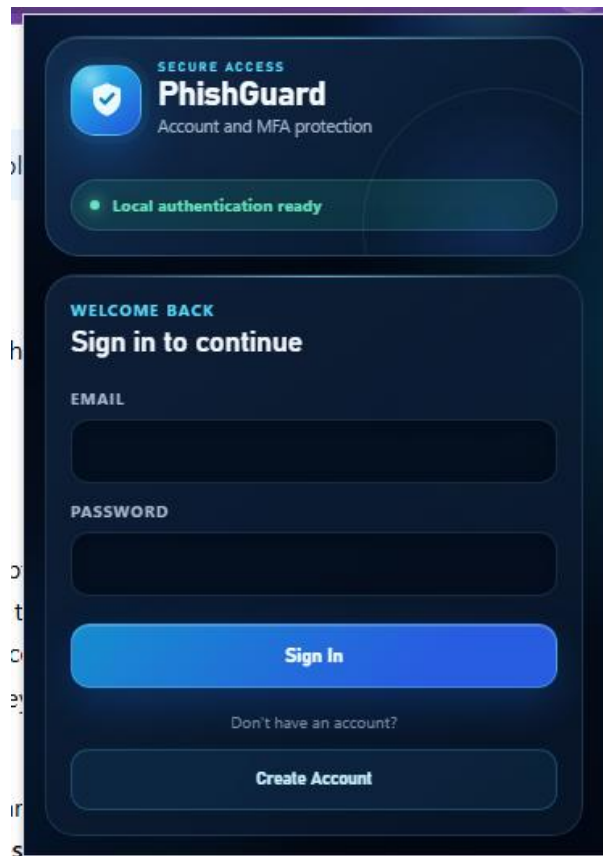


Figure 60: PhishGuard Sign-in Interface

The sign-in interface is the authentication entry point for returning users. The first stage verifies the email address and account password, but successful password verification does not immediately grant access to protected PhishGuard functions. Instead, the system proceeds to the mandatory MFA stage so that possession of the password alone is insufficient to establish an authenticated session. This two-stage flow reduces the impact of a stolen or guessed password and matches the account-security design implemented in the backend. The interface also keeps registration and recovery options accessible without mixing them into the main phishing-scanning workflow, allowing authentication to remain a clearly separated security boundary before the extension exposes user-specific history, trust settings, reports, or vault data.

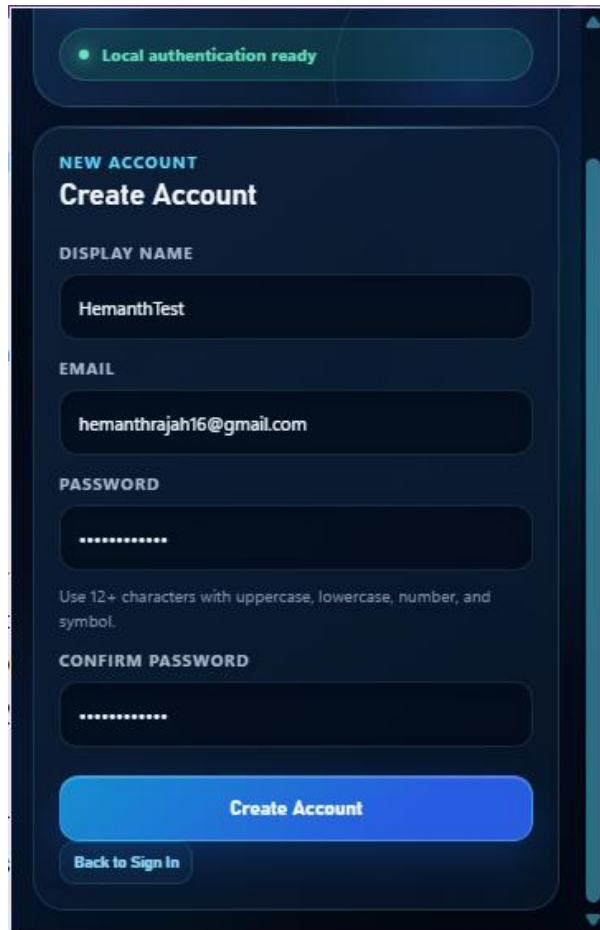
The image shows a mobile application interface for creating a new account. At the top, a green status bar indicates 'Local authentication ready'. Below this, the section is titled 'NEW ACCOUNT' and 'Create Account'. There are four input fields: 'DISPLAY NAME' with the value 'HemanthTest', 'EMAIL' with the value 'hemanthrajah16@gmail.com', 'PASSWORD' with masked characters, and 'CONFIRM PASSWORD' also with masked characters. A password requirement note states: 'Use 12+ characters with uppercase, lowercase, number, and symbol.' At the bottom, there is a large blue 'Create Account' button and a smaller 'Back to Sign In' button.

Figure 61: PhishGuard Account Creation Interface

The account-creation interface collects the minimum identity information required for the authenticated PhishGuard features: a display name, email address, and account password. Password requirements are communicated directly beside the form so that weak credentials can be rejected before an account is created. After successful registration, the user is not taken directly into the main extension; the system moves to mandatory TOTP enrolment so that MFA is established as part of the initial account setup. The current workflow therefore allows normal account creation without an additional beta-access field while still enforcing the security controls required by the project. This separation also helps distinguish the account password used for authentication from the independent master password later used to unlock the encrypted Password Vault.

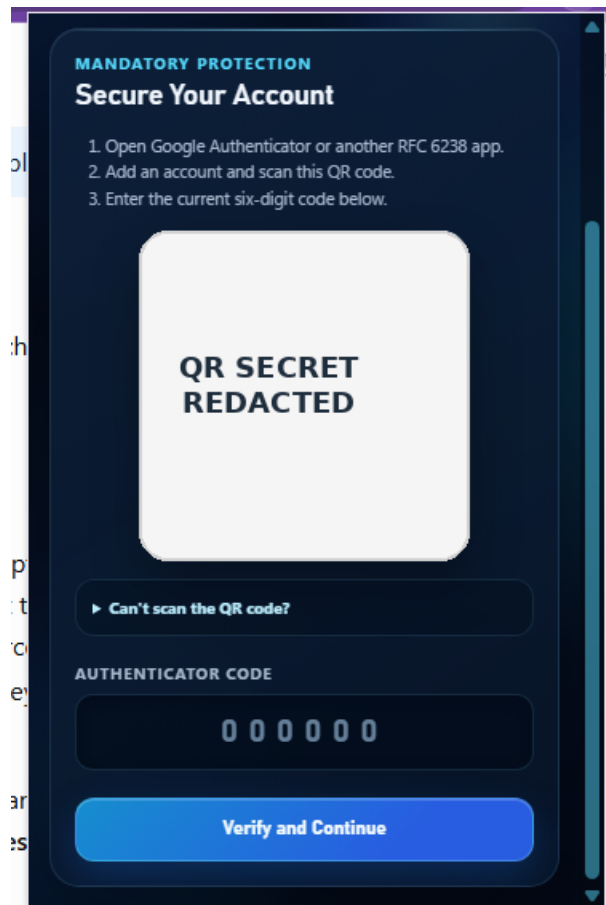


Figure 62: Mandatory MFA Enrollment after Account Creation

Immediately after registration, PhishGuard requires the user to enrol a time-based one-time password authenticator before normal account access is enabled. The enrolment screen provides the information required by a standard authenticator application and then asks the user to confirm possession of the newly configured factor with a valid code. This prevents a newly created account from remaining in a password-only state. The QR enrolment secret has been redacted in the report because disclosure would allow another person to reproduce the same authenticator factor. The implementation therefore demonstrates both the user-facing onboarding flow and the project's treatment of MFA secrets as security-sensitive data that should not be exposed in screenshots, logs, or generated reports.

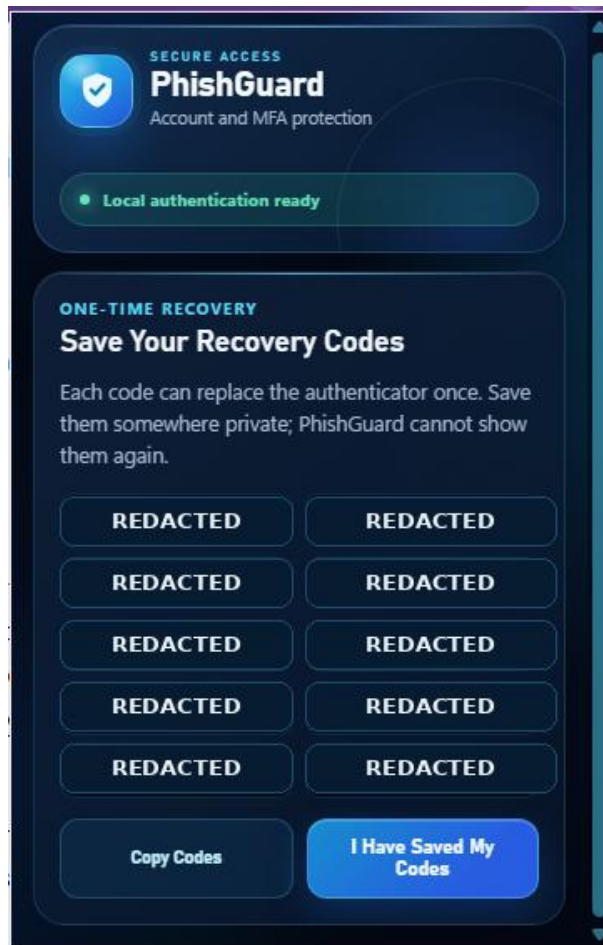


Figure 63: One-Time Recovery Codes after Account Creation

After MFA enrolment is completed, PhishGuard generates a set of one-time recovery codes for use when the authenticator device is unavailable. The recovery-code screen appears during account setup so the user can save the codes before relying on MFA for future sign-ins. Each code is intended for single use, meaning a successfully consumed recovery code cannot be used again. The values shown in the original application were redacted before inclusion in the report because they are alternative authentication credentials and must be protected in the same way as passwords or authenticator secrets. This recovery mechanism improves account availability without weakening the requirement for a second factor, because recovery still requires possession of a separate secret rather than bypassing MFA entirely.

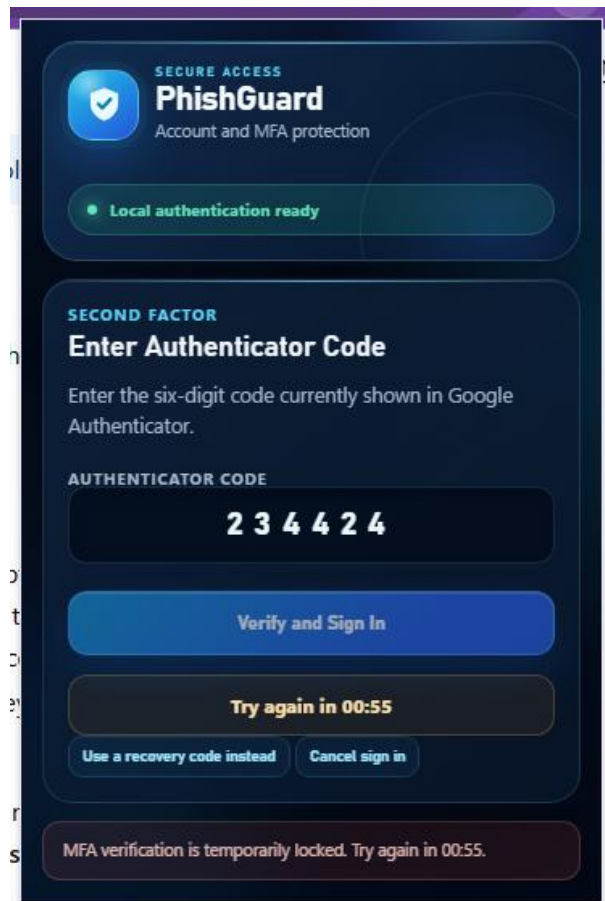


Figure 64: Progressive MFA Lockout after Repeated Incorrect Codes

This interface demonstrates the response to repeated incorrect MFA codes after the account password has already been verified. Rather than allowing unlimited guessing, PhishGuard records failed MFA attempts and applies a temporary lockout once the configured attempt limit is reached. The remaining wait time is shown to the user so the restriction is understandable instead of appearing as an unexplained authentication failure. Subsequent repeated failures can escalate to longer lockout periods, which slows automated guessing while avoiding permanent account lockout caused by an attacker. Importantly, the MFA lockout is applied after successful password verification, so ordinary wrong-password attempts do not directly consume the account's MFA attempt allowance.

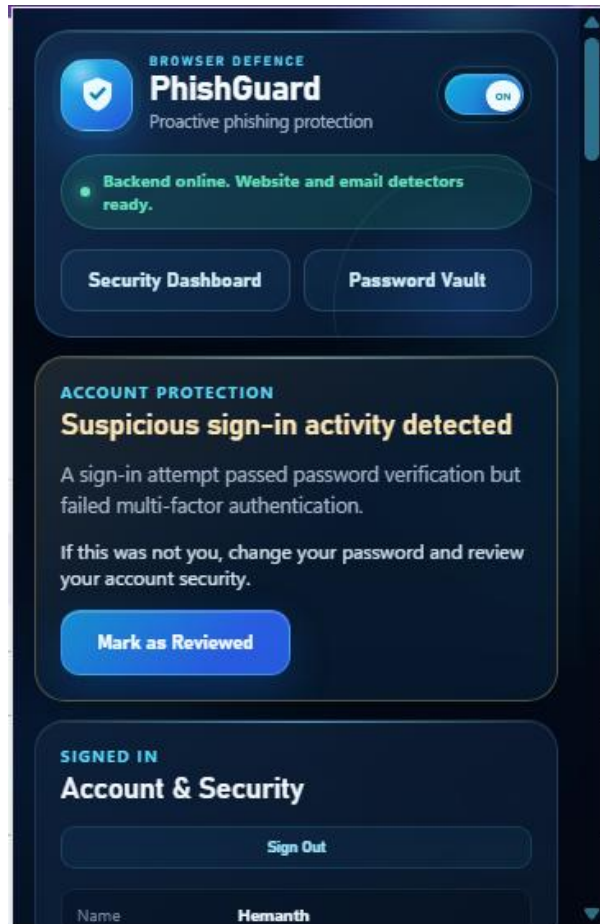


Figure 65: Suspicious Sign-In Activity Security Alert

The suspicious sign-in alert is shown to the legitimate user after a later successful authentication when an earlier attempt passed password verification but failed the MFA stage. This distinction is important: the earlier attempt demonstrated knowledge of the account password, but it did not complete multi-factor authentication and therefore did not obtain a normal authenticated PhishGuard session. The alert explains the security significance in user-friendly language and recommends changing the password if the activity was not recognised. The Mark as Reviewed action allows the user to acknowledge the event without deleting the underlying security record immediately. This feature converts backend authentication telemetry into a practical account-protection signal while avoiding misleading claims that the account itself was successfully accessed.

4.5.3 Website Protection and Manual Scanning

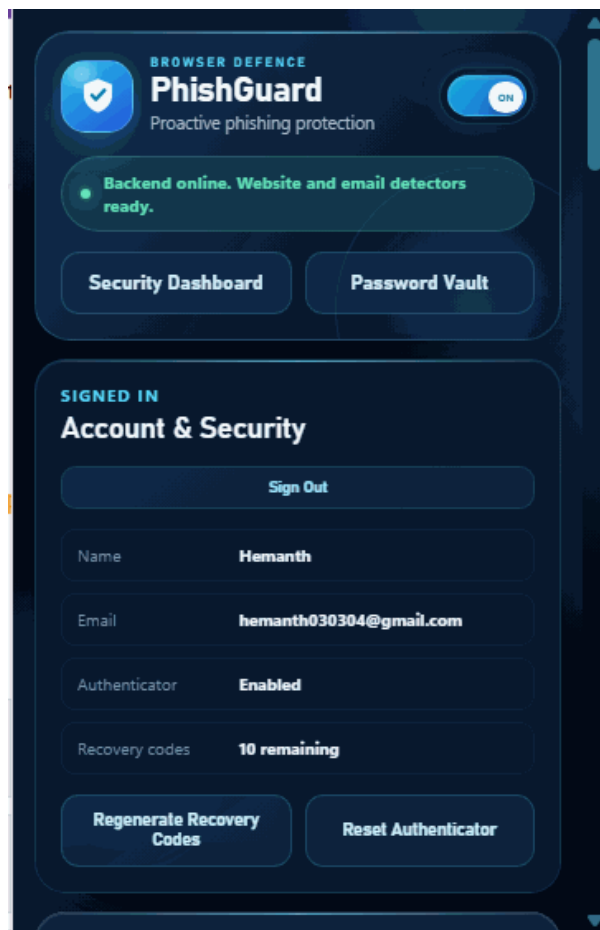


Figure 66: Authenticated PhishGuard Popup

After password and MFA verification are complete, the authenticated popup becomes the main day-to-day control surface for PhishGuard. It displays the current protection status and provides direct navigation to the Security Dashboard and Password Vault while keeping common scanning actions within the compact extension window. The popup also acts as the bridge between the active browser tab, the hosted backend, and the user's local settings or scan history. Account and security controls are available from the same authenticated context, but they are visually separated from the phishing-analysis cards to reduce accidental actions. The design therefore prioritises the most frequent protection functions while still giving the user access to deeper reporting, trust management, settings, and credential-security features when required.

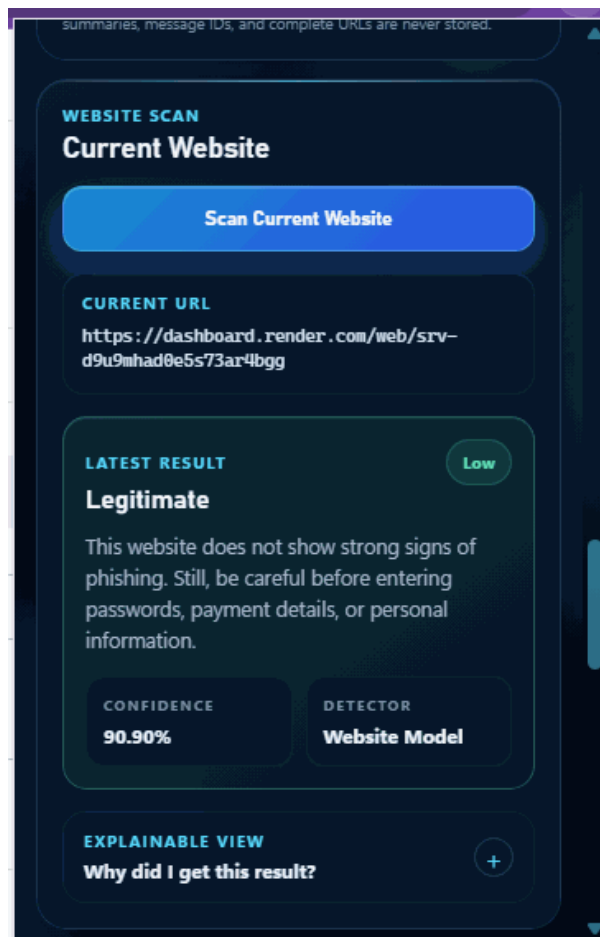


Figure 67: Current Website Scan Result

The current-site result presents the analysis of the active browser tab in a form intended for quick decision making. PhishGuard displays the final contextual risk and the principal evidence contributing to that result rather than exposing only a raw machine-learning probability. Internally, the frozen website model score remains separate from contextual checks and any user-specific trust adjustment, allowing the system to explain why the visible result may differ from the base model output. This separation is important for evaluation because it preserves the original model metrics while allowing practical browser-security signals to affect the user-facing decision. When the result remains below the configured intervention threshold, the user can continue browsing normally while still reviewing the supporting reasons in the extension.

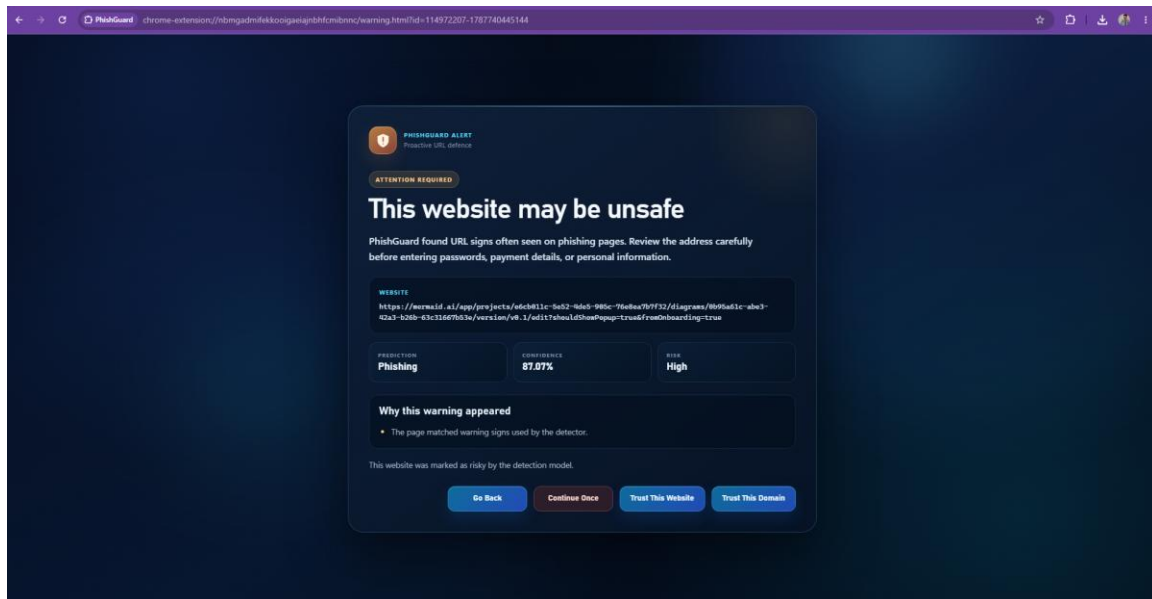


Figure 68: High-Risk Website Warning Page

When the final website risk reaches the configured intervention threshold, PhishGuard interrupts normal navigation with a dedicated warning page. The page communicates that the destination may be unsafe, presents the risk level and supporting evidence, and gives the user explicit choices instead of silently blocking the page. Go Back supports the safest response, while Continue Once allows the user to proceed temporarily when there is a legitimate reason to do so. Separating the warning page from the popup makes the intervention difficult to overlook at the moment a risky navigation decision is being made. The design therefore combines automated detection with informed user control rather than treating the classifier output as an irreversible access-control decision.

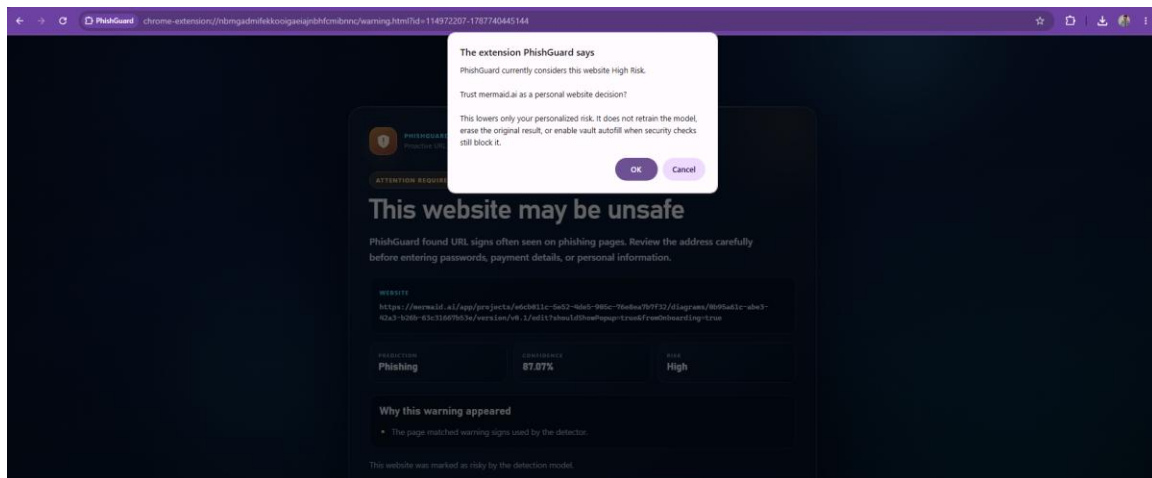


Figure 69: Trust Website Confirmation from the Warning Page

Persistent website trust is treated as a deliberate personal security decision rather than as an extension of the Continue Once action. When the user chooses to trust a warned destination, PhishGuard asks for confirmation before saving the trust rule. This additional step reduces the chance that a persistent trust decision is created accidentally while responding quickly to a warning. Saved trust is user-specific and affects the contextual handling of matching websites; it does not retrain the Random Forest model or change the raw model probability. Severe evidence can also restrict how much trust is allowed to reduce the final risk, which prevents a previously trusted destination from automatically overriding strong phishing indicators.

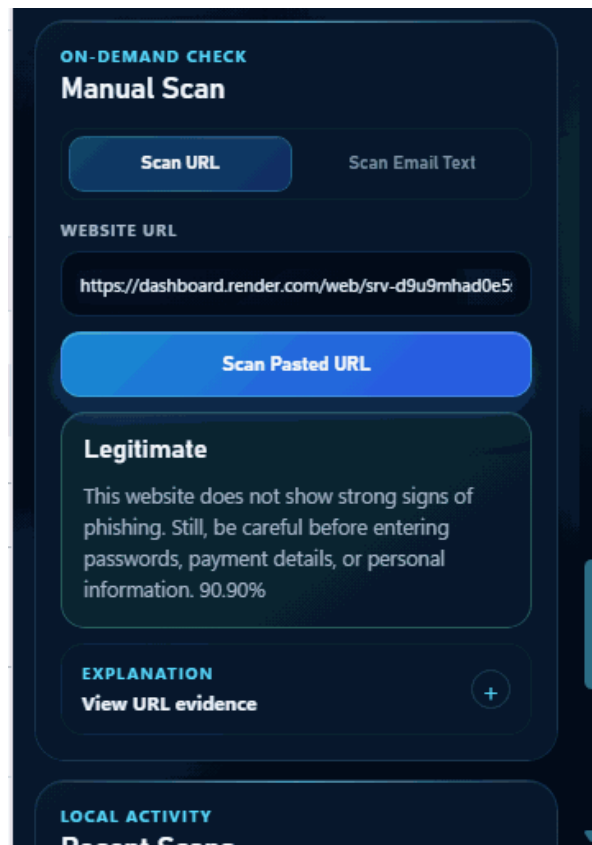


Figure 70: Manual Website Scan Result

Manual website scanning allows the user to analyse a URL without first navigating to it. The manual workflow is useful for links copied from messages, documents, social media, or other sources where the user wants a second opinion before opening the destination. The result uses the same risk terminology and evidence structure as the automatic active-tab scan, so the interpretation remains consistent across both workflows. The backend still performs the same website-model and contextual analysis rather than using a simplified manual-only classifier. This reduces duplicated logic and ensures that a URL receives comparable treatment whether it is analysed proactively through the manual form or automatically after becoming the current browser tab.

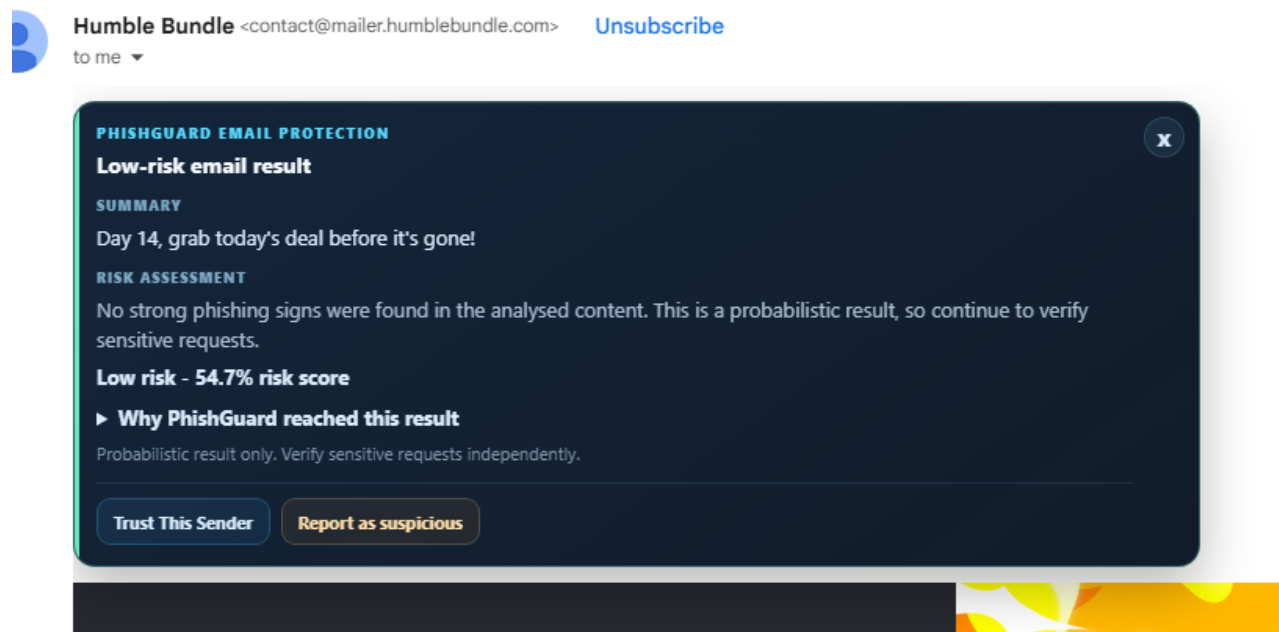


Figure 71: Low-Risk Email Scan Result

This Gmail-integrated banner shows how PhishGuard presents an email-assessment result directly inside the message view. In this example, the analysed email from Humble Bundle is classified as Low risk with a 54.7% contextual risk score, and the banner explains that no strong phishing signs were found in the analysed content. The interface includes a concise summary, a risk-assessment statement, an expandable explanation section, and two user actions—Trust This Sender and Report as Suspicious—so the user can either reinforce legitimate communication or flag doubtful content. Presenting the result inside the open email supports timely decision-making without forcing the user to leave Gmail or interpret a score without context.

4.5.4 Dashboard, Reporting and Personal Trust

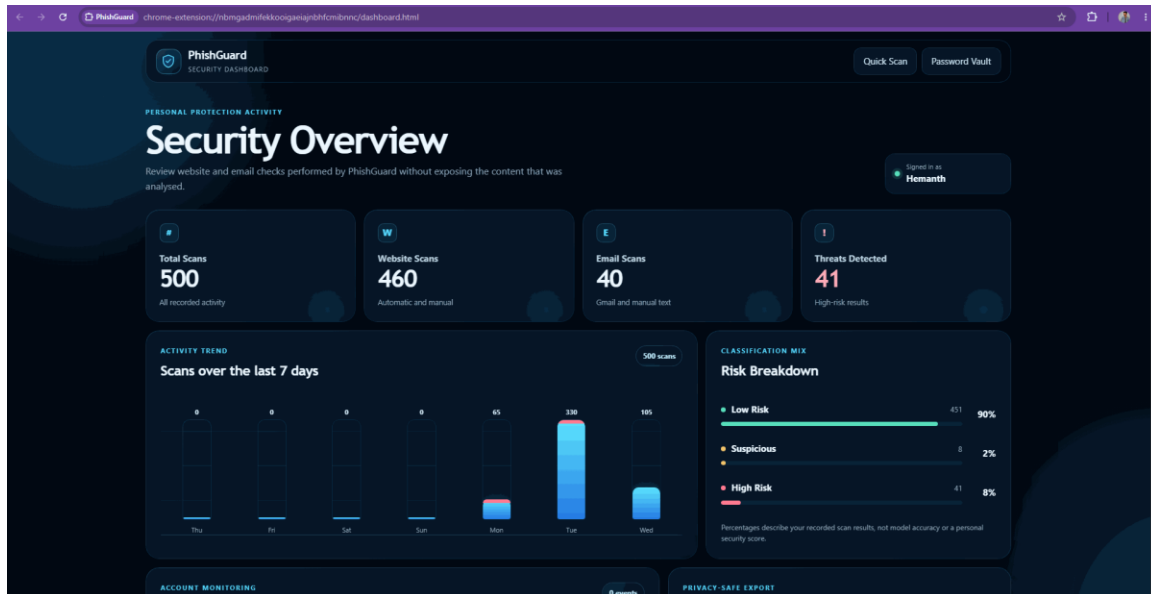


Figure 72: Security Dashboard Overview

The Security Dashboard provides a broader view of PhishGuard activity than can fit inside the extension popup. Summary cards and recent activity indicators allow the user to review the volume and distribution of scan results before opening individual records. Search and filtering functions support investigation of specific websites, emails, dates, or risk levels without requiring the backend to maintain a complete browsing-history database. Routine scan history remains user-scoped in the extension's Chrome storage, which aligns the dashboard with the project's privacy objective of avoiding unnecessary central collection of browsing behavior. From this page the user can also move into detailed scan records, reports, security events, trust management, and configuration functions.

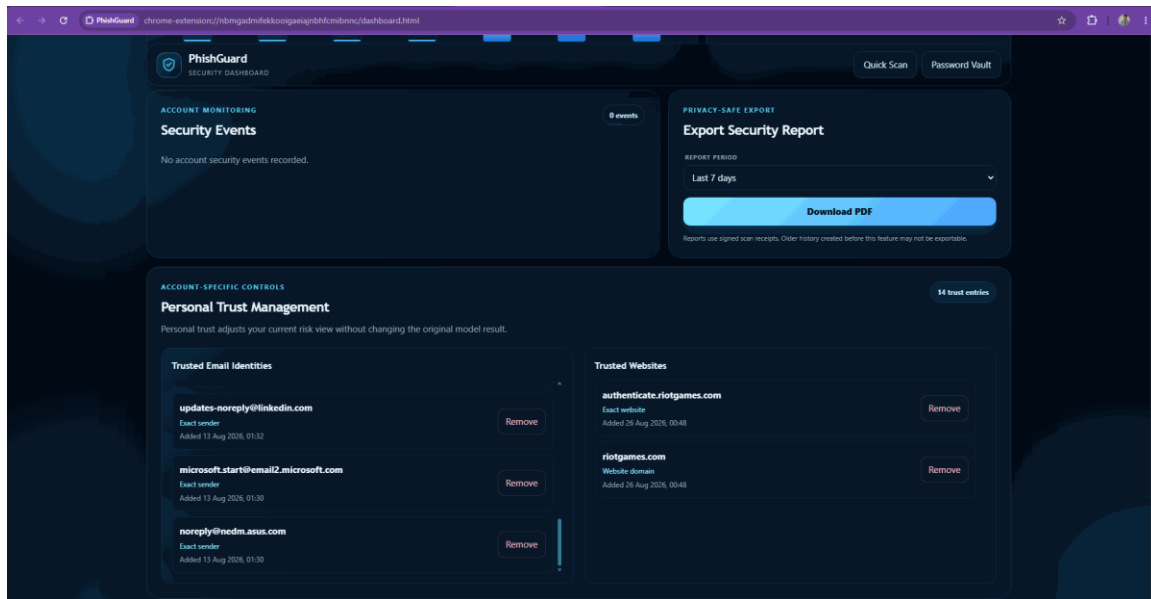


Figure 73: Security Events and Personal Trust Management

This dashboard section brings together two user-specific controls that affect future security decisions: account-security events and personal trust. Security events provide a reviewable record of meaningful authentication activity, such as a password being accepted followed by failed MFA attempts. Personal trust entries allow the user to manage previously trusted websites, senders, or domains and remove them when they are no longer appropriate. These trust decisions remain separate from the frozen machine-learning models, so deleting or adding trust does not alter the trained classifier. Presenting both functions in a management view also makes the controls reversible and auditable, which is safer than applying hidden or permanent overrides that the user cannot later inspect.

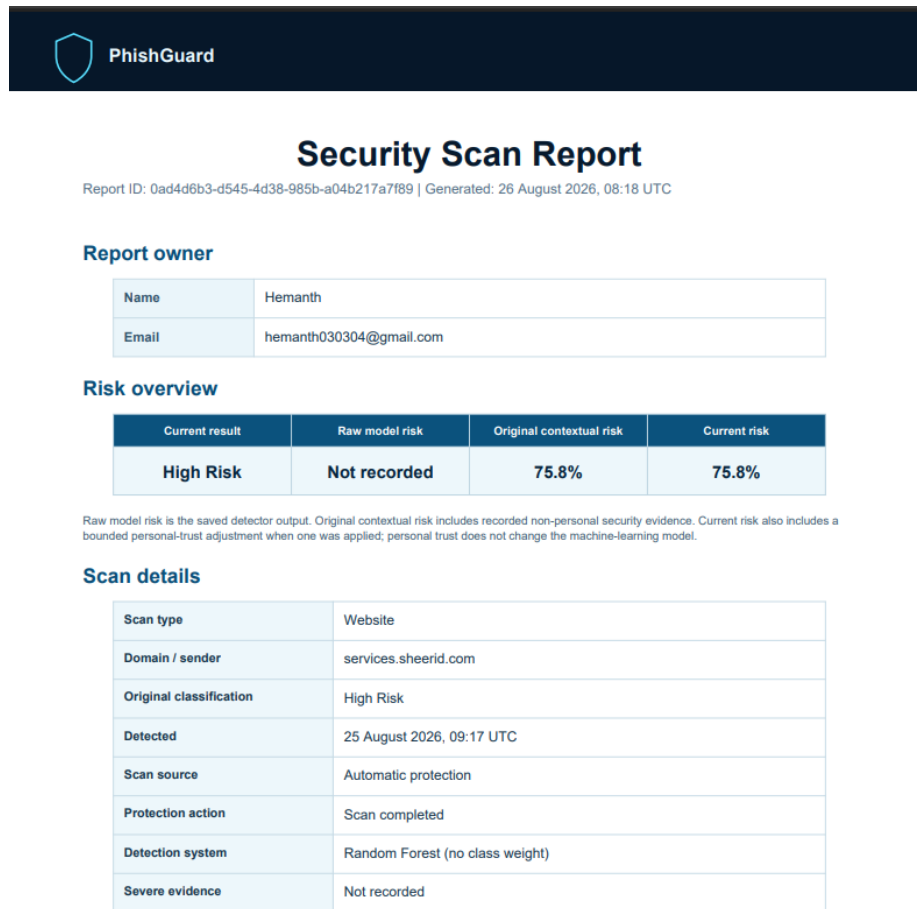


Figure 74: Generated Security Scan Report

The generated PDF converts PhishGuard scan evidence into a formal document that can be stored, reviewed, or submitted as security evidence outside the extension interface. The report identifies the authenticated owner, selected reporting period or scan, relevant metadata, overall risk assessment, and the reasons that contributed to the decision. Report generation is handled through an authenticated backend request so the server can validate ownership before returning a PDF. The report intentionally excludes reusable authentication secrets and other unnecessary sensitive values. By separating base-model information from final contextual reasoning where appropriate, the PDF also supports the project's explainability objective and provides a clearer audit trail than a single risk label alone.

4.5.5 Encrypted Password Vault

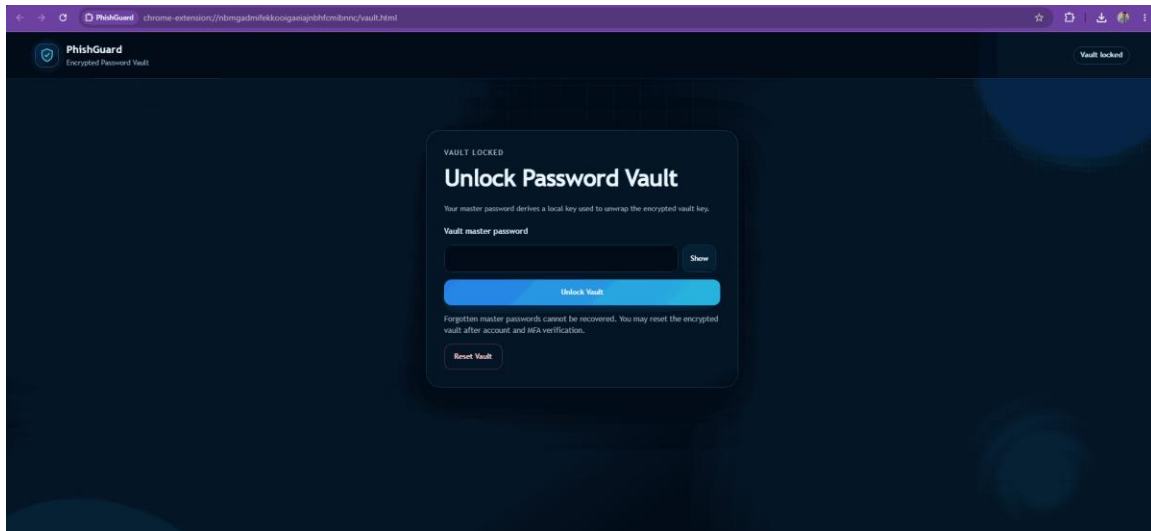


Figure 75: Password Vault Unlock

The Password Vault uses a security boundary that is separate from normal PhishGuard account authentication. Even after the user has signed into the extension, encrypted credentials remain unavailable until the vault is unlocked with the independent master password. The browser derives the required key material locally and uses it to unwrap the vault encryption key; the plaintext master password is not sent to the backend for verification or storage. This design limits the impact of a backend database disclosure because the server stores encrypted vault material rather than a directly usable credential collection. Locking the vault when it is not needed also reduces the period during which decrypted credential data exists in the active extension session.

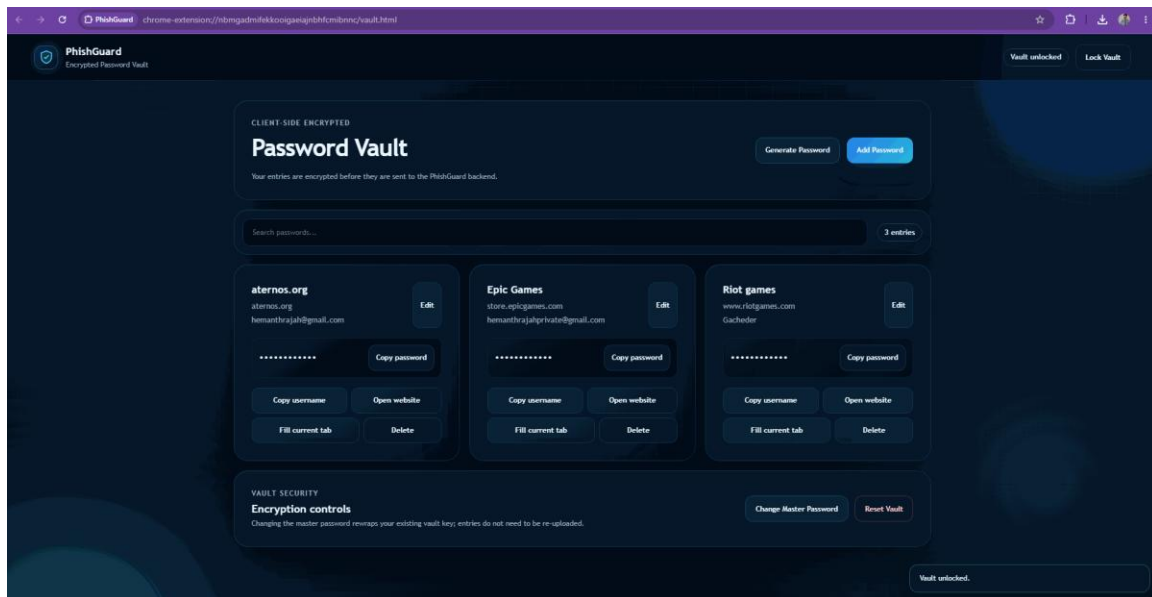


Figure 76: Password Vault Dashboard

The dedicated Password Vault dashboard provides the larger workspace required for secure credential management. Users can search saved entries, add or update logins, generate strong passwords, lock the vault, change the master password, and request credential filling for a matching browser tab. These operations are kept separate from the ordinary phishing-scanning popup because decrypted credential handling requires more deliberate interaction and stronger visual separation. Fill Current Tab is not treated as a simple convenience action; it uses the active destination and the vault's phishing-aware checks before credentials are inserted. This design connects password management with the core anti-phishing objective by reducing the chance that stored credentials are filled into an unsafe or mismatched destination.

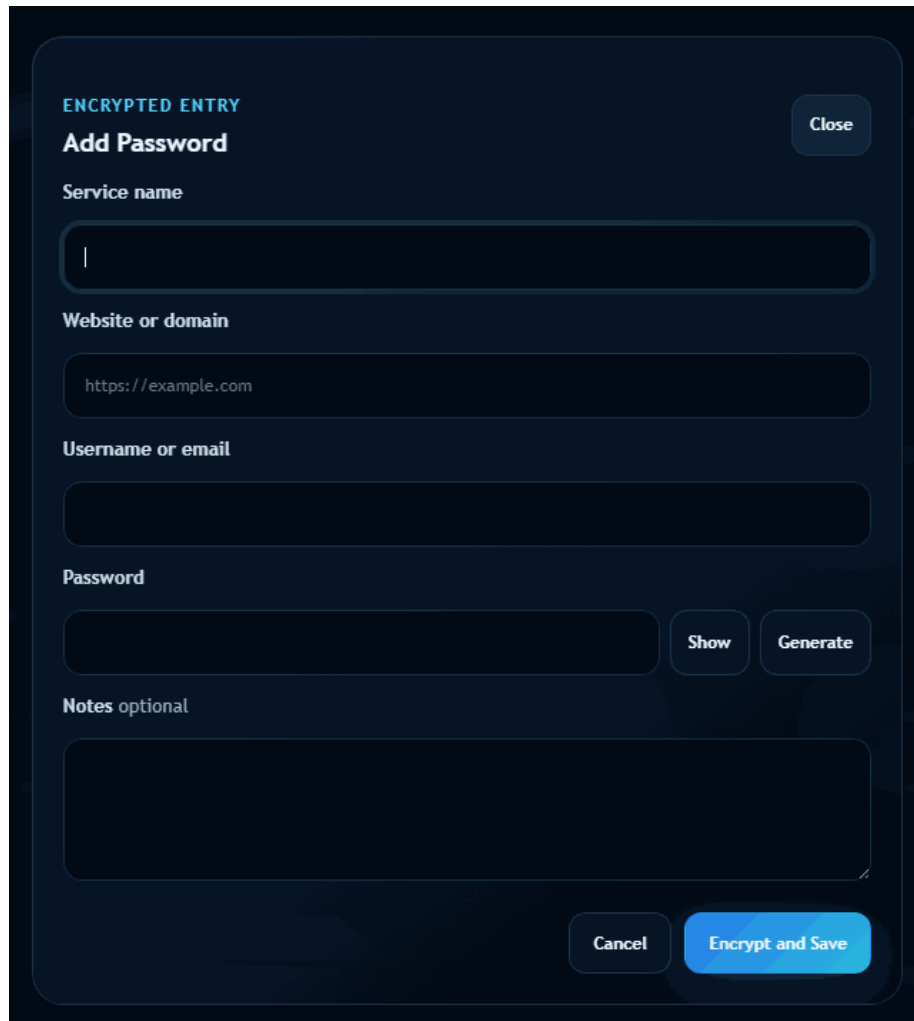
The image shows a dark-themed user interface for adding a password to an encrypted vault. At the top, it says 'ENCRYPTED ENTRY' in blue, followed by 'Add Password' in white. There is a 'Close' button in the top right. The form contains several input fields: 'Service name' (empty), 'Website or domain' (containing 'https://example.com'), 'Username or email' (empty), and 'Password' (empty). To the right of the password field are 'Show' and 'Generate' buttons. Below these is a 'Notes optional' section with a large text area. At the bottom right, there are 'Cancel' and 'Encrypt and Save' buttons.

Figure 77: Add Password to the Encrypted Vault

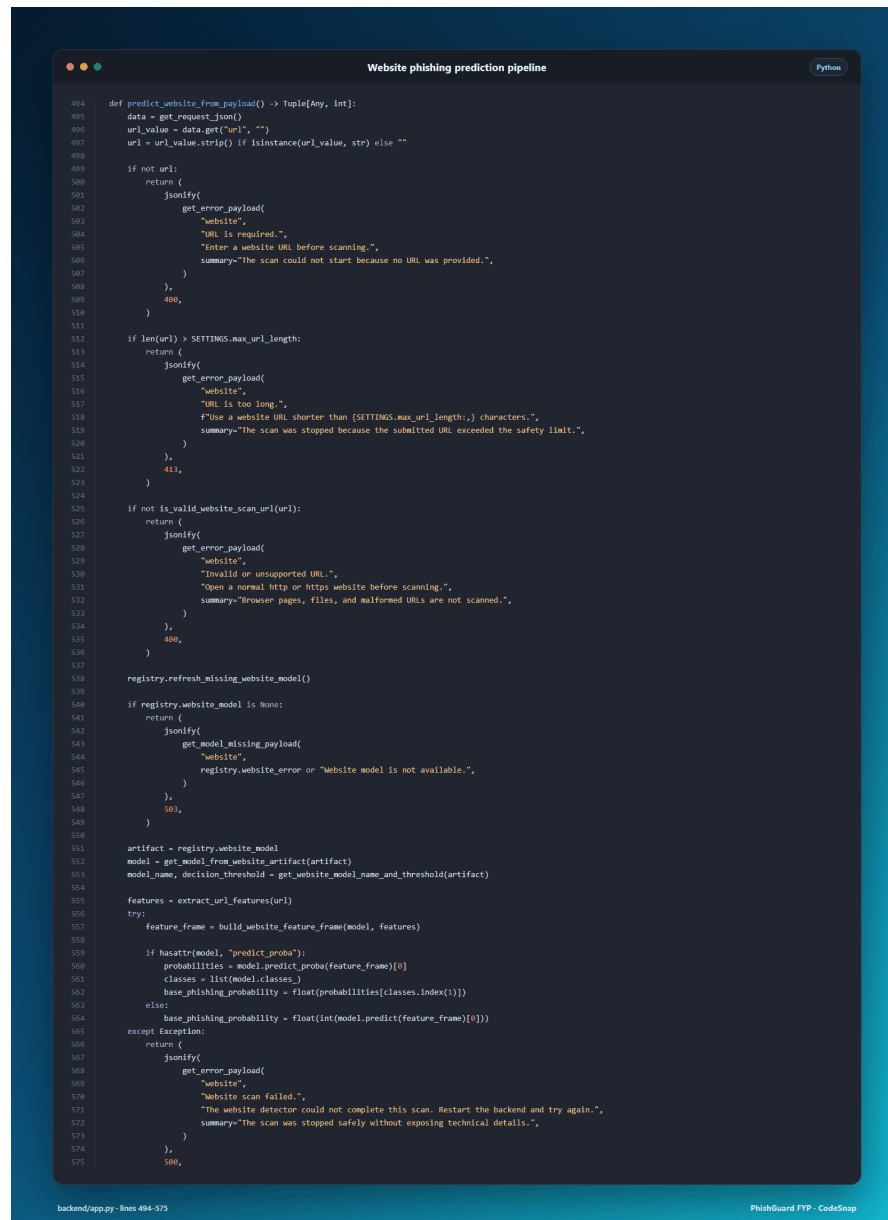
The Add Password form allows the user to create a new encrypted vault entry containing the destination, username, password, and supporting entry information. Before persistence, the credential payload is encrypted inside the browser using the unlocked vault key, so the backend receives ciphertext together with the required cryptographic metadata rather than plaintext login details. The system intentionally distinguishes saving a credential from filling it: a user may choose to save information even when the current page is not eligible for automatic filling, while later fill operations remain subject to destination-safety checks. This separation preserves user control without allowing the password manager to bypass the phishing-protection logic implemented elsewhere in PhishGuard.

4.6 Sample Code Implementation

The earlier report draft contained a much larger collection of source-code screenshots. For the final report, this section is limited to 22 representative excerpts selected to cover the most significant implementation paths: model loading and feature consistency, website and email detection,

authentication and MFA, browser protection, Gmail integration, reporting, and encrypted-vault cryptography. The temporary access-registration layer is no longer part of the active system and is therefore not included as implementation evidence. Near-duplicate helpers, styling excerpts, migrations, and secondary UI code are omitted so that the evidence remains focused and readable.

4.6.1 Backend Website Prediction API



```

404 def predict_website_from_payload() -> tuple[Any, int]:
405     data = get_request_json()
406     url_value = data.get("url", "")
407     url = url_value.strip() if isinstance(url_value, str) else ""
408
409     if not url:
410         return (
411             jsonify(
412                 get_error_payload(
413                     "website",
414                     "URL is required.",
415                     "Enter a website URL before scanning.",
416                     summary="The scan could not start because no URL was provided.",
417                 )
418             ),
419             400,
420         )
421
422     if len(url) > SETTINGS.max_url_length:
423         return (
424             jsonify(
425                 get_error_payload(
426                     "website",
427                     "URL is too long.",
428                     f"Use a website URL shorter than {SETTINGS.max_url_length} characters.",
429                     summary="The scan was stopped because the submitted URL exceeded the safety limit.",
430                 )
431             ),
432             413,
433         )
434
435     if not is_valid_website_scan_url(url):
436         return (
437             jsonify(
438                 get_error_payload(
439                     "website",
440                     "Invalid or unsupported URL.",
441                     "Open a normal http or https website before scanning.",
442                     summary="Browser pages, files, and malformed URLs are not scanned.",
443                 )
444             ),
445             400,
446         )
447
448     registry.refresh_missing_website_model()
449
450     if registry.website_model is None:
451         return (
452             jsonify(
453                 get_model_missing_payload(
454                     "website",
455                     registry.website_error or "Website model is not available.",
456                 )
457             ),
458             503,
459         )
460
461     artifact = registry.website_model
462     model = get_model_from_website_artifact(artifact)
463     model_name, decision_threshold = get_website_model_name_and_threshold(artifact)
464
465     features = extract_url_features(url)
466     try:
467         feature_frame = build_website_feature_frame(model, features)
468
469         if hasattr(model, "predict_proba"):
470             probabilities = model.predict_proba(feature_frame)[0]
471             classes = list(model.classes_)
472             base_phishing_probability = float(probabilities[classes.index(1)])
473         else:
474             base_phishing_probability = float(int(model.predict(feature_frame)[0]))
475     except Exception:
476         return (
477             jsonify(
478                 get_error_payload(
479                     "website",
480                     "Website scan failed.",
481                     "The website detector could not complete this scan. Restart the backend and try again.",
482                     summary="The scan was stopped safely without exposing technical details.",
483                 )
484             ),
485             500,
486         )

```

Figure 78: Website Phishing Prediction Pipeline

This backend excerpt represents the main website-scanning execution path used by the production API. The request is validated before URL features are extracted in the exact format expected by the saved website model. The frozen Random Forest then produces the base phishing probability, after which contextual website analysis evaluates additional combinations of security-relevant

signals without altering the trained model itself. The response construction stage keeps the base probability, final contextual risk, risk label, and human-readable evidence logically separate so the extension and reports can explain the decision. Centralising this sequence in the backend also means the extension does not need to ship the full model artifact, which simplifies updates and keeps production inference consistent across installed clients.

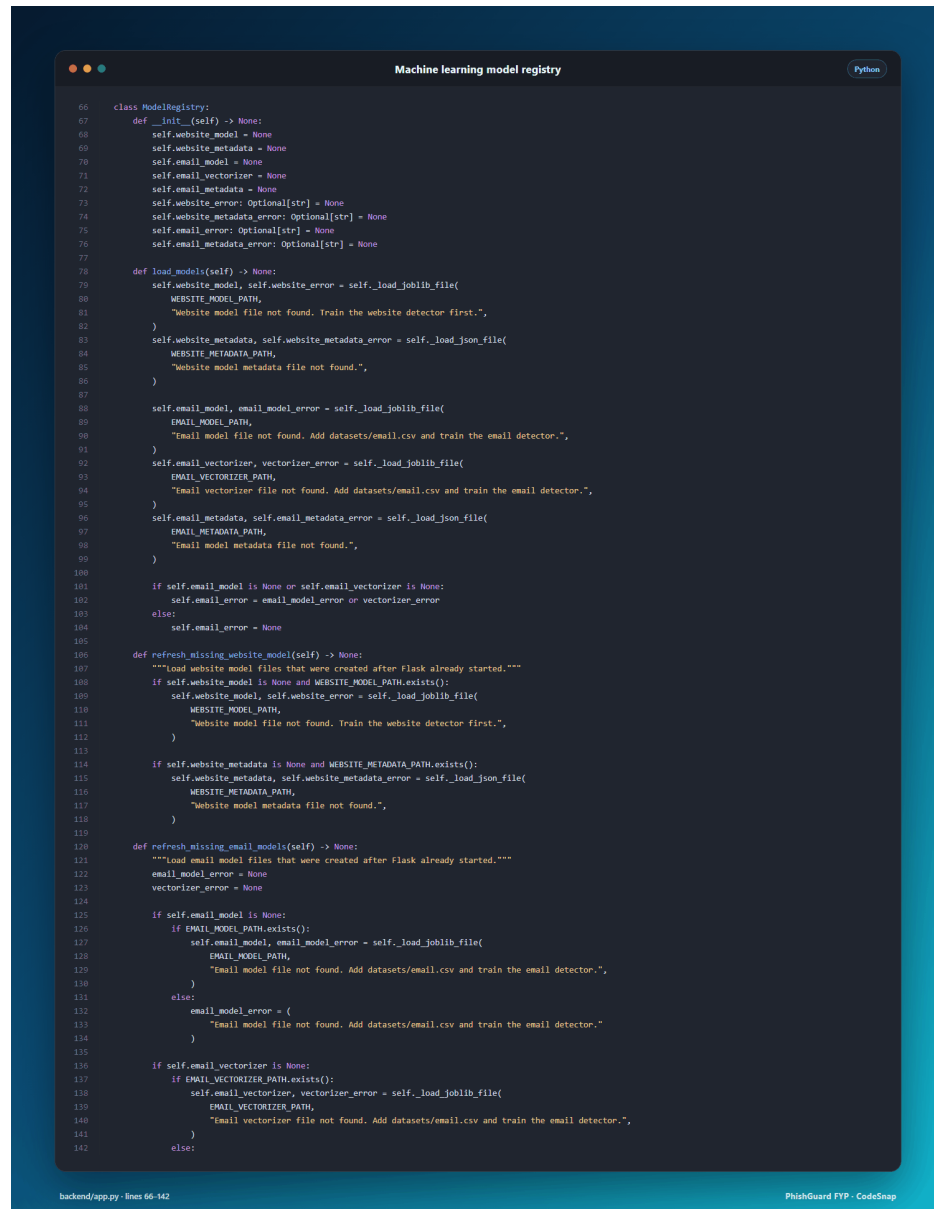
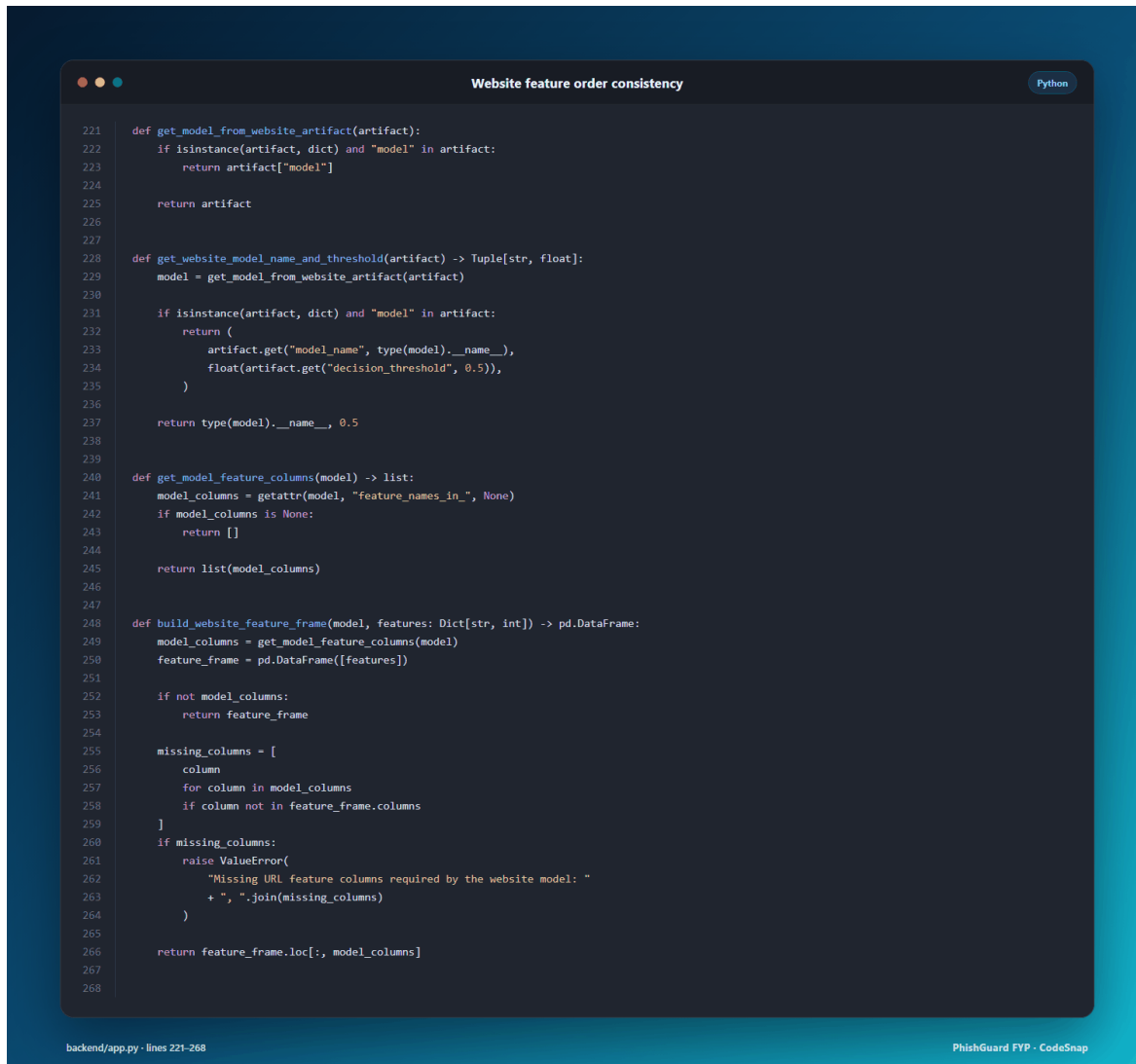


Figure 79: Machine-Learning Model Registry

The model registry provides a central backend location for loading and exposing the deployed website and email model artifacts. Instead of allowing individual API routes to load arbitrary files independently, the registry validates that the required artifacts and associated metadata are available before prediction requests are served. This reduces duplicated loading logic and helps

prevent the website and email endpoints from accidentally using inconsistent model versions. The registry also supports clearer startup diagnostics because missing or incompatible artifacts can be detected when the backend initialises rather than during an end-user scan. In the final architecture, this component therefore forms the controlled boundary between persisted machine-learning artifacts and the API functions that consume them.



```

221 def get_model_from_website_artifact(artifact):
222     if isinstance(artifact, dict) and "model" in artifact:
223         return artifact["model"]
224
225     return artifact
226
227
228 def get_website_model_name_and_threshold(artifact) -> Tuple[str, float]:
229     model = get_model_from_website_artifact(artifact)
230
231     if isinstance(artifact, dict) and "model" in artifact:
232         return (
233             artifact.get("model_name", type(model).__name__),
234             float(artifact.get("decision_threshold", 0.5)),
235         )
236
237     return type(model).__name__, 0.5
238
239
240 def get_model_feature_columns(model) -> list:
241     model_columns = getattr(model, "feature_names_in_", None)
242     if model_columns is None:
243         return []
244
245     return list(model_columns)
246
247
248 def build_website_feature_frame(model, features: Dict[str, int]) -> pd.DataFrame:
249     model_columns = get_model_feature_columns(model)
250     feature_frame = pd.DataFrame([features])
251
252     if not model_columns:
253         return feature_frame
254
255     missing_columns = [
256         column
257         for column in model_columns
258         if column not in feature_frame.columns
259     ]
260     if missing_columns:
261         raise ValueError(
262             "Missing URL feature columns required by the website model: "
263             + ", ".join(missing_columns)
264         )
265
266     return feature_frame.loc[:, model_columns]
267
268

```

backend/app.py - lines 221-268

PhishGuard FYP - CodeSnap

Figure 80: Website Feature-Order Consistency Check

Machine-learning inference depends not only on using the correct features but also on presenting them in the same order used during training. This excerpt validates the runtime website feature vector against the feature order stored with the deployed model metadata. If the extractor and artifact disagree, the backend can reject the inconsistent configuration instead of silently producing predictions from misaligned columns. This is particularly important because several raw length-based features were intentionally removed during model refinement to reduce false positives on

long legitimate URLs. The consistency check therefore acts as a regression safeguard between the shared feature extractor, the saved Random Forest artifact, and the live prediction route.

4.6.2 Website Model and Contextual Risk Analysis

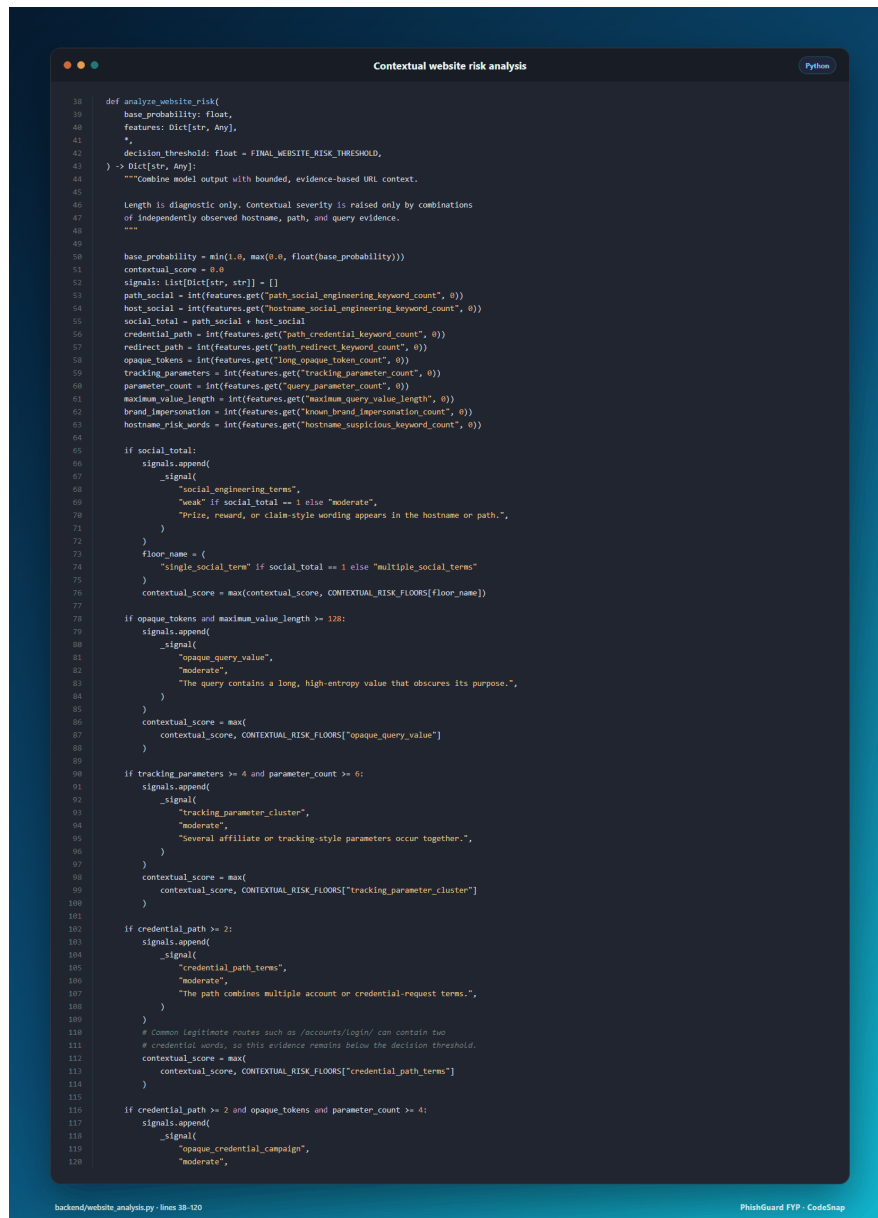
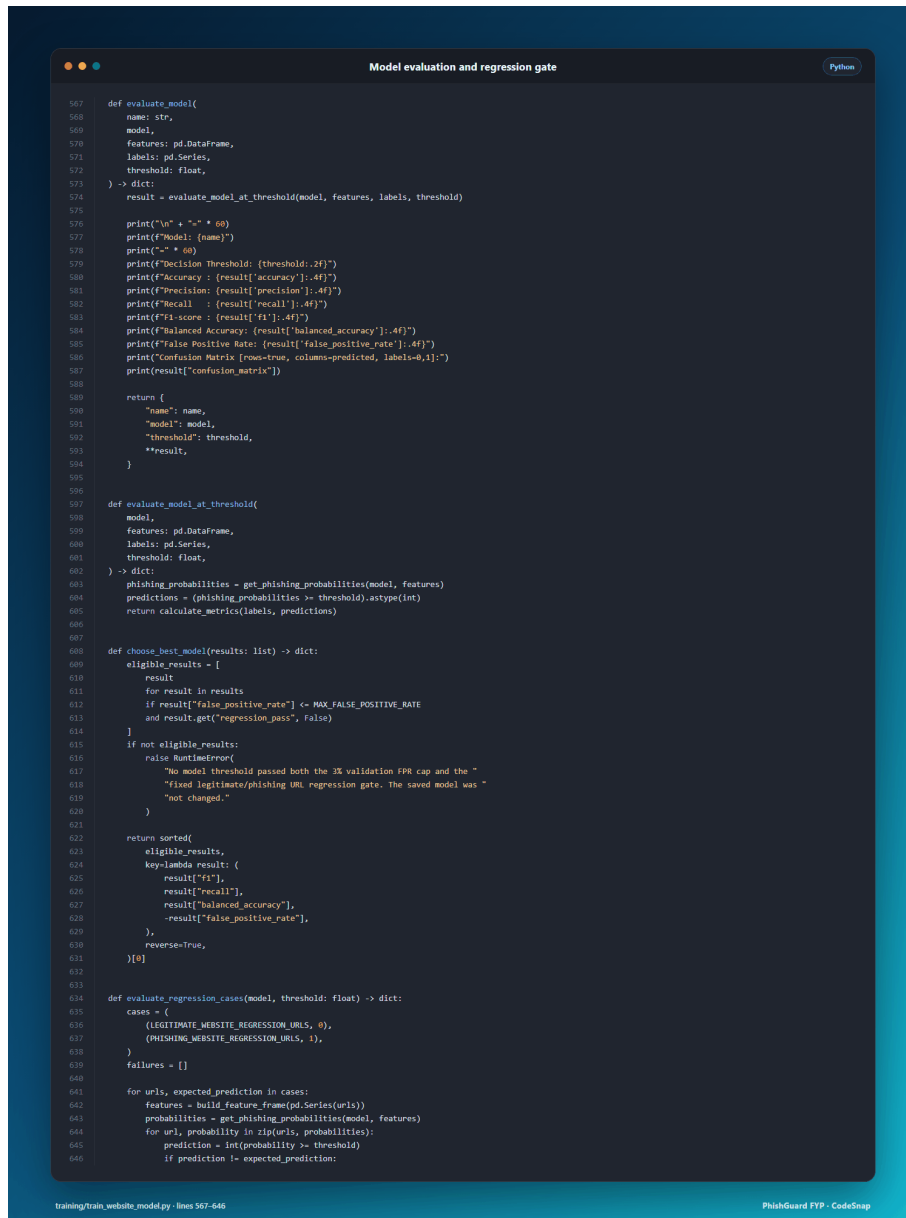


Figure 81: Contextual Website Risk Analysis

The contextual website layer evaluates security evidence after the frozen Random Forest probability has been calculated. Its purpose is to capture meaningful combinations of signals that may be difficult for the supervised model to represent reliably without changing the model's held-out metrics or retraining it for every rule adjustment. The logic can raise the final user-facing risk when several suspicious structural indicators occur together and can apply bounded personal-trust reductions when the evidence permits. Severe indicators restrict the amount of reduction that trust

can provide, preventing a saved trust decision from neutralising strong phishing evidence. Because the raw model probability is preserved separately, the system can report both the classifier output and the practical contextual decision transparently.



```

567 def evaluate_model(
568     name: str,
569     model,
570     features: pd.DataFrame,
571     labels: pd.Series,
572     threshold: float,
573 ) -> dict:
574     result = evaluate_model_at_threshold(model, features, labels, threshold)
575
576     print("\n" + "-" * 60)
577     print(f"Model: {name}")
578     print("-" * 60)
579     print(f"Decision Threshold: {threshold:.2f}")
580     print(f"Accuracy : {result['accuracy']:.4f}")
581     print(f"Precision: {result['precision']:.4f}")
582     print(f"Recall : {result['recall']:.4f}")
583     print(f"F1-score : {result['f1']:.4f}")
584     print(f"Balanced Accuracy: {result['balanced_accuracy']:.4f}")
585     print(f"False Positive Rate: {result['false_positive_rate']:.4f}")
586     print("Confusion Matrix (row=true, column=predicted, labels=0,1):")
587     print(result["confusion_matrix"])
588
589     return {
590         "name": name,
591         "model": model,
592         "threshold": threshold,
593         **result,
594     }
595
596 def evaluate_model_at_threshold(
597     model,
598     features: pd.DataFrame,
599     labels: pd.Series,
600     threshold: float,
601 ) -> dict:
602     phishing_probabilities = get_phishing_probabilities(model, features)
603     predictions = (phishing_probabilities >= threshold).astype(int)
604     return calculate_metrics(labels, predictions)
605
606 def choose_best_model(results: list) -> dict:
607     eligible_results = [
608         result
609         for result in results
610         if result["false_positive_rate"] <= MAX_FALSE_POSITIVE_RATE
611         and result.get("regression_pass", False)
612     ]
613     if not eligible_results:
614         raise RuntimeError(
615             "No model threshold passed both the IX validation FPR cap and the "
616             "fixed legitimate/phishing URL regression gate. The saved model was "
617             "not changed."
618         )
619
620     return sorted(
621         eligible_results,
622         key=lambda result: (
623             result["f1"],
624             result["recall"],
625             result["balanced_accuracy"],
626             -result["false_positive_rate"],
627         ),
628         reverse=True,
629     )[0]
630
631 def evaluate_regression_cases(model, threshold: float) -> dict:
632     cases = (
633         (LEGITIMATE_WEBSITE_REGRESSION_URLS, 0),
634         (PHISHING_WEBSITE_REGRESSION_URLS, 1),
635     )
636     failures = []
637
638     for urls, expected_prediction in cases:
639         features = build_feature_frame(pd.Series(urls))
640         probabilities = get_phishing_probabilities(model, features)
641         for url, probability in zip(urls, probabilities):
642             prediction = int(probability >= threshold)
643             if prediction != expected_prediction:

```

Figure 82: Website Model Evaluation and Regression Gate

The website-model training pipeline evaluates candidate artifacts using held-out classification metrics and a separate set of realistic regression cases before a model is accepted for deployment. Accuracy alone is not treated as sufficient because a model can achieve a good aggregate score while still misclassifying important categories of legitimate or phishing URLs. The regression checks therefore include behaviours that previously caused practical issues, such as long legitimate Google URLs and representative phishing-style credential destinations. The selected artifact is

saved together with its feature metadata and decision threshold only after the required checks pass. This release gate helps keep later training changes from silently reintroducing false positives or removing behavior relied upon by the browser extension.

4.6.3 Email Model and Contextual Risk Analysis

```

181 def detect_email(
182     model,
183     vectorizer,
184     email_input: Dict[str, Any],
185     decision_threshold: float,
186 ) -> Dict[str, Any]:
187     transformed = vectorizer.transform([clean_email_text(email_input["text"])])
188     base_probability = get_base_phishing_probability(model, transformed)
189     analysis = analyze_email_metadata(
190         sender=email_input["sender"],
191         sender_display_name=email_input["sender_display_name"],
192         subject=email_input["subject"],
193         text=email_input["body"],
194         raw_links=email_input["links"],
195     )
196     risk_result = adjust_email_risk(
197         base_probability,
198         analysis,
199         email_input.get("personal_trust"),
200     )
201     log_privacy_safe_diagnostics(email_input, analysis, risk_result)
202     final_risk_score = risk_result["final_risk_score"]
203     risk_classification = get_email_risk_classification(
204         final_risk_score,
205         decision_threshold,
206     )
207     reasons = build_email_explanations(
208         analysis,
209         base_probability,
210         final_risk_score,
211         decision_threshold,
212         risk_result,
213     )
214     evidence_breakdown = build_email_evidence_breakdown(
215         analysis,
216         base_probability,
217         final_risk_score,
218         decision_threshold,
219         risk_result,
220     )
221     original_risk_result = {
222         "risk_result": risk_result,
223         "final_risk_score": risk_result["original_contextual_risk_score"],
224         "applied_adjustments": [
225             item
226             for item in risk_result["applied_adjustments"]
227             if item.get("signal") != "personal_trust"
228         ],
229         "personal_trust": {},
230     }
231     original_evidence_breakdown = build_email_evidence_breakdown(
232         analysis,
233         base_probability,
234         risk_result["original_contextual_risk_score"],
235         decision_threshold,
236         original_risk_result,
237     )
238     summary = None
239     if email_input.get("summary_enabled", True):
240         summary = build_local_email_summary(
241             email_input["subject"],
242             email_input["body"],
243             analysis["brand_names"],
244             email_input["sender_display_name"],
245         )
246     return {
247         "prediction": int(final_risk_score >= decision_threshold),
248         "base_phishing_probability": round(base_probability, 4),
249         "original_contextual_risk_score": risk_result[
250             "original_contextual_risk_score"
251         ],
252         "final_risk_score": final_risk_score,
253         "risk_score_percent": f"{(final_risk_score * 100):.2f}%",
254         "risk_adjustment": risk_result["adjustment"],
255         "applied_adjustments": risk_result["applied_adjustments"],
256         "adjustments": risk_result["adjustments"],
257         "personal_trust": risk_result["personal_trust"],
258         "base_score_evidence": bool(analysis.get("personal_trust_override")),
259         "risk_classification": risk_classification,
260         "reasons": reasons,
261         "evidence_breakdown": evidence_breakdown,
262         "original_evidence_breakdown": original_evidence_breakdown,
263         "summary": summary,
264         "summary_enabled": email_input.get("summary_enabled", True),
265         "user_message": build_email_user_message(risk_classification),
266         "sender_domain": analysis["sender_domain"],
267         "link_domain": analysis["link_domain"][:4],
268         "brand_names": analysis["brand_names"][:4],
269         "domain_reputation": analysis["domain_reputation"],
270         "authentication_available": analysis["authentication_available"],

```

Figure 83: Email Detection Orchestration

This backend function coordinates the complete email-analysis path rather than treating the text classifier as the final answer. The visible email content is first prepared for the frozen calibrated text model, which produces the base phishing probability. Additional metadata and link-related evidence are then evaluated, followed by any permitted user-specific sender or domain trust adjustment. The function combines these inputs into the final contextual risk while retaining the

original model probability and the individual reasons used to explain increases or reductions. This layered design allows the Gmail banner, manual scanner, dashboard, and PDF reports to present a consistent decision without hiding the effect of contextual security evidence or personal trust inside an opaque model score.

```

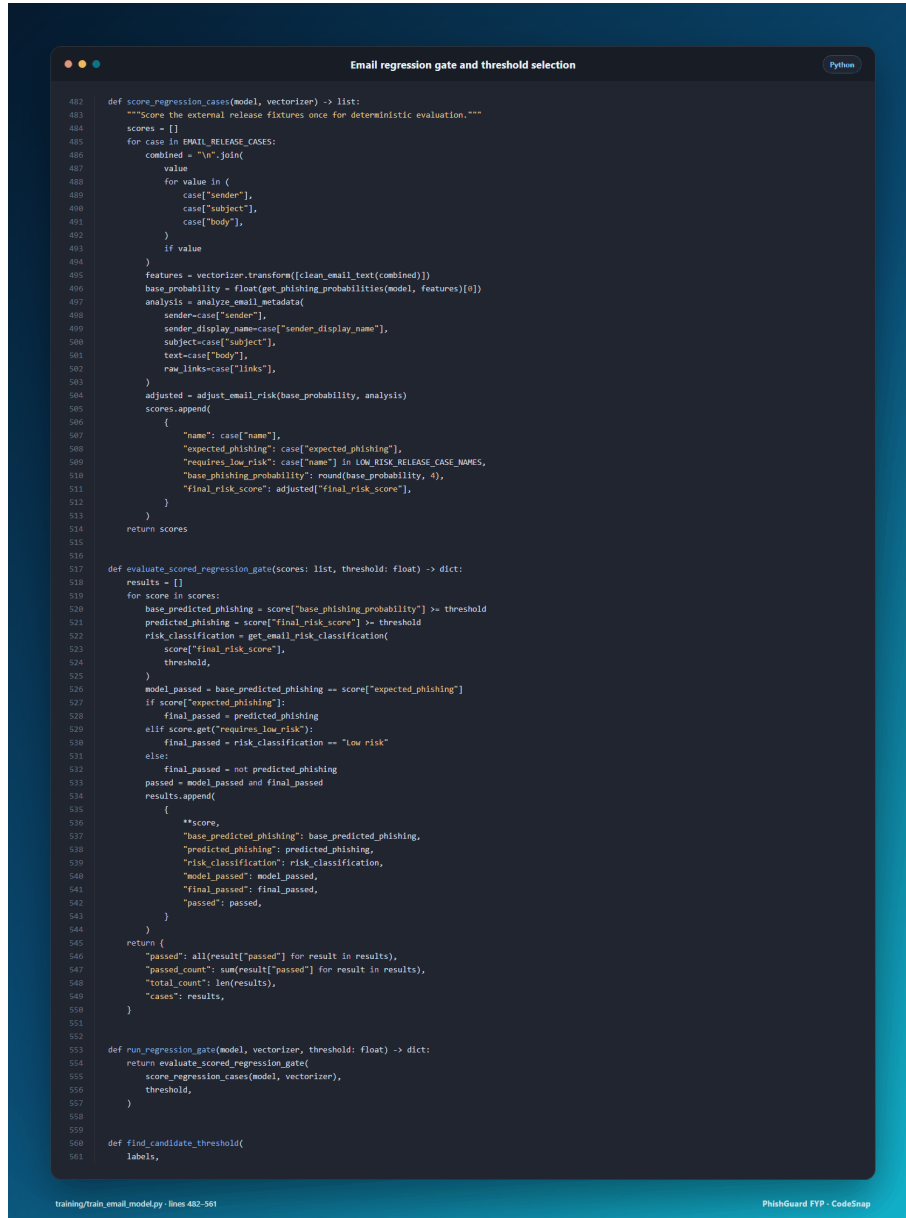
741         "link_risk_adjustment",
742     )
743
744     consistency_adjustment = 0.0
745     if (
746         analysis["trusted_consistency"]
747         and not positive_adjustment
748         and not analysis["concrete_suspicious_evidence"]
749     ):
750         consistency_adjustment = -MAX_CONSISTENCY_REDUCTION
751         applied.append(
752             {
753                 "signal": "trusted_consistency",
754                 "adjustment": consistency_adjustment,
755                 "explanation": RISK_ADJUSTMENT_RULES["trusted_consistency"]["evidence"],
756             }
757         )
758
759     corporate_adjustment = 0.0
760     if (
761         analysis["corporate_consistency"]
762         and not analysis["trusted_consistency"]
763         and not positive_adjustment
764     ):
765         corporate_adjustment = -MAX_CORPORATE_CONSISTENCY_REDUCTION
766         applied.append(
767             {
768                 "signal": "corporate_consistency",
769                 "adjustment": corporate_adjustment,
770                 "explanation": RISK_ADJUSTMENT_RULES["corporate_consistency"]["evidence"],
771             }
772         )
773
774     personal_trust_status = evaluate_personal_trust(
775         analysis, personal_trust, base_probability
776     )
777     personal_trust_adjustment = (
778         -personal_trust_status["reduction"]
779         if personal_trust_status["applied"]
780         else 0.0
781     )
782     if personal_trust_adjustment:
783         applied.append(
784             {
785                 "signal": "personal_trust",
786                 "adjustment": personal_trust_adjustment,
787                 "explanation": personal_trust_status["reason"],
788             }
789         )
790
791     pre_personal_trust_adjustment = (
792         positive_adjustment + consistency_adjustment + corporate_adjustment
793     )
794     original_contextual_risk_score = clamp(
795         base_probability + pre_personal_trust_adjustment
796     )
797
798     total_adjustment = (
799         positive_adjustment
800         + consistency_adjustment
801         + corporate_adjustment
802         + personal_trust_adjustment
803     )
804     adjustment_breakdown = {
805         **{key: round(value, 4) for key, value in categories.items()},
806         "brand_consistency_adjustment": round(consistency_adjustment, 4),
807         "corporate_consistency_adjustment": round(corporate_adjustment, 4),
808         "personal_trust_adjustment": round(personal_trust_adjustment, 4),
809         "total_adjustment": round(total_adjustment, 4),
810     }
811
812     return {
813         "base_phishing_probability": round(clamp(base_probability), 4),
814         "original_contextual_risk_score": round(
815             original_contextual_risk_score, 4
816         ),
817         "adjustment": round(total_adjustment, 4),
818         "final_risk_score": round(
819             clamp(base_probability + total_adjustment),
820             4,
821         ),
822         "applied_adjustments": applied,
823         "adjustments": adjustment_breakdown,
824         "personal_trust": personal_trust_status,
825     }
826

```

Figure 84: Hybrid Email Risk Adjustment

The hybrid email-risk logic controls how contextual evidence and personal trust modify the frozen text-model probability. Exact-sender trust is intentionally stronger than broader domain trust because it represents a more specific user decision, while public-provider domains are not treated as automatically safe. At the same time, severe evidence such as impersonation, deceptive destinations, or dangerous credential-related conflicts can block personal-trust reductions entirely.

This prevents a previously trusted sender from becoming a blanket bypass for strong phishing signals. The code keeps the base probability unchanged and calculates a separate final contextual risk, allowing the report and interface to show that personalisation affected the operational decision without claiming that the underlying machine-learning model itself changed.



```

482 def score_regression_cases(model, vectorizer) -> list:
483     """Score the external release fixtures once for deterministic evaluation."""
484     scores = []
485     for case in EMAIL_RELEASE_CASES:
486         combined = " ".join(
487             value
488             for value in (
489                 case["sender"],
490                 case["subject"],
491                 case["body"],
492             )
493             if value
494         )
495         features = vectorizer.transform([clean_email_text(combined)])
496         base_probability = float(get_phishing_probabilities(model, features)[0])
497         analysis = analyze_email_metadata(
498             sender=case["sender"],
499             sender_display_name=case["sender_display_name"],
500             subject=case["subject"],
501             text=case["body"],
502             raw_links=case["links"],
503         )
504         adjusted = adjust_email_risk(base_probability, analysis)
505         scores.append(
506             {
507                 "name": case["name"],
508                 "expected_phishing": case["expected_phishing"],
509                 "requires_low_risk": case["name"] in LOW_RISK_RELEASE_CASE_NAMES,
510                 "base_phishing_probability": round(base_probability, 4),
511                 "final_risk_score": adjusted["final_risk_score"],
512             }
513         )
514     return scores
515
516
517 def evaluate_scored_regression_gate(scores: list, threshold: float) -> dict:
518     results = []
519     for score in scores:
520         base_predicted_phishing = score["base_phishing_probability"] >= threshold
521         predicted_phishing = score["final_risk_score"] >= threshold
522         risk_classification = get_email_risk_classification(
523             score["final_risk_score"],
524             threshold,
525         )
526         model_passed = base_predicted_phishing == score["expected_phishing"]
527         if score["expected_phishing"]:
528             final_passed = predicted_phishing
529         elif score.get("requires_low_risk"):
530             final_passed = risk_classification == "Low risk"
531         else:
532             final_passed = not predicted_phishing
533         passed = model_passed and final_passed
534         results.append(
535             {
536                 "score": score,
537                 "base_predicted_phishing": base_predicted_phishing,
538                 "predicted_phishing": predicted_phishing,
539                 "risk_classification": risk_classification,
540                 "model_passed": model_passed,
541                 "final_passed": final_passed,
542                 "passed": passed,
543             }
544         )
545     return {
546         "passed": all(result["passed"] for result in results),
547         "passed_count": sum(result["passed"] for result in results),
548         "total_count": len(results),
549         "cases": results,
550     }
551
552
553 def run_regression_gate(model, vectorizer, threshold: float) -> dict:
554     return evaluate_scored_regression_gate(
555         score_regression_cases(model, vectorizer),
556         threshold,
557     )
558
559
560 def find_candidate_threshold(
561     labels,

```

Figure 85: Email Regression Gate and Threshold Selection

The email-training pipeline evaluates the calibrated LinearSVC model at the selected operational threshold and then applies an independent regression gate before deployment. Standard held-out metrics measure general classification performance, while the regression fixtures test representative modern legitimate and phishing messages that are important to the deployed browser workflow. This second layer is useful because a model can improve aggregate metrics

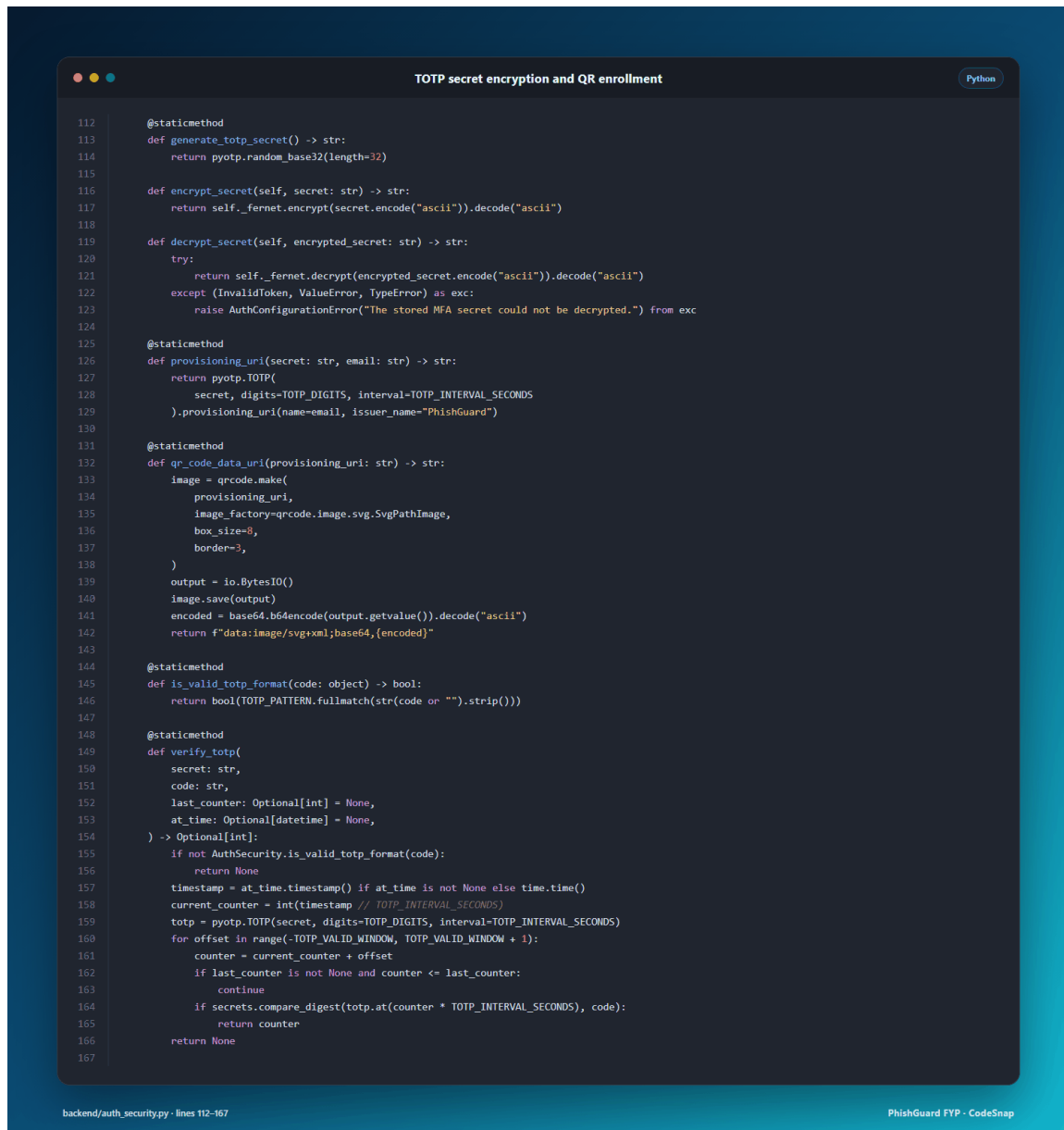
while regressing on specific newsletter, account-notification, or phishing patterns encountered by real users. The release process therefore freezes the model only when both the quantitative evaluation and the regression expectations are satisfied. The threshold remains part of the saved deployment configuration so the backend and documented evaluation use the same decision boundary.

4.6.4 Authentication, MFA and Security Events



Figure 86: Password Validation and Argon2 Hashing

This authentication excerpt shows the server-side handling of the PhishGuard account password. Registration first validates the password against the configured policy, after which the accepted password is transformed using Argon2 rather than stored in plaintext or with reversible encryption. Login verifies the submitted password against the stored Argon2 hash, so the database does not need to retain the original credential. This account password is intentionally separate from the Password Vault master password: the former authenticates access to PhishGuard services, while the latter is used locally for client-side vault key derivation. Maintaining this separation prevents the backend authentication credential from becoming the encryption key for stored vault passwords.



```

112     @staticmethod
113     def generate_totp_secret() -> str:
114         return pyotp.random_base32(length=32)
115
116     def encrypt_secret(self, secret: str) -> str:
117         return self._fernet.encrypt(secret.encode("ascii")).decode("ascii")
118
119     def decrypt_secret(self, encrypted_secret: str) -> str:
120         try:
121             return self._fernet.decrypt(encrypted_secret.encode("ascii")).decode("ascii")
122         except (InvalidToken, ValueError, TypeError) as exc:
123             raise AuthConfigurationError("The stored MFA secret could not be decrypted.") from exc
124
125     @staticmethod
126     def provisioning_uri(secret: str, email: str) -> str:
127         return pyotp.TOTP(
128             secret, digits=TOTP_DIGITS, interval=TOTP_INTERVAL_SECONDS
129         ).provisioning_uri(name=email, issuer_name="PhishGuard")
130
131     @staticmethod
132     def qr_code_data_uri(provisioning_uri: str) -> str:
133         image = qrcode.make(
134             provisioning_uri,
135             image_factory=qrcode.image.svg.SvgPathImage,
136             box_size=8,
137             border=3,
138         )
139         output = io.BytesIO()
140         image.save(output)
141         encoded = base64.b64encode(output.getvalue()).decode("ascii")
142         return f"data:image/svg+xml;base64,{encoded}"
143
144     @staticmethod
145     def is_valid_totp_format(code: object) -> bool:
146         return bool(TOTP_PATTERN.fullmatch(str(code or "").strip()))
147
148     @staticmethod
149     def verify_totp(
150         secret: str,
151         code: str,
152         last_counter: Optional[int] = None,
153         at_time: Optional[datetime] = None,
154     ) -> Optional[int]:
155         if not AuthSecurity.is_valid_totp_format(code):
156             return None
157         timestamp = at_time.timestamp() if at_time is not None else time.time()
158         current_counter = int(timestamp // TOTP_INTERVAL_SECONDS)
159         totp = pyotp.TOTP(secret, digits=TOTP_DIGITS, interval=TOTP_INTERVAL_SECONDS)
160         for offset in range(-TOTP_VALID_WINDOW, TOTP_VALID_WINDOW + 1):
161             counter = current_counter + offset
162             if last_counter is not None and counter <= last_counter:
163                 continue
164             if secrets.compare_digest(totp.at(counter * TOTP_INTERVAL_SECONDS), code):
165                 return counter
166         return None
167

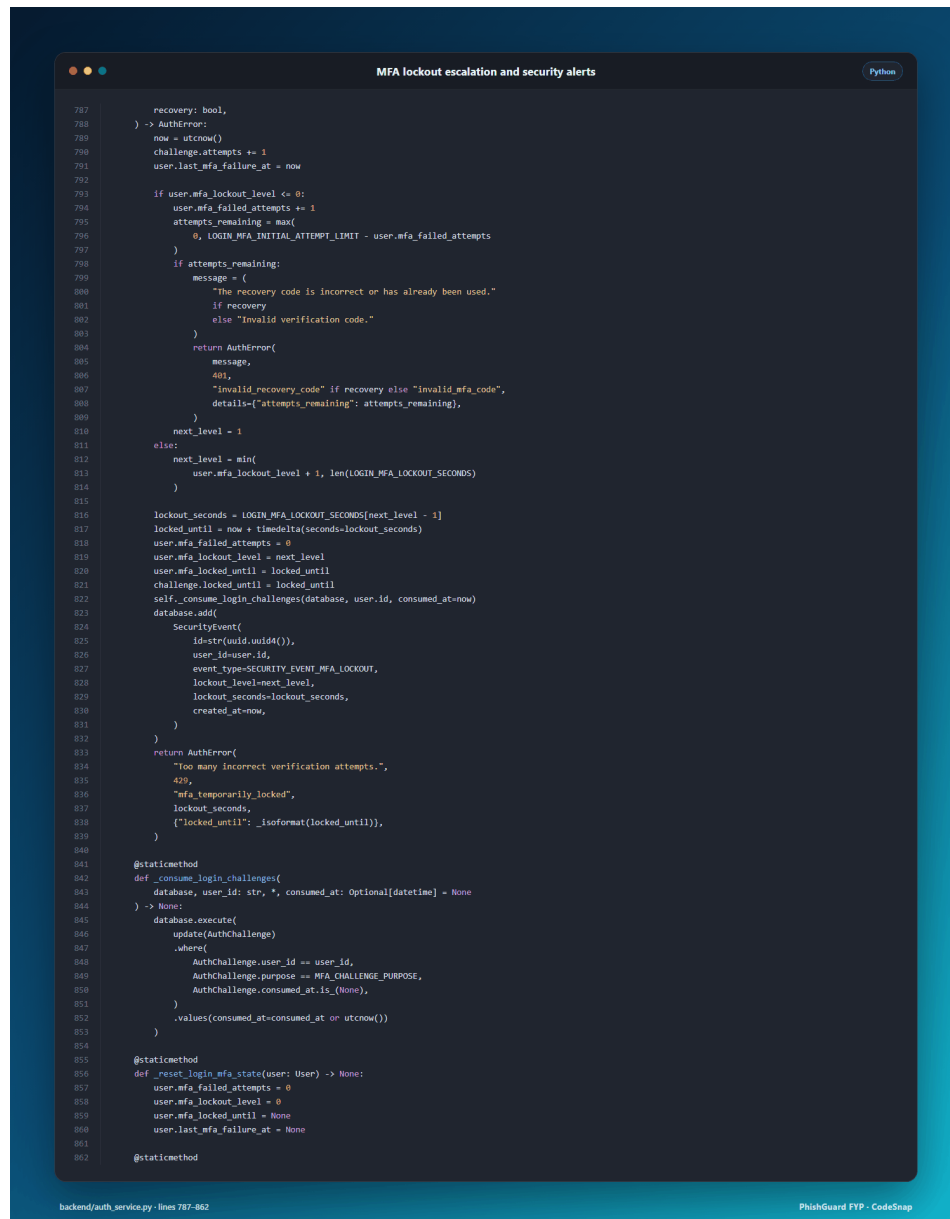
```

backend/auth_security.py · lines 112-167

PhishGuard FYP - CodeSnap

Figure 87: TOTP Secret Protection and Authenticator Enrollment

The MFA enrolment code generates the TOTP material required by standard authenticator applications and prepares the information displayed to the user during setup. The authenticator secret is treated as sensitive persistent data and is protected before database storage rather than being stored as ordinary readable account information. A newly generated factor is not considered fully established until the user proves possession by submitting a valid time-based code. Recovery codes are created separately so the user has an alternative recovery mechanism without exposing the TOTP secret itself. Together, these controls implement the mandatory second-factor workflow shown in the system screenshots while keeping enrolment secrets out of normal logs, reports, and later user interfaces.



```

787         recovery: bool,
788     ) -> AuthError:
789         now = utcnow()
790         challenge.attempts += 1
791         user.last_mfa_failure_at = now
792
793         if user.mfa_lockout_level <= 0:
794             user.mfa_failed_attempts += 1
795             attempts_remaining = max(
796                 0, LOGIN_MFA_INITIAL_ATTEMPT_LIMIT - user.mfa_failed_attempts
797             )
798             if attempts_remaining:
799                 message = (
800                     "The recovery code is incorrect or has already been used."
801                     if recovery
802                     else "Invalid verification code."
803                 )
804                 return AuthError(
805                     message,
806                     401,
807                     "Invalid recovery code" if recovery else "Invalid mfa code",
808                     details={"attempts_remaining": attempts_remaining},
809                 )
810             next_level = 1
811         else:
812             next_level = min(
813                 user.mfa_lockout_level + 1, len(LOGIN_MFA_LOCKOUT_SECONDS)
814             )
815
816         lockout_seconds = LOGIN_MFA_LOCKOUT_SECONDS[next_level - 1]
817         locked_until = now + timedelta(seconds=lockout_seconds)
818         user.mfa_failed_attempts = 0
819         user.mfa_lockout_level = next_level
820         user.mfa_locked_until = locked_until
821         challenge.locked_until = locked_until
822         self._consume_login_challenges(database, user.id, consumed_at=now)
823         database.add(
824             SecurityEvent(
825                 id=str(uuid.uuid4()),
826                 user_id=user.id,
827                 event_type=SECURITY_EVENT_MFA_LOCKOUT,
828                 lockout_level=next_level,
829                 lockout_seconds=lockout_seconds,
830                 created_at=now,
831             )
832         )
833         return AuthError(
834             "Too many incorrect verification attempts.",
835             429,
836             "mfa temporarily locked",
837             lockout_seconds,
838             {"locked_until": _isoformat(locked_until)},
839         )
840
841     @staticmethod
842     def _consume_login_challenges(
843         database, user_id: str, *, consumed_at: Optional[datetime] = None
844     ) -> None:
845         database.execute(
846             update(AuthChallenge)
847             .where(
848                 AuthChallenge.user_id == user_id,
849                 AuthChallenge.purpose == MFA_CHALLENGE_PURPOSE,
850                 AuthChallenge.consumed_at.is_(None),
851             )
852             .values(consumed_at=consumed_at or utcnow())
853         )
854
855     @staticmethod
856     def _reset_login_mfa_state(user: User) -> None:
857         user.mfa_failed_attempts = 0
858         user.mfa_lockout_level = 0
859         user.mfa_locked_until = None
860         user.last_mfa_failure_at = None
861
862     @staticmethod

```

backend/auth_service.py - lines 787-862

PhishGuard FYP - CodeSnap

Figure 88: Progressive MFA Lockout and Security Events

This service logic records failed MFA attempts only after the primary account password has already been accepted. Once the configured failure limit is reached, a temporary lockout is applied and the lockout level can increase after repeated cycles, slowing further guessing without permanently disabling the account. The same security path can create a privacy-safe event indicating that the correct password was followed by failed MFA attempts. That event is later displayed to the legitimate user after successful authentication, allowing suspicious activity to be reviewed without claiming that the earlier attempt completed login. The code therefore connects rate limiting, MFA attempt management, temporary lockouts, and user-visible security awareness within one account-protection workflow.

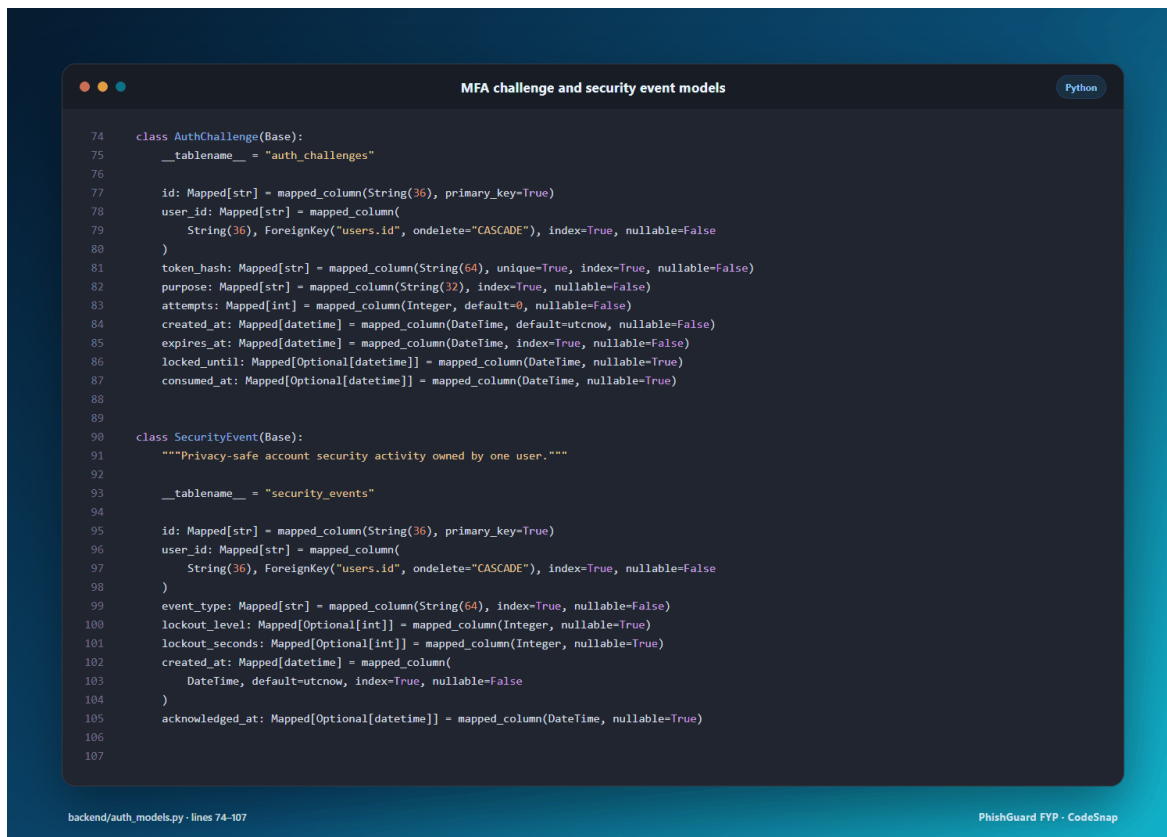


Figure 89: MFA Challenge and Security-Event Models

These database models persist only the state required to manage short-lived authentication challenges and later security review. The MFA challenge structure records ownership, purpose, attempt state, expiry, lockout information, and whether the challenge has already been consumed, which prevents the same challenge from being reused indefinitely. Security-event records capture the event type, timing, lockout information, and acknowledgement state needed by the dashboard or popup. Reusable plaintext authentication secrets are not stored in these event records. The separation between challenge state and security-event history also allows temporary authentication data to expire while retaining only the minimum account-protection evidence needed for later user review.

```

147 def auth_status():
148     return _success(get_auth_status())
149
150
151 @auth_blueprint.post("/register")
152 def register():
153     def action():
154         payload = _get_json()
155         return get_auth_service().register(
156             payload.get("display_name"),
157             payload.get("email"),
158             payload.get("password"),
159             payload.get("confirm_password"),
160             _client_key(),
161         )
162     return _run_auth_action(action, 201)
163
164
165
166 @auth_blueprint.post("/login")
167 def login():
168     def action():
169         payload = _get_json()
170         return get_auth_service().begin_password_login(
171             payload.get("email"), payload.get("password"), _client_key()
172         )
173     return _run_auth_action(action)
174
175
176
177 @auth_blueprint.post("/mfa/setup")
178 def mfa_setup():
179     def action():
180         payload = _get_json()
181         return get_auth_service().get_mfa_setup(
182             payload.get("temporary_auth_token")
183         )
184     return _run_auth_action(action)
185
186
187
188 @auth_blueprint.post("/mfa/verify")
189 def verify_mfa_enrollment():
190     def action():
191         payload = _get_json()
192         return get_auth_service().complete_mfa_enrollment(
193             payload.get("temporary_auth_token"),
194             payload.get("code"),
195             _client_key(),
196         )
197     return _run_auth_action(action)
198
199
200
201 @auth_blueprint.post("/login/mfa")
202 def verify_login_mfa():
203     def action():
204         payload = _get_json()
205         return get_auth_service().complete_login_mfa(
206             payload.get("temporary_auth_token"),
207             payload.get("code"),
208             payload.get("recovery_code"),
209             _client_key(),
210         )
211     return _run_auth_action(action)
212
213
214
215 @auth_blueprint.post("/recovery/acknowledge")
216 def acknowledge_recovery_codes():

```

Figure 90: Registration, Login and MFA API Routes

These routes expose the main authenticated account workflow used by the Chrome extension: account registration, password login, MFA verification, recovery-code login, session validation, and related account-security operations. The current registration route accepts the normal account fields and no longer requires a separate beta-access code, matching the simplified direct-download installation process shown in Figures 58 to 60. Password verification and MFA remain separate stages, so the backend can issue a short-lived challenge after correct credentials rather than creating a fully authenticated session immediately. Successful MFA or recovery verification then completes the login flow under the existing session controls. This route separation keeps each authentication stage explicit and easier to test than a single endpoint containing every branch.

4.6.5 Chrome Extension Website Protection

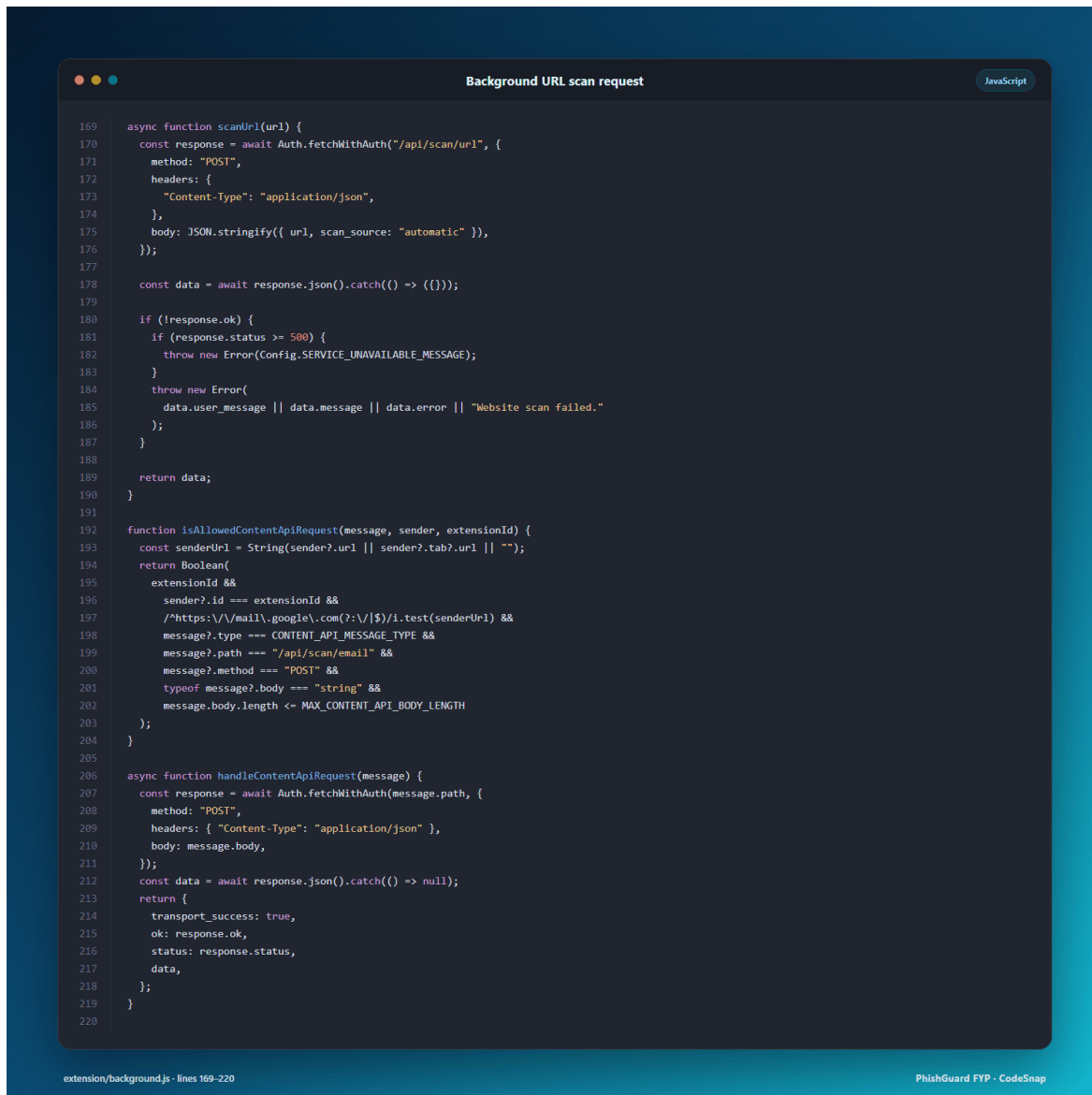


Figure 91: Background Website Scan Request

The Manifest V3 background service worker acts as the extension-side coordinator for authenticated website-scan requests. When a supported tab needs analysis, the extension sends the URL to the configured backend API and waits for the structured risk response instead of embedding the machine-learning model directly inside the browser package. The request path also handles service unavailability and other API errors so a temporary backend problem does not cause an uncontrolled extension failure. Centralising remote scanning in the service worker limits duplicated networking logic across popup and content-script components. It also allows the backend model and contextual analysis to be updated independently of the user's local extension files as long as the API contract remains compatible.

```

105: async function continueOnce() {
106:   if (!warningData.originalUrl) {
107:     return;
108:   }
109:
110:   const { allowedUrls = [] } = await getStorage(["allowedUrls"]);
111:   allowedUrls[warningData.originalUrl] = WebsiteTrust.createDefaultAllowance();
112:   await recordWarningAction("once.continue");
113:   await setStorage({ allowedUrls });
114:   window.location.href = warningData.originalUrl;
115: }
116:
117: async function trustWebsite(trustScope) {
118:   if (!warningData.originalUrl || !websiteTrust) {
119:     return;
120:   }
121:
122:   const hostname = WebsiteTrust.canonicalizeHostname(warningData.originalUrl);
123:   const registrableDomain = WebsiteTrust.canonicalizeDomain(hostname);
124:   warningData.registrableDomain = hostname;
125:   if (!websiteTrust.getRegistrableDomain(hostname)) {
126:     const target = trustScope === "domain" ? registrableDomain : hostname;
127:     if (!target) {
128:       elements.trustMessage.hidden = false;
129:       elements.trustMessage.textContent = "This website identity cannot be trusted.";
130:       return;
131:     }
132:
133:     const highRisk = String(warningData.risk_level || "").toLowerCase().includes("high");
134:     const trustLabel = trustScope === "domain" ? "domain" : "website";
135:     if (!window.confirm(
136:       `${highRisk ? "PhishGuard currently considers this website High Risk." : ""}Trust ${target} as a personal ${trustLabel} decision? (Within this lowers only your personalized risk. It does not retrain the model, erase the original result, or enable vault autofill when
137:     )) {
138:       return;
139:     }
140:
141:     const stored = await getStorage([
142:       WebsiteTrust.TRUSTED_WEBSITES_STORAGE_KEY,
143:       "allowedUrls",
144:       ScanHistory.HISTORY_STORAGE_KEY,
145:     ]);
146:
147:     const currentEntries = WebsiteTrust.normalizeTrustedWebsites(
148:       stored[WebsiteTrust.TRUSTED_WEBSITES_STORAGE_KEY]
149:     );
150:
151:     const nextEntries = WebsiteTrust.addTrustedWebsite(
152:       currentEntries,
153:       hostname,
154:       Date.now(),
155:     );
156:
157:     const trustEntry = {
158:       trustScope,
159:       registrableDomain,
160:       source: "warning_page",
161:     };
162:
163:     const trustedEntry = nextEntries.find(
164:       (entry) => entry.trust_scope === trustScope && entry.domain === target
165:     );
166:     if (!trustedEntry) {
167:       const nextEntry = {
168:         trustScope,
169:         registrableDomain,
170:         source: "warning_page",
171:       };
172:       const updates = {
173:         [WebsiteTrust.TRUSTED_WEBSITES_STORAGE_KEY]: nextEntries,
174:         allowedUrls,
175:       };
176:       if (warningData.historyScand) {
177:         const assessment = WebsiteTrust.calculateWebsitePersonalizedRisk(
178:           warningData,
179:           trustedEntry
180:         );
181:         updates[ScanHistory.HISTORY_STORAGE_KEY] = ScanHistory.updateHistoryPersonalization(
182:           stored[ScanHistory.HISTORY_STORAGE_KEY],
183:           warningData.historyScand,
184:           {
185:             ...assessment,

```

Figure 92: Warning Page Continue and Trust Actions

The warning-page script implements three deliberately different user responses to a high-risk website. Go Back abandons the risky navigation, Continue Once allows only a temporary continuation for the current navigation context, and Trust Website creates a persistent personal trust decision after confirmation. Treating these as separate actions prevents a quick one-time override from silently becoming a long-term allow rule. The saved trust decision is later evaluated by the contextual layer and remains bounded by severe security evidence. This code therefore preserves user autonomy while ensuring that convenience actions do not automatically disable future phishing protection or retrain the underlying website model.

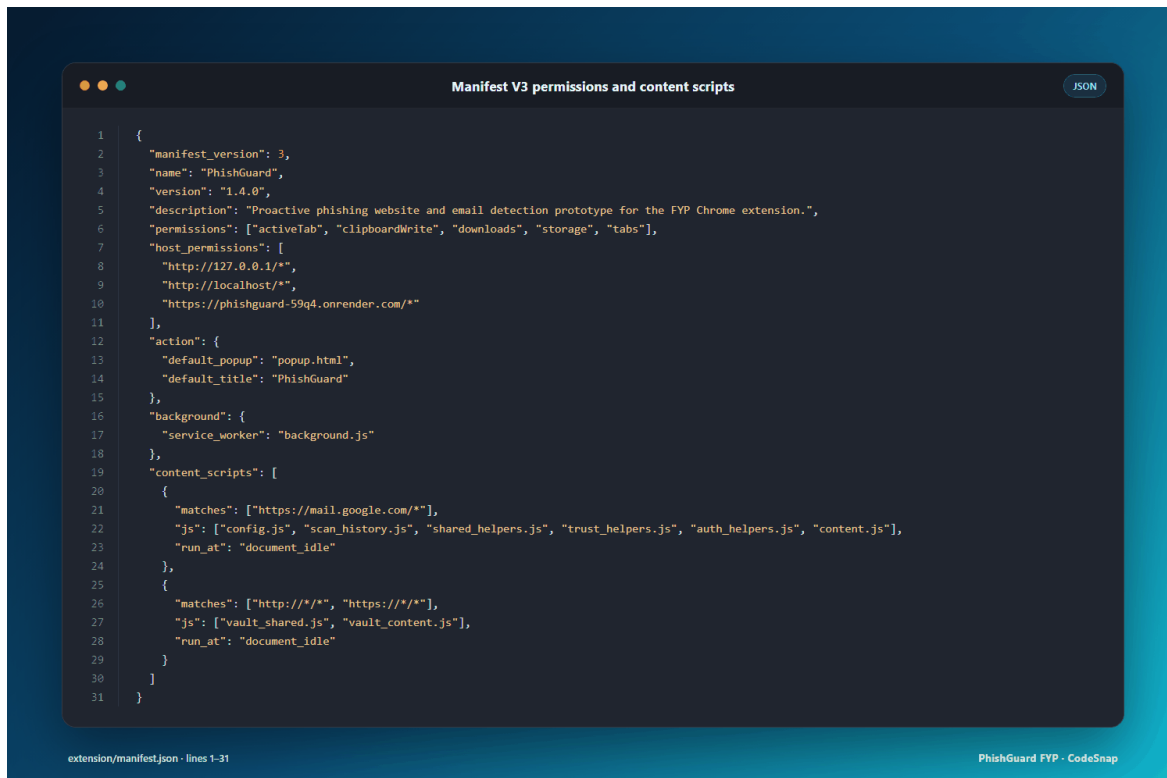


Figure 93: Manifest V3 Permissions and Content Scripts

The extension manifest defines the browser capabilities required by the final PhishGuard build and therefore provides a compact view of its security boundary. The configuration registers the popup, background service worker, warning and dashboard resources, Gmail and vault content scripts, and the host permissions required to communicate with the configured backend and supported web contexts. Permissions such as storage, tabs, activeTab, downloads, and clipboard access are declared explicitly rather than being obtained dynamically without visibility. Using Manifest V3 also places the background logic in a service worker rather than a permanently running background page. Keeping the declared permissions aligned with implemented features supports the project's principle of requesting only the browser access needed for protection, reporting, and credential-management functions.

4.6.6 Gmail Email Protection

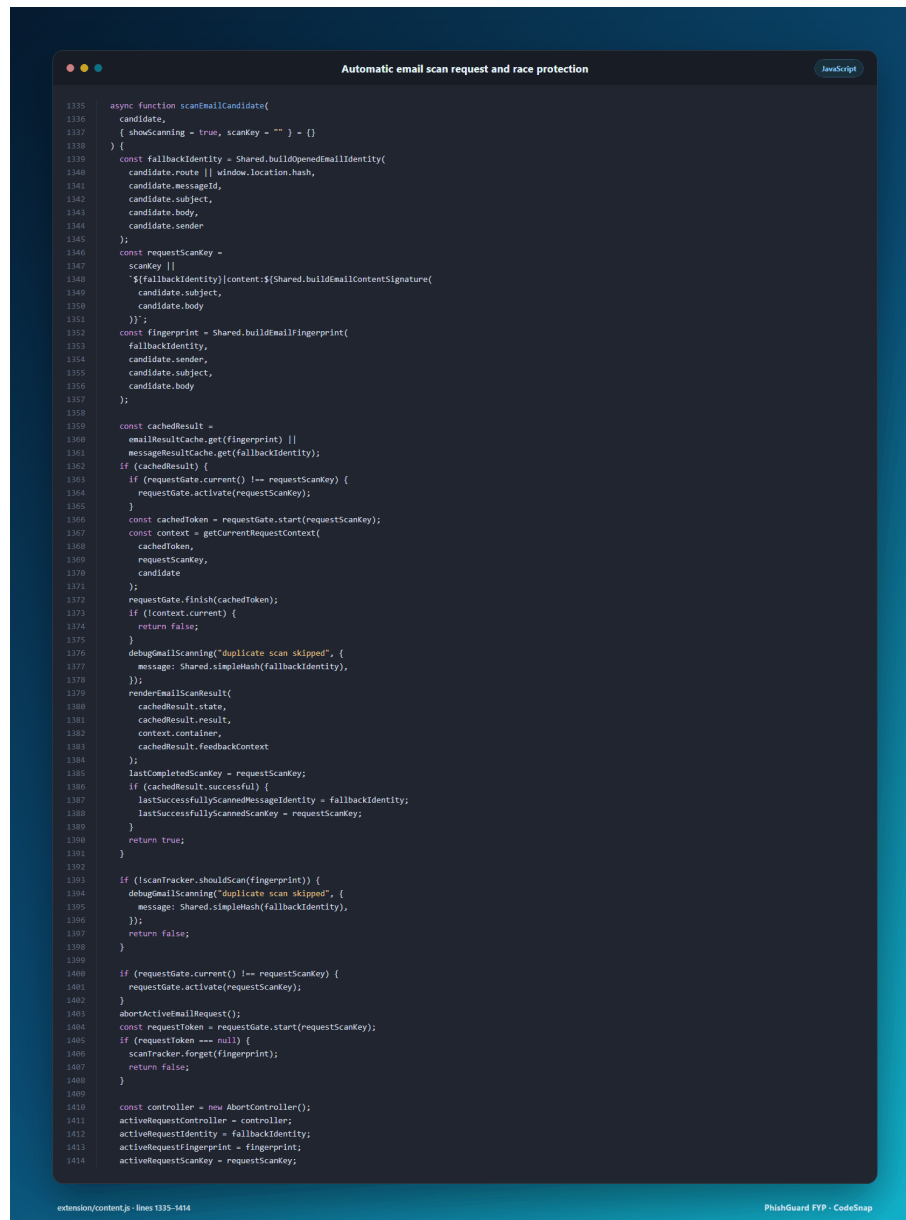


Figure 94: Automatic Gmail Scan Request and Race Protection

Automatic Gmail analysis must account for the fact that users can open, close, or switch between messages faster than a backend request completes. This content-script logic associates each scan with the identity of the email that was open when the request began and uses cancellation or stale-response checks when the active message changes. A late result for a previously opened email is therefore prevented from replacing the banner for the message currently on screen. Debouncing also reduces unnecessary repeated scans caused by Gmail's dynamic DOM updates. These controls are important for correctness because a technically accurate phishing score would still be misleading if it were displayed on the wrong message due to asynchronous race conditions.

4.6.7 Security Dashboard and PDF Reporting

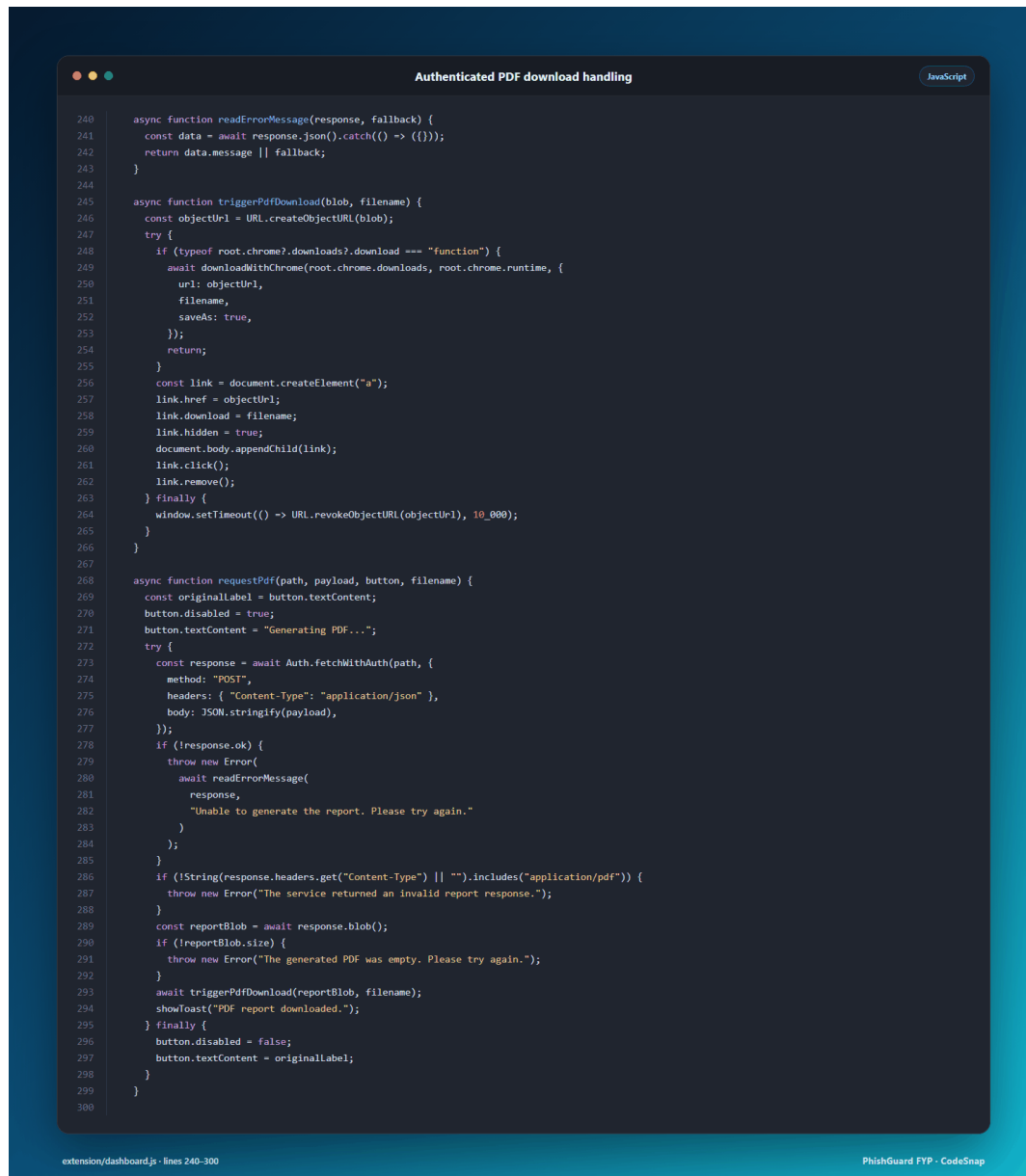


Figure 95: Authenticated PDF Download Handling

The Security Dashboard requests scan reports through an authenticated backend route and then handles the successful PDF response as binary data rather than ordinary JSON. Structured error responses are processed separately so authentication, ownership, or validation failures can be reported to the user without attempting to download an invalid file. The backend verifies that the requested scan or reporting range belongs to the authenticated user before generating the document. Range requests are also constrained by valid account and date boundaries. This separation between authenticated request handling, error processing, and browser download logic

prevents report generation from becoming an unauthorised path to another user's scan data while still providing a straightforward export experience.

4.6.8 Client-Side Encrypted Password Vault

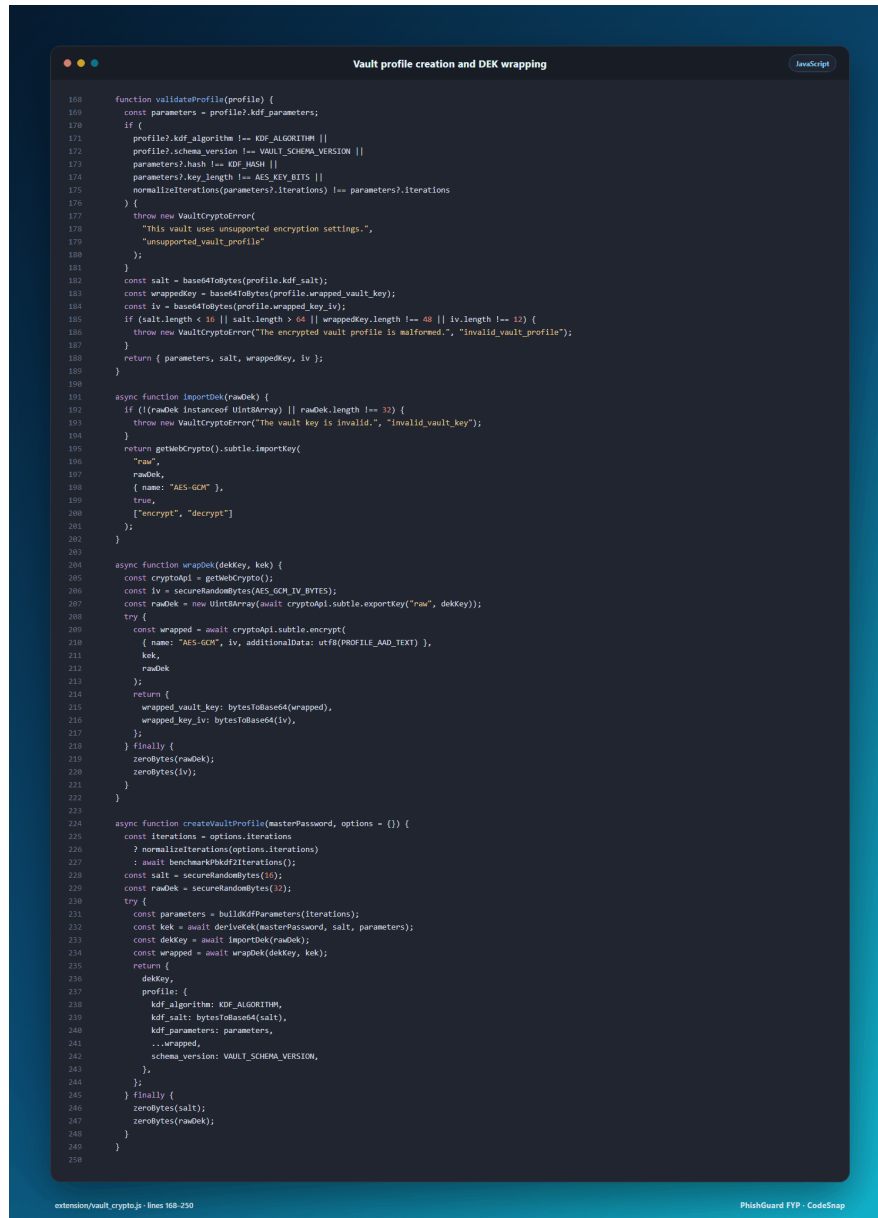


Figure 96: Vault Profile Creation and Data-Encryption-Key Wrapping

When a vault is created, the browser generates a random data-encryption key for protecting credential entries instead of encrypting every password directly with the user's master password. The master password is processed through the configured password-based key-derivation function to obtain key material that wraps the random vault key. The backend stores the wrapped key, salt, IV, KDF parameters, and schema version, but it does not receive the plaintext master password or unwrapped vault key. This layered design allows the master password to be changed by rewrapping

the vault key rather than decrypting and re-encrypting every credential entry. It also establishes a clear client-side cryptographic boundary between persistent server storage and usable plaintext credentials.

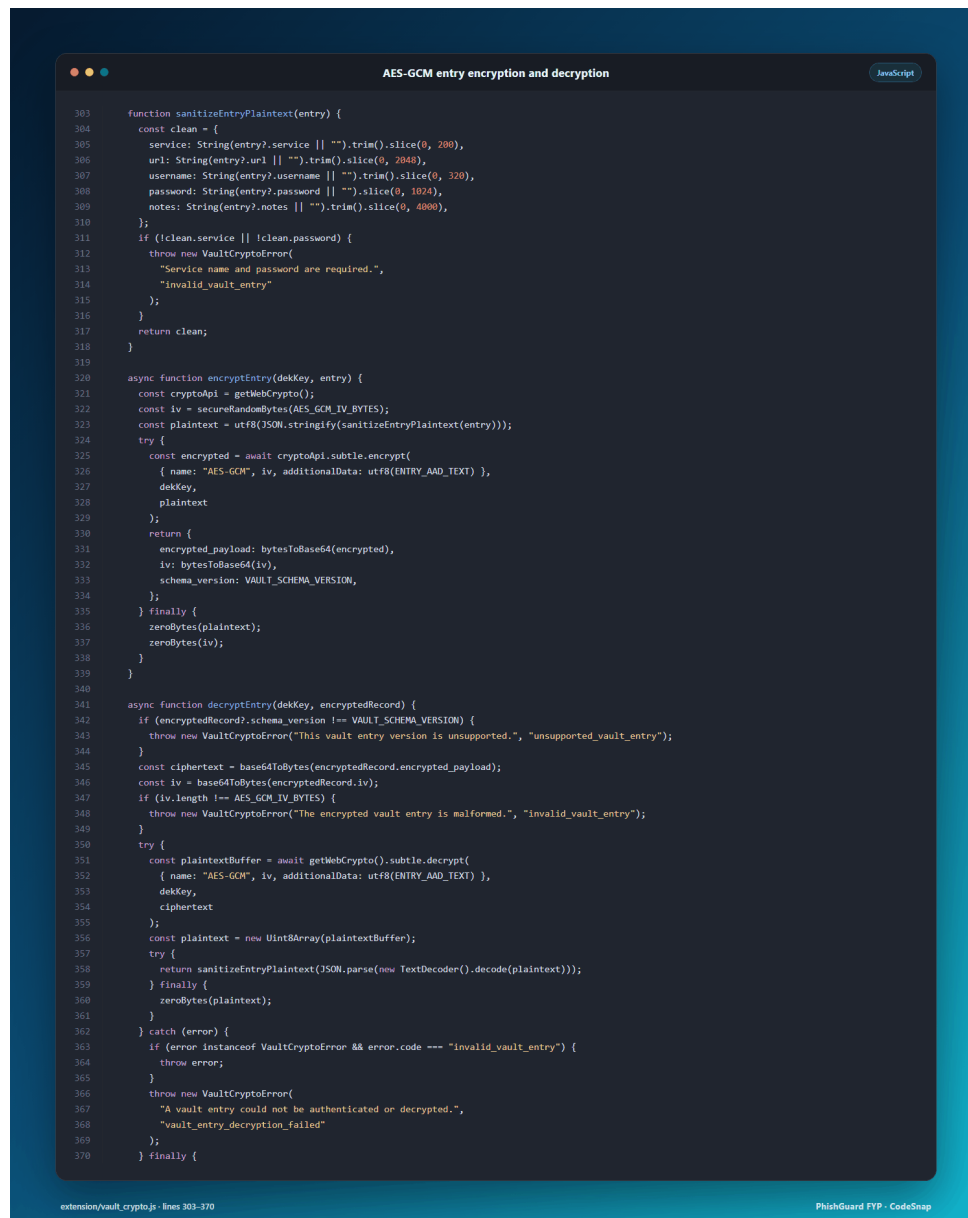


Figure 97: AES-GCM Vault Entry Encryption and Decryption

Individual vault entries are encrypted in the browser using AES-GCM before they are sent to the backend. A fresh IV is used for each encrypted payload so repeated credentials do not produce identical ciphertext, while the authentication tag provided by GCM allows tampering to be detected during decryption. The decrypted username, password, and related entry fields exist only after the user has unlocked the vault and the browser has successfully recovered the local data-encryption key. The backend stores the ciphertext, IV, version information, and ownership

relationship but does not need the ability to decrypt the entry. This design therefore protects both confidentiality and integrity of saved credentials while keeping cryptographic operations within the user's browser session.

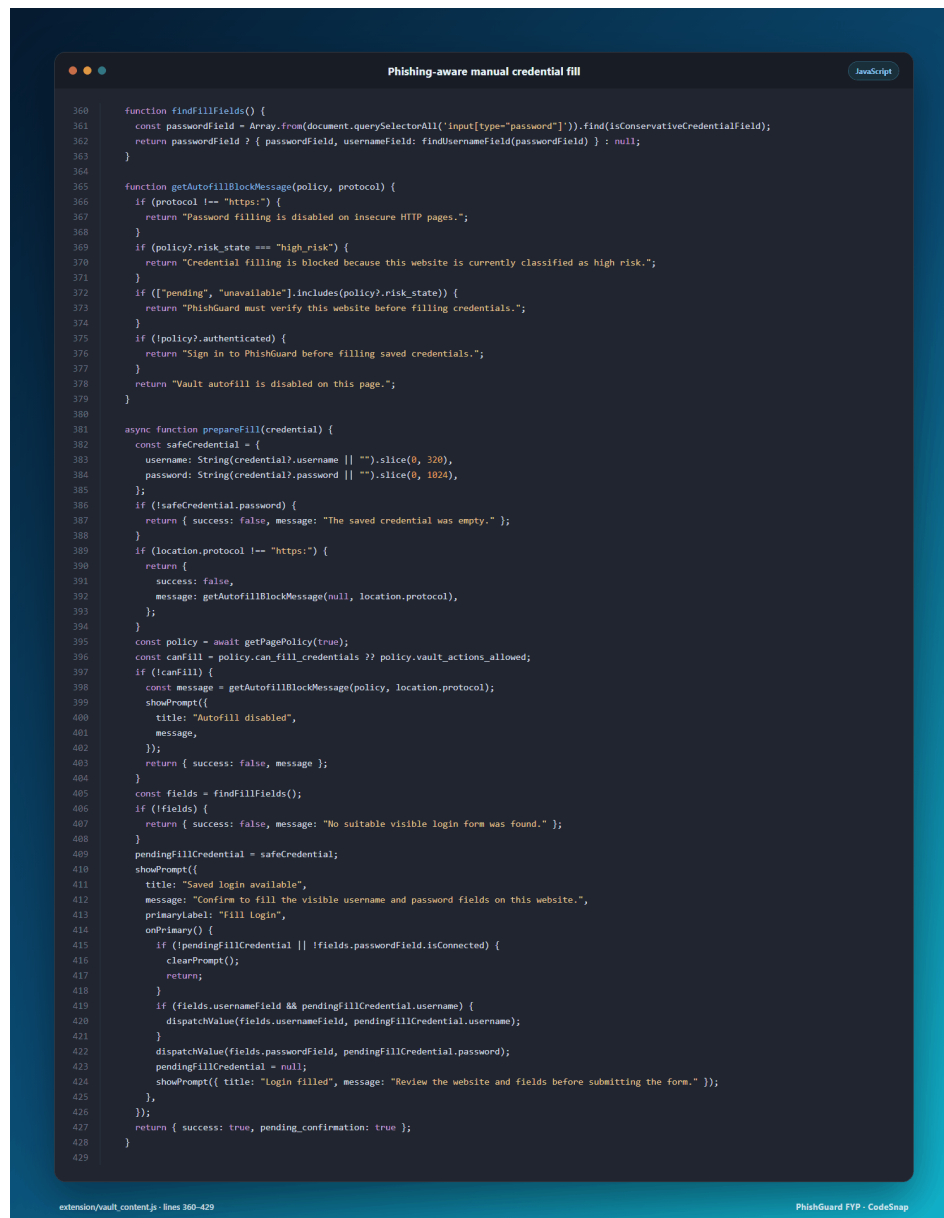


Figure 98: Phishing-Aware Manual Credential Fill

Credential filling is deliberately more restrictive than simply storing an entry in the vault. Before Fill Current Tab inserts a username and password, the extension identifies the intended browser tab, checks whether the saved credential matches the destination, and applies the relevant destination-safety policy. If the page is considered unsafe or does not satisfy the expected matching conditions, the fill operation can be refused even though the credential remains safely stored in the vault. This prevents the password manager from becoming a mechanism that automatically

supplies valid credentials to a suspicious phishing page. The implementation therefore connects the encrypted-vault feature directly to PhishGuard's anti-phishing objective instead of treating password filling as an independent convenience feature.



```

64 def validate_profile_payload(payload: object) -> dict[str, Any]:
65     if not isinstance(payload, dict):
66         raise AuthError("A valid vault profile is required.", 400, "invalid_vault_payload")
67     allowed_fields = {
68         "kdf_algorithm",
69         "kdf_salt",
70         "kdf_parameters",
71         "wrapped_vault_key",
72         "wrapped_key_iv",
73         "schema_version",
74     }
75     if set(payload) - allowed_fields:
76         raise AuthError(
77             "The vault profile contains unsupported fields.",
78             400,
79             "invalid_vault_payload",
80         )
81     if payload.get("kdf_algorithm") != VAULT_KDF_ALGORITHM:
82         raise AuthError(
83             "The vault profile uses an unsupported key-derivation algorithm.",
84             400,
85             "unsupported_vault_algorithm",
86         )
87     parameters = payload.get("kdf_parameters")
88     if not isinstance(parameters, dict):
89         raise AuthError(
90             "Vault key-derivation parameters are required.",
91             400,
92             "invalid_vault_payload",
93         )
94     allowed_parameters = {"iterations", "hash", "key_length"}
95     if set(parameters) != allowed_parameters:
96         raise AuthError(
97             "Vault key-derivation parameters are invalid.",
98             400,
99             "invalid_vault_payload",
100        )
101        iterations = parameters.get("iterations")
102        if (
103            isinstance(iterations, bool)
104            or not isinstance(iterations, int)
105            or not MIN_PBKDF2_ITERATIONS <= iterations <= MAX_PBKDF2_ITERATIONS
106            or parameters.get("hash") != "SHA-256"
107            or parameters.get("key_length") != 256
108        ):
109            raise AuthError(
110                "Vault key-derivation parameters are outside the supported range.",
111                400,
112                "invalid_vault_payload",
113            )
114        schema_version = payload.get("schema_version")
115        if schema_version != VAULT_SCHEMA_VERSION:
116            raise AuthError(
117                "The vault profile schema version is unsupported.",
118                400,
119                "unsupported_vault_schema",
120            )
121        return {
122            "kdf_algorithm": VAULT_KDF_ALGORITHM,
123            "kdf_salt": _decode_base64(
124                payload.get("kdf_salt"), "kdf_salt", minimum_bytes=16, maximum_bytes=64
125            ),
126            "kdf_parameters": json.dumps(parameters, separators=(",", ":"), sort_keys=True),
127            "wrapped_vault_key": _decode_base64(
128                payload.get("wrapped_vault_key"),
129                "wrapped_vault_key",
130                minimum_bytes=48,
131                maximum_bytes=128,
132            ),
133            "wrapped_key_iv": _decode_base64(
134                payload.get("wrapped_key_iv"),
135                "wrapped_key_iv",
136                minimum_bytes=12,
137                maximum_bytes=12,
138            ),
139            "schema_version": VAULT_SCHEMA_VERSION,
140        }
141    def validate_entry_payload(payload: object) -> dict[str, Any]:

```

Figure 99: Ciphertext-Only Backend Vault Validation

The backend vault routes enforce ownership, payload structure, and schema-version requirements without decrypting the credential contents. Incoming vault records are expected to contain ciphertext and the associated cryptographic metadata produced by the browser, and requests are scoped to the authenticated user before data is created, updated, or returned. Because the server does not need the master password or plaintext vault key, it can validate persistence rules while preserving the client-side encryption boundary. This reduces the sensitivity of a backend database

compromise compared with storing directly readable usernames and passwords. Versioned payload validation also provides a controlled path for future cryptographic or schema changes without silently accepting malformed encrypted records.

4.7 Summary

Chapter 4 demonstrates that the Part 1 architecture was converted into a deployable PhishGuard prototype and documents the implementation through a deliberately reduced set of representative screenshots and code excerpts. The completed system includes website and email phishing detection, contextual risk analysis, Manifest V3 browser integration, Gmail protection, mandatory MFA and security events, personal trust, privacy-safe reporting, a browser-encrypted password vault, and a direct-download installation workflow from the public website.

The final evidence also shows that user control and explainability are preserved across the system. Raw model outputs remain separate from contextual and trust effects, warning actions distinguish temporary continuation from persistent trust, vault credentials remain encrypted before backend storage, and users can download the extension package, load it unpacked in Chrome, create an account normally, and complete mandatory MFA before using the protection features. These implemented controls provide the basis for the testing and evaluation presented in Chapter 5.

CHAPTER 5: RESULT AND DISCUSSION

5.1 Introduction

This chapter evaluates the completed PhishGuard prototype through automated software testing, model evaluation, regression testing, and user acceptance testing. The FYP guideline requires at least two testing techniques; this project uses automated unit/regression testing as the technical technique and User Acceptance Testing (UAT) as the user-facing technique. Automated tests provide repeatable evidence that individual modules and security boundaries behave as intended, while UAT evaluates whether intended users can understand and operate the finished system in realistic scenarios.

The latest source package contains 156 Python test functions and twelve dedicated Node.js test programs. The project documentation records the Python release suite as passing in its pinned development environment. In this report-generation environment, the JavaScript test programs were independently executed and all twelve passed; the Python suite could not be independently re-executed because the container does not contain the project-specific Flask, PyOTP, and tldextract dependencies. The final report therefore distinguishes source-project verification records from the independent document-preparation check rather than presenting an unexecuted result as newly verified.

5.2 Testing Design / Plan

5.2.1 Automated Unit and Regression Testing

Automated testing is appropriate because PhishGuard contains many deterministic security and data-processing behaviours that can be evaluated repeatedly without requiring a human tester. The Python suite covers backend authorization, MFA, model contracts, contextual risk, reporting, domain normalization, vault ownership, deployment configuration, static website requirements, training helpers, and regression cases. Node.js programs cover extension authentication helpers, background URL filtering, API configuration, Gmail triggers and polling, history/dashboard helpers, trusted sender/website helpers, and vault cryptographic/content logic.

ID	Area	Test Description	Expected Result
PY-001	Authentication & MFA	Registration validation and password hashing	The stated behavior is enforced without violating existing security or regression constraints.

ID	Area	Test Description	Expected Result
PY-002	Authentication & MFA	Mfa enrollment qr and enablement gate	The stated behavior is enforced without violating existing security or regression constraints.
PY-003	Authentication & MFA	Two stage totp login and protected access	The stated behavior is enforced without violating existing security or regression constraints.
PY-004	Authentication & MFA	Malformed expired and bruteforced totp are rejected	The stated behavior is enforced without violating existing security or regression constraints.
PY-005	Authentication & MFA	Password failures are source limited without account lockout	The stated behavior is enforced without violating existing security or regression constraints.
PY-006	Authentication & MFA	Progressive mfa lockouts reset after success	The stated behavior is enforced without violating existing security or regression constraints.
PY-007	Authentication & MFA	Security events are owned acknowledgeable and non sensitive	The stated behavior is enforced without violating existing security or regression constraints.
PY-008	Authentication & MFA	Expired challenge and totp replay remain blocked	The stated behavior is enforced without violating existing security or regression constraints.
PY-009	Authentication & MFA	Recovery code is one time and regeneration invalidates old codes	The stated behavior is enforced without violating existing security or regression constraints.
PY-010	Authentication & MFA	Authenticator reset replaces secret and revokes sessions	The stated behavior is enforced without violating existing security or regression constraints.
PY-011	Authentication & MFA	Logout revokes the backend session	The stated behavior is enforced without violating existing security or regression constraints.
PY-012	Authentication & MFA	Expired session is rejected	The stated behavior is enforced without violating existing security or regression constraints.
PY-013	Backend scan APIs	Health route	The stated behavior is enforced without violating existing security or regression constraints.

ID	Area	Test Description	Expected Result
PY-014	Backend scan APIs	Cors allows extension origins but not arbitrary websites	The stated behavior is enforced without violating existing security or regression constraints.
PY-015	Backend scan APIs	Oversized url is rejected before model prediction	The stated behavior is enforced without violating existing security or regression constraints.
PY-016	Backend scan APIs	Malformed email link metadata is rejected	The stated behavior is enforced without violating existing security or regression constraints.
PY-017	Backend scan APIs	Website bundle uses saved threshold	The stated behavior is enforced without violating existing security or regression constraints.
PY-018	Backend scan APIs	Scan url returns expected json structure	The stated behavior is enforced without violating existing security or regression constraints.
PY-019	Backend scan APIs	Low risk website scan has truthful explanation	The stated behavior is enforced without violating existing security or regression constraints.
PY-020	Backend scan APIs	Website contextual score can raise a low raw model result	The stated behavior is enforced without violating existing security or regression constraints.
PY-021	Backend scan APIs	Invalid url input returns safe error	The stated behavior is enforced without violating existing security or regression constraints.
PY-022	Backend scan APIs	Status exposes safe website model metadata	The stated behavior is enforced without violating existing security or regression constraints.
PY-023	Backend scan APIs	Missing email model returns clear response	The stated behavior is enforced without violating existing security or regression constraints.
PY-024	Backend scan APIs	Empty email input returns safe error	The stated behavior is enforced without violating existing security or regression constraints.
PY-025	Backend scan APIs	Short email input returns safe error	The stated behavior is enforced without violating existing security or regression constraints.

ID	Area	Test Description	Expected Result
PY-026	Backend scan APIs	Scan email legitimate sample returns clear result	The stated behavior is enforced without violating existing security or regression constraints.
PY-027	Backend scan APIs	Scan email phishing sample returns clear result	The stated behavior is enforced without violating existing security or regression constraints.
PY-028	Backend scan APIs	Email response separates summary and risk assessment	The stated behavior is enforced without violating existing security or regression constraints.
PY-029	Backend scan APIs	Email summary can be skipped without changing scan contract	The stated behavior is enforced without violating existing security or regression constraints.
PY-030	Backend scan APIs	Personal trust adjustment is reported separately	The stated behavior is enforced without violating existing security or regression constraints.
PY-031	Deployment configuration	Development defaults to project relative sqlite	The stated behavior is enforced without violating existing security or regression constraints.
PY-032	Deployment configuration	Relative sqlite url is independent of working directory	The stated behavior is enforced without violating existing security or regression constraints.
PY-033	Deployment configuration	Neon style database urls use psycopg driver	The stated behavior is enforced without violating existing security or regression constraints.
PY-034	Deployment configuration	Unsupported database scheme is rejected	The stated behavior is enforced without violating existing security or regression constraints.
PY-035	Deployment configuration	Unknown backend environment is rejected	The stated behavior is enforced without violating existing security or regression constraints.
PY-036	Deployment configuration	Production has no implicit cors origins	The stated behavior is enforced without violating existing security or regression constraints.
PY-037	Deployment configuration	Development cors patterns are narrow	The stated behavior is enforced without violating existing security or regression constraints.

ID	Area	Test Description	Expected Result
PY-038	Deployment configuration	Sqlite schema can be initialized explicitly	The stated behavior is enforced without violating existing security or regression constraints.
PY-039	Deployment configuration	Initial migration builds authentication schema	The stated behavior is enforced without violating existing security or regression constraints.
PY-040	Deployment configuration	Model paths do not depend on working directory	The stated behavior is enforced without violating existing security or regression constraints.
PY-041	Deployment configuration	Backend runtime has no hardcoded windows project path	The stated behavior is enforced without violating existing security or regression constraints.
PY-042	Domain normalization	Authorised domains and subdomains are accepted	The stated behavior is enforced without violating existing security or regression constraints.
PY-043	Domain normalization	Lookalike and parent domain attacks are rejected	The stated behavior is enforced without violating existing security or regression constraints.
PY-044	Domain normalization	Newsletter brand lookalikes are rejected	The stated behavior is enforced without violating existing security or regression constraints.
PY-045	Domain normalization	Case trailing dot and port are canonicalised	The stated behavior is enforced without violating existing security or regression constraints.
PY-046	Domain normalization	Unicode idna cannot match by visual similarity	The stated behavior is enforced without violating existing security or regression constraints.
PY-047	Domain normalization	Url host is separate from path and query	The stated behavior is enforced without violating existing security or regression constraints.
PY-048	Domain normalization	Public suffix registrable domain	The stated behavior is enforced without violating existing security or regression constraints.
PY-049	Email contextual analysis	Legitimate linkedin notifications remain low risk	The stated behavior is enforced without violating existing security or regression constraints.

ID	Area	Test Description	Expected Result
PY-050	Email contextual analysis	Fake linkedin with unrelated link is high risk	The stated behavior is enforced without violating existing security or regression constraints.
PY-051	Email contextual analysis	Linkedin lookalike sender is not trusted	The stated behavior is enforced without violating existing security or regression constraints.
PY-052	Email contextual analysis	Legitimate github notification remains low risk	The stated behavior is enforced without violating existing security or regression constraints.
PY-053	Email contextual analysis	Google security and fake google are separated	The stated behavior is enforced without violating existing security or regression constraints.
PY-054	Email contextual analysis	Generic security words do not force phishing	The stated behavior is enforced without violating existing security or regression constraints.
PY-055	Email contextual analysis	Consistency reduction is limited and requires links	The stated behavior is enforced without violating existing security or regression constraints.
PY-056	Email contextual analysis	Missing sender metadata is unknown not a mismatch	The stated behavior is enforced without violating existing security or regression constraints.
PY-057	Email contextual analysis	Suspicious links override trusted looking branding	The stated behavior is enforced without violating existing security or regression constraints.
PY-058	Email contextual analysis	Adjustment rules are bounded and documented	The stated behavior is enforced without violating existing security or regression constraints.
PY-059	Email contextual analysis	Display name alone never reduces risk	The stated behavior is enforced without violating existing security or regression constraints.
PY-060	Email contextual analysis	One legitimate link cannot neutralize suspicious links	The stated behavior is enforced without violating existing security or regression constraints.
PY-061	Email contextual analysis	Redirect target is audited and prevents consistency reduction	The stated behavior is enforced without violating existing security or regression constraints.

ID	Area	Test Description	Expected Result
PY-062	Email contextual analysis	Urgent payment and credentials override consistency	The stated behavior is enforced without violating existing security or regression constraints.
PY-063	Email contextual analysis	Score clamping and duplicate evidence	The stated behavior is enforced without violating existing security or regression constraints.
PY-064	Email contextual analysis	Summary is plain text and bounded	The stated behavior is enforced without violating existing security or regression constraints.
PY-065	Email contextual analysis	Short malformed summary uses safe fallback	The stated behavior is enforced without violating existing security or regression constraints.
PY-066	Email contextual analysis	Personal trust reduction is bounded and gradual	The stated behavior is enforced without violating existing security or regression constraints.
PY-067	Email contextual analysis	Domain trust is smaller than exact sender trust	The stated behavior is enforced without violating existing security or regression constraints.
PY-068	Email contextual analysis	Exact sender trust does not apply to another address	The stated behavior is enforced without violating existing security or regression constraints.
PY-069	Email contextual analysis	Domain trust requires confirmation	The stated behavior is enforced without violating existing security or regression constraints.
PY-070	Email contextual analysis	Public provider cannot be broadly trusted	The stated behavior is enforced without violating existing security or regression constraints.
PY-071	Email contextual analysis	Strong text score alone does not cancel explicit trust	The stated behavior is enforced without violating existing security or regression constraints.
PY-072	Email contextual analysis	Generic credential language does not override trust without attack evidence	The stated behavior is enforced without violating existing security or regression constraints.
PY-073	Email contextual analysis	Evidence breakdown separates increasing and reducing signals	The stated behavior is enforced without violating existing security or regression constraints.

ID	Area	Test Description	Expected Result
PY-074	Email contextual analysis	Dangerous download overrides personal trust	The stated behavior is enforced without violating existing security or regression constraints.
PY-075	Email contextual analysis	Suspicious links and credentials override personal trust	The stated behavior is enforced without violating existing security or regression constraints.
PY-076	Email contextual analysis	Personal trust score is clamped	The stated behavior is enforced without violating existing security or regression constraints.
PY-077	Email model regression	Normal meeting email is legitimate	The stated behavior is enforced without violating existing security or regression constraints.
PY-078	Email model regression	Urgent password email is phishing	The stated behavior is enforced without violating existing security or regression constraints.
PY-079	Email release gate	Independent newsletter and phishing release gate	The stated behavior is enforced without violating existing security or regression constraints.
PY-080	Email release gate	Order identifier does not claim apu brand	The stated behavior is enforced without violating existing security or regression constraints.
PY-081	Email release gate	Modern email model and final risk gate	The stated behavior is enforced without violating existing security or regression constraints.
PY-082	Email summaries	Linkedin notification summary	The stated behavior is enforced without violating existing security or regression constraints.
PY-083	Email summaries	Github notification summary	The stated behavior is enforced without violating existing security or regression constraints.
PY-084	Email summaries	University announcement summary	The stated behavior is enforced without violating existing security or regression constraints.
PY-085	Email summaries	Security alert summary	The stated behavior is enforced without violating existing security or regression constraints.

ID	Area	Test Description	Expected Result
PY-086	Email summaries	Order confirmation summary	The stated behavior is enforced without violating existing security or regression constraints.
PY-087	Email summaries	Newsletter summary ignores footer	The stated behavior is enforced without violating existing security or regression constraints.
PY-088	Email summaries	Short email uses fallback	The stated behavior is enforced without violating existing security or regression constraints.
PY-089	Email summaries	Footer only email uses fallback	The stated behavior is enforced without violating existing security or regression constraints.
PY-090	Email summaries	Phishing credential request summary is neutral	The stated behavior is enforced without violating existing security or regression constraints.
PY-091	Email summaries	Quoted reply is not used by summary	The stated behavior is enforced without violating existing security or regression constraints.
PY-092	Email summaries	Asus summary uses body details instead of repeating subject	The stated behavior is enforced without violating existing security or regression constraints.
PY-093	Extension helper integration	Background url filtering helpers	The stated behavior is enforced without violating existing security or regression constraints.
PY-094	Extension helper integration	Email history and manual scan helpers	The stated behavior is enforced without violating existing security or regression constraints.
PY-095	Extension helper integration	Gmail navigation trigger helpers	The stated behavior is enforced without violating existing security or regression constraints.
PY-096	Extension helper integration	Gmail polling fallback helpers	The stated behavior is enforced without violating existing security or regression constraints.
PY-097	Extension helper integration	Authentication and user scoped storage helpers	The stated behavior is enforced without violating existing security or regression constraints.

ID	Area	Test Description	Expected Result
PY-098	Extension helper integration	Api environment and failure helpers	The stated behavior is enforced without violating existing security or regression constraints.
PY-099	Extension helper integration	Security dashboard and scan history helpers	The stated behavior is enforced without violating existing security or regression constraints.
PY-100	Extension helper integration	Personal trusted website helpers	The stated behavior is enforced without violating existing security or regression constraints.
PY-101	Extension production config	Manifest has exact render and local development permissions	The stated behavior is enforced without violating existing security or regression constraints.
PY-102	Extension production config	Runtime consumers have no hardcoded api origins or native fetch	The stated behavior is enforced without violating existing security or regression constraints.
PY-103	Extension production config	Expected api routes use the shared request helpers	The stated behavior is enforced without violating existing security or regression constraints.
PY-104	Extension production config	Unavailable scan payload is not labelled legitimate	The stated behavior is enforced without violating existing security or regression constraints.
PY-105	PDF security reports	Individual report requires authentication and receipt ownership	The stated behavior is enforced without violating existing security or regression constraints.
PY-106	PDF security reports	Email report separates model score and contextual evidence	The stated behavior is enforced without violating existing security or regression constraints.
PY-107	PDF security reports	Personalized report preserves original score and bounds trust	The stated behavior is enforced without violating existing security or regression constraints.
PY-108	PDF security reports	Severe website evidence caps trust and scan id is bound	The stated behavior is enforced without violating existing security or regression constraints.
PY-109	PDF security reports	Legacy low risk report uses truthful explanation fallback	The stated behavior is enforced without violating existing security or regression constraints.

ID	Area	Test Description	Expected Result
PY-110	PDF security reports	Custom and preset ranges filter signed records	The stated behavior is enforced without violating existing security or regression constraints.
PY-111	PDF security reports	Range before account creation is clamped not rejected	The stated behavior is enforced without violating existing security or regression constraints.
PY-112	PDF security reports	Invalid and empty ranges are rejected cleanly	The stated behavior is enforced without violating existing security or regression constraints.
PY-113	PDF security reports	Large report spans pages and excludes supplied secrets	The stated behavior is enforced without violating existing security or regression constraints.
PY-114	Static website	Landing page has required semantic sections	The stated behavior is enforced without violating existing security or regression constraints.
PY-115	Static website	Download is not an imaginary store link	The stated behavior is enforced without violating existing security or regression constraints.
PY-116	Static website	Responsive and reduced motion styles exist	The stated behavior is enforced without violating existing security or regression constraints.
PY-117	Static website	Static site has no remote runtime dependencies	The stated behavior is enforced without violating existing security or regression constraints.
PY-118	Static website	Static site defines a restrictive content policy	The stated behavior is enforced without violating existing security or regression constraints.
PY-119	Training helpers	Email label normalization	The stated behavior is enforced without violating existing security or regression constraints.
PY-120	Training helpers	Generic spam is not mapped to phishing	The stated behavior is enforced without violating existing security or regression constraints.
PY-121	Training helpers	Email template normalization removes variable fields	The stated behavior is enforced without violating existing security or regression constraints.

ID	Area	Test Description	Expected Result
PY-122	Training helpers	Email regression gate is independent from supplements	The stated behavior is enforced without violating existing security or regression constraints.
PY-123	Training helpers	Path digits are not website model features	The stated behavior is enforced without violating existing security or regression constraints.
PY-124	Training helpers	Raw query length is not a model feature	The stated behavior is enforced without violating existing security or regression constraints.
PY-125	Training helpers	Brand impersonation features are explainable	The stated behavior is enforced without violating existing security or regression constraints.
PY-126	Training helpers	Contextual query features distinguish path evidence from search terms	The stated behavior is enforced without violating existing security or regression constraints.
PY-127	Vault backend	Vault requires an mfa authenticated session	The stated behavior is enforced without violating existing security or regression constraints.
PY-128	Vault backend	Setup and profile store only opaque encryption material	The stated behavior is enforced without violating existing security or regression constraints.
PY-129	Vault backend	Entry crud never requires plaintext fields	The stated behavior is enforced without violating existing security or regression constraints.
PY-130	Vault backend	Cross user read update and delete are blocked	The stated behavior is enforced without violating existing security or regression constraints.
PY-131	Vault backend	Plaintext shaped malformed and oversized payloads fail	The stated behavior is enforced without violating existing security or regression constraints.
PY-132	Vault backend	Profile update replaces wrapping material without entries	The stated behavior is enforced without violating existing security or regression constraints.
PY-133	Vault backend	User deletion cascades to encrypted vault data	The stated behavior is enforced without violating existing security or regression constraints.

ID	Area	Test Description	Expected Result
PY-134	Vault backend	Reset requires account password fresh totp and confirmation	The stated behavior is enforced without violating existing security or regression constraints.
PY-135	Vault integration	Manifest loads conservative vault content script	The stated behavior is enforced without violating existing security or regression constraints.
PY-136	Vault integration	Vault page uses local scripts and shared authenticated api	The stated behavior is enforced without violating existing security or regression constraints.
PY-137	Vault integration	Plaintext vault material is not persisted by extension code	The stated behavior is enforced without violating existing security or regression constraints.
PY-138	Vault integration	Backend vault routes do not accept plaintext entry fields	The stated behavior is enforced without violating existing security or regression constraints.
PY-139	Vault integration	Vault migration follows existing auth revision	The stated behavior is enforced without violating existing security or regression constraints.
PY-140	Website contextual analysis	Four category long url release gate	The stated behavior is enforced without violating existing security or regression constraints.
PY-141	Website contextual analysis	Existing long safe and phishing model gates pass contextual scoring	The stated behavior is enforced without violating existing security or regression constraints.
PY-142	Website contextual analysis	Prize campaign regression is high risk without changing model score	The stated behavior is enforced without violating existing security or regression constraints.
PY-143	Website contextual analysis	Promotional redirect campaign is high risk without length rule	The stated behavior is enforced without violating existing security or regression constraints.
PY-144	Website contextual analysis	Long or tracking heavy url is not high risk by length alone	The stated behavior is enforced without violating existing security or regression constraints.
PY-145	Website contextual analysis	Severe ip and punycode credential structures override low model scores	The stated behavior is enforced without violating existing security or regression constraints.

ID	Area	Test Description	Expected Result
PY-146	Website contextual analysis	Low risk long url explanation is truthful	The stated behavior is enforced without violating existing security or regression constraints.
PY-147	Website model regression	Feature order matches saved model and backend prediction	The stated behavior is enforced without violating existing security or regression constraints.
PY-148	Website model regression	Metadata matches saved model artifact	The stated behavior is enforced without violating existing security or regression constraints.
PY-149	Website model regression	Common trusted domains remain legitimate	The stated behavior is enforced without violating existing security or regression constraints.
PY-150	Website model regression	Long social urls remain legitimate	The stated behavior is enforced without violating existing security or regression constraints.
PY-151	Website model regression	Long search engine urls remain legitimate	The stated behavior is enforced without violating existing security or regression constraints.
PY-152	Website model regression	Shared training regression gate matches saved model	The stated behavior is enforced without violating existing security or regression constraints.
PY-153	Website model regression	Long youtube and github urls remain legitimate	The stated behavior is enforced without violating existing security or regression constraints.
PY-154	Website model regression	Suspicious lookalikes remain phishing	The stated behavior is enforced without violating existing security or regression constraints.
PY-155	Website model regression	Long phishing urls remain phishing	The stated behavior is enforced without violating existing security or regression constraints.
PY-156	Website model regression	Brand impersonation urls remain phishing	The stated behavior is enforced without violating existing security or regression constraints.

Table 12: Python Automated Test Plan

ID	Test Program	Coverage	Expected Result
JS-01	extension_auth_helpers.test.js	Extension Auth Helpers	Program exits successfully and all assertions pass.
JS-02	extension_background_helpers.test.js	Extension Background Helpers	Program exits successfully and all assertions pass.
JS-03	extension_config.test.js	Extension Config	Program exits successfully and all assertions pass.
JS-04	extension_email_helpers.test.js	Extension Email Helpers	Program exits successfully and all assertions pass.
JS-05	extension_gmail_polling.test.js	Extension Gmail Polling	Program exits successfully and all assertions pass.
JS-06	extension_gmail_trigger.test.js	Extension Gmail Trigger	Program exits successfully and all assertions pass.
JS-07	extension_scan_history.test.js	Extension Scan History	Program exits successfully and all assertions pass.
JS-08	extension_trust_helpers.test.js	Extension Trust Helpers	Program exits successfully and all assertions pass.
JS-09	extension_trusted_website_helpers.test.js	Extension Trusted Website Helpers	Program exits successfully and all assertions pass.
JS-10	extension_vault_content.test.js	Extension Vault Content	Program exits successfully and all assertions pass.
JS-11	extension_vault_crypto.test.js	Extension Vault Crypto	Program exits successfully and all assertions pass.
JS-12	extension_vault_helpers.test.js	Extension Vault Helpers	Program exits successfully and all assertions pass.

Table 13: Node.js Extension Test Plan

5.2.2 User Acceptance Testing Design

Human-participant UAT was designed as a task-based evaluation of the final PhishGuard build rather than as a proposed or developer-only test. Three intended users completed the same individual testing form independently: keevanrajah (26, Associate Technical Analyst, four years of cybersecurity experience), Vicknesh Arumugam (24, InfraSecOps Engineer, two years of cybersecurity experience), and Amresh (29, Cybersecurity Consultant, six years of cybersecurity

experience). Testing was performed in Google Chrome on Windows 11 or Windows 11 Pro. Each participant attempted the same sixteen practical tasks, producing 48 comparable task executions across installation, account creation and MFA, website and Gmail protection, trust controls, dashboard and PDF reporting, security-event review, Password Vault use, settings, and logout.

For every practical task, the form provided a fixed procedure and expected result, followed by Pass/Fail and Notes fields. Table 14 reproduces the sixteen tasks that were actually attempted by all three participants. Only legitimate websites, sanitized phishing fixtures, controlled suspicious pages, test emails, and dedicated test credentials were used; participants were not asked to visit live malicious infrastructure or enter real credentials into suspicious pages. After the practical tasks, each participant rated twelve usability statements from 1 (Strongly Disagree) to 5 (Strongly Agree), selected one overall decision from Accepted, Accepted with Minor Changes, or Not Accepted, provided free-text feedback, and confirmed that the responses reflected personal use of PhishGuard. The completed execution results are reported in Section 5.3.6 and the signed/confirmed forms are reproduced in Section 5.3.7.

ID	Actual UAT Task	Acceptance Condition
UAT-01	Install & load extension	Extension loads and opens successfully without any beta/activation code.
UAT-02	Create account & enrol MFA	Registration succeeds; MFA is mandatory and recovery codes are shown.
UAT-03	Sign in	Authenticated features become available only after MFA succeeds.
UAT-04	Scan normal website	A timely low-risk result and understandable explanation are shown without unnecessary blocking.
UAT-05	Scan controlled suspicious URL	PhishGuard shows a high-risk/suspicious result and warning with understandable evidence.
UAT-06	Continue Once	The current navigation proceeds once and the site is not saved as trusted.
UAT-07	Trust & remove website trust	Trust is visible and reversible; normal handling returns after removal.
UAT-08	Manual website scan	The result and explanation are consistent with automatic website protection.
UAT-09	Scan Gmail message	The banner updates for the current message and presents understandable email-risk evidence.
UAT-10	Dismiss banner & sender trust	Dismissal stays scoped to Email A and sender trust is saved only for the current user.
UAT-11	Review dashboard & trust	Dashboard information loads correctly and trust/security records can be reviewed and managed.
UAT-12	Download PDF reports	Readable PDFs download with the expected user, risk, evidence, and explanation details.
UAT-13	Review security alert	A suspicious sign-in/security event appears without claiming the failed-MFA attempt gained access.

ID	Actual UAT Task	Acceptance Condition
UAT-14	Unlock & use Password Vault	The vault requires the correct master password and hides plaintext credentials while locked.
UAT-15	Generate password & Fill Current Tab	A matching tab is found and credentials fill only when destination checks permit it.
UAT-16	Settings & logout	Setting behaviour is understandable and logout removes access to authenticated features.

Table 14: UAT Scenario Design

5.2.3 Testing Environment and Acceptance Criteria

The intended acceptance environment is Google Chrome with the unpacked PhishGuard Manifest V3 extension loaded, the production API configuration pointing to Render, and the Render service connected to Neon PostgreSQL. Gmail testing requires a normal web Gmail session and should use non-sensitive test or ordinary emails. PDF testing uses Chrome download behavior. Password-vault testing should use dedicated test credentials and HTTPS login pages where possible.

Area	Acceptance Criterion
Technical automated suite	All release-gate assertions pass; no security regression is accepted.
Website model regression	Long legitimate URLs remain below blocking risk while known suspicious regression fixtures remain detected by model/contextual gates.
Email regression gate	Modern legitimate fixtures pass low-risk checks and phishing/lookalike fixtures remain High Risk.
UAT task completion	At least 80% of required tasks completed without developer intervention.
UAT usability	Mean Likert score at or above 4.0/5 for clarity, ease of use, warning usefulness, and confidence, subject to actual collected results.
Critical UAT defects	No unresolved defect that prevents authentication, scanning, warning, report generation, or vault safety flow.

Table 15: Testing Acceptance Criteria

5.3 System Testing and Discussion

5.3.1 Automated Python Test Execution Record

The latest project documentation identifies 156 Python tests in the final release suite. Static inspection of the supplied test source confirms 156 individual Python test functions across seventeen files, matching the documented count. The report records the release verification status supplied with the project as Pass for these cases. Final submission should retain the terminal test output or an additional screenshot if the supervisor requires direct execution evidence.

ID	Area	Actual Output	Pass/Fail
PY-001	Authentication & MFA	As expected in the latest recorded release verification	Pass
PY-002	Authentication & MFA	As expected in the latest recorded release verification	Pass
PY-003	Authentication & MFA	As expected in the latest recorded release verification	Pass
PY-004	Authentication & MFA	As expected in the latest recorded release verification	Pass
PY-005	Authentication & MFA	As expected in the latest recorded release verification	Pass
PY-006	Authentication & MFA	As expected in the latest recorded release verification	Pass
PY-007	Authentication & MFA	As expected in the latest recorded release verification	Pass
PY-008	Authentication & MFA	As expected in the latest recorded release verification	Pass
PY-009	Authentication & MFA	As expected in the latest recorded release verification	Pass
PY-010	Authentication & MFA	As expected in the latest recorded release verification	Pass
PY-011	Authentication & MFA	As expected in the latest recorded release verification	Pass
PY-012	Authentication & MFA	As expected in the latest recorded release verification	Pass
PY-013	Backend scan APIs	As expected in the latest recorded release verification	Pass
PY-014	Backend scan APIs	As expected in the latest recorded release verification	Pass
PY-015	Backend scan APIs	As expected in the latest recorded release verification	Pass
PY-016	Backend scan APIs	As expected in the latest recorded release verification	Pass
PY-017	Backend scan APIs	As expected in the latest recorded release verification	Pass
PY-018	Backend scan APIs	As expected in the latest recorded release verification	Pass
PY-019	Backend scan APIs	As expected in the latest recorded release verification	Pass
PY-020	Backend scan APIs	As expected in the latest recorded release verification	Pass
PY-021	Backend scan APIs	As expected in the latest recorded release verification	Pass
PY-022	Backend scan APIs	As expected in the latest recorded release verification	Pass
PY-023	Backend scan APIs	As expected in the latest recorded release verification	Pass
PY-024	Backend scan APIs	As expected in the latest recorded release verification	Pass

ID	Area	Actual Output	Pass/Fail
PY-025	Backend scan APIs	As expected in the latest recorded release verification	Pass
PY-026	Backend scan APIs	As expected in the latest recorded release verification	Pass
PY-027	Backend scan APIs	As expected in the latest recorded release verification	Pass
PY-028	Backend scan APIs	As expected in the latest recorded release verification	Pass
PY-029	Backend scan APIs	As expected in the latest recorded release verification	Pass
PY-030	Backend scan APIs	As expected in the latest recorded release verification	Pass
PY-031	Deployment configuration	As expected in the latest recorded release verification	Pass
PY-032	Deployment configuration	As expected in the latest recorded release verification	Pass
PY-033	Deployment configuration	As expected in the latest recorded release verification	Pass
PY-034	Deployment configuration	As expected in the latest recorded release verification	Pass
PY-035	Deployment configuration	As expected in the latest recorded release verification	Pass
PY-036	Deployment configuration	As expected in the latest recorded release verification	Pass
PY-037	Deployment configuration	As expected in the latest recorded release verification	Pass
PY-038	Deployment configuration	As expected in the latest recorded release verification	Pass
PY-039	Deployment configuration	As expected in the latest recorded release verification	Pass
PY-040	Deployment configuration	As expected in the latest recorded release verification	Pass
PY-041	Deployment configuration	As expected in the latest recorded release verification	Pass
PY-042	Domain normalization	As expected in the latest recorded release verification	Pass
PY-043	Domain normalization	As expected in the latest recorded release verification	Pass
PY-044	Domain normalization	As expected in the latest recorded release verification	Pass
PY-045	Domain normalization	As expected in the latest recorded release verification	Pass
PY-046	Domain normalization	As expected in the latest recorded release verification	Pass
PY-047	Domain normalization	As expected in the latest recorded release verification	Pass
PY-048	Domain normalization	As expected in the latest recorded release verification	Pass

ID	Area	Actual Output	Pass/Fail
PY-049	Email contextual analysis	As expected in the latest recorded release verification	Pass
PY-050	Email contextual analysis	As expected in the latest recorded release verification	Pass
PY-051	Email contextual analysis	As expected in the latest recorded release verification	Pass
PY-052	Email contextual analysis	As expected in the latest recorded release verification	Pass
PY-053	Email contextual analysis	As expected in the latest recorded release verification	Pass
PY-054	Email contextual analysis	As expected in the latest recorded release verification	Pass
PY-055	Email contextual analysis	As expected in the latest recorded release verification	Pass
PY-056	Email contextual analysis	As expected in the latest recorded release verification	Pass
PY-057	Email contextual analysis	As expected in the latest recorded release verification	Pass
PY-058	Email contextual analysis	As expected in the latest recorded release verification	Pass
PY-059	Email contextual analysis	As expected in the latest recorded release verification	Pass
PY-060	Email contextual analysis	As expected in the latest recorded release verification	Pass
PY-061	Email contextual analysis	As expected in the latest recorded release verification	Pass
PY-062	Email contextual analysis	As expected in the latest recorded release verification	Pass
PY-063	Email contextual analysis	As expected in the latest recorded release verification	Pass
PY-064	Email contextual analysis	As expected in the latest recorded release verification	Pass
PY-065	Email contextual analysis	As expected in the latest recorded release verification	Pass
PY-066	Email contextual analysis	As expected in the latest recorded release verification	Pass
PY-067	Email contextual analysis	As expected in the latest recorded release verification	Pass
PY-068	Email contextual analysis	As expected in the latest recorded release verification	Pass
PY-069	Email contextual analysis	As expected in the latest recorded release verification	Pass
PY-070	Email contextual analysis	As expected in the latest recorded release verification	Pass
PY-071	Email contextual analysis	As expected in the latest recorded release verification	Pass
PY-072	Email contextual analysis	As expected in the latest recorded release verification	Pass

ID	Area	Actual Output	Pass/Fail
PY-073	Email contextual analysis	As expected in the latest recorded release verification	Pass
PY-074	Email contextual analysis	As expected in the latest recorded release verification	Pass
PY-075	Email contextual analysis	As expected in the latest recorded release verification	Pass
PY-076	Email contextual analysis	As expected in the latest recorded release verification	Pass
PY-077	Email model regression	As expected in the latest recorded release verification	Pass
PY-078	Email model regression	As expected in the latest recorded release verification	Pass
PY-079	Email release gate	As expected in the latest recorded release verification	Pass
PY-080	Email release gate	As expected in the latest recorded release verification	Pass
PY-081	Email release gate	As expected in the latest recorded release verification	Pass
PY-082	Email summaries	As expected in the latest recorded release verification	Pass
PY-083	Email summaries	As expected in the latest recorded release verification	Pass
PY-084	Email summaries	As expected in the latest recorded release verification	Pass
PY-085	Email summaries	As expected in the latest recorded release verification	Pass
PY-086	Email summaries	As expected in the latest recorded release verification	Pass
PY-087	Email summaries	As expected in the latest recorded release verification	Pass
PY-088	Email summaries	As expected in the latest recorded release verification	Pass
PY-089	Email summaries	As expected in the latest recorded release verification	Pass
PY-090	Email summaries	As expected in the latest recorded release verification	Pass
PY-091	Email summaries	As expected in the latest recorded release verification	Pass
PY-092	Email summaries	As expected in the latest recorded release verification	Pass
PY-093	Extension helper integration	As expected in the latest recorded release verification	Pass
PY-094	Extension helper integration	As expected in the latest recorded release verification	Pass
PY-095	Extension helper integration	As expected in the latest recorded release verification	Pass
PY-096	Extension helper integration	As expected in the latest recorded release verification	Pass

ID	Area	Actual Output	Pass/Fail
PY-097	Extension helper integration	As expected in the latest recorded release verification	Pass
PY-098	Extension helper integration	As expected in the latest recorded release verification	Pass
PY-099	Extension helper integration	As expected in the latest recorded release verification	Pass
PY-100	Extension helper integration	As expected in the latest recorded release verification	Pass
PY-101	Extension production config	As expected in the latest recorded release verification	Pass
PY-102	Extension production config	As expected in the latest recorded release verification	Pass
PY-103	Extension production config	As expected in the latest recorded release verification	Pass
PY-104	Extension production config	As expected in the latest recorded release verification	Pass
PY-105	PDF security reports	As expected in the latest recorded release verification	Pass
PY-106	PDF security reports	As expected in the latest recorded release verification	Pass
PY-107	PDF security reports	As expected in the latest recorded release verification	Pass
PY-108	PDF security reports	As expected in the latest recorded release verification	Pass
PY-109	PDF security reports	As expected in the latest recorded release verification	Pass
PY-110	PDF security reports	As expected in the latest recorded release verification	Pass
PY-111	PDF security reports	As expected in the latest recorded release verification	Pass
PY-112	PDF security reports	As expected in the latest recorded release verification	Pass
PY-113	PDF security reports	As expected in the latest recorded release verification	Pass
PY-114	Static website	As expected in the latest recorded release verification	Pass
PY-115	Static website	As expected in the latest recorded release verification	Pass
PY-116	Static website	As expected in the latest recorded release verification	Pass
PY-117	Static website	As expected in the latest recorded release verification	Pass
PY-118	Static website	As expected in the latest recorded release verification	Pass
PY-119	Training helpers	As expected in the latest recorded release verification	Pass
PY-120	Training helpers	As expected in the latest recorded release verification	Pass

ID	Area	Actual Output	Pass/Fail
PY-121	Training helpers	As expected in the latest recorded release verification	Pass
PY-122	Training helpers	As expected in the latest recorded release verification	Pass
PY-123	Training helpers	As expected in the latest recorded release verification	Pass
PY-124	Training helpers	As expected in the latest recorded release verification	Pass
PY-125	Training helpers	As expected in the latest recorded release verification	Pass
PY-126	Training helpers	As expected in the latest recorded release verification	Pass
PY-127	Vault backend	As expected in the latest recorded release verification	Pass
PY-128	Vault backend	As expected in the latest recorded release verification	Pass
PY-129	Vault backend	As expected in the latest recorded release verification	Pass
PY-130	Vault backend	As expected in the latest recorded release verification	Pass
PY-131	Vault backend	As expected in the latest recorded release verification	Pass
PY-132	Vault backend	As expected in the latest recorded release verification	Pass
PY-133	Vault backend	As expected in the latest recorded release verification	Pass
PY-134	Vault backend	As expected in the latest recorded release verification	Pass
PY-135	Vault integration	As expected in the latest recorded release verification	Pass
PY-136	Vault integration	As expected in the latest recorded release verification	Pass
PY-137	Vault integration	As expected in the latest recorded release verification	Pass
PY-138	Vault integration	As expected in the latest recorded release verification	Pass
PY-139	Vault integration	As expected in the latest recorded release verification	Pass
PY-140	Website contextual analysis	As expected in the latest recorded release verification	Pass
PY-141	Website contextual analysis	As expected in the latest recorded release verification	Pass
PY-142	Website contextual analysis	As expected in the latest recorded release verification	Pass
PY-143	Website contextual analysis	As expected in the latest recorded release verification	Pass
PY-144	Website contextual analysis	As expected in the latest recorded release verification	Pass

ID	Area	Actual Output	Pass/Fail
PY-145	Website contextual analysis	As expected in the latest recorded release verification	Pass
PY-146	Website contextual analysis	As expected in the latest recorded release verification	Pass
PY-147	Website model regression	As expected in the latest recorded release verification	Pass
PY-148	Website model regression	As expected in the latest recorded release verification	Pass
PY-149	Website model regression	As expected in the latest recorded release verification	Pass
PY-150	Website model regression	As expected in the latest recorded release verification	Pass
PY-151	Website model regression	As expected in the latest recorded release verification	Pass
PY-152	Website model regression	As expected in the latest recorded release verification	Pass
PY-153	Website model regression	As expected in the latest recorded release verification	Pass
PY-154	Website model regression	As expected in the latest recorded release verification	Pass
PY-155	Website model regression	As expected in the latest recorded release verification	Pass
PY-156	Website model regression	As expected in the latest recorded release verification	Pass

Table 16: Python Automated Test Execution Summary

5.3.2 Node.js Extension Test Execution

All twelve dedicated Node.js test programs were executed directly against the supplied latest source package during preparation of this report draft. Every program completed successfully. This independently confirms the extension-side helper logic for authentication, background scanning, production API configuration, Gmail triggers and polling, email/history helpers, trusted senders, trusted websites, scan history/dashboard, and password-vault logic.

ID	Test Program	Actual Result	Pass/Fail
JS-01	extension_auth_helpers.test.js	All assertions completed successfully	Pass
JS-02	extension_background_helpers.test.js	All assertions completed successfully	Pass
JS-03	extension_config.test.js	All assertions completed successfully	Pass
JS-04	extension_email_helpers.test.js	All assertions completed successfully	Pass
JS-05	extension_gmail_polling.test.js	All assertions completed successfully	Pass

ID	Test Program	Actual Result	Pass/Fail
JS-06	extension_gmail_trigger.test.js	All assertions completed successfully	Pass
JS-07	extension_scan_history.test.js	All assertions completed successfully	Pass
JS-08	extension_trust_helpers.test.js	All assertions completed successfully	Pass
JS-09	extension_trusted_website_helpers.test.js	All assertions completed successfully	Pass
JS-10	extension_vault_content.test.js	All assertions completed successfully	Pass
JS-11	extension_vault_crypto.test.js	All assertions completed successfully	Pass
JS-12	extension_vault_helpers.test.js	All assertions completed successfully	Pass

Table 17: Node.js Extension Test Execution

5.3.3 Website Model Evaluation

The frozen website model is a Random Forest without class weighting and uses a calibrated decision threshold of 0.52. The latest comparison run used 507,360 cleaned URLs, consisting of 393,027 legitimate and 114,333 phishing samples. The held-out model result is intentionally conservative: the false-positive rate is low at 2.59%, but phishing recall is 33.98%. This trade-off is important because an extension that interrupts ordinary browsing too frequently will rapidly lose user trust, yet the low recall means the URL-only model cannot be described as sufficient by itself.

Metric	Result
Accuracy	83.15%
Precision	79.23%
Recall	33.98%
F1-score	47.56%
Balanced accuracy	65.70%
False-positive rate	2.59%

Table 18: Website Model Held-out Metrics

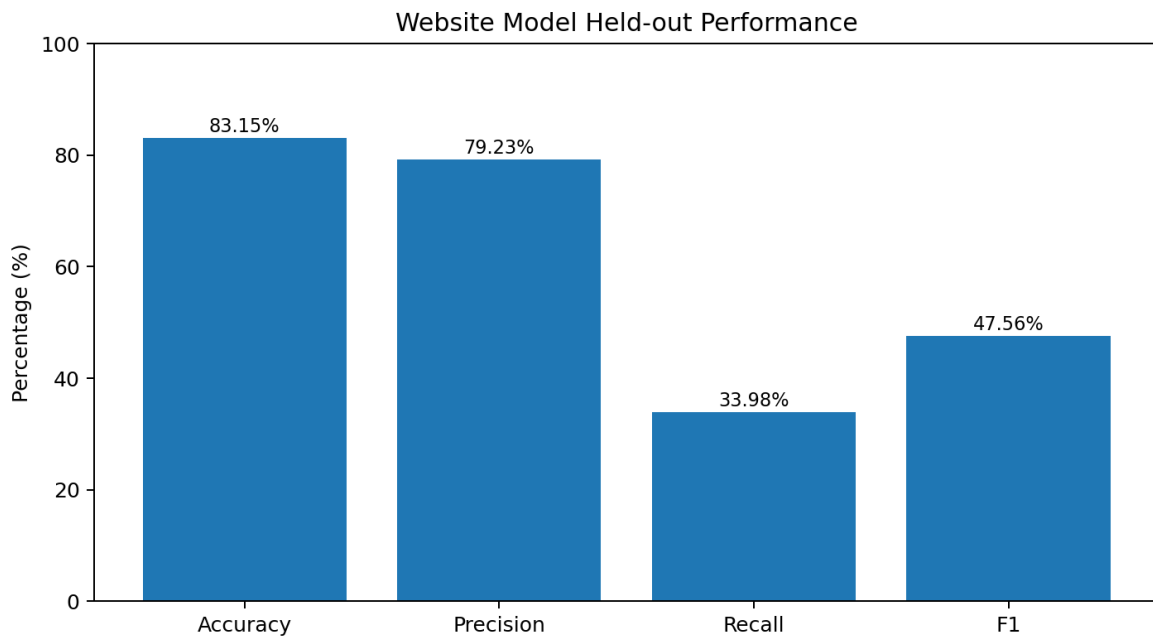


Figure 100: Website Model Held-out Performance

Figure 100 summarises the held-out performance of the frozen Random Forest website classifier and shows the main trade-off selected for the final PhishGuard model. The classifier achieves 83.15% accuracy and 79.23% precision, indicating that most URLs labelled as phishing are genuinely phishing in the held-out set. Its false-positive rate is only 2.59%, which is important for a browser extension because excessive warnings on legitimate websites could reduce user trust and encourage users to ignore protection. However, phishing recall is lower at 33.98%, producing an F1-score of 47.56% and balanced accuracy of 65.70%. This shows that the frozen model is conservative: it limits false alarms but does not identify every phishing sample using URL features alone. PhishGuard therefore preserves the Random Forest probability for honest model evaluation and applies a separate contextual-risk layer during runtime. Strong structural evidence such as credential-oriented IP destinations, punycode, suspicious redirect/query structures, and brand-impersonation characteristics can raise the final practical risk, while personal trust remains separately bounded when severe evidence exists. Consequently, Figure 100 reports only the frozen classifier's held-out performance; contextual improvements are treated as runtime security handling rather than being added to the supervised-learning metrics.

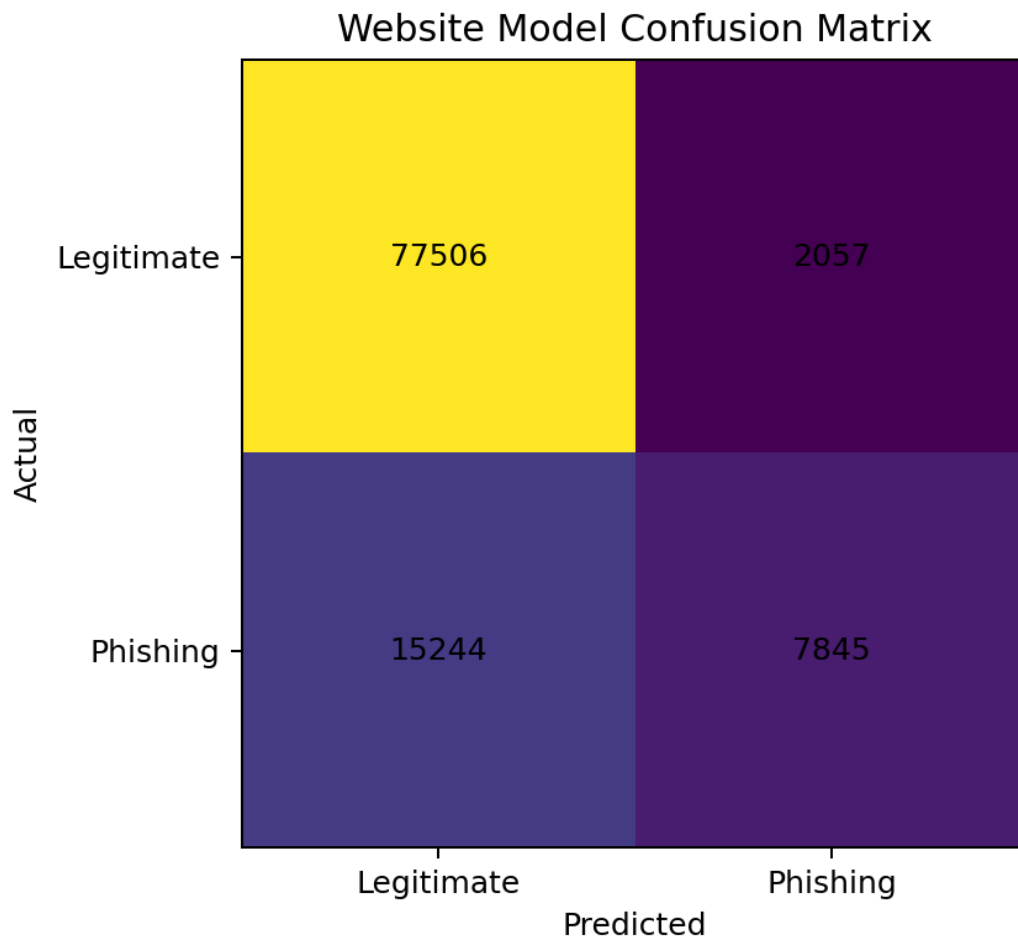


Figure 101: Website Model Confusion Matrix

The held-out confusion matrix contains 77,506 true legitimate classifications, 2,057 false positives, 15,244 false negatives, and 7,845 true phishing classifications.

Because the frozen model misses some structurally suspicious campaign URLs, a separate bounded contextual runtime layer was added without changing the model artifact or its historical metrics. Two supplied prize-campaign regression URLs demonstrate this design: one receives a raw model probability of 39.51% and the other 15.65%, yet both become 86.00% final contextual risk because multiple independent prize/redirect/opaque-query/affiliate indicators occur together. Conversely, long Google Search, Google Images, Microsoft OAuth, shopping/cloud, and tracking-heavy legitimate regression fixtures remain non-blocking. This approach improves a targeted false-negative class without claiming that the Random Forest recall itself increased.

5.3.4 Email Model Evaluation

The frozen email model is a calibrated balanced LinearSVC using TF-IDF word unigrams and bigrams with a threshold of 0.70. Preparation removes exact cleaned duplicates, normalized-template duplicates, conflicting families, and groups related templates before the holdout split. This reduces leakage and makes the reported metrics more meaningful than a random row-level split over highly similar emails.

Metric	Result
Accuracy	97.18%
Precision	98.94%
Recall	93.27%
F1-score	96.02%
Balanced accuracy	96.35%
False-positive rate	0.57%
False-negative rate	6.73%
Brier score	0.0161
Log loss	0.0596
Expected calibration error	0.0149

Table 19: Email Model Leakage-safe Held-out Metrics

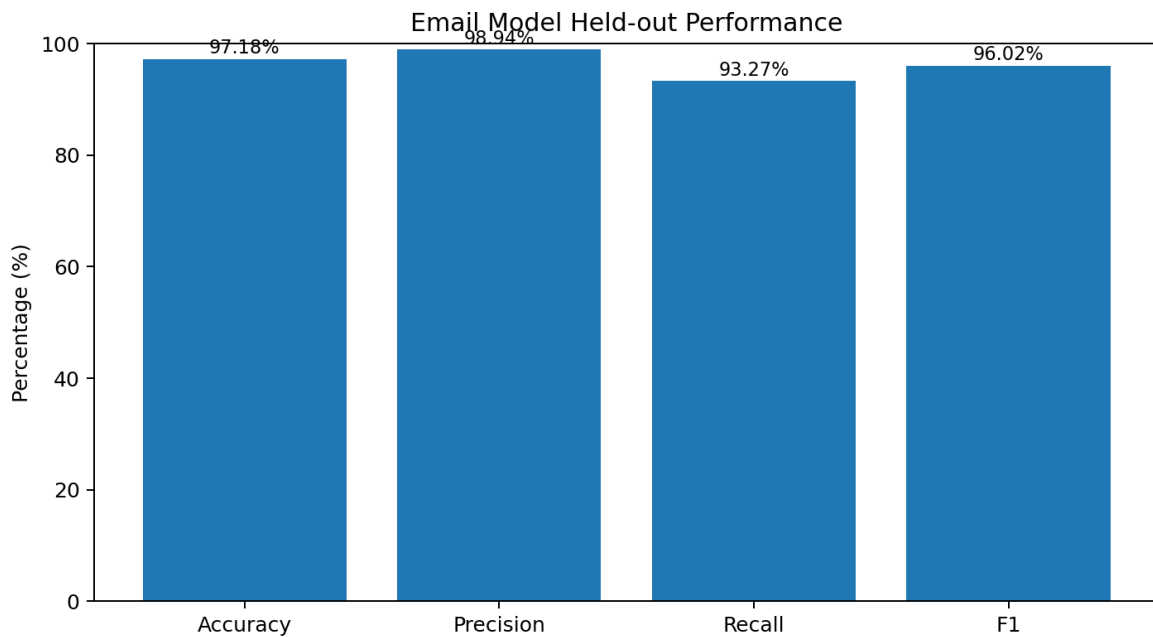


Figure 102: Email Model Held-out Performance

Figure 102 shows that the frozen calibrated LinearSVC email classifier performs strongly and consistently on the leakage-safe held-out set. The model achieves 97.18% accuracy, 98.94% precision, 93.27% recall, 96.02% F1-score, and 96.35% balanced accuracy. The very high precision is especially important for a user-facing email warning system because it indicates that legitimate messages are rarely labelled as phishing; the measured false-positive rate is only 0.57%. Recall is slightly lower than precision, with a 6.73% false-negative rate, showing that some phishing messages can still be missed by the text model alone. Probability-quality results are also strong (Brier score 0.0161, log loss 0.0596, expected calibration error 0.0149), supporting the use of the calibrated probability at the selected 0.70 operating threshold. These figures describe the frozen text classifier only; runtime sender, link, contextual-evidence, and personal-trust logic is evaluated separately so improvements in final PhishGuard risk are not incorrectly reported as changes to the underlying model metrics.

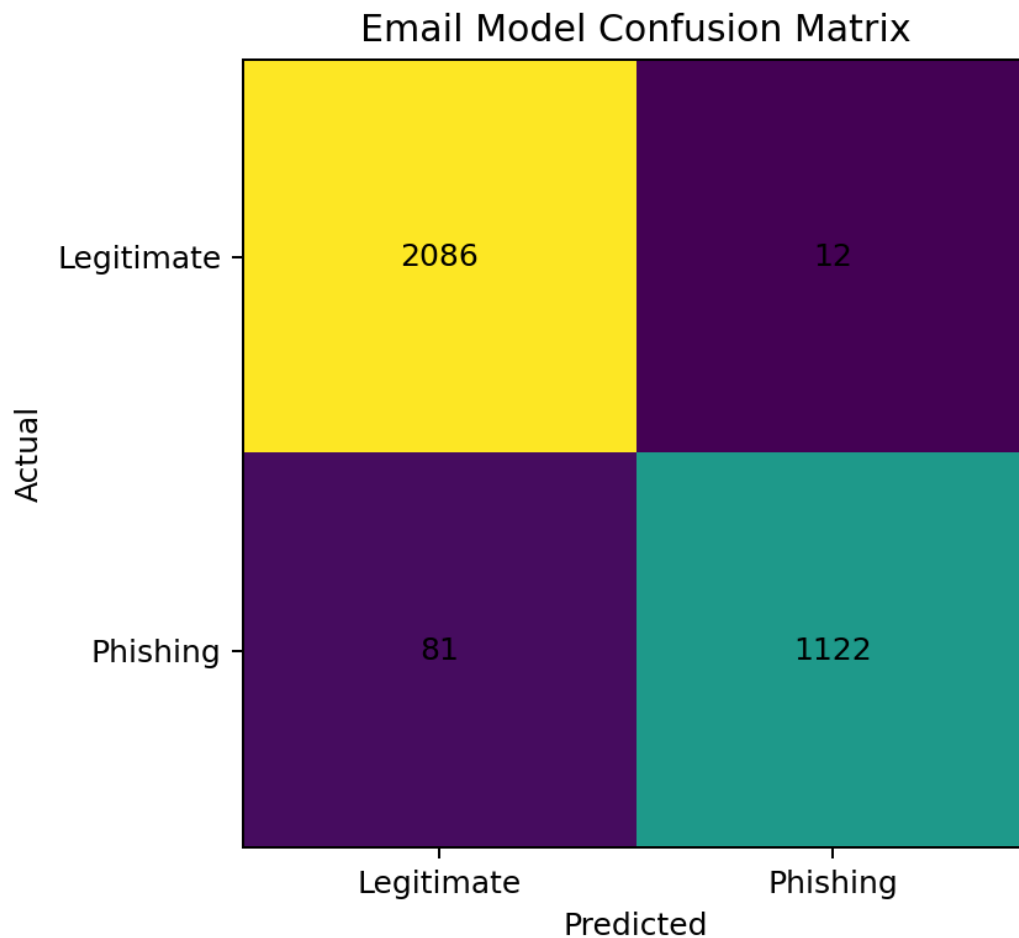


Figure 103: Email Model Confusion Matrix

The confusion matrix contains 2,086 true legitimate classifications, 12 false positives, 81 false negatives, and 1,122 true phishing classifications.

The runtime email system does not equate the calibrated text probability with the final PhishGuard risk. Sender/link consistency, concrete destination evidence, and personal trust are applied as bounded contextual signals. Exact-sender personal trust can reduce a future score by 0.50, 0.55, or 0.60 depending on the established state, whereas confirmed domain trust is smaller. Strong text probability alone no longer cancels explicit trust, but dangerous downloads, suspicious credential links, sender impersonation, and other high-severity evidence can still override or limit the reduction. The independent email release gate contains 37 modern legitimate and phishing/lookalike regression cases and is documented as passing 37/37.

5.3.5 Security and Regression Discussion

The strongest aspect of the final test design is that it contains regressions derived from actual problems found during development instead of only idealized unit cases. Long legitimate URLs were initially prone to false positives, so raw path/query length was removed from the saved website feature list and dedicated long-URL regression tests were added. Later, two prize-campaign URLs exposed a false-negative gap in the frozen Random Forest, which led to a transparent contextual layer and regression tests requiring these patterns to become High Risk without hard-coding the supplied hostnames.

A similar pattern occurred for email scanning. Legitimate notifications and newsletters could contain security, verify, password, invoice, or urgency vocabulary and therefore received excessive risk under earlier contextual logic. The final implementation separates base model probability from contextual evidence, makes personal trust visible and bounded, requires concrete sender/destination conflicts before overriding trust, and adds a modern release gate. Gmail-specific regressions also cover route changes, stale responses, polling, message identity, and banner dismissal.

Security controls are also explicitly tested. Authentication tests cover wrong-password source throttling, MFA enrollment, progressive lockouts, TOTP replay, recovery-code one-time use, session expiry, logout, and ownership of security events. Vault tests confirm client-side encryption boundaries, cross-user access blocking, rejection of plaintext-shaped backend payloads, authenticated reset, inactivity locking, safe hostname matching, and conservative fill behavior. PDF tests cover report ownership, date-range clamping, multi-page reports, personal trust overlays, and exclusion of supplied secrets.

5.3.6 User Acceptance Testing Execution and Results

All three completed UAT forms were returned on 31 August 2026. Across the sixteen practical acceptance tasks, every tester recorded Pass for every task, producing 48 successful task executions out of 48 attempted tasks (100%). The tasks covered the complete user workflow from direct-download installation and Load unpacked setup through MFA, website and Gmail protection, dashboard/report export, security-event review, and encrypted Password Vault use. All three participants selected Accepted as the overall acceptance decision, with no participant selecting Accepted with Minor Changes or Not Accepted.

The usability ratings were also strongly positive. The first tester, keevanrajah, rated all twelve statements 5/5 for a mean of 5.00. Vicknesh produced a mean of 4.75, with 4/5 ratings for warning clarity, manual website scanning, and overall comfort using PhishGuard. Amresh produced a mean of 4.83, with 4/5 ratings for warning clarity and email/Gmail risk information. The combined mean across all 36 usability responses was 4.86/5.00. Qualitative feedback highlighted the real-time website warning, Gmail in-message scanning, MFA compatibility, and smooth browser integration as useful features. Suggested improvements included clearer confirmation after website/sender/domain trust actions and future investigation of certificate or SSL-related legitimacy signals. One tester reported no recommended improvement. These suggestions are recorded as future refinement opportunities rather than as claims about capabilities already implemented in the final build.

Tester	Task Completion	Mean Rating	Acceptance
keevanrajah	16/16 Pass	5.00 / 5	Accepted
Vicknesh Arumugam	16/16 Pass	4.75 / 5	Accepted
Amresh	16/16 Pass	4.83 / 5	Accepted
Overall	48/48 Pass (100%)	4.86 / 5	3/3 Accepted

Table 20: UAT Execution Summary

5.3.7 Completed User Acceptance Testing Forms

The following three completed individual UAT forms are reproduced in full as evidence of the user acceptance testing described in Chapter 5. All normal narrative text and table text have been standardized to 12-point Times New Roman for consistency with the report; the tester signature/confirmation box content is retained without font normalization.

PHISHGUARD

Individual User Acceptance Testing (UAT) Form

Final Year Project Part 2

Tester Profile

Name / Pseudonym	keevanrajah	Date	15-08-2026
Age Group	26	Occupation	Associate Technical Analyst
Primary Browser	Google Chrome	Cybersecurity Experience	4 years
Device / OS	Windows 11	Organization Name	Tech Mahindra

Instructions

- Use the separate PhishGuard Installation and UAT Guide to install and open the system.
- Use only legitimate websites and the controlled/sanitized test material provided. Do not enter real credentials into suspicious pages.
- Perform each task below. If anything unexpected occurs, describe it briefly in the Notes column.
- Complete the usability ratings, overall acceptance, comments, and confirmation based on your own experience.

A. Practical Acceptance Tasks

ID / Task	Procedure	Expected Result	Result	Notes
UAT-01 Install & load extension	Download ZIP, extract it, open chrome://extensions, enable Developer mode and select Load unpacked.	Extension loads and opens successfully without any beta/activation code.	☒ Pass ☐ Fail	-
UAT-02 Create account & enrol MFA	Create a test account with a valid email and strong test password, then enrol TOTP MFA.	Registration succeeds; MFA is mandatory and recovery codes are shown.	☒ Pass ☐ Fail	-
UAT-03 Sign in	Sign in using the test account password followed by the correct MFA code.	Authenticated features become available only after MFA succeeds.	☒ Pass ☐ Fail	-
UAT-04 Scan normal website	Open a known legitimate HTTPS website and allow Website Protection to analyse it.	A timely low-risk result and understandable explanation are shown without unnecessary blocking.	☒ Pass ☐ Fail	-
UAT-05 Scan controlled suspicious URL	Use the provided sanitized phishing fixture or controlled suspicious page only.	PhishGuard shows a high-risk/suspicious result and warning with understandable evidence.	☒ Pass ☐ Fail	-
UAT-06 Continue Once	From the controlled warning page, choose Continue Once.	The current navigation proceeds once and the site is not saved as trusted.	☒ Pass ☐ Fail	-
UAT-07 Trust & remove website trust	Save a trust decision for the controlled test destination, rescan, then remove the trust.	Trust is visible and reversible; normal handling returns after removal.	☒ Pass ☐ Fail	-
UAT-08 Manual website scan	Paste a provided legitimate or controlled suspicious URL into Manual Scan.	The result and explanation are consistent with automatic website protection.	☒ Pass ☐ Fail	-

ID / Task	Procedure	Expected Result	Result	Notes
UAT-09 Scan Gmail message	Open a provided legitimate Gmail test email and then a controlled suspicious test email.	The banner updates for the current message and presents understandable email-risk evidence.	☒ Pass ☐ Fail	-
UAT-10 Dismiss banner & sender trust	Dismiss Email A's banner, open Email B, and save sender trust for the legitimate test sender.	Dismissal stays scoped to Email A and sender trust is saved only for the current user.	☒ Pass ☐ Fail	-
UAT-11 Review dashboard & trust	Open the Security Dashboard and review overview, recent scans, search/filter, scan details, Security Events and Personal Trust.	Dashboard information loads correctly and trust/security records can be reviewed and managed.	☒ Pass ☐ Fail	-
UAT-12 Download PDF reports	Download an individual scan report and at least one valid date-range report.	Readable PDFs download with the expected user, risk, evidence and explanation details.	☒ Pass ☐ Fail	-
UAT-13 Review security alert	Follow the provided safe password-correct/MFA-failed test procedure, then later sign in successfully.	A suspicious sign-in/security event appears without claiming the failed-MFA attempt gained access.	☒ Pass ☐ Fail	-
UAT-14 Unlock & use Password Vault	Unlock the vault with a test master password, add a non-sensitive test credential, lock it, then reopen it.	Vault requires the correct master password and hides plaintext credentials while locked.	☒ Pass ☐ Fail	-
UAT-15 Generate password & Fill Current Tab	Generate a test password, save a login for the controlled HTTPS page, then use Fill Current Tab.	A matching tab is found and credentials fill only when destination checks permit it.	☒ Pass ☐ Fail	-
UAT-16 Settings & logout	Review protection/settings controls, change one non-critical setting, then log out.	Setting behavior is understandable and logout removes access to authenticated features.	☒ Pass ☐ Fail	-

B. Usability and User Experience Rating

Tick one rating for each statement: 1 = Strongly Disagree, 2 = Disagree, 3 = Neutral, 4 = Agree, 5 = Strongly Agree.

Statement	1	2	3	4	5
PhishGuard was easy to understand and navigate.	☐	☐	☐	☐	☒
The installation and first-time setup process was clear.	☐	☐	☐	☐	☒

Statement	1	2	3	4	5
Website scan results were displayed quickly enough for normal browsing.	☐	☐	☐	☐	☒
The website warning clearly explained why a page was considered risky.	☐	☐	☐	☐	☒
Manual website scanning was easy to use.	☐	☐	☐	☐	☒
Email/Gmail risk information was clear and useful.	☐	☐	☐	☐	☒
The Security Dashboard was easy to understand.	☐	☐	☐	☐	☒
PDF reports contained useful and understandable information.	☐	☐	☐	☐	☒
Personal trust controls were clear and manageable.	☐	☐	☐	☐	☒
Account security and MFA behavior was understandable.	☐	☐	☐	☐	☒
The Password Vault was easy to unlock and use.	☐	☐	☐	☐	☒
Overall, I would be comfortable using PhishGuard for phishing protection.	☐	☐	☐	☐	☒

C. Overall Acceptance

- ☒ Accepted - the system is suitable for its intended use.
- ☐ Accepted with Minor Changes - the system is usable, but minor improvements are recommended.
- ☐ Not Accepted - important issues prevent acceptable use.

Most useful feature(s)	The real-time website phishing warning was useful because it clearly showed when a website could be risky before continuing.
Problem(s) / confusing behavior	-
Recommended improvement(s)	Add a clearer confirmation message after a website, sender, or domain is trusted

Tester Confirmation

I confirm that I personally attempted the UAT tasks above. The ratings, comments, acceptance decision and confirmation reflect my own experience using PhishGuard.

Tester Name / Pseudonym	keevanrajah	Date	15-08-2026
Signature / Confirmation		Email (optional)	Keevan2000@gmail.com
Organization Signature	Tech Mahindra ICT Services (Malaysia) Sdn. Bhd ("Tech Mahindra") Authorized Signatory  Suchitra Kerkar SVP, Global HR Head BPS		

PHISHGUARD**Individual User Acceptance Testing (UAT) Form**

Final Year Project Part 2

Tester Profile

Name / Pseudonym	Vicknesh Arumugam	Date	16-08-2026
Age Group	24	Occupation	InfraSecOps Engineer
Primary Browser	Google Chrome	Cybersecurity Experience	2 years
Device / OS	Windows 11 Pro	Organization Name	Tridimas Sdn Bhd

Instructions

- Use the separate PhishGuard Installation and UAT Guide to install and open the system.
- Use only legitimate websites and the controlled/sanitized test material provided. Do not enter real credentials into suspicious pages.
- Perform each task below. If anything unexpected occurs, describe it briefly in the Notes column.
- Complete the usability ratings, overall acceptance, comments, and confirmation based on your own experience.

A. Practical Acceptance Tasks

ID / Task	Procedure	Expected Result	Result	Notes
UAT-01 Install & load extension	Download ZIP, extract it, open chrome://extensions, enable Developer mode and select Load unpacked.	Extension loads and opens successfully without any beta/activation code.	☒ Pass ☐ Fail	-
UAT-02 Create account & enrol MFA	Create a test account with a valid email and strong test password, then enrol TOTP MFA.	Registration succeeds; MFA is mandatory and recovery codes are shown.	☒ Pass ☐ Fail	-
UAT-03 Sign in	Sign in using the test account password followed by the correct MFA code.	Authenticated features become available only after MFA succeeds.	☒ Pass ☐ Fail	-
UAT-04 Scan normal website	Open a known legitimate HTTPS website and allow Website Protection to analyse it.	A timely low-risk result and understandable explanation are shown without unnecessary blocking.	☒ Pass ☐ Fail	-
UAT-05 Scan controlled suspicious URL	Use the provided sanitized phishing fixture or controlled suspicious page only.	PhishGuard shows a high-risk/suspicious result and warning with understandable evidence.	☒ Pass ☐ Fail	-
UAT-06 Continue Once	From the controlled warning page, choose Continue Once.	The current navigation proceeds once and the site is not saved as trusted.	☒ Pass ☐ Fail	-
UAT-07 Trust & remove website trust	Save a trust decision for the controlled test destination, rescan, then remove the trust.	Trust is visible and reversible; normal handling returns after removal.	☒ Pass ☐ Fail	-

ID / Task	Procedure	Expected Result	Result	Notes
UAT-08 Manual website scan	Paste a provided legitimate or controlled suspicious URL into Manual Scan.	The result and explanation are consistent with automatic website protection.	☒ Pass ☐ Fail	-
UAT-09 Scan Gmail message	Open a provided legitimate Gmail test email and then a controlled suspicious test email.	The banner updates for the current message and presents understandable email-risk evidence.	☒ Pass ☐ Fail	-
UAT-10 Dismiss banner & sender trust	Dismiss Email A's banner, open Email B, and save sender trust for the legitimate test sender.	Dismissal stays scoped to Email A and sender trust is saved only for the current user.	☒ Pass ☐ Fail	-
UAT-11 Review dashboard & trust	Open the Security Dashboard and review overview, recent scans, search/filter, scan details, Security Events and Personal Trust.	Dashboard information loads correctly and trust/security records can be reviewed and managed.	☒ Pass ☐ Fail	-
UAT-12 Download PDF reports	Download an individual scan report and at least one valid date-range report.	Readable PDFs download with the expected user, risk, evidence and explanation details.	☒ Pass ☐ Fail	-
UAT-13 Review security alert	Follow the provided safe password-correct/MFA-failed test procedure, then later sign in successfully.	A suspicious sign-in/security event appears without claiming the failed-MFA attempt gained access.	☒ Pass ☐ Fail	-
UAT-14 Unlock & use Password Vault	Unlock the vault with a test master password, add a non-sensitive test credential, lock it, then reopen it.	Vault requires the correct master password and hides plaintext credentials while locked.	☒ Pass ☐ Fail	-
UAT-15 Generate password & Fill Current Tab	Generate a test password, save a login for the controlled HTTPS page, then use Fill Current Tab.	A matching tab is found and credentials fill only when destination checks permit it.	☒ Pass ☐ Fail	-
UAT-16 Settings & logout	Review protection/settings controls, change one non-critical setting, then log out.	Setting behavior is understandable and logout removes access to authenticated features.	☒ Pass ☐ Fail	-

B. Usability and User Experience Rating

Tick one rating for each statement: 1 = Strongly Disagree, 2 = Disagree, 3 = Neutral, 4 = Agree, 5 = Strongly Agree.

Statement	1	2	3	4	5
PhishGuard was easy to understand and navigate.	☐	☐	☐	☐	☒

Statement	1	2	3	4	5
The installation and first-time setup process was clear.	☐	☐	☐	☐	☒
Website scan results were displayed quickly enough for normal browsing.	☐	☐	☐	☐	☒
The website warning clearly explained why a page was considered risky.	☐	☐	☐	☒	☐
Manual website scanning was easy to use.	☐	☐	☐	☒	☐
Email/Gmail risk information was clear and useful.	☐	☐	☐	☐	☒
The Security Dashboard was easy to understand.	☐	☐	☐	☐	☒
PDF reports contained useful and understandable information.	☐	☐	☐	☐	☒
Personal trust controls were clear and manageable.	☐	☐	☐	☐	☒
Account security and MFA behavior was understandable.	☐	☐	☐	☐	☒
The Password Vault was easy to unlock and use.	☐	☐	☐	☐	☒
Overall, I would be comfortable using PhishGuard for phishing protection.	☐	☐	☐	☒	☐

C. Overall Acceptance

☒ Accepted - the system is suitable for its intended use.

☐ Accepted with Minor Changes - the system is usable, but minor improvements are recommended.

☐ Not Accepted - important issues prevent acceptable use.

Most useful feature(s)	Real-time scanning is useful for end users, the MFA feature surprised me as it worked flawlessly with even Microsoft Authenticator, the smoothness during the usage of extension is good as well as integration with latest and older versions of browsers.
Problem(s) / confusing behavior	-
Recommended improvement(s)	Can include SSL inspection as part of the legitimacy check, as a lot of fake websites use similar but not the same SSL certificate. It definitely can improve detection speeds.

Tester Confirmation

I confirm that I personally attempted the UAT tasks above. The ratings, comments, acceptance decision and confirmation reflect my own experience using PhishGuard.

Tester Name / Pseudonym	Vicknesh Arumugam	Date	16-08-2026
Signature / Confirmation	VICKNESH	Email (optional)	vckneshx@gmail.com
Organization Signature			

PHISHGUARD**Individual User Acceptance Testing (UAT) Form**

Final Year Project Part 2

Tester Profile

Name / Pseudonym	Amresh	Date	16/08/2026
Age Group	29	Occupation	Cybersecurity Consultant
Primary Browser	Google Chrome	Cybersecurity Experience	6 years
Device / OS	Windows 11	Organization Name	Atilla Cybertech

Instructions

- Use the separate PhishGuard Installation and UAT Guide to install and open the system.
- Use only legitimate websites and the controlled/sanitized test material provided. Do not enter real credentials into suspicious pages.
- Perform each task below. If anything unexpected occurs, describe it briefly in the Notes column.
- Complete the usability ratings, overall acceptance, comments, and confirmation based on your own experience.

A. Practical Acceptance Tasks

ID / Task	Procedure	Expected Result	Result	Notes
UAT-01 Install & load extension	Download ZIP, extract it, open chrome://extensions, enable Developer mode and select Load unpacked.	Extension loads and opens successfully without any beta/activation code.	☒ Pass ☐ Fail	-
UAT-02 Create account & enrol MFA	Create a test account with a valid email and strong test password, then enrol TOTP MFA.	Registration succeeds; MFA is mandatory and recovery codes are shown.	☒ Pass ☐ Fail	-
UAT-03 Sign in	Sign in using the test account password followed by the correct MFA code.	Authenticated features become available only after MFA succeeds.	☒ Pass ☐ Fail	-
UAT-04 Scan normal website	Open a known legitimate HTTPS website and allow Website Protection to analyse it.	A timely low-risk result and understandable explanation are shown without unnecessary blocking.	☒ Pass ☐ Fail	-
UAT-05 Scan controlled suspicious URL	Use the provided sanitized phishing fixture or controlled suspicious page only.	PhishGuard shows a high-risk/suspicious result and warning with understandable evidence.	☒ Pass ☐ Fail	-
UAT-06 Continue Once	From the controlled warning page, choose Continue Once.	The current navigation proceeds once and the site is not saved as trusted.	☒ Pass ☐ Fail	-
UAT-07 Trust & remove website trust	Save a trust decision for the controlled test destination, rescan, then remove the trust.	Trust is visible and reversible; normal handling returns after removal.	☒ Pass ☐ Fail	-

ID / Task	Procedure	Expected Result	Result	Notes
UAT-08 Manual website scan	Paste a provided legitimate or controlled suspicious URL into Manual Scan.	The result and explanation are consistent with automatic website protection.	☒ Pass ☐ Fail	-
UAT-09 Scan Gmail message	Open a provided legitimate Gmail test email and then a controlled suspicious test email.	The banner updates for the current message and presents understandable email-risk evidence.	☒ Pass ☐ Fail	-
UAT-10 Dismiss banner & sender trust	Dismiss Email A's banner, open Email B, and save sender trust for the legitimate test sender.	Dismissal stays scoped to Email A and sender trust is saved only for the current user.	☒ Pass ☐ Fail	-
UAT-11 Review dashboard & trust	Open the Security Dashboard and review overview, recent scans, search/filter, scan details, Security Events and Personal Trust.	Dashboard information loads correctly and trust/security records can be reviewed and managed.	☒ Pass ☐ Fail	-
UAT-12 Download PDF reports	Download an individual scan report and at least one valid date-range report.	Readable PDFs download with the expected user, risk, evidence and explanation details.	☒ Pass ☐ Fail	-
UAT-13 Review security alert	Follow the provided safe password-correct/MFA-failed test procedure, then later sign in successfully.	A suspicious sign-in/security event appears without claiming the failed-MFA attempt gained access.	☒ Pass ☐ Fail	-
UAT-14 Unlock & use Password Vault	Unlock the vault with a test master password, add a non-sensitive test credential, lock it, then reopen it.	Vault requires the correct master password and hides plaintext credentials while locked.	☒ Pass ☐ Fail	-
UAT-15 Generate password & Fill Current Tab	Generate a test password, save a login for the controlled HTTPS page, then use Fill Current Tab.	A matching tab is found and credentials fill only when destination checks permit it.	☒ Pass ☐ Fail	-
UAT-16 Settings & logout	Review protection/settings controls, change one non-critical setting, then log out.	Setting behaviour is understandable and logout removes access to authenticated features.	☒ Pass ☐ Fail	-

B. Usability and User Experience Rating

Tick one rating for each statement: 1 = Strongly Disagree, 2 = Disagree, 3 = Neutral, 4 = Agree, 5 = Strongly Agree.

Statement	1	2	3	4	5
PhishGuard was easy to understand and navigate.	☐	☐	☐	☐	☒

Statement	1	2	3	4	5
The installation and first-time setup process was clear.	☐	☐	☐	☐	☒
Website scan results were displayed quickly enough for normal browsing.	☐	☐	☐	☐	☒
The website warning clearly explained why a page was considered risky.	☐	☐	☐	☒	☐
Manual website scanning was easy to use.	☐	☐	☐	☐	☒
Email/Gmail risk information was clear and useful.	☐	☐	☐	☒	☐
The Security Dashboard was easy to understand.	☐	☐	☐	☐	☒
PDF reports contained useful and understandable information.	☐	☐	☐	☐	☒
Personal trust controls were clear and manageable.	☐	☐	☐	☐	☒
Account security and MFA behaviour was understandable.	☐	☐	☐	☐	☒
The Password Vault was easy to unlock and use.	☐	☐	☐	☐	☒
Overall, I would be comfortable using PhishGuard for phishing protection.	☐	☐	☐	☐	☒

C. Overall Acceptance

☒ Accepted - the system is suitable for its intended use.

☐ Accepted with Minor Changes - the system is usable, but minor improvements are recommended.

☐ Not Accepted - important issues prevent acceptable use.

Most useful feature(s)	The Gmail email scanning feature was useful because the phishing-risk result appeared directly inside the opened email with an explanation.
Problem(s) / confusing behaviour	-
Recommended improvement(s)	-

Tester Confirmation

I confirm that I personally attempted the UAT tasks above. The ratings, comments, acceptance decision and confirmation reflect my own experience using PhishGuard.

Tester Name / Pseudonym	Amresh	Date	16/08/2026
Signature / Confirmation		Email (optional)	-
Organization Signature			

5.3.8 Testing Discussion

The current technical evidence indicates that PhishGuard has been developed with a strong regression-oriented engineering process. The website and email detectors are not evaluated only through accuracy: precision, recall, false-positive behaviour, class balance, confusion matrices, leakage controls, calibration, and realistic regression cases are documented. This is particularly important for phishing protection because a high overall accuracy can hide either a large false-negative security gap or an excessive false-positive usability problem.

The completed UAT complements the automated evidence by showing that three target users were able to operate the full PhishGuard workflow successfully. The 48/48 practical-task passes demonstrate that the tested installation, authentication, scanning, trust, reporting, security-event, and vault workflows were usable in the participants' environments, while the 4.86/5 combined usability mean indicates strong perceived clarity and usability. The feedback also identifies two reasonable refinement areas: trust actions could provide even clearer confirmation, and future research could investigate certificate or SSL-related legitimacy signals alongside the current URL and contextual evidence. Because the sample contains only three testers and all three have cybersecurity-related occupations, the results demonstrate acceptance within this small technically experienced group and should not be generalized to all end users without broader future testing.

5.4 Summary

Chapter 5 combines automated technical testing, model evaluation, regression evidence, and completed user acceptance testing. The latest source contains 156 Python test functions and twelve extension Node.js programs; all twelve Node programs were independently executed successfully during report preparation, while the latest project verification record reports the Python suite as passing. The frozen website model achieves a low false-positive rate but limited phishing recall and is therefore supported by a transparent contextual layer, while the email model performs substantially more strongly on its leakage-safe holdout. UAT was completed by three target users: all 48 practical task executions passed, all three testers accepted the system for its intended use, and the combined usability mean was 4.86/5. The results therefore provide user-facing evidence that complements the automated and model-level evaluation, while the small technically experienced tester sample remains an acknowledged limitation.

CHAPTER 6: CONCLUSION

6.1 Critical Evaluation

The final project has achieved the main objective of designing and developing a supervised-learning-based phishing protection system for both websites and emails. PhishGuard is no longer only an investigation concept: it is a functioning Manifest V3 extension connected to a hosted Flask backend, two trained and versioned supervised-learning models, a deployed PostgreSQL database, and a set of security and usability features that support realistic use. The extension can automatically inspect supported website navigations, manually scan URLs, scan the currently opened visible Gmail message, explain risk, warn before high-risk navigation, record privacy-safe history, and export security reports.

The project also demonstrates a useful engineering principle: model performance and user-facing security decisions should not be treated as identical. The website Random Forest is intentionally preserved with its measured 33.98% recall rather than having its historical metrics rewritten after runtime changes. Contextual analysis catches important combinations of suspicious structure that the URL-only model misses, and personal trust is represented separately from both raw model and contextual risk. The email detector uses the same separation between base model probability and final contextual risk. This improves transparency and makes the project easier to evaluate academically.

A further strength is security integration. Mandatory MFA, recovery codes, progressive MFA lockouts, opaque sessions, security events, explicit CORS configuration, protected report ownership, and client-side vault encryption demonstrate that a phishing extension itself should not create a new weak point. The password vault is outside the core research objective, but it creates a coherent security narrative: PhishGuard can warn about phishing and also restrict credential filling on unsafe destinations. The vault design also shows a clear trust boundary because plaintext credentials and the vault master password are not stored by the Flask backend.

The project contributes to ordinary users, students, and working adults by offering protection at the point where browsing and email decisions are made. It also contributes academically by providing an explainable applied comparison between a URL-feature Random Forest and a TF-IDF LinearSVC email classifier, together with real-world regression handling, user-specific trust, and browser delivery. These characteristics support the original survey findings that users value direct warnings, fast detection, ease of use, and understandable explanations. The completed UAT further supports practical usability within the tested group: three participants completed all sixteen tasks successfully, all three accepted the system, and the combined usability mean was 4.86/5.

6.2 Limitation

Website model recall: The frozen URL-only Random Forest recall is 33.98%, so it misses many phishing URLs when evaluated without the runtime contextual layer. The contextual layer improves specific structural false-negative classes but does not change the model recall metric.

Dataset generalization: Both models rely on historical/public datasets and supplements. Independent modern cross-dataset validation is still limited, and distribution drift remains a long-term concern.

Gmail-specific automatic email extraction: Automatic DOM extraction is implemented for Gmail only. Manual pasted email text is available elsewhere, but other webmail interfaces are not automatically supported.

No live domain reputation / WHOIS path: The runtime deliberately excludes third-party domain-age and reputation queries to avoid privacy disclosure, rate limits, latency, and unreliable demonstrations. This removes a potentially valuable signal.

Process-local source rate limiter: Password/source throttling is process-local and is not suitable for multi-instance production deployment without Redis or another shared limiter.

MFA and recovery limitations: Loss of both the authenticator and all recovery codes has no self-service bypass. Changing the MFA encryption key without preserving the original also makes stored TOTP secrets unreadable.

Extension token storage: The opaque bearer session is stored in chrome.storage.local. It is isolated from ordinary page JavaScript, but browser-profile compromise remains a threat.

Vault threat model: The vault protects data at rest and in transit to the backend, but plaintext necessarily exists in the browser during display, copy, save, generation, or fill. Malicious extensions, page scripts, keyloggers, or a compromised browser may still capture it.

Local history: Security Dashboard scan history and signed receipts are stored locally in the extension and are not a complete cross-device audit system. Clearing extension storage removes this history.

Browser-level validation and UAT sample size: Automated helper tests do not reproduce every Chrome and Gmail runtime condition. UAT was completed successfully by three testers, but all three had cybersecurity-related occupations, so the acceptance results represent a small technically experienced sample rather than the wider general-user population.

Prototype distribution: The static public website and Chrome Web Store listing are not deployed/published as part of the final prototype.

6.3 Recommendation

1. Collect a larger independent modern URL dataset and investigate new feature/model combinations that improve phishing recall without returning to the long-legitimate-URL false-positive problem.
2. Validate the email classifier on an independent modern corpus from a different source and time period, and continue maintaining leakage-safe grouped splits and realistic transactional/newsletter regression gates.
3. Add optional reputation, certificate, redirect-chain, and domain-age signals through privacy-conscious asynchronous services rather than making third-party lookup a mandatory runtime dependency.
4. Extend automatic webmail support beyond Gmail through provider-specific adapters while retaining minimal permissions and visible-content-only processing.
5. Replace process-local source throttling with a shared production limiter such as Redis or a database-backed distributed mechanism before any multi-instance deployment.
6. Add WebAuthn/passkey support for account authentication and optionally for local vault unlock without creating a server-side vault decryption path.
7. Add automated end-to-end Chrome tests for warning navigation, Gmail DOM behaviour, PDF downloads, trust synchronization, and vault fill, complementing the current helper-level Node tests.
8. Introduce managed secret rotation and backup procedures for MFA encryption keys and production configuration if the prototype is operated beyond an academic demonstration.
9. Expand future UAT to a larger and more diverse non-technical user group, and refine trust-action confirmation wording based on the completed tester feedback. Certificate or SSL-related signals may also be investigated as optional contextual evidence without treating certificate presence alone as proof of legitimacy.
10. Keep model, contextual-risk, personal-trust, and user-navigation decisions separately represented in future versions so improvements remain auditable and do not obscure the real ML evaluation.

REFERENCES

Anti-Phishing Working Group. (2025). *Phishing activity trends report: Fourth quarter 2024*.

https://docs.apwg.org/reports/apwg_trends_report_q4_2024.pdf

Asiri, S., Xiao, Y., Alzahrani, S., Li, S., & Li, T. (2023). A survey of intelligent detection designs of HTML URL phishing attacks. *IEEE Access*, 11, 6421-6443. <https://doi.org/10.1109/ACCESS.2023.3237798>

Asiri, S., Xiao, Y., Alzahrani, S., & Li, T. (2024). PhishingRTDS: A real-time detection system for phishing attacks using a deep learning model. *Computers & Security*, 141, 103843.

<https://doi.org/10.1016/j.cose.2024.103843>

Breiman, L. (2001). Random forests. *Machine Learning*, 45, 5-32. <https://doi.org/10.1023/A:1010933404324>

Fares, H., Kilani, J., Fagroud, F. E., Toumi, H., Lakrami, F., Baddi, Y., & Akin, N. (2024). Machine learning approach for email phishing detection. *Procedia Computer Science*, 251, 746-751.

<https://doi.org/10.1016/j.procs.2024.11.179>

Federal Bureau of Investigation, Internet Crime Complaint Center. (2025). *2024 IC3 annual report*.

https://www.ic3.gov/AnnualReport/Reports/2024_IC3Report.pdf

Google Chrome Developers. (n.d.). *Chrome Extensions API reference*. Retrieved August 27, 2026, from

<https://developer.chrome.com/docs/extensions/reference/api>

Jain, S., Jotwani, V., & Agrawal, S. (2024). Comparative analysis of agile frameworks: Evaluating their impact on software development lifecycle efficiency. *International Journal of Intelligent Systems and Applications in Engineering*, 12(23s), 3126-3137. <https://ijisae.org/index.php/IJISAE/article/view/7627>

Kavya, S., Karthik, S., Elakkiya, K., Manikandan, R., & Sudhakar, S. (2024). Staying ahead of phishers: A review of recent advances and challenges in phishing attack detection. *Artificial Intelligence Review*.

<https://doi.org/10.1007/s10462-024-11055-z>

- Linh, D. M., Ha, H. D., Chau, H. M., Vu, Q. S., & Tran, T.-N. (2024). Real-time phishing detection using deep learning methods by extensions. *International Journal of Electrical and Computer Engineering*, 14(3), 3021-3035. <https://doi.org/10.11591/ijece.v14i3.pp3021-3035>
- M'Raihi, D., Machani, S., Pei, M., & Rydell, J. (2011). *TOTP: Time-based one-time password algorithm* (RFC 6238). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc6238>
- Meléndez, R., Ptaszynski, M., & Masui, F. (2024). Comparative investigation of traditional machine-learning models and transformer models for phishing email detection. *Electronics*, 13(24), 4877. <https://doi.org/10.3390/electronics13244877>
- Naqvi, B., Perova, K., Farooq, A., Makhdoom, I., Oyedeji, S., & Porras, J. (2023). Mitigation strategies against phishing attacks: A systematic literature review. *Computers & Security*, 132, 103387. <https://doi.org/10.1016/j.cose.2023.103387>
- National Institute of Standards and Technology. (2007). *Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC* (NIST Special Publication 800-38D). <https://doi.org/10.6028/NIST.SP.800-38D>
- National Institute of Standards and Technology. (2025). *Digital identity guidelines: Authentication and authenticator management* (NIST Special Publication 800-63B-4). <https://doi.org/10.6028/NIST.SP.800-63B-4>
- Omari, K. (2023). Phishing detection using gradient boosting classifier. *Procedia Computer Science*, 230, 120-127. <https://doi.org/10.1016/j.procs.2023.12.067>
- OWASP Foundation. (n.d.-a). *Authentication cheat sheet*. Retrieved August 27, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- OWASP Foundation. (n.d.-b). *Multifactor authentication cheat sheet*. Retrieved August 27, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Multifactor_Authentication_Cheat_Sheet.html

OWASP Foundation. (n.d.-c). *Password storage cheat sheet*. Retrieved August 27, 2026, from

https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P.,

Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., &

Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning*

Research, 12, 2825-2830. <https://www.jmlr.org/papers/v12/pedregosa11a.html>

Rashid, F., Doyle, B., Han, S. C., & Seneviratne, S. (2024). Phishing URL detection generalisation using unsupervised domain adaptation. *Computer Networks*, 245, 110398.

<https://doi.org/10.1016/j.comnet.2024.110398>

Safi, A., & Singh, S. (2023). A systematic literature review on phishing website detection techniques. *Journal of King Saud University - Computer and Information Sciences*, 35(2), 590-611.

<https://doi.org/10.1016/j.jksuci.2023.01.004>

Shafin, S. S. (2025). An explainable feature selection framework for web phishing detection with machine learning. *Data Science and Management*, 8, 127-136. <https://doi.org/10.1016/j.dsm.2024.08.004>

Sitorus, G. B. M. T., Efendi, R. A. G., Napitupulu, J. F., Pranoto, H., & Hermawan, E. S. (2025). Bridging the gap: A comparative evaluation of Agile, Waterfall, and hybrid methodologies in modern software projects. *Procedia Computer Science*, 269, 1259-1268. <https://doi.org/10.1016/j.procs.2025.09.067>

Tetteh, S. G. (2024). Empirical study of agile software development methodologies: A comparative analysis. *Asian Journal of Research in Computer Science*, 17(5), 30-42.

<https://doi.org/10.9734/ajrcos/2024/v17i5436>

Turan, M. S., Barker, E., Burr, W. E., & Chen, L. (2010). *Recommendation for password-based key derivation: Part 1: Storage applications* (NIST Special Publication 800-132). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-132>

- Wilk-Jakubowski, J. L., & Pawlik, A. (2025). Machine learning and neural networks for phishing detection: A review. *Electronics*, 14(18), 3744. <https://doi.org/10.3390/electronics14183744>
- Wu, C.-Y., Kuo, C.-C., & Yang, C.-S. (2023). Phishing detection with browser extension based on machine learning. In *2023 18th Asia Joint Conference on Information Security (AsiaJCIS)* (pp. 81-87). IEEE. <https://doi.org/10.1109/AsiaJCIS60284.2023.00023>

APPENDICES

Appendix A: PPF – Title Registration Proposal

Project Title Proposal		
Project Title: Development of a Browser Extension for Phishing Website and Email Detection Using Supervised Learning	Status	APPROVED
Proposed By	HEMANTHRAJAH A/L N KRISHNARAJAH	
Description	Introduction Phishing scams are among the most prevalent and effective types of cybercrimes since they involve deceiving websites and emails to steal valuable information in the form of login details, financial, and personal information. As online communication and Internet-based services become more and more dependable, attackers are increasingly perfecting phishing methods to evade the more traditional rule-based security systems. Traditional detection methods, including the blacklist-based filtering and the static signature matching method, fail most of the time to combat the fresh phishing attacks. The recent progress in Artificial Intelligence (AI) and Machine Learning (ML) has made the security solutions more adaptive and intelligent. In particular, supervised learning methods have demonstrated good prospects of detecting phishing patterns through learning on labelled data sets of legitimate and malicious information. Machine learning models can effectively identify phishing attacks by the characteristics of the URLs of the websites, the text of the webpage, email, and metadata and enhance the detection future as time goes. In this project, it is proposed to design and develop a phishing detection system, which will be based on supervised learning models to detect phishing websites and phishing emails. The system will offer an effective, scalable and data-driven method to phishing detection that will help users and organisations to reduce the cyber risks related to phishing. Aim Design and develop a supervised learning-based system capable of accurately and efficiently detecting phishing websites and phishing emails by analysing their structural, textual, and behavioural features, in order to enhance cybersecurity and protect users from online threats. Objectives: *To analyze the common characteristics and techniques used in phishing websites and emails. *To collect and preprocess relevant datasets for the detection of phishing websites and emails. *To develop supervised machine learning models to classify websites and emails as either legitimate or phishing. *To evaluate the performance of the developed models using standard classification metrics. Targeted Users The end users of this system are: Internet users in general, who are also prone to phishing attacks via mail and malicious websites. Small and Medium Enterprises (SMEs) that prefer a phishing detection solution that is affordable and intelligent. IT Security Teams and System Administrators that tend to use automated tools to help them detect phishing threats. Researchers and Students who might want to use machine learning methods to solve real world cybersecurity issues. Expected Outcome The reason behind the success of this project is that it is expected to achieve a working phishing detection system that will show how a supervised learning method can be used to identify phishing websites and emails. The project will also offer comparative analysis of various models of supervised learning, their strengths, and limitations in the case of phishing.	
SDG	SDG9	
Keywords	Information security Cybersecurity Cybersecurity Phishing Detection Personalized Learning Cybersecurity	
Preferred Supervisor(s)	SHAHAB ALIZADEH, DR. KAMALAKANNAN MACHAP, SEVAKUMAR A/L JEVARETNAM, TAN LI JUNE, DHASON PADMA KUMAR	
Assigned Supervisor	dr.kamalakannan	
Assigned Second Marker	manimegalai	
Remarks		
Name	Remarks	Date
TS. UMAPATHY EAGANATHAN	Good.	Mar 3, 2026, 9:25:47 AM

Figure 104: PPF

Appendix B: Ethics Form

Supervisor Remarks

Name	Remarks	Date
DR. KAMALAKANNAN MACHAP	Significant ethics were followed in the work.	Apr 17, 2026, 7:47:32 AM

Ethics Form (Fast-Track)

1

2

3

4

Participant Confidentiality

Nature of Research

Target Participants

Support Information

Participant Confidentiality

Will you describe the main procedures to participants in advance, so that they are informed about what to expect?

☒ Yes

☐ No

☐ N/A

Will you tell participants that their participation is voluntary?

☒ Yes

☐ No

☐ N/A

Will you obtain written consent for participation?

☒ Yes

☐ No

☐ N/A

If the research is observational, will you ask participants for their consent to being observed?

☒ Yes

☐ No

☐ N/A

Will you tell participants that they may withdraw from the research at any time and for any reason?

☒ Yes

☐ No

☐ N/A

With questionnaires and interviews will you give participants the option of omitting questions they do not want to answer?

☒ Yes

☐ No

☐ N/A

Will you tell participants that their data will be treated with full confidentiality and that, if published, it will not be identifiable as theirs?

☒ Yes

☐ No

☐ N/A

Will you give participants the opportunity to be debriefed i.e. to find out more about the study and its results?

☒ Yes

☐ No

☐ N/A

Note: If you have ticked No to any of questions above, you should complete the Full Ethics Approval Form.

Full Ethics Form

Figure 105: Ethics Form Page 1

Supervisor Remarks		
Name	Remarks	Date
DR. KAMALAKANNAN MACHAP	Significant ethics were followed in the work.	Apr 17, 2026, 7:47:32 AM

Ethics Form (Fast-Track)							
1 Participant Confidentiality		2 Nature of Research		3 Target Participants		4 Support Information	
Nature of Research							
Will your project/assignment deliberately mislead participants in any way?		☐ Yes	☒ No			☐ N/A	
Is there any realistic risk of any participants experiencing either physical or psychological distress or discomfort? This should include details of what you will tell participants to do if they should experience any problems (e.g., who they can contact for help). You may also need to consider risk assessment issues.		☐ Yes	☒ No			☐ N/A	
Is the nature of the research such that contentious or sensitive issues might be involved? This includes research which could induce psychological stress, anxiety or humiliation, or cause more than minimal pain.		☐ Yes	☒ No			☐ N/A	
Does your research involve the use of sensitive materials? E.g., records of personal or sensitive confidential information.		☐ Yes	☒ No			☐ N/A	
Does your research require external agency approval?		☐ Yes	☒ No			☐ N/A	
Does your research use hazardous or controlled substance?		☐ Yes	☒ No			☐ N/A	
Does your research require you to visit participants in their home or non-public space?		☐ Yes	☒ No			☐ N/A	
Does your research investigate illegal activities or behaviours?		☐ Yes	☒ No			☐ N/A	
Will your research data be used in the future after the conclusion of your project?		☐ Yes	☒ No			☐ N/A	
Will your research involve in processing sensitive data belonging to an organisation/persons?		☐ Yes	☒ No			☐ N/A	
Will your research be collecting photographs, videos, and audio recordings of the participants?		☐ Yes	☒ No			☐ N/A	
Will the participants' personal particulars be known to any third party?		☐ Yes	☒ No			☐ N/A	
Will the participants' data confidentiality be made known to the public?		☐ Yes	☒ No			☐ N/A	
Will the research be conducted where the safety of the researchers maybe in question?		☐ Yes	☒ No			☐ N/A	
Will be the research be conducted outside of Malaysia and/or UK?		☐ Yes	☒ No			☐ N/A	
Note: If you have ticked YES to any of questions above, you should complete the Full Ethics Approval Form. Full Ethics Form							

Figure 106: Ethics Form Page 2

Supervisor Remarks

Name	Remarks	Date
DR. KAMALAKANNAN MACHAP	Significant ethics were followed in the work.	Apr 17, 2026, 7:47:32 AM

Ethics Form (Fast-Track)

1

Participant Confidentiality

2

Nature of Research

3

Target Participants

4

Support Information

Target Participants

Do participants fall into any of the following special groups?

☐ Yes

☒ No

☐ N/A

- Children (under 18 years of age)
- People with communication or learning difficulties
- Patients
- People in custody
- People who could be regarded as vulnerable
- People engaged in illegal activities (e.g., drug taking)
- Groups of people whose relationship among each other allow one to have influence over the other such as: Carers and patients with chronic conditions; teachers and their students; prison authorities and prisoners; employers and employees
- Groups where permission of a gatekeeper is normally required for initial access to members.

Note: You may also need to obtain satisfactory clearance from the relevant authorities.

Does the project/assignment involve external funding or external collaboration where the funding body or external collaborative partner requires the University to provide evidence that the project/assignment had been subject to ethical scrutiny?

☐ Yes

☒ No

☐ N/A

Note: If you have ticked Yes to any of questions above, you should complete the Full Ethics Approval Form. There is an obligation on student and supervisor to bring to the attention of the APU School Research Ethics Committee any issues with ethical implications not clearly covered by the above checklist.

Full Ethics Form

Figure 107: Ethics Form Page 3

227 | Page

Supervisor Remarks

Name	Remarks	Date
DR. KAMALAKANNAN MACHAP	Significant ethics were followed in the work.	Apr 17, 2026, 7:47:32 AM

Ethics Form (Fast-Track)

1Participant Confidentiality

2Nature of Research

3Target Participants

4Support Information

Support Information

I consider that this project/assignment has no significant ethical implications requiring a full ethics submission to the APU Research Ethics Committee.

☒

I am aware of APU liability policy and will make the necessary arrangement for insurance coverage of all researchers and participants of the project/assignment.

☒

Description

Participants for this study will consist of internet users, including students and working adults, who frequently use websites and email platforms. Participants will be recruited through online invitation methods, including the sharing of survey links via communication platforms such as WhatsApp, Telegram, and email. Participation will be voluntary, and all participants will be informed of the study's purpose prior to their involvement.

Data will be collected through a structured questionnaire and, if necessary, brief interviews to gather information regarding phishing awareness, previous

Consent Form
(Maximum 3 files)

Information Sheet

Additional Files

I also confirm that:

Note: Please check either 1 of the checkboxes.

i) All key documents e.g., consent form, information sheet, questionnaire/interview, and all material such as emails and posters for the purpose of recruitment of participants are appended to this application.

☐

ii) Any key documents e.g., consent form, information sheet, questionnaire/interview schedules which need to be finalised following initial investigations will be submitted for approval by the project/assignment supervisor/module lecturer before they are used in primary data collection.

☒

Figure 108: Ethics Form Page 4

228 | Page

Appendix C: Log Sheets

Meeting Logs

APPROVED

Meeting Log

Title

Development of a Browser Extension for Phishing Website and Email Detection Using Supervised Learning

Supervisor Name

dr.kamalakannan

Created At

22-04-2026 05:04 AM

Meeting Name

Meeting 1

Meeting Schedule Date

31-03-2026 03:35 pm

Location

ONLINE MEETING

Content

Items For Discussion

Discussed the required format and structure of the Investigation Report based on the university guideline. The supervisor reviewed the initial project direction and advised the developer to revise the project title so that it is clearer, more focused, and better aligned with the actual scope of the study. The project aim and objectives were also discussed, and guidance was given to make the aim more specific and the objectives more measurable and relevant to the proposed browser extension for phishing website and email detection. The supervisor also reminded the developer to ensure that Chapter 1 clearly establishes the problem background, scope, and project direction.

Content of Discussion

The supervisor explained the expected format of the Investigation Report and advised the developer to follow the guideline carefully. The initial project title was reviewed and it was suggested that the title should be revised to make it clearer, more focused, and more aligned with the actual scope of the study. The project aim and objectives were also discussed, and the supervisor advised that the aim should clearly state the overall purpose of the project while the objectives should be concise, measurable, and relevant to phishing website and email detection. Guidance was also given to ensure that Chapter 1 properly introduces the problem background and project scope.

Action List

Revise the project title. Refine the project aim and objectives. Update Chapter 1 based on the supervisor's comments. Follow the Investigation Report format more closely. Investigation Report format and structure, project title, project aim, project objectives, and overall project direction.

Download

Meeting Log Remarks

Name	Remarks	Date
DR. KAMALAKANNAN MACHAP	IR details were mentioned in the log sheets and successfully approved.	Apr 22, 2026, 4:09:14 AM

Figure 109: Meeting Log 1

Meeting Logs

APPROVED

Meeting Log

Title

Development of a Browser Extension for Phishing Website and Email Detection Using Supervised Learning

Supervisor Name

dr.kamalakannan

Created At

22-04-2026 12:06 PM

Meeting Name

Meeting 2

Meeting Schedule Date

14-04-2026 01:50 pm

Location

ONLINE MEETING

Content

Items For Discussion

Survey questionnaire, ethics-related survey content, Chapter 1, and Chapter 2.

Content of Discussion

The survey questionnaire was presented to the supervisor for review. The questions were discussed in terms of relevance, clarity, and suitability for collecting user awareness, experience, and expectations related to phishing and browser-based protection. The supervisor approved the survey for distribution. Chapter 1 and Chapter 2 were also shown and reviewed. Feedback was given to improve the academic flow of the writing, maintain stronger alignment between the literature review and the proposed system, and ensure that only recent and credible references were used. The supervisor was satisfied with the overall progress and advised the developer to continue with data collection and analysis.

Action List

Distribute the survey. Continue improving Chapter 1 and Chapter 2. Update references where necessary. Start preparing for survey analysis and Chapter 3.

Download

Meeting Log Remarks

Name	Remarks	Date
DR. KAMALAKANNAN MACHAP	Reviewed chapters 1 , 2 and survey questions.	Apr 22, 2026, 4:11:24 AM

Figure 110: Meeting Log 2

Meeting Logs

APPROVED

Meeting Log

Title

Development of a Browser Extension for Phishing Website and Email Detection Using Supervised Learning

Supervisor Name

dr.kamalakannan

Created At

22-04-2026 12:08 PM

Meeting Name

Meeting 3

Meeting Schedule Date

20-04-2026 03:35 pm

Location

ONLINE MEETING

Content

Items For Discussion

Unfinished Chapter 3, report format, expected word count, survey progress, and number of respondents.

Content of Discussion

The unfinished draft of Chapter 3 was presented, including the selected methodology, methodology comparison, data gathering design, and the direction of the survey analysis. The developer asked again about the expected format and word count of the Investigation Report to ensure that the final document would meet academic expectations. The survey findings were also discussed, including the fact that 44 valid respondents had been collected. The supervisor confirmed that this number was sufficient for the investigation stage and advised that the analysis should not only describe the survey results but also connect them clearly to system requirements, usability expectations, and the proposed browser extension design. Additional advice was given to make sure the report remains consistent in formatting, structure, and referencing.

Action List

Complete Chapter 3. Strengthen the survey analysis and link findings to system requirements. Recheck formatting and word count. Continue with Chapter 4 and final revisions.

Download

Meeting Log Remarks

Name	Remarks	Date
DR. KAMALAKANNAN MACHAP	IR details were mentioned in the log sheets and successfully approved.	Apr 22, 2026, 4:11:51 AM

Figure 111: Meeting Log 3

Meeting Logs

APPROVED

Meeting Log

Title

Development of a Browser Extension for Phishing Website and Email Detection Using Supervised Learning

Supervisor Name

dr.kamalakannan

Created At

22-04-2026 12:08 PM

Meeting Name

Meeting 4

Meeting Schedule Date

22-04-2026 12:00 pm

Location

ONLINE MEETING

Content

Items For Discussion

Final completed Investigation Report, overall structure, formatting, and readiness for submission.

Content of Discussion

The final completed Investigation Report was presented to the supervisor for final review. The supervisor checked the overall structure of the document, including the chapters, survey analysis, references, appendices, and formatting consistency. Minor comments were discussed regarding final polishing and submission readiness. The supervisor confirmed that the report was complete, properly aligned with the approved project direction, and ready for submission. Final approval was given to proceed with the Investigation Report submission.

Action List

Make any final minor corrections if needed. Finalize the document formatting. Prepare the report for submission. Submit the completed Investigation Report.

Download

Meeting Log Remarks

Name	Remarks	Date
DR. KAMALAKANNAN MACHAP	IR details were mentioned in the log sheets and successfully approved.	Apr 22, 2026, 4:10:42 AM

Figure 112: Meeting Log 4

Meeting Log — Meeting 5

Title: Development of a Browser Extension for Phishing Website and Email Detection Using Supervised Learning

Meeting No: Meeting 5

Created At: 2026-08-28 12:16

Supervisor Name: DR. KAMALAKANNAN MACHAP

Remarks History

SUPERVISOR	REMARK	DATE
Dr. Kamalakannan Machap	Report has reviewed and required to complete chapters 5 & 6. .	2026-08-28 12:20

Items for Discussion

Record of Discussion

Action List

Noted by students before the mandatory supervisory meeting

Demonstration of the current PhishGuard system and overall implementation progress.

Review of the Chrome extension, website phishing detection, email phishing detection, authentication and MFA, Security Dashboard, PDF reporting, personal trust features, and Password Vault.

Discussion of the connection between the browser extension, hosted backend, machine-learning models, and database.

Figure 113: Meeting Log 5

Meeting Log — Meeting 6

Title: Development of a Browser Extension for Phishing Website and Email Detection Using Supervised Learning

Meeting No: Meeting 6

Created At: 2026-08-28 12:17

Supervisor Name: DR. KAMALAKANNAN MACHAP

Remarks History

SUPERVISOR	REMARK	DATE
Dr. Kamalakannan Machap	Discussed and reviewed the chapter 4	2026-08-28 12:22

Items for Discussion

Record of Discussion

Action List

Noted by students before the mandatory supervisory meeting

Review of Chapter 4: Design and Implementation.

Review of system architecture, database design, interface design, implementation screenshots, and source-code evidence.

Discussion of the structure and clarity of Section 4.5 Implementation.

Figure 114: Meeting Log 6

Meeting Log — Meeting 7

Title: Development of a Browser Extension for Phishing Website and Email Detection Using Supervised Learning

Meeting No: Meeting 7

Created At: 2026-09-01 11:33

Supervisor Name: DR. KAMALAKANNAN MACHAP

Remarks History

SUPERVISOR	REMARK	DATE
Dr. Kamalakannan Machap	Reviewed the report for final submission. Feedback provided to complete the remaining work before submission.	2026-09-01 11:35

Items for Discussion	Record of Discussion	Action List
Noted by students before the mandatory supervisory meeting		
Final review of the completed FYP report and overall documentation quality. Review of Chapter 4 design and implementation evidence, Chapter 5 testing, Chapter 6, and final appendices. Confirmation that the report structure, figures, tables, references, screenshots, and supporting documents were generally satisfactory.		

Figure 115: Meeting Log 7

Appendix D: A3 Project Poster



Figure 116: PhishGuard A3 Project Poster

Appendix E: Gantt Charts

Part 1: Investigation Report Gantt Chart

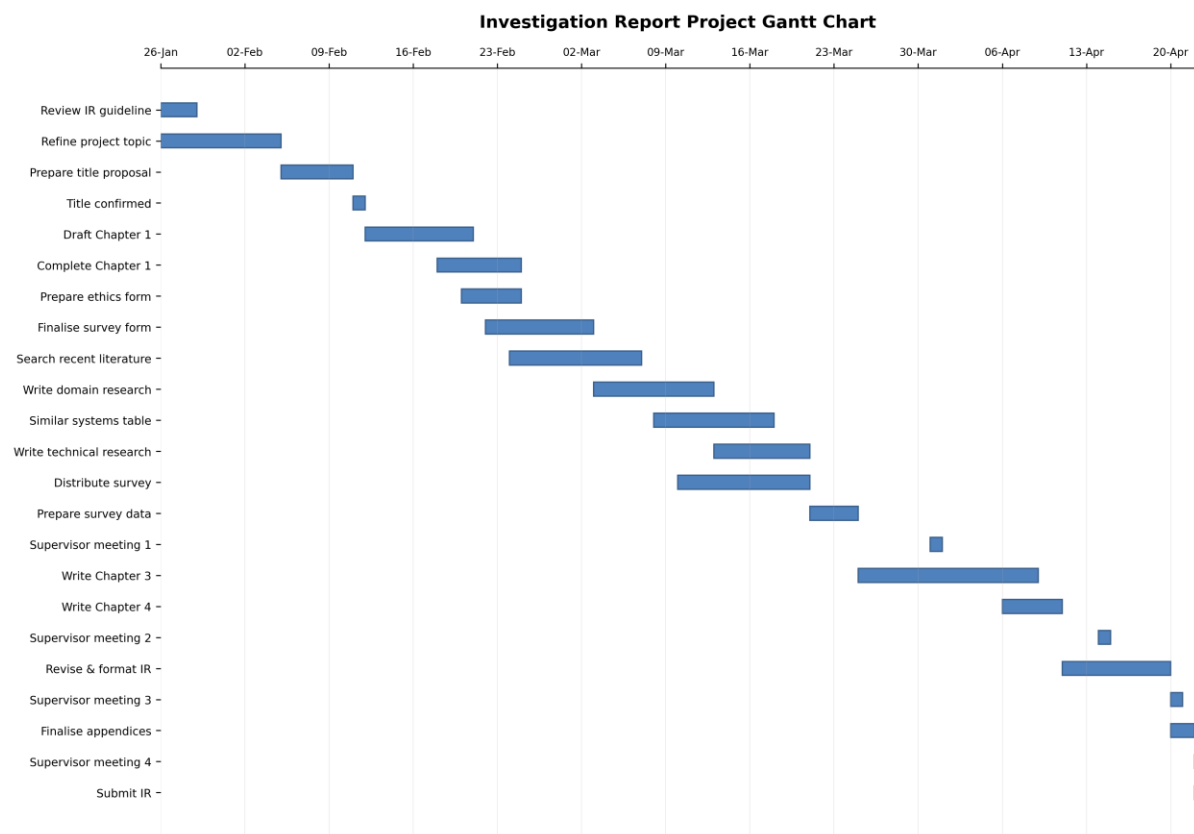


Figure 117: FYP Part 1 (Investigation Report) Gantt Chart

Figure 117 presents the original Part 1 Investigation Report schedule used during the research, literature review, survey, methodology, supervisor-review, and submission period. It is retained separately from the implementation schedule so the two FYP stages can be reviewed independently.

Part 2: Development, Testing and Final Submission Gantt Chart

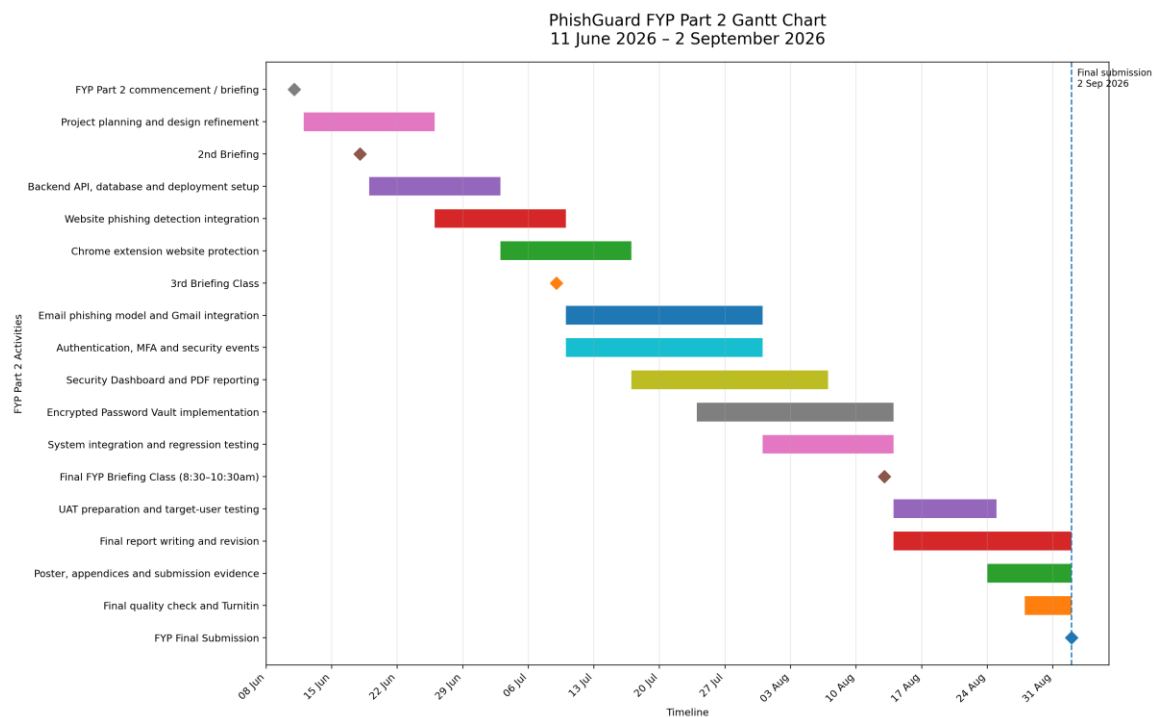


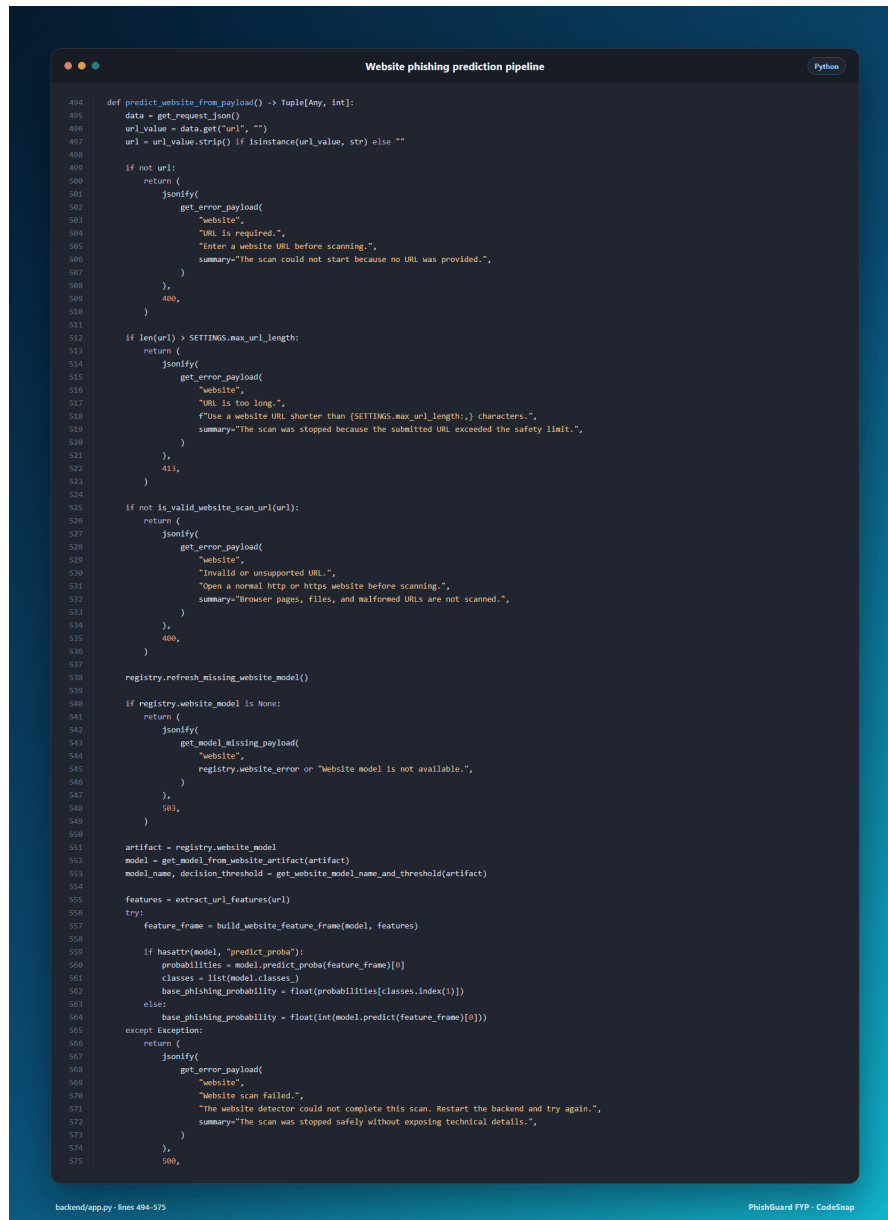
Figure 118: FYP Part 2 Gantt Chart

Figure 118 presents the FYP Part 2 implementation and submission timeline from 11 June 2026 to the final submission on 2 September 2026. It covers architecture refinement, website and email protection, authentication and MFA, dashboard/reporting, the encrypted Password Vault, integration and regression testing, UAT, final documentation, appendices, Turnitin, and submission preparation.

Appendix F: Selected Source Code Excerpts

The following three representative source-code excerpts are reproduced from the implementation evidence discussed in Section 4.6. They show the website prediction API, email-detection orchestration, and the authenticated account and MFA API routes.

F.1 Website Prediction API



```

404 def predict_website_from_payload() -> Tuple[Any, int]:
405     data = get_request_json()
406     url_value = data.get("url", "")
407     url = url_value.strip() if isinstance(url_value, str) else ""
408
409     if not url:
410         return (
411             jsonify(
412                 get_error_payload(
413                     "website",
414                     "URL is required.",
415                     "Enter a website URL before scanning.",
416                     summary="The scan could not start because no URL was provided.",
417                 ),
418             ),
419             400,
420         )
421
422     if len(url) > SETTINGS.max_url_length:
423         return (
424             jsonify(
425                 get_error_payload(
426                     "website",
427                     "URL is too long.",
428                     f"Use a website URL shorter than {SETTINGS.max_url_length} characters.",
429                     summary="The scan was stopped because the submitted URL exceeded the safety limit.",
430                 ),
431             ),
432             413,
433         )
434
435     if not is_valid_website_scan_url(url):
436         return (
437             jsonify(
438                 get_error_payload(
439                     "website",
440                     "Invalid or unsupported URL.",
441                     "Open a normal http or https website before scanning.",
442                     summary="Browser pages, files, and malformed URLs are not scanned.",
443                 ),
444             ),
445             400,
446         )
447
448     registry.refresh_missing_website_model()
449
450     if registry.website_model is None:
451         return (
452             jsonify(
453                 get_model_missing_payload(
454                     "website",
455                     registry.website_error or "Website model is not available.",
456                 ),
457             ),
458             503,
459         )
460
461     artifact = registry.website_model
462     model = get_model_from_website_artifact(artifact)
463     model_name, decision_threshold = get_website_model_name_and_threshold(artifact)
464
465     features = extract_url_features(url)
466     try:
467         feature_frame = build_website_feature_frame(model, features)
468
469         if hasattr(model, "predict_proba"):
470             probabilities = model.predict_proba(feature_frame)[0]
471             classes = list(model.classes_)
472             base_phishing_probability = float(probabilities[classes.index(1)])
473         else:
474             base_phishing_probability = float(int(model.predict(feature_frame)[0]))
475     except Exception:
476         return (
477             jsonify(
478                 get_error_payload(
479                     "website",
480                     "Website scan failed.",
481                     "The website detector could not complete this scan. Restart the backend and try again.",
482                     summary="The scan was stopped safely without exposing technical details.",
483                 ),
484             ),
485             500,
486         )

```

backend/app.py: line 484-573

PhishGuard FTP - CodeSnip

Figure 119: Website Prediction API Source Code

F.2 Email Detection Orchestration



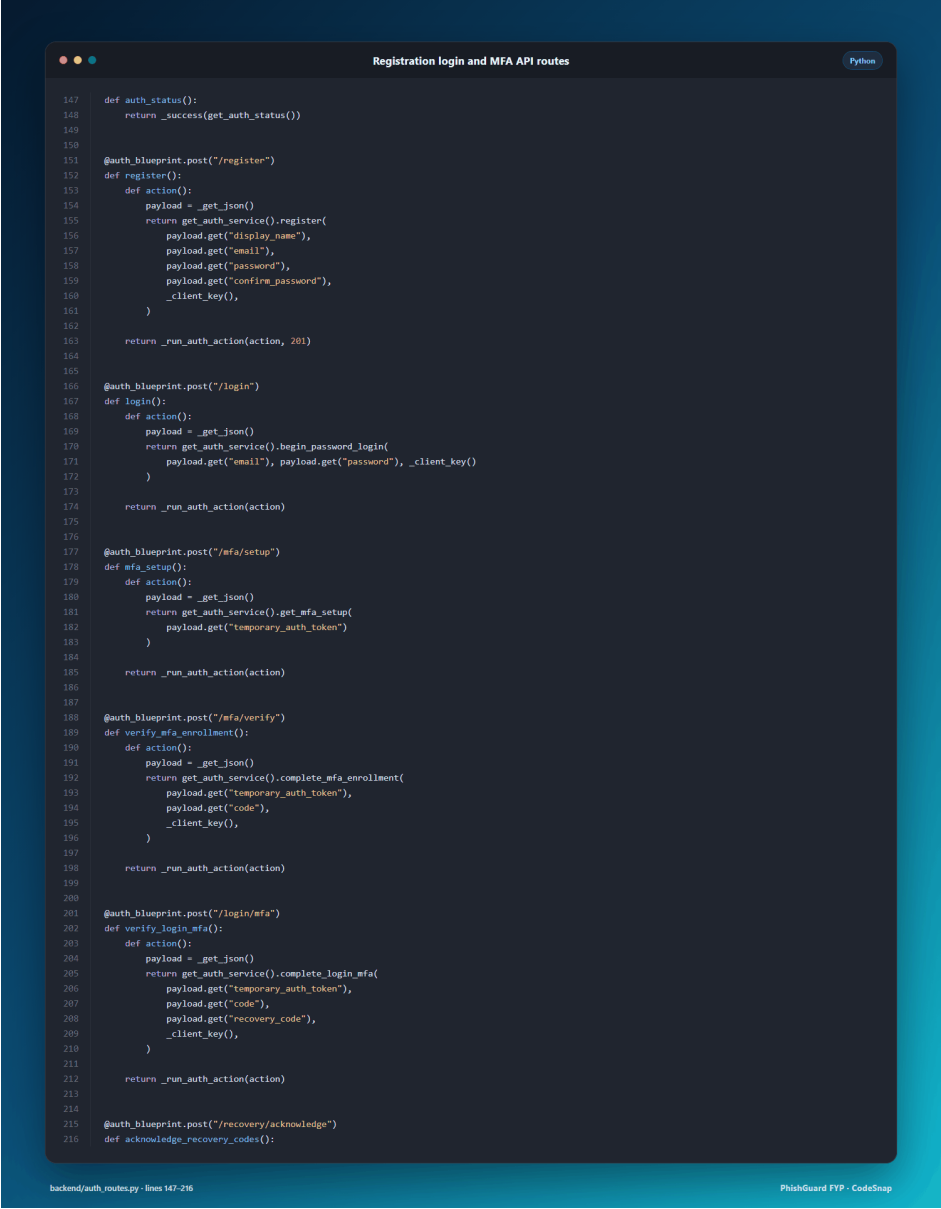
```

181 def detect_email(
182     model,
183     vectorizer,
184     email_input: Dict[str, Any],
185     decision_threshold: float,
186 ) -> Dict[str, Any]:
187     transformed = vectorizer.transform([clean_email_text(email_input["text"])])
188     base_probability = get_base_phishing_probability(model, transformed)
189     analysis = analyze_email_metadata(
190         sender=email_input["sender"],
191         sender_display_name=email_input["sender_display_name"],
192         subject=email_input["subject"],
193         text=email_input["body"],
194         raw_links=email_input["links"],
195     )
196     risk_result = adjust_email_risk(
197         base_probability,
198         analysis,
199         email_input.get("personal_trust"),
200     )
201     log_privacy_safe_diagnostic(email_input, analysis, risk_result)
202     final_risk_score = risk_result["final_risk_score"]
203     risk_classification = get_email_risk_classification(
204         final_risk_score,
205         decision_threshold,
206     )
207     reasons = build_email_explanations(
208         analysis,
209         base_probability,
210         final_risk_score,
211         decision_threshold,
212         risk_result,
213     )
214     evidence_breakdown = build_email_evidence_breakdown(
215         analysis,
216         base_probability,
217         final_risk_score,
218         decision_threshold,
219         risk_result,
220     )
221     original_risk_result = {
222         "risk_result": risk_result,
223         "final_risk_score": risk_result["original_contextual_risk_score"],
224         "applied_adjustments": [
225             item
226             for item in risk_result["applied_adjustments"]
227             if item.get("signal") != "personal_trust"
228         ],
229         "personal_trust": {},
230     }
231     original_evidence_breakdown = build_email_evidence_breakdown(
232         analysis,
233         base_probability,
234         risk_result["original_contextual_risk_score"],
235         decision_threshold,
236         original_risk_result,
237     )
238     summary = None
239     if email_input.get("summary_enabled", True):
240         summary = build_local_email_summary(
241             email_input["subject"],
242             email_input["body"],
243             analysis["brand_names"],
244             email_input["sender_display_name"],
245         )
246     return {
247         "prediction": int(final_risk_score >= decision_threshold),
248         "base_phishing_probability": round(base_probability, 4),
249         "original_contextual_risk_score": risk_result[
250             "original_contextual_risk_score"
251         ],
252         "final_risk_score": final_risk_score,
253         "risk_score_percent": f"{(final_risk_score * 100):.2f}%",
254         "risk_adjustment": risk_result["adjustment"],
255         "applied_adjustments": risk_result["applied_adjustments"],
256         "adjustments": risk_result["adjustments"],
257         "personal_trust": risk_result["personal_trust"],
258         "has_sovereign_evidence": bool(analysis.get("personal_trust_override")),
259         "risk_classification": risk_classification,
260         "reasons": reasons,
261         "evidence_breakdown": evidence_breakdown,
262         "original_evidence_breakdown": original_evidence_breakdown,
263         "summary": summary,
264         "summary_enabled": email_input.get("summary_enabled", True),
265         "user_message": build_email_user_message(risk_classification),
266         "sender_domain": analysis["sender_domain"],
267         "link_domains": analysis["link_domains"][:4],
268         "brand_names": analysis["brand_names"][:4],
269         "domain_reputation": analysis["domain_reputation"],
270         "authentication_available": analysis["authentication_available"],

```

Figure 120: Email Detection Orchestration Source Code

F.3 Authentication and MFA API Routes



```

147 def auth_status():
148     return _success(get_auth_status())
149
150
151 @auth_blueprint.post("/register")
152 def register():
153     def action():
154         payload = _get_json()
155         return get_auth_service().register(
156             payload.get("display_name"),
157             payload.get("email"),
158             payload.get("password"),
159             payload.get("confirm_password"),
160             _client_key(),
161         )
162
163     return _run_auth_action(action, 201)
164
165
166 @auth_blueprint.post("/login")
167 def login():
168     def action():
169         payload = _get_json()
170         return get_auth_service().begin_password_login(
171             payload.get("email"), payload.get("password"), _client_key()
172         )
173
174     return _run_auth_action(action)
175
176
177 @auth_blueprint.post("/mfa/setup")
178 def mfa_setup():
179     def action():
180         payload = _get_json()
181         return get_auth_service().get_mfa_setup(
182             payload.get("temporary_auth_token")
183         )
184
185     return _run_auth_action(action)
186
187
188 @auth_blueprint.post("/mfa/verify")
189 def verify_mfa_enrollment():
190     def action():
191         payload = _get_json()
192         return get_auth_service().complete_mfa_enrollment(
193             payload.get("temporary_auth_token"),
194             payload.get("code"),
195             _client_key(),
196         )
197
198     return _run_auth_action(action)
199
200
201 @auth_blueprint.post("/login/mfa")
202 def verify_login_mfa():
203     def action():
204         payload = _get_json()
205         return get_auth_service().complete_login_mfa(
206             payload.get("temporary_auth_token"),
207             payload.get("code"),
208             payload.get("recovery_code"),
209             _client_key(),
210         )
211
212     return _run_auth_action(action)
213
214
215 @auth_blueprint.post("/recovery/acknowledge")
216 def acknowledge_recovery_codes():

```

backend/auth_routes.py lines 147-216

PhishGuard FIP · CodeSnap

Figure 121: Authentication and MFA API Routes Source Code

Appendix G: Respondent Demographic Profile

To preserve confidentiality, respondents are represented using pseudonymous identifiers rather than real names. The demographic profile below is derived directly from the valid survey responses.

Participant	Gender	Age Group	Occupation
Respondent 01	Male	18 - 24	Student
Respondent 02	Male	18 - 24	Student
Respondent 03	Male	18 - 24	Student
Respondent 04	Male	18 - 24	Employed
Respondent 05	Male	18 - 24	Student
Respondent 06	Male	18 - 24	Student
Respondent 07	Male	18 - 24	Student
Respondent 08	Male	18 - 24	Employed
Respondent 09	Male	18 - 24	Student
Respondent 10	Female	18 - 24	Student
Respondent 11	Female	18 - 24	Student
Respondent 12	Male	18 - 24	Student
Respondent 13	Female	18 - 24	Student
Respondent 14	Male	18 - 24	Student
Respondent 15	Male	18 - 24	Student
Respondent 16	Female	18 - 24	Student
Respondent 17	Female	18 - 24	Student
Respondent 18	Female	45 and above	Employed
Respondent 19	Female	18 - 24	Student
Respondent 20	Male	18 - 24	Employed
Respondent 21	Female	18 - 24	Student
Respondent 22	Female	18 - 24	Student
Respondent 23	Male	25 - 34	Unemployed
Respondent 24	Female	35 - 44	Employed
Respondent 25	Female	25 - 34	Employed
Respondent 26	Male	18 - 24	Employed
Respondent 27	Female	18 - 24	Student
Respondent 28	Male	18 - 24	Student
Respondent 29	Male	45 and above	Employed
Respondent 30	Female	45 and above	Self-employed
Respondent 31	Female	45 and above	Unemployed
Respondent 32	Female	45 and above	Employed

Respondent 33	Female	35 - 44	Employed
Respondent 34	Female	35 - 44	Employed
Respondent 35	Male	25 - 34	Employed
Respondent 36	Male	45 and above	Self-employed
Respondent 37	Male	18 - 24	Student
Respondent 38	Male	45 and above	Employed
Respondent 39	Female	35 - 44	Employed
Respondent 40	Male	18 - 24	Student
Respondent 41	Male	18 - 24	Student
Respondent 42	Male	45 and above	Self-employed
Respondent 43	Male	18 - 24	Student
Respondent 44	Female	45 and above	Unemployed

Table 21: Respondent Demographic Profile